Continuous-Space Diffusion and Flow Language Models: A Survey

TokenRhythm Technologies

Abstract

Continuous-space diffusion and flow language models generate text by iteratively refining real-valued representations before decoding them into discrete tokens. Adapting continuous generative processes to discrete language must address rounding ambiguity and embedding collapse under squared-error regression, uninformative noise schedules, distribution mismatch between clean-text autoencoders and learned priors, and mode averaging in few-step sampling. We organize the literature by three coupled design dimensions: the representation being refined, the transition process used to update it, and the text interface that converts states into tokens. This framework supports comparisons across token-level and latent-space generators, which we trace across controllable generation and editing, sequence-to-sequence tasks, multimodal conditioning, discourse planning, code generation, and reasoning. Comparing published results against autoregressive and discrete-diffusion baselines, we find that continuous models can match same-scale autoregressive sample quality on prompt continuation, but still lag in held-out likelihood bounds and require substantially more training compute under current scaling laws. Their clearest empirical gains appear in gradient-guided structural control, plan-level diversity, image-to-text flow captioning, and task-specific reasoning after targeted fine-tuning. In the few-step regime, trajectory-consistent distillation and stochastic bridges prevent entropy collapse, yet standalone continuous generators have not surpassed optimized autoregressive baselines in end-to-end latency at matched quality. The wall-clock speedups reported to date rely on autoregressive verification of continuous drafts. We conclude with open problems in likelihood scaling, readout fidelity, few-step serving, representation reuse, and post-training.

Project Page: https://github.com/TokenRhythm/awesome-cdllm

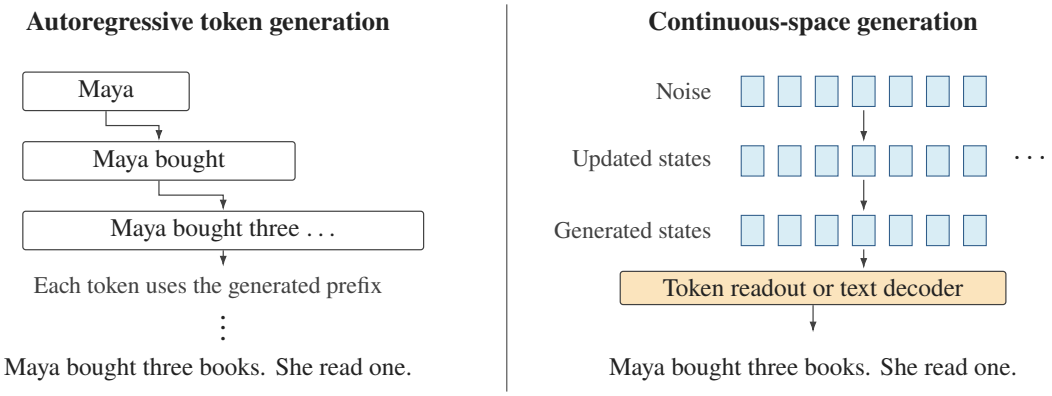


Figure 1. Two routes to text. An autoregressive model appends one token at a time to its prefix; in an illustrative full-sequence continuous path, a *transition* operator refines a real-valued *state* through a sequence of updates and decodes tokens through a *text interface* such as a vocabulary head, rounding, or a separate parallel or autoregressive decoder [213, 239]. Hybrid designs may also advance blockwise or commit tokens at intermediate steps.

Contents

1 Introduction **3**
- 1.1 Development of the field and empirical standing . . . 4
- 1.2 Scope and relation to prior surveys . . . 5
- 1.3 How to read the survey . . . 6

2 Background: From Text to Continuous Generation **7**
- 2.1 Problem formulation and notation . . . 7
- 2.2 Diffusion and flow matching, and what the denoiser predicts . . . 9
- 2.3 Why continuous text generation is hard . . . 11
- 2.4 Terminology and boundary with discrete diffusion . . . 13

3 The State: Text Representations **15**
- 3.1 Token embeddings and vocabulary states . . . 15
- 3.2 Contextual and pretrained encodings . . . 19
- 3.3 Compressed text representations . . . 19
- 3.4 Sentence and segment representations . . . 21

4 The Transition: Learning the Denoiser **22**
- 4.1 Prediction targets . . . 22
- 4.2 Training stages and module adaptation . . . 25
- 4.3 Noise schedules and self-conditioning . . . 28

5 Sampling, the Text Interface, and Generation Order **30**
- 5.1 Samplers and few-step routes . . . 31
- 5.2 From states to tokens . . . 35
- 5.3 Generation order and parallelism . . . 37
- 5.4 Conditioning and guidance . . . 40

6 Tasks, Evidence, and Evaluation **42**
- 6.1 Tasks and capabilities . . . 42
- 6.2 Evaluation measures . . . 47
- 6.3 Cross-paper empirical comparisons . . . 49

7 Open Problems **70**
- 7.1 The likelihood and compute gap . . . 71
- 7.2 Recovering precise text from generated states . . . 71
- 7.3 Few-step quality and end-to-end latency . . . 72
- 7.4 Reusing representations and extending length . . . 72
- 7.5 Interpreting and optimizing revisable computation . . . 73

8 Conclusion **73**

A Mathematical Details **75**

B Reconstruction and Generated-State Diagnostics **77**

C Boundary with Discrete Diffusion and Hybrid States **78**

D Experimental Protocols and Evaluation Settings **80**

E Literature Selection and Scope **82**

F Method and Configuration Lookup **82**

G Extended Catalog of Systems and Applications **88**

References **91**

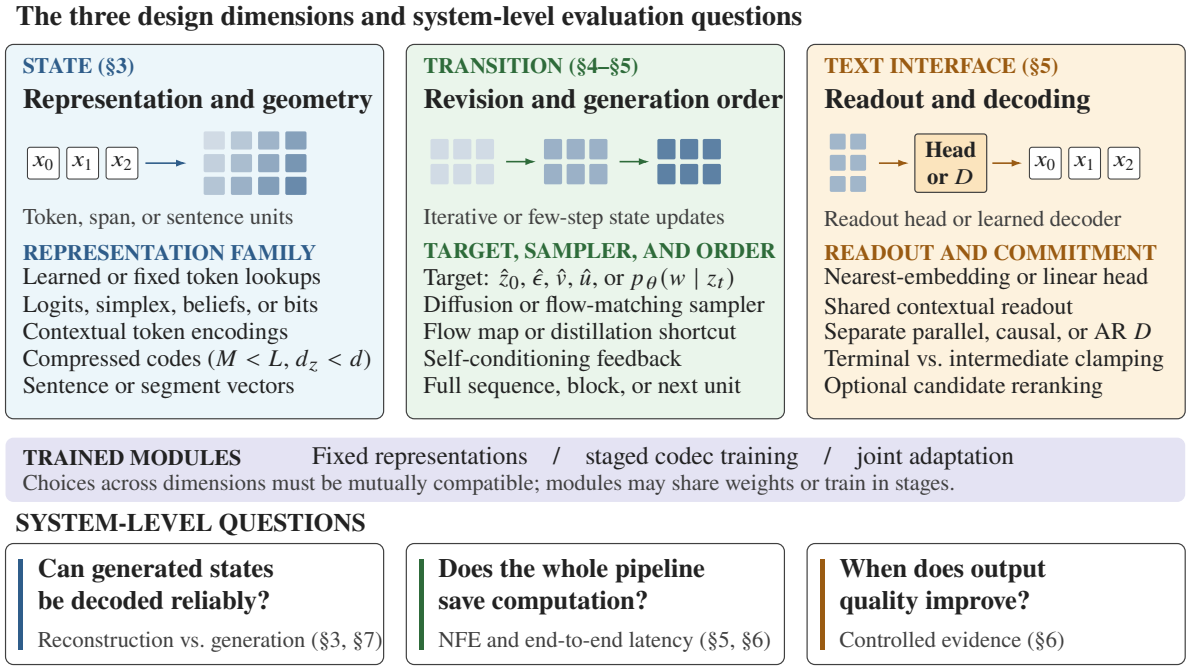


Figure 2. Overview of the three design dimensions and system-level evaluation questions in this survey (Section 1.3). Left to right: the *state* specifies what one generated unit represents and what geometry constrains it; the *transition* specifies how the denoiser revises the state and in what order positions or latent units are generated; and the *text interface* specifies where discrete token decisions enter, whether a shared head or a separate decoder produces the tokens, and whether decoding is parallel or autoregressive. Choices across these three dimensions must be mutually compatible so that prediction targets match the state geometry and the decoder tolerates the error distribution of generated states, while the three questions at the bottom guide system evaluation in Sections 3, 5, and 6.

1 Introduction

An autoregressive language model generates text by appending one token at a time to a prefix, committing each token as soon as it is emitted. Continuous-space diffusion and flow language models instead maintain a real-valued *state*, such as one vector per token position, a shorter array of compressed codes, or a single sentence vector, and refine that state over a sequence of updates [213, 239]. Updating continuous states in parallel across a sequence, span, or block decouples the number of denoiser evaluations from the token length of the generated unit and allows global revision of a draft. Because the intermediate state is differentiable, an external classifier or reward model can also steer generation directly by gradient for syntactic and attribute control [213]. Moreover, representing text as continuous vectors allows language models to share a generative formulation with images, audio, and speech, or to operate directly inside the representation space of a pretrained encoder or codec [211, 230, 317]. Figure 1 illustrates the autoregressive and continuous generation paths.

Because text is discrete and the evolving state is continuous, transferring image diffusion or flow matching directly to language encounters several recurring failure modes (Section 2.3). Under squared-error regression, *rounding ambiguity and embedding collapse* arise when a denoiser predicting $\mathbb{E}[z_0 \mid z_t]$ averages over plausible tokens to land between valid embeddings, or when a jointly trained embedding table shrinks toward a common point unless anchored by token-discrimination losses [111, 213]. *Uninformative noise schedules* borrowed from vision spend most of the training budget at noise levels where each token remains trivially recoverable from its own perturbed vector [111, 409]. In autoencoder spaces, *latent–prior mismatch* occurs when the encoder's clean-text latent geometry diverges from the distribution generated by the learned prior [195, 253]. Finally, *few-step mode averaging* degrades coarse deterministic samplers when conditional-mean predictions or uncalibrated velocity averages blend incompatible continuations into repetitive, high-frequency text [38, 287].

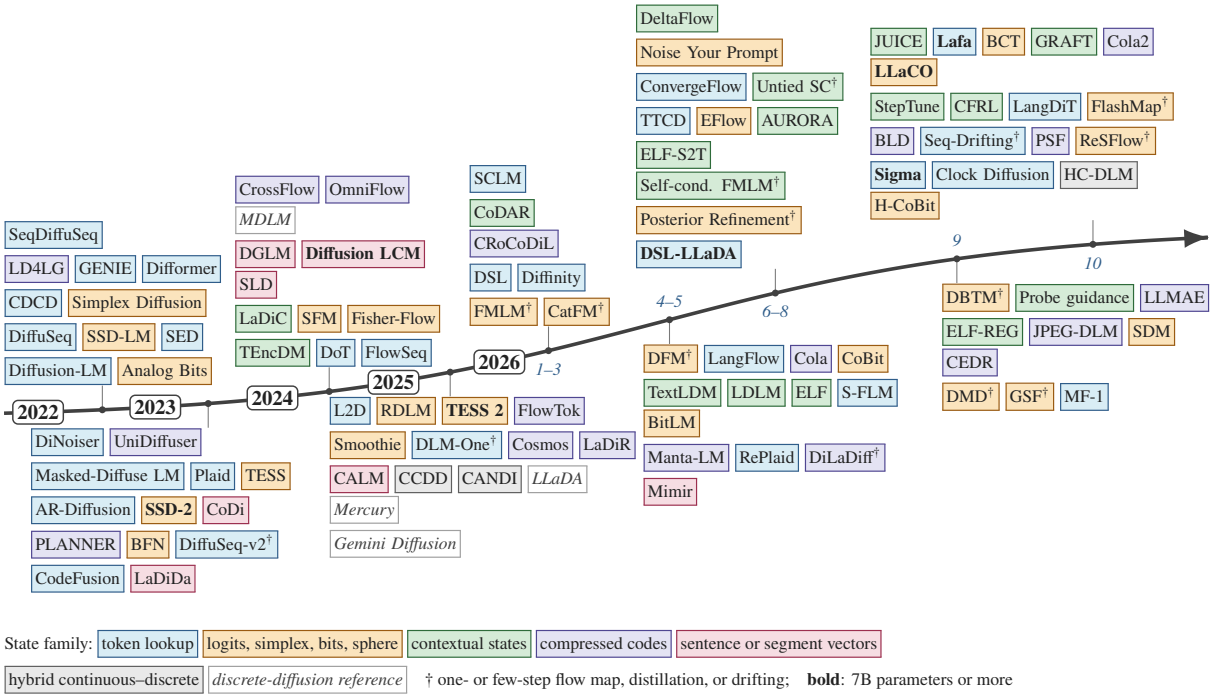


Figure 3. Timeline of continuous-space diffusion and flow language models from 2022 to October 2026, ordered by first public posting. Each box is a representative system or study, colored by the state family it refines (Section 3); gray italic boxes are discrete-diffusion reference points (Section 2.4). † marks one- or few-step flow-map, distillation, or drifting methods, and bold names mark systems with 7B or more parameters. The horizontal axis is not to scale: 2026 is split into January–March, April–May, June–August, September, and October, the months in which most recent work appeared.

1.1 Development of the field and empirical standing

Figure 3 traces the development of continuous-space language generation since 2022. Early systems such as Diffusion-LM, DiffuSeq, SED, CDCD, and SSD-LM established token-aligned continuous generation by corrupting learned or fixed token embeddings or vocabulary-sized logit vectors with Gaussian noise, introducing gradient steering, source conditioning, and token-anchoring losses to stabilize rounding [87, 119, 134, 213, 341]. In 2023, Plaid introduced end-to-end variational likelihood training that made continuous models directly comparable with autoregressive language models through held-out likelihood bounds, reported as perplexity or bits per character, while LD4LG and PLANNER moved continuous generation into compressed latent codes decoded by pretrained autoregressive models [128, 239, 441]. During 2024–25, large-scale pretraining shifted toward discrete masked diffusion: MDLM simplified the continuous-time masked objective, LLaDA trained an 8B masked model from scratch, and Dream adapted a 7B autoregressive backbone to match same-scale autoregressive models on standard benchmarks [268, 308, 408]. Over the same period, Mercury, Gemini Diffusion, and Seed Diffusion demonstrated high-throughput diffusion text generation, with Gemini Diffusion introduced as an experimental demo and Seed Diffusion as an experimental preview [123, 159, 339]. BD3-LM bridged blockwise autoregression and within-block diffusion, and continuous methods explored pretrained contextual states, sentence-level concept vectors, and simplex states adapted from masked or autoregressive backbones [36, 176, 190, 317, 344].

From 2025 onward, work on continuous representations focused on trajectory compression and latent codecs. One line of work compresses continuous trajectories into finite-interval flow maps or distilled few-step generators, as in FMLM, Categorical Flow Maps, DLM-One, and DiLaDiff [63, 86, 191, 195, 304]. Another pairs latent flow priors with staged or co-adapted text codecs, as in Cola, AURORA, and TextLDM [131, 166, 220]. Contextual-state flows such as ELF report stronger few-step generation quality than discrete

baselines, LangFlow develops ODE-based likelihood evaluation and RePlaid narrows the likelihood-bound gap to discrete diffusion, and Sigma scales Plaid-style likelihood-trained embedding diffusion to 3B and 8B parameters by adapting autoregressive checkpoints to blockwise diffusion [65, 153, 405, 406]. In subsequent model adaptations, Lafa converts 0.5B–8B autoregressive checkpoints to blockwise asynchronous flows on 9–27B instruction tokens [56], LLaCO adapts an 8B autoregressive–diffusion hybrid checkpoint to spherical diffusion on 1.7B tokens [164], and Clock Diffusion adds position-dependent noise schedules to support key–value caching and variable-length generation [312]. In latent compression, multi-token latents are paired with parallel decompression in JPEG-DLM [426], branching autoregressive decoding in BLD [103], parallel block-autoregressive decoding of prefix states in Predictive-State Flows [140], and block-causal masked-diffusion decoding in Cola2 [132]. In parallel, distribution-matching distillation, Gumbel Straight Flow, Discrete Beckmann Transport, and Seq-Drifting target one- and few-step generation [183, 228, 348, 358], while ELF-REG, CEDR, and pathwise reinforcement learning extend continuous flows to mathematical reasoning, code, and verifier-rewarded post-training [72, 137, 218].

Comparing continuous models against autoregressive and discrete-diffusion baselines requires separating sample quality, held-out likelihood bounds, and model scale (Section 6.3). On open-ended text, continuous models have not shown a general advantage over equal-compute autoregressive baselines, though well-calibrated token-embedding diffusion can match an equal-scale autoregressive model in prompt-continuation sample quality [87]. In held-out likelihood evaluations, continuous models' variational upper bounds (NLL $\leq \mathcal{B}_{\text{var}}$) and corrected probability-flow ODE estimates yield higher perplexities than matched autoregressive baselines [65, 406]. Variational-bound scaling studies additionally find a large training-compute multiple to match exact autoregressive negative log-likelihood (Section 7.1) [128, 406]. Regarding scale, the largest continuous generator trained from scratch is the 7B sentence-level Two-Tower LCM, trained on 2.7T tokens over frozen SONAR sentence embeddings and decoded through the frozen SONAR decoder [190]. Token-level continuous models trained from scratch include 1–2B systems, Sigma's nominal 3B training-loss ablation, and hyperspherical SwaFlow (up to 3.741B including embeddings, with 2T-token scaling runs and a final 4T-token model) [17, 405]. Other 3B–13B token-level systems adapt pretrained autoregressive or masked backbones, including SSD-2 up to 13B [135] and TESS 2, DSL-LLaDA, Lafa, and LLaCO at 7–8B [56, 164, 344, 420]. At 3B and 8B parameters, blockwise Sigma closely matches both its compute-matched masked diffusion counterpart NLD and its starting autoregressive checkpoint on mathematical reasoning and code generation after continued pretraining (Table 17) [106, 405].

Section 6.3 compares published systems across downstream tasks, few-step budgets, and end-to-end serving latency. Favorable head-to-head results over autoregressive baselines concentrate in settings that exploit full-sequence differentiability or latent planning (Tables 12 and 13), including gradient-guided structural control, plan- and sentence-level diversity, and reasoning after task-specific training [72, 218, 344]. Image-to-text FlowTok reaches scores comparable to other non-autoregressive captioners [141]. Under few-step budgets, naive coarse solvers and several undistilled one-step flow maps collapse in entropy. Preserving diversity can be achieved through trajectory-consistent distillation, as in FMLM and progressive Discrete Flow Maps, or stochastic endpoint bridges (Table 14; Section 7.1) [38, 191, 287]. Finally, reducing denoiser evaluations does not by itself lower wall-clock latency once condition encoding, latent decoding, and verification passes are included (Table 16). Standalone continuous generators still trail key–value-cached autoregressive baselines in end-to-end latency at matched quality: matched-quality speedups at batch size one come from pairing continuous block drafters with autoregressive verification in Lafa and FlashMap [56, 326], whereas compressed-latent generators raise batched throughput at some cost to sample quality [103, 140, 426].

1.2 Scope and relation to prior surveys

This survey reviews the mechanisms and empirical performance of continuous-space text diffusion and flow models for readers familiar with Transformer language models. We focus on generative systems that iteratively refine a real-valued representation across the generated unit, spanning token embeddings, vocabulary logits, simplex or belief vectors, analog bits, contextual encodings, compressed latent codes,

and sentence or segment vectors, before decoding that representation into natural language or code. Closely related discrete diffusion models, hybrid discrete–continuous systems, latent-reasoning methods without a generative diffusion or flow prior, and pre-diffusion latent-variable translation models are summarized for comparison in Appendix C. The main literature corpus was assembled through 24 October 2026; citation corrections and version updates in this revision use public sources available by 28 October 2026. Appendix E outlines the literature selection criteria, and Appendix D summarizes the evaluation protocols behind our cross-paper comparisons.

Table 1 compares prior surveys and tutorials with the present work. Early 2023 surveys separated discrete from continuous text diffusion and cataloged the initial token-embedding models [217, 470], while Yi et al. organized text diffusion by downstream task [415]. In 2025, Li et al. devoted one section to continuous models within a broader survey centered on discrete diffusion, and Yu et al. surveyed discrete diffusion models exclusively [212, 422]. More recently, *The Latent Space* reviewed latent-space computation broadly [424], the appendix of ELF provided a tutorial and multi-axis comparison of continuous text generators [153], and Dieleman synthesized the representation, schedule, and distillability arguments for revisiting continuous models [86]. Building on these works, our contributions are:

1. an organizing decomposition into three coupled design dimensions covering state representation, transition mechanism, and text interface, carried consistently from the technical sections to the empirical synthesis tables (Figure 2, Table 5);

2. a self-contained treatment of parameterization identities, sampler updates, and the failure modes of continuous text generation that connects Gaussian diffusion, flow matching, flow maps, and distilled generators (Section 2; Appendix A);

3. a systematic analysis of state representations, training recipes, samplers, generation orders, and decoders across the core lookup table and extended configuration catalog in Appendices F–G (Sections 3–5);

4. task-level reviews of downstream applications anchored on representative empirical comparisons (Section 6.1); and

5. cross-paper synthesis tables comparing continuous models against autoregressive and discrete-diffusion baselines across sample quality, likelihood bounds, few-step distillation, and end-to-end latency, followed by open problems in scaling, readout, serving, representation reuse, and post-training (Sections 6.2–7; Appendix D).

1.3 How to read the survey

Sections 3–5 follow the three design dimensions of Figure 2: Section 3 examines the *state* representation in terms of channel content, sequence length, and geometric constraints; Sections 4 and 5.1 examine how the *transition* is trained and sampled; and Sections 5.2 and 5.3 examine the *text interface* and generation order. Separating these three dimensions distinguishes systems that share a surface label. For instance, Diffusion LCM and CALM both generate one vector per text unit autoregressively. However, Diffusion LCM operates over frozen SONAR sentence embeddings and decodes through the frozen SONAR decoder, whereas CALM feeds the decoded tokens back into the next step, leading to different dependency structures and runtime costs (Section 5.3) [190, 323]. Section 6 reviews downstream applications, evaluation metrics, and cross-paper empirical comparisons, and Section 7 discusses open problems.

Table 2 provides a task- and goal-indexed guide to the survey. Readers new to the area can start with Sections 1–2, the opening paragraphs of Sections 3–5, and Section 6.3. Readers looking up a specific system or configuration can consult the standardized method lookup table in Appendix F, which follows the terminology of Table 5, alongside the extended catalog in Appendix G and the taxonomy map in Figure 5.

Table 1. Related surveys and syntheses, their organizational axes, and how this survey complements them through the state–transition–text-interface decomposition, end-to-end cost accounting, and cross-paper synthesis tables.

Survey	Year	Scope	How continuous models are organized	What this survey adds
Li, Zhou, Zhao, and Wen [217]	2023	Non-autoregressive text diffusion	Discrete versus continuous; pretrained-LM integration; conditioning	Covers 2023–2026 systems; analyzes state, transition, and decoder mechanisms
Zou et al. [470]	2023	Diffusion models in NLP	Discrete versus continuous; comparison with autoregression; applications	Controlled comparisons across sample quality, likelihood bounds, and compute
Yi et al. [415]	2024	Diffusion in text generation	By task: conditional (including fine-grained control), unconstrained, multi-mode	Connects each downstream task family to the state, transition, or decoder choice it depends on
Li, Chen, Guo, and Shen [212]	2025	Diffusion LMs at large; one section on continuous models from Diffusion-LM to ELF, Cola, and BitLM	Chronological list; notes the shift to discrete models	Covers 2022–2026 continuous systems, failure modes, and cross-paper evaluation tables
Yu, Li, and Wang [422]	2026	Discrete diffusion LLMs and multimodal models	Discrete only	The continuous-space complement, with an explicit boundary to discrete work
The Latent Space [424]	2026	Latent-space computation: reasoning, planning, memory	By mechanism (architecture, representation, computation, optimization) and capability	Evaluates latent codes as generative objects, contrasting clean reconstruction with prior decoding
ELF, Appendix A [153]	2026	Tutorial and table inside a method paper	Multi-axis table of continuous systems	Adds generation orders, end-to-end cost accounting, and cross-paper evaluation tables
Dieleman [86]	2026	Expert synthesis (blog)	Embeddings, losses, schedules, self-conditioning; distillability argument	System-wide catalog and empirical evaluation of few-step distillation (Section 6.3)

2 Background: From Text to Continuous Generation

Every system in this survey must turn a discrete sentence into a real-valued state, learn to revise noisy versions of that state, and turn the final state back into words. This section fixes the notation for those steps (Section 2.1) and writes down the diffusion and flow-matching constructions and the identities that connect what different systems predict (Section 2.2). It then explains why the discreteness of text defeats a straightforward transfer of image diffusion (Section 2.3) and fixes the vocabulary and the boundary with discrete diffusion used in the rest of the survey (Section 2.4).

2.1 Problem formulation and notation

Let $x = (x_1, \ldots, x_L) \in \mathcal{V}^L$ denote a discrete sequence of L tokens from a vocabulary $\mathcal{V}$, illustrated in Figures 4 and 6 on the two-sentence passage "Maya bought three books. She read one." A continuous language model maps x to a clean real-valued state $z_0 \in \mathbb{R}^{M \times d}$ comprising M positions of channel width d. A token lookup sets $M = L$ and $z_{0,i} = e(x_i)$, whereas a learned encoder E_ϕ incorporates surrounding context and may compress the sequence into $M \leq L$ positions, either deterministically or through a

Table 2. Reader goals and where the survey answers them. Each goal points to a section whose opening synthesis states the main trade-offs before cataloging individual methods.

Reader goal	Where to start	Answered in
Fewest updates at usable quality	Hybrid corruption and ODE solvers in DiffuSeq-v2; flow-map distillation in FMLM; inference-time feedback repair in Untied Self-Conditioning; one-step distillation and drifting in DMD and Seq-Drifting	§5.1; Table 14
Parity with autoregressive models	Matched-scale MAUVE in CDCD; likelihood bounds in Plaid and RePlaid; instruction and math/code adaptation in TESS 2 and Sigma; verified flow drafting in Lafa; end-to-end latency accounting	Tables 12, 13, 16; §7.1
Controllability: attributes, syntax, style	Noisy-state classifier gradients in Diffusion-LM; classifier guidance on expected embeddings in SSD-LM and ParaGuide; reward guidance in TESS 2	§5.4; §6.1.1
Long or flexible-length output	Blockwise generation in SSD-LM and AURORA; target-length prediction in Difformer; sentence-level stopping in Diffusion LCM; cached semi-autoregressive windows in Clock Diffusion	§5.3
Decode a sentence or segment vector	Frozen SONAR decoding in Diffusion LCM; soft-prompt conditioning in DGLM; code-trained decoders in PLANNER; iterative inversion in Vec2Text	§5.2; §3.4
Reuse a pretrained encoder or codec	Frozen contextual encoder states in TEncDM; staged language-model codecs in LD4LG; co-adapted causal codecs in Cola	§3.2; §3.3
Multimodal conditioning	Direct image-to-text flow in CrossFlow; joint multimodal refinement in OmniFlow; visually conditioned sentence vectors in V-LCM	§6.1.3
Source-conditioned transformation	Partial target noising in DiffuSeq; encoder–decoder conditioning in SeqDiffuSeq; noise-scale calibration in DiNoiser	§6.1.2
Plans, dialogue, and discourse	Paragraph-level latent codes in PLANNER; sentence and segment trajectories in Diffusion LCM and SLD	§6.1.4
Knowledge, code, and reasoning	Commonsense fact vectors in DiffuCOMET; syntax-aware program embeddings in CodeFusion; rationale and math generation in Diffusion of Thought, TESS 2, ELF-REG, and CEDR	§6.1.5; §6.1.6

posterior distribution $q_\phi(z \mid x)$ (Section 3). Fixed-window generators pre-allocate M positions prior to sampling, whereas block, chunk, and sentence interfaces pair continuous refinement with a continuation and stopping rule (Section 5.3).

In conditional generation, the model generates x given an observed condition c, such as a prefix, a source document, an instruction, an attribute label, or fixed infilling spans. During training, the clean state z_0 provides the supervision target for noisy intermediate states. At inference, iterative updates transport a random or source-initialized starting state to a terminal representation, which a decoder maps back to discrete tokens via nearest-embedding rounding, a linear vocabulary head, or a conditional decoder $p_\psi(x \mid z)$ (Figure 8). Table 3 summarizes the notation used throughout the survey and defines the primary computational cost measures. The number of function evaluations (NFE) counts total denoiser forward passes per generated unit, including two-pass self-conditioning, unbatched guidance branches, and terminal logit passes, reported alongside any separate decoder or reranker passes. End-to-end latency measures the elapsed wall-clock time from the input condition to the delivered tokens (Section 5.4).

Across the schematic figures, blue denotes encoders and continuous states (token embeddings, contextual encodings, or latent codes), green denotes the continuous denoiser or flow generator, and orange denotes the text-interface decoder or readout.

Table 3. Notation. Diffusion time t runs toward noise and flow time s toward data throughout; where a cited equation uses flow time for both of its arguments, as in the categorical flow map of Section 5.1, the text says so.

Symbol	Meaning
$x = (x_1, \dots, x_L)$; $\mathcal{V}$	token sequence of length L; vocabulary
$z_0 \in \mathbb{R}^{M \times d}$	clean state: M positions of width d ($M = L$ for token-aligned states); $e(\cdot)$ token lookup; E_ϕ encoder; $q_\phi(z \mid x)$ stochastic encoder
c	condition: source, prefix, instruction, label, or fixed positions
$t \in [0, T]$; $\alpha(t), \sigma(t)$	diffusion time, increasing toward noise; signal and noise scales of $z_t = \alpha(t) z_0 + \sigma(t)\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$
$s \in [0, 1]$; u; v_θ	flow time, increasing toward data; conditional path velocity; learned velocity field
$\hat{z}_0, \hat{\epsilon}, \hat{v}, \hat{s}_\theta$; $s(z_t, t)$; q_θ	predictions of the clean state, the noise, the v-target, and the plug-in score estimate; true marginal score $\nabla_{z_t} \log p_t(z_t)$; predicted token posterior
$X_{s,t}$	flow map from the state at time s to the state at time t
$\lambda(t)$; $p(t)$	training loss weight; training time distribution
$p_\psi(x \mid z)$ or D	probabilistic or deterministic decoder from states or codes to tokens
NFE; end-to-end latency	total denoiser forward passes per generated unit, reported alongside any separate decoder or reranker passes; wall-clock time from condition to delivered tokens

2.2 Diffusion and flow matching, and what the denoiser predicts

Gaussian diffusion corrupts the clean state by

$$z_t = \alpha(t) z_0 + \sigma(t)\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{1}$$

where t increases from clean data toward noise, $\alpha(t)$ decreases, and $\sigma(t)$ increases. Variance-preserving schedules keep $\alpha^2 + \sigma^2 = 1$, as in Diffusion-LM's square-root schedule, whereas variance-exploding schedules keep $\alpha = 1$ and let σ grow, as in CDCD's $\sigma(t) = t$ [87, 213]. Training samples a time, corrupts the state, and asks a denoiser, given z_t, t, and c, to recover information about z_0. Its attention layers connect positions even when its output head predicts each position separately. Sampling starts at the noisy end and applies a finite sequence of updates.

At nondegenerate noise levels ($\alpha(t) > 0, \sigma(t) > 0$) on a Gaussian path, the standard continuous vector targets are affine functions of one another, so the network's output target is a choice of parameterization. With $\alpha = \alpha(t)$ and $\sigma = \sigma(t)$, we distinguish the true marginal score $s(z_t, t) = \nabla_{z_t} \log p_t(z_t)$ from the model's plug-in score estimate $\hat{s}_\theta(z_t, t)$:

$$\begin{aligned}
s(z_t, t) = \nabla_{z_t} \log p_t(z_t) &= -\frac{\mathbb{E}[\epsilon \mid z_t]}{\sigma} = \frac{\alpha \mathbb{E}[z_0 \mid z_t] - z_t}{\sigma^2}, \\
\hat{\epsilon}_\theta(z_t, t) &= \frac{z_t - \alpha \hat{z}_{0,\theta}(z_t, t)}{\sigma}, \qquad \hat{s}_\theta(z_t, t) = -\frac{\hat{\epsilon}_\theta(z_t, t)}{\sigma} = \frac{\alpha \hat{z}_{0,\theta}(z_t, t) - z_t}{\sigma^2}, \\
\hat{v}_\theta(z_t, t) &= \alpha \hat{\epsilon}_\theta(z_t, t) - \sigma \hat{z}_{0,\theta}(z_t, t) \quad (\hat{z}_{0,\theta} = \alpha z_t - \sigma \hat{v}_\theta \text{ when } \alpha^2 + \sigma^2 = 1), \\
z_s &= (1 - s)\epsilon + s z_0, \quad u = z_0 - \epsilon, \quad \hat{z}_{0,\theta} = z_s + (1 - s)\hat{u}_\theta, \quad \hat{\epsilon}_\theta = z_s - s\hat{u}_\theta \quad (\alpha = s,\ \sigma = 1 - s).
\end{aligned} \tag{2}$$

The first line is Tweedie's formula for the true marginal score. The second and third lines convert algebraically between a network's clean-state prediction $\hat{z}_0$, noise prediction $\hat{\epsilon}$, plug-in score estimate $\hat{s}_\theta$, and the v-parameterization $v = \alpha\epsilon - \sigma z_0$ of progressive distillation [311] whenever $\alpha(t) > 0$ and $\sigma(t) > 0$. The plug-in estimate equals the true score, $\hat{s}_\theta(z_t, t) = s(z_t, t)$, only when $\hat{z}_{0,\theta}(z_t, t) = \mathbb{E}[z_0 \mid z_t]$ (or equivalently $\hat{\epsilon}_\theta(z_t, t) = \mathbb{E}[\epsilon \mid z_t]$), the Bayes-optimal predictor under squared-error regression. Appendix A.2 states this identity for token-aligned states, where the conditional mean is the expected embedding $\sum_{w \in \mathcal{V}} p(w \mid z_t) e_w$. A predicted token posterior $p_\theta(\cdot \mid z_t) \in \Delta^{|\mathcal{V}|-1}$ uniquely determines that expected embedding $\hat{z}_0 = \sum_{w \in \mathcal{V}} p_\theta(w \mid z_t) e_w \in \mathbb{R}^d$. The converse does not hold in general once $|\mathcal{V}| > d + 1$, because the embedding matrix $E^\top \in \mathbb{R}^{d \times |\mathcal{V}|}$ has a nontrivial nullspace and distinct token distributions can share the same mean vector (Section 4.1; Appendix A.2). The final line covers

the flow-matching case: with flow time $s \in (0, 1)$ running toward data and the linear interpolant, the conditional velocity is $u = z_0 - \epsilon$, and a velocity prediction $\hat{u}_\theta$ determines $\hat{z}_{0,\theta}$ and $\hat{\epsilon}_\theta$. Consequently, a model described in the literature as predicting "velocity" may be either a flow-matching velocity model or a v-parameterized diffusion model run with the deterministic sampler of denoising diffusion implicit models (DDIM), which differ in path and sampler (Section 4.1). Under any of these parameterizations, every sampler update can be written in terms of $\hat{z}_0$. The ancestral step of denoising diffusion probabilistic models (DDPM), the deterministic DDIM step $z_{t'} = \alpha(t')\hat{z}_0 + \sigma(t')\hat{\epsilon}$, and Euler or higher-order steps of the probability-flow ordinary differential equation (ODE) are all functions of the current state and $\hat{z}_0$ [149, 242, 336]. For the linear path the Euler step on $\hat{u}$ equals the deterministic DDIM step exactly, since $z_s + (s' - s)(\hat{z}_0 - \hat{\epsilon}) = s'\hat{z}_0 + (1 - s')\hat{\epsilon}$. Section 5.1 writes these updates out, and Section 5.2 shows that clamping, projection, and expected-embedding substitution are alternative replacements for $\hat{z}_0$ inside them.

Flow matching instead specifies a conditional path from noise at $s = 0$ to data at $s = 1$ and regresses a network directly on its velocity:

$$\begin{aligned} \mathcal{L}_{\mathrm{CFM}}(\theta) &= \mathbb{E}_{s\sim p(s),\, z_0\sim p_{\mathrm{data}},\, \epsilon\sim\mathcal{N}(0,I)} \left[\lambda(s)\, \|v_\theta(z_s, s, c) - u_s(z_s \mid z_0, \epsilon)\|^2\right], \\ z_s &= (1 - s)\epsilon + s z_0, \qquad u_s(z_s \mid z_0, \epsilon) = z_0 - \epsilon, \end{aligned} \tag{3}$$

where the displayed path is the linear one and u_s is its conditional velocity evaluated along the sampled draw. Because the network learns a marginal velocity field aggregated over many noise–data pairs, it has no access to the specific target of a new draw at inference time. Under the fixed-path assumptions of conditional flow matching, this loss shares the same parameter gradient as regression on the marginal velocity [223]. Since the marginal field generates the marginal probability path (Theorem 1 in [223]), training requires no trajectory simulation. At inference, however, generation must still integrate the learned marginal field, whose trajectories can remain curved even when individual conditional paths are straight. Diffusion and flow matching are closely related along Gaussian paths once time orientation ($t = 1 - s$ increasing toward noise) and nondegeneracy ($\alpha(t) > 0, \sigma(t) > 0$) are fixed. For any smooth nondegenerate Gaussian path $z_t = \alpha(t)z_0 + \sigma(t)\epsilon$, the marginal velocity in diffusion time, $u_t(z_t) = \mathbb{E}[\dot{\alpha}(t)z_0 + \dot{\sigma}(t)\epsilon \mid z_t]$, is an affine function of the state and the true marginal score,

$$u_t(z_t) = \frac{\dot{\alpha}(t)}{\alpha(t)} z_t - \sigma(t)\left(\dot{\sigma}(t) - \frac{\sigma(t)\dot{\alpha}(t)}{\alpha(t)}\right) \nabla_{z_t} \log p_t(z_t), \tag{4}$$

where dots denote derivatives in t and generation integrates the probability-flow ODE from T to 0. At nondegenerate noise levels ($\alpha(t) > 0, \sigma(t) > 0$) a score, ϵ-, x_0-, or v-predictor therefore defines a probability-flow velocity field, and whenever $\alpha(t)\dot{\sigma}(t) - \sigma(t)\dot{\alpha}(t) \neq 0$ (equivalently $\frac{\mathrm{d}}{\mathrm{d}t}[\sigma(t)/\alpha(t)] \neq 0$) a velocity predictor conversely defines a plug-in score and $\hat{z}_0$ estimate, up to the loss weighting λ and the time reparameterization [223]. Realizing the same family of marginals through a Markov forward SDE $\mathrm{d}z_t = f(t)z_t\,\mathrm{d}t + g(t)\,\mathrm{d}w_t$ requires the additional compatibility condition that the noise-to-signal ratio $\sigma(t)/\alpha(t)$ be nondecreasing in t, so that $g(t)^2 = \alpha(t)^2 \frac{\mathrm{d}}{\mathrm{d}t}\left(\sigma(t)^2/\alpha(t)^2\right) \geq 0$ (Appendix A.1). Practical differences between systems termed "Gaussian diffusion" and "continuous flow" thus reduce to the choice of path, loss weighting, parameterization, and whether sampling integrates an ODE or a Markov SDE. The lookup table in Appendix F records the process family alongside the generation order, listing the path, weighting, and sampler as qualifiers.

Several extensions of diffusion and flow matching appear throughout recent continuous text generators. *Rectified flow* instantiates flow matching with the linear interpolant and introduces a reflow step that re-pairs noise and data along the learned coupling to straighten trajectories [231]. *Stochastic interpolants* generalize the path to $z_s = I(s, \epsilon, z_0) + \gamma(s)w$ with an optional latent noise term, encompassing both deterministic flows and diffusions [4]. For the exact solution operator $X_{s,t}$ of a well-posed flow ODE, a *flow map* transports the state from time s to time t along the same trajectory and satisfies the diagonal identity $X_{s,s}(z) = z$ and the semigroup composition property $X_{t,u}(X_{s,t}(z)) = X_{s,u}(z)$. In a learned two-time network $X^\theta_{s,t}$, the residual parameterization $X^\theta_{s,t}(z) = z + (t - s)\bar{u}_\theta(z, s, t)$ enforces $X^\theta_{s,s}(z) = z$ algebraically at $s = t$, while self-distillation and consistency losses train the network to approximate semigroup composition. A consistency model corresponds to the endpoint case $t = 1$, mapping points

along a shared trajectory to the same terminal state [338]. Flow-map matching and MeanFlow instead learn two-time maps via an average velocity derived from differentiating the interval displacement. Flow-map matching anchors the start point: $\bar{u}(z, s, t) = (X_{s,t}(z) - z)/(t - s)$ satisfies the Lagrangian identity $\bar{u}(z, s, t) = v(X_{s,t}(z), t) - (t - s)\partial_t \bar{u}(z, s, t)$, obtained by differentiating $X_{s,t}(z) = z + (t - s)\bar{u}(z, s, t)$ in t [48]. MeanFlow anchors the state at the later time t of an interval $[r, t]$ (where sampling runs from t down to r), so its average velocity $\bar{u}(z_t, r, t)$ satisfies the Eulerian identity $\bar{u}(z_t, r, t) = v(z_t, t) - (t - r)\frac{\mathrm{d}}{\mathrm{d}t}\bar{u}(z_t, r, t)$ with total derivative $\frac{\mathrm{d}}{\mathrm{d}t}\bar{u} = v \cdot \nabla_z \bar{u} + \partial_t \bar{u}$ along the trajectory [116]. Because a flow map advances the state across a finite interval per model call rather than predicting an instantaneous velocity, it directly addresses few-step generation without requiring multi-step numerical integration. Flow Map Language Models (FMLM) parameterize such a map with a simplex-valued output head whose intermediate states need not lie on the simplex [191] (Appendix A.4 derives the categorical map used there and in Section 5.1 from the linear path).

A complementary acceleration family, *distribution-matching distillation* (DMD), trains a one- or few-step student so that the marginal distributions of its noised outputs match the teacher's across noise levels, guided by the difference between the teacher's score and a student-score network [416]. In continuous text generation, DLM-One instantiates this approach via Score identity Distillation (SiD) in continuous embedding or logit space [63]. Once a score or velocity model is trained, backward integration can follow either a reverse-time stochastic differential equation (SDE) or a deterministic probability-flow ODE. Under an exact score, the reverse-time SDE and the probability-flow ODE share every time-marginal distribution despite following different individual trajectories. Under learned scores and finite step sizes, however, the two solvers can produce different output distributions [337]. Specifying the denoiser objective therefore leaves open the choice of sampler, time grid, and stochasticity (Section 5.1; Appendix A). Bayesian Flow Networks (BFN) adopt a distinct Bayesian formulation: the continuous state tracks categorical belief parameters updated by Bayes' rule under noisy Gaussian evidence, starting from a uniform prior and drawing randomness from the evidence stream (Section 3.1) [124].

2.3 Why continuous text generation is hard

Unlike continuous pixel intensities in images, discrete tokens do not admit valid intermediate values: a vector lying midway between two token embeddings does not represent a word, and an average of two distinct sentence representations need not correspond to any grammatical sentence. This mismatch between continuous trajectories and discrete language produces the failure modes analyzed below.

Rounding and embedding collapse Under squared-error regression, the Bayes-optimal denoiser predicts the conditional expectation of the clean embedding, $\mathbb{E}[z_0 \mid z_t, c]$ (Appendix A.2), whereas L_1 regression, as in DiffSLT, targets the coordinatewise conditional median. When several tokens are plausible at a position, the conditional mean lies inside their convex hull and near none of them, forcing nearest-embedding rounding to make an arbitrary choice and turning small terminal errors into wrong words. Diffusion-LM mitigates this ambiguity by adding a token-rounding term to its training objective and clamping each intermediate prediction to its nearest token embedding during sampling [213]. A second vulnerability appears when the token embeddings are learned jointly with the denoiser. Because squared-error regression can be trivially lowered by shrinking all embeddings toward a common point, unconstrained training collapses the lookup table so that noise dominates every state. Fixing vector norms via L_2 normalization or LayerNorm controls overall scale, but distinct tokens can still collapse toward a shared direction unless training freezes the lookup table (as in SED [341]) or enforces token discrimination. Common remedies include a vocabulary anchor loss in Difformer, token-posterior cross-entropy over L_2-normalized embeddings in CDCD, an endpoint reconstruction term within Plaid's variational bound, or a spherical uniformity regularizer in Sigma [87, 111, 128, 405]. Davis et al. give an approximate analysis under a Bayes-optimal denoiser: the input-path gradient of token cross-entropy repels the true token's embedding from the posterior mean, while Plaid's squared-error gradient contains an opposing attractive component; the squared-error objective alone admits a collapsed codebook [81]. Unit-norm embeddings trained by

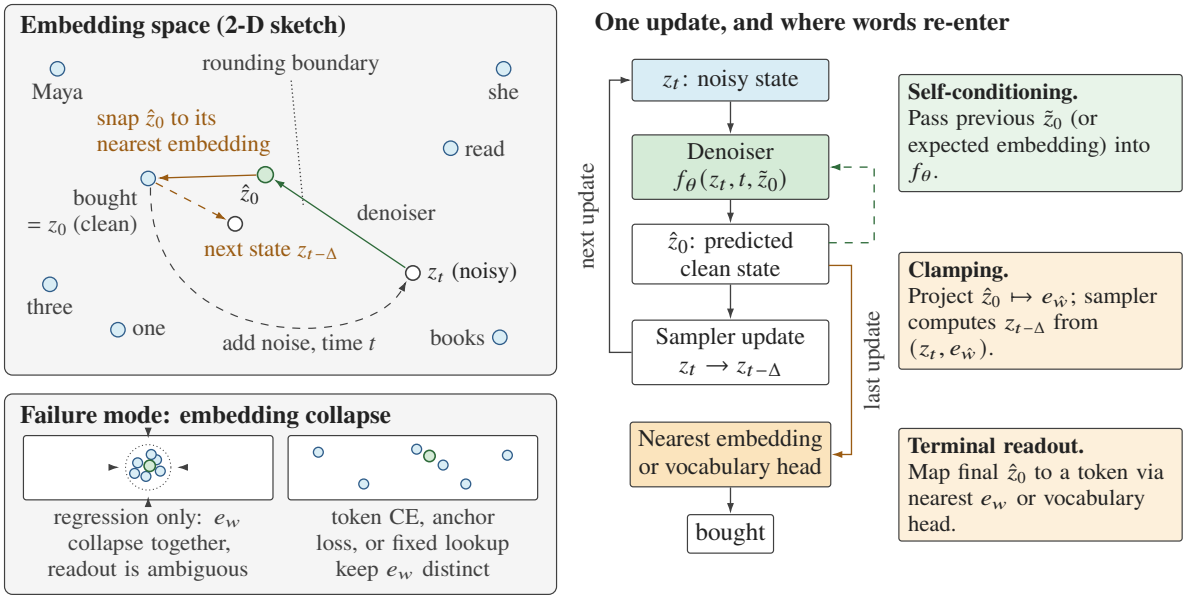


Figure 4. The core mechanism of token-embedding diffusion and its failure mode. The figure follows one sentence from clean embeddings through a noisy state z_t to the denoiser's prediction $\hat{z}_0$, which lies between embeddings; rounding picks the nearest token, clamping snaps the prediction before the next update, and self-conditioning feeds it back as input. In the collapse panel at bottom left, learning the embedding table by regression alone lets embeddings drift together so the prediction stops carrying token information; norm constraints fix scale, while token cross-entropy, anchor losses, uniformity regularization, or a fixed lookup preserve token distinctions (Sections 3.1 and 4.1).

cross-entropy alone consequently spread until their decoding error rate approaches that of one-hot states, motivating geometry-aware noise schedules [81]. Figure 4 illustrates both mechanisms; Sections 3.1, 4.1, and 5.2 examine the resulting state geometries, training objectives, and readout rules.

Uninformative noise levels The noise schedule determines which corruption regimes the denoiser practices during training. At insufficient noise, the clean token can remain the nearest neighbor to its perturbed vector, allowing local recovery with little use of the source condition; DiNoiser finds that this regime persists to larger noise scales as embedding dimensionality increases. Its analysis also shows that presenting the denoiser with larger noise levels strengthens reliance on the source condition [409]. As detailed in Section 4.3, existing remedies rescale the forward noise (Difformer), truncate small noise levels during training and increase the denoiser's noise-level input at inference (DiNoiser), warp training time to approximately equalize prediction-entropy increments (CDCD), or learn position-specific schedules (SeqDiffuSeq) [87, 111, 409, 425]. Interpreting schedule ablations also requires distinguishing the forward corruption path (α, σ), the training time distribution and loss weighting $(\lambda(t), p(t))$, and the inference discretization grid. For instance, Plaid compares likelihood and heuristic loss weightings with a fixed embedding table, while width-matched schedule sweeps in AURORA modify $(\lambda(t), p(t))$ with the forward path held fixed (Section 4.3) [128, 220].

Latent–prior mismatch When generation takes place in a learned autoencoder or codec space, token rounding is replaced by a distribution shift at the decoder interface. The codec is trained to reconstruct clean text from encoder outputs, but at inference its decoder must read latent codes generated by a learned prior. In LDLM's decoder-noise ablation (Table 4), injecting stronger noise into the decoder's training inputs lowers external generative perplexity (GenPPL; Section 6.2) yet collapses distributional similarity (MAUVE [286]) and token entropy [253], because excessive noise prevents accurate hidden-state and token reconstruction. LaDiff exhibits the complementary effect: stronger codec regularization lowers clean-text reconstruction accuracy but substantially improves distributional similarity when decoding

Table 4. Effect of decoder input noise σ_{dec} during LDLM training on 128-token OpenWebText generation, scored with GPT-2 Large across five 1,000-sample seeds (mean ± standard deviation; MAUVE reported on a 0–100 scale) [253]. Increasing decoder-input noise lowers GenPPL while sharply reducing MAUVE and token entropy.

Decoder noise	External GenPPL ↓	MAUVE ↑	Token entropy
$\sigma_{\text{dec}} = 3$	66.3 ± 0.8	89.1 ± 3.1	4.25 ± 0.01
$\sigma_{\text{dec}} = 4$	38.1 ± 0.6	0.4 ± 0.1	2.19 ± 0.01

samples drawn from the trained prior [195]. Clean-text reconstruction and prior-driven generation thus evaluate the decoder on different input distributions. Codec systems address latent-space conditioning and prior–decoder compatibility through choices such as constraining latent-vector norms (LD4LG, except its FLAN-T5 variant), fitting the prior to variational posterior means (PLANNER), or co-adapting the codec alongside the prior after reconstruction pretraining (Cola) [131, 239, 441]. Related remedies retune noise schedules per latent width (AURORA) or fine-tune the decoder on a mixture of clean and prior-denoised latents (BLD; Sections 3.3 and 5.3) [103, 220]. Figure 7 and Appendix B summarize the corresponding diagnostic protocols.

Few-step mode averaging Compressing generation into a few updates exposes a further problem: coarse updates average across incompatible continuations. For a denoiser supervised by squared error or token-posterior expectation, each step moves the state toward the conditional mean $\mathbb{E}[z_0 \mid z_t, c]$. Over many small updates, the trajectory commits gradually and resolves to a sharp token sequence. Over a few coarse updates from high noise, however, the conditional mean averages across many plausible but mutually incompatible continuations, and a deterministic solver that follows this average converges toward high-frequency, repetitive tokens shared across modes. Because external perplexity rewards predictable repetition, a collapsed sampler can score deceptively well on GenPPL alone while failing unigram-entropy and distributional checks [105]. This failure recurs across single-step categorical flow maps and coarse Euler trajectories of pretrained flow models (Section 5.1) [38, 287]. One set of remedies reinjects intermediate stochasticity by sampling and re-noising discrete endpoints, as in SSD-LM's projection step and Marginal-Conditioned Bridges (MCB) [38, 134]. Another learns finite-interval transport directly through trajectory-consistent flow-map distillation (FMLM), distribution matching (DLM-One and reverse-KL DMD), or trajectory-free drifting (Seq-Drifting) [63, 191, 228, 358]. The published Categorical Flow Maps model reports one-step LM1B entropy 4.03 [304].

A subtler form of conditional-mean averaging also operates at a model's full sampling budget, thinning the lexical tail without depressing unigram entropy. Under a squared-error denoising head, lower-probability rare words are systematically pulled toward more probable competitors, causing rare words to appear at a fraction of their human-text rate across model scales while leaving external perplexity and unigram entropy largely unaffected [401]. Section 5.1, Table 14, and Section 7.1 examine these few-step and tail-averaging dynamics in detail.

2.4 Terminology and boundary with discrete diffusion

Table 5 summarizes the terminology used throughout the survey, and Figure 5 maps representative systems by state family, transition and generation order, and decoder. In this terminology, whether the evolving states are real-valued or categorical and whether each vector encodes a single token, contextual features, or a compressed span belong to the state representation. Whether units are generated jointly or sequentially belongs to the transition, and whether tokens are emitted in parallel or autoregressively belongs to the text interface. The word *latent* in particular guarantees neither compression nor parallel generation: a contextual code can keep one position per token, and sentence or fixed-token-chunk vectors can be generated autoregressively, as in Diffusion LCM and CALM, respectively (Section 5.3) [190, 323]. Image latent diffusion supplies the encoder–generator–decoder decomposition that codec systems borrow [303], but its compression rationale is perceptual detail, whereas text must preserve identities, quantities, and

Table 5. Terminology used throughout the survey, grouped by state and code representation, transition dynamics and generation order, and text-interface decoding.

Term	Meaning
state, continuous state	the real-valued representation being refined when token-aligned: one vector per token, or a vocabulary-sized logit, simplex, or belief vector per position
latent code, code	the generated representation in autoencoder or codec systems: compressed code arrays or sentence and segment vectors produced by an encoder
codec	the encoder–decoder pair mapping between discrete text and latent codes
denoiser; latent prior	the network mapping a noisy state, noise level, and condition to a prediction; in codec systems, the generative model over codes is referred to as the *latent prior*
prediction target	the quantity supervised during denoiser training: clean state x_0, noise ϵ, $v = \alpha\epsilon - \sigma z_0$, score, flow-matching velocity, or token-posterior cross-entropy (Eq. 2)
update; sampler	a single numerical step of generation; the inference procedure specifying the time grid, solver, and noise injection
transition	design dimension (Section 1.3) encompassing the revision mechanism: training objective, schedule, sampler, and generation order
generation order	the order in which positions or latent units are generated: full sequence, blockwise autoregression, next-vector autoregression, or a next-token inner loop
text interface; decoder	design dimension specifying where discrete decisions enter and whether tokens are decoded in parallel or autoregressively; the module itself is the *decoder* (vocabulary head or rounding, shared contextual readout, or separate parallel, causal-block, AR, or discrete-diffusion decoder)
few-step; flow map; CatFM	generation in a small number of updates; two-time interval operators $X_{s,t}$; Categorical Flow Maps (CFM is reserved for the conditional flow-matching objective)
NFE; end-to-end latency	denoiser function evaluations per generated unit (Table 3), reported alongside any separate decoder or reranker passes; total wall-clock time to delivered tokens
autoregressive (AR)	sequential dependence in which each generated unit (token, block, or vector) conditions on previously generated units

order; Section 3.3 therefore examines compression using text-specific diagnostics.

Discrete diffusion language models at scale At multi-billion-parameter scales, discrete diffusion models that maintain categorical token states throughout generation serve as the primary diffusion baselines. D3PM formulated discrete diffusion via token transition matrices including the absorbing mask process, SEDD made discrete score matching tractable by estimating probability ratios, and MDLM simplified continuous-time masked diffusion to a weighted masked-language-modeling cross-entropy [37, 238, 308]. Building on that recipe, LLaDA trained an 8B masked model from scratch and Dream adapted a 7B autoregressive checkpoint, both matching same-scale autoregressive peers on standard benchmarks [268, 408], while Mercury, Gemini Diffusion, and Seed Diffusion demonstrated high-throughput diffusion text generation, with Gemini Diffusion introduced as an experimental demo and Seed Diffusion as an experimental preview [123, 159, 339]. BD3-LM bridges autoregression and diffusion by generating token blocks causally with within-block masked diffusion, providing the discrete prototype for the blockwise orders in Section 5.3 [36].

Several theoretical results connect these discrete processes directly to continuous-state dynamics. Duo shows that taking the position-wise argmax of Gaussian-corrupted one-hot vectors yields the marginals of uniform-state discrete diffusion under a transformed noise schedule, allowing curriculum and distillation techniques to transfer across paradigms; this marginal correspondence does not in general identify their sample trajectories [309]. Haxholli extends this correspondence to convex-interpolant flows: when a continuous source factorizes across positions with coordinate-permutation-invariant per-position laws, almost surely tie-free coordinates, atomless coordinate gaps at the interpolation thresholds, and an argmax-compatible lifted coupling, the position-wise argmax of a continuous convex-interpolant flow toward one-hot vertices follows a discrete convex-interpolant flow with a uniform source [139]. The

choice of continuous source distribution governs the induced discrete noise schedule. A Gaussian source delays token commitment as vocabulary size grows because the target coordinate must overtake the maximum of many noise coordinates. By contrast, a centered negative-exponential source yields a vocabulary-independent schedule; a single 10k-step OpenWebText pilot reports lower one-step generative perplexity than the Gaussian source, accompanied by a modest decrease in sample entropy [139]. Simplex Diffusion Models establish a third bridge via Dirichlet corruption on the simplex, recovering discrete diffusion with a full-support prior in the high-temperature limit and deterministic interpolation with Gaussian fluctuations in the low-temperature limit [84]. Unlike discrete masked models, which use real-valued network activations only to parameterize transitions between categorical token states, the systems surveyed below refine the continuous representation itself. Discrete and hybrid models (such as LADD [325]) are summarized for comparison in Appendix C.

Continuous latent variables coordinated parallel word predictions before text diffusion. TextFlow and FlowSeq (2019) placed invertible flow priors over character- or target-aligned latent sequences read by parallel decoders [248, 469]. LaNMT and Lee et al. made inference iterative, by alternating latent and token updates or by learning a continuous refinement of a fixed-length latent array [193, 332]. These systems learned a latent distribution, improved inference under a fixed model, or learned an update direction, but none learned transport along a noise-to-data path, which is what distinguishes the models surveyed here. FlowSeq (2024) is the flow-matching namesake (Section 5.1) [154], and Appendix C summarizes these earlier latent-variable systems for comparison.

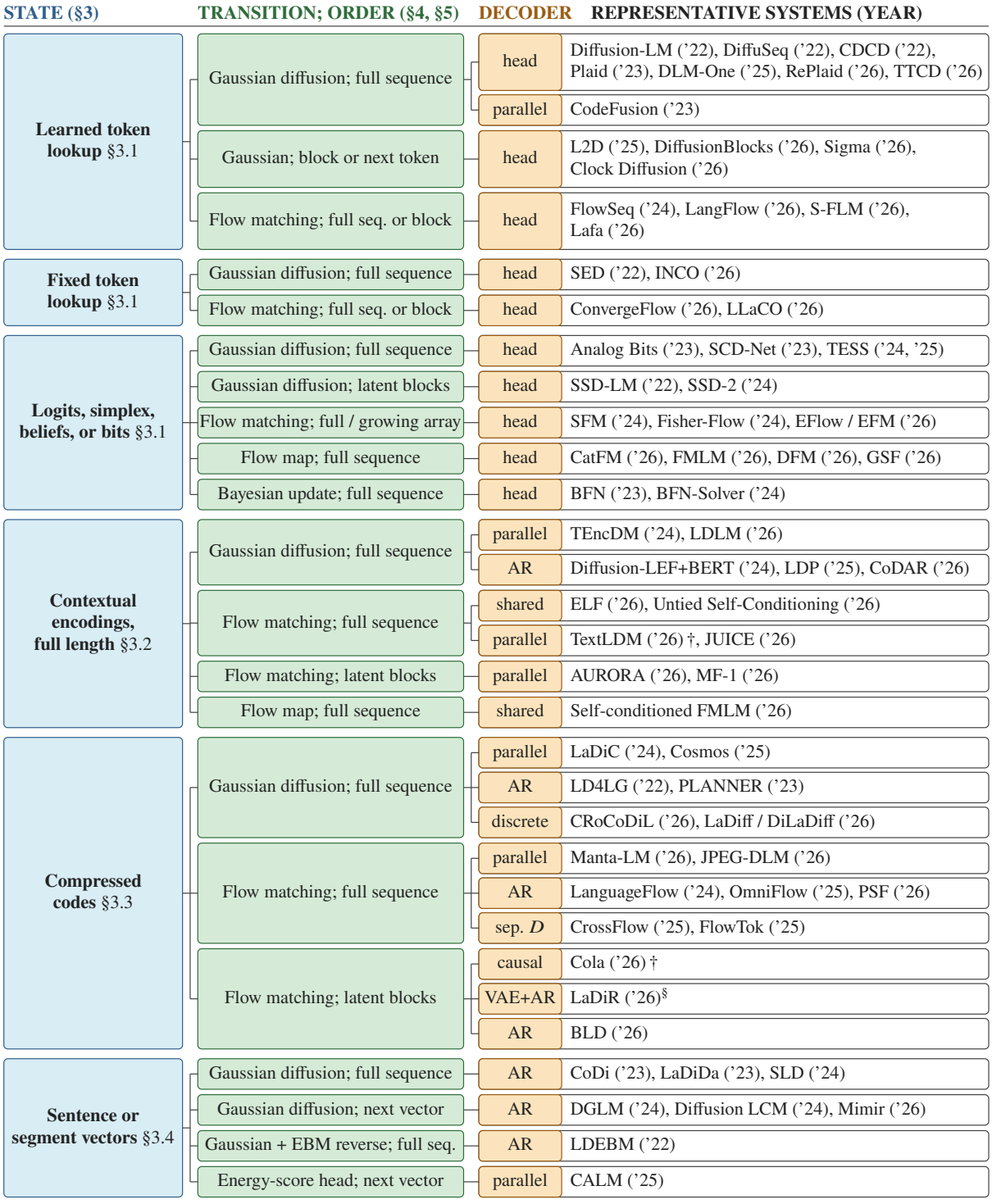
3 The State: Text Representations

Because text arrives as a sequence of discrete symbols, a continuous language generator must first map those symbols into real-valued coordinates that noise can corrupt and a denoiser can recover. Common representations assign one vector per token, one vector per sentence, or a shorter array of codes summarizing a passage. This choice determines the shape of the array updated by the denoiser, the linguistic detail that the decoder must reconstruct afterward, and the failure modes that the training objective must prevent. Figure 6 illustrates these encodings on a single sentence. This section examines token lookups (Section 3.1), contextual encodings (Section 3.2), compressed codes (Section 3.3), and sentence or segment vectors (Section 3.4). Table 6 summarizes their modeling and decoding requirements, and Appendix F lists every configuration.

3.1 Token embeddings and vocabulary states

Mapping each vocabulary symbol x_i to a continuous vector $z_{0,i}$ and adding Gaussian noise around it keeps one state position per token ($M = L$), but directly exposes the rounding and embedding-collapse vulnerabilities analyzed in Section 2.3. Because the encoding defines the continuous target distribution, its vector dimension, geometry, and compatibility with the denoiser's input and output parameterizations affect sample quality [31]. Token-level systems address these constraints by anchoring a low-dimensional lookup table through frozen weights or token-discrimination objectives paired with norm control. Alternatively, they lift the state into vocabulary-sized coordinates where token probabilities are explicit and readout reduces to an argmax, or constrain the state onto a sphere or bit hypercube so that endpoints remain separated.

Anchoring learned embedding tables Diffusion-LM learns the embedding table jointly with the denoiser and prevents collapse through an end-to-end objective that includes a token-rounding cross-entropy term. During sampling, it clamps each intermediate clean-state prediction to its nearest token embedding and conditions the reverse posterior on that embedding [213]. SED eliminates collapse by fixing the lookup table prior to diffusion training. Stage 1 trains a BERT model with a bottlenecked input embedding layer and extracts its lookup table; stage 2 freezes that table while the denoiser regresses clean embeddings by squared error ($\|\hat{x}_0 - x_0\|_2^2$). A separate linear readout head is trained by token



Decoder: head rounding or vocabulary head; shared shared contextual readout; parallel separate parallel decoder; AR separate AR decoder; causal causal block decoder; VAE+AR thought VAE + in-model AR answer; discrete discrete-diffusion decoder; sep. D separate Transformer decoder.

† compresses channel width ($M = L, d_z < d$); other compressed codecs reduce position count ($M < L$).

§ VAE decoder reconstructs thought blocks in stage 1; the reasoning backbone generates final answers autoregressively.

Figure 5. Representative design-space map of continuous-state language models, organized by state family (Section 3), transition mechanism and generation order (Sections 4 and 5), and text-interface decoder (Section 5.2). Representative systems illustrate each branch; the full configuration lookup and extended catalog appear in Appendices F and G. Token-level states are read primarily by a vocabulary head or nearest-embedding rounding, whereas contextual, compressed, and sentence-level states rely on a shared contextual readout or a separate parallel, causal-block, autoregressive, or discrete-diffusion decoder.

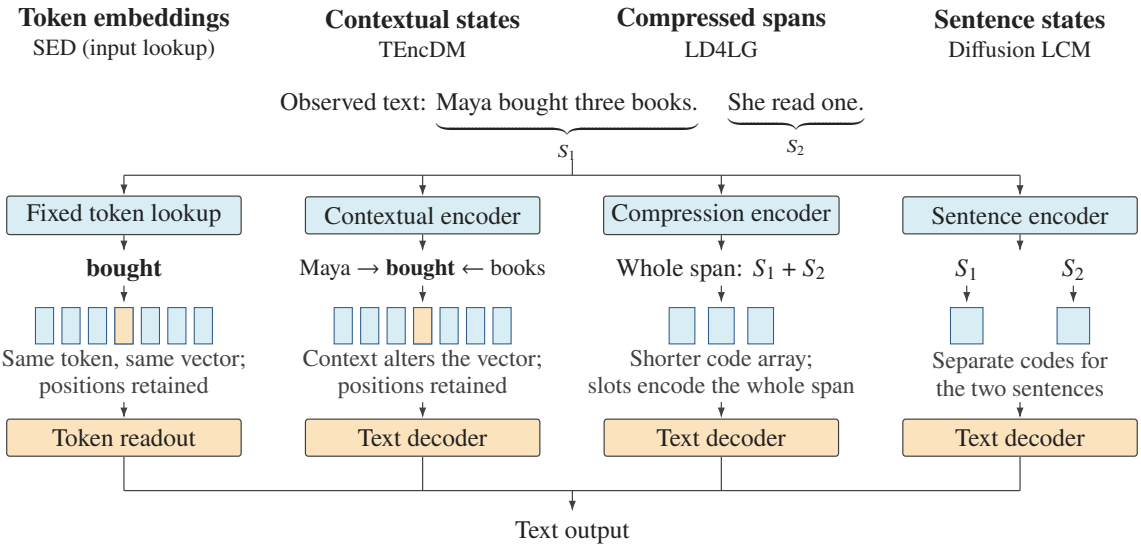


Figure 6. Continuous encodings of the same text: SED's fixed low-dimensional input token-embedding lookup extracted from a bottleneck-trained BERT model, TEncDM's full-length contextual encoder states with one vector per token at full channel width, LD4LG's shortened code array with $M < L$ positions, and Diffusion LCM's sentence vectors [190, 239, 317, 341]. Unlike TEncDM, token-aligned codecs such as TextLDM retain $M = L$ positions while narrowing the channel width to $d_z < d_{\text{model}}$ [166]. Across the panels, the highlighted "bought" slot depends only on token identity in an input lookup, incorporates surrounding context in a contextual encoder, and ceases to correspond to a single word once the passage is compressed into $M < L$ slots or encoded at the sentence level. Section 5 compares how each state is decoded.

cross-entropy on clean embeddings sampled from $q_V(x_0 \mid w)$, so this reconstruction term updates only the readout matrix [341]. SED also introduces self-conditioning for text embeddings by feeding its previous clean-state estimate back into the next denoiser call (Section 4.3). CDCD retains a jointly learned table and pairs L_2 normalization with token-posterior cross-entropy supervision [87]. Plaid similarly learns embeddings within a variational likelihood bound, forming its clean-state estimate as a softmax-weighted average of vocabulary embeddings and anchoring the table with an endpoint cross-entropy term at $t = 0$ (Section 4.1) [128]. At multi-billion-parameter scale, Sigma extends Plaid's formulation to 3B and 8B parameters using a 16-dimensional embedding table constrained to the unit sphere [405]. Even with fixed unit norms, the learned rows collapse into a subspace of roughly four effective dimensions unless regularized. Adding a spherical uniformity loss spreads the embeddings across the sphere and improves few-step sampling (Section 5.1) [405].

In sequence-to-sequence tasks, source conditioning retains the token-aligned target state while varying how source context enters the denoiser. DiffuSeq concatenates source and target embeddings into a single sequence and corrupts only the target portion, leaving the clean source visible at every step [119], whereas encoder–decoder variants such as SeqDiffuSeq [425] and Difformer [111] encode the source once and let a bidirectional target denoiser cross-attend to it. DiffuSeq-v2 keeps the concatenated layout but replaces a time-dependent subset of noised target vectors with a learned absorbing vector, training the denoiser to recover both masked and Gaussian-perturbed positions to support few-step sampling (Section 5.1) [120].

Vocabulary-coordinate and simplex states Operating in vocabulary coordinates avoids nearest-neighbor rounding: each position holds a $|\mathcal{V}|$-dimensional vector whose clean endpoint is a scaled one-hot vector or a point on the probability simplex, reducing token readout to an argmax or categorical sample. Simplex Diffusion introduced this approach for categorical data by running independent non-negative Cox–Ingersoll–Ross processes on each coordinate and normalizing by their sum, yielding a trajectory on the simplex with a Dirichlet stationary distribution [301]. Simplex Diffusion Models (SDMs) scale a Dirichlet construction to subword vocabularies: for a clean token x_i and a full-support prior π, their forward process $P_t \sim \mathrm{Dir}\big(c_t(\alpha_t e_{x_i} + (1 - \alpha_t)\pi)\big)$ decouples the mean from the concentration c_t and

admits closed-form reverse bridges [84]. Each update samples a candidate token from the predicted posterior and steps toward it on the simplex with controlled stochastic churn. In GSM8K code-generation evaluation after TinyGSM training, SDMs without self-conditioning outperform both self-conditioned masked diffusion and a reimplemented Dirichlet Flow Matching baseline at 512 sampling steps, though they still trail a greedy autoregressive baseline [84, 340]. SSD-LM retains Gaussian corruption over almost-one-hot logit vectors, projecting predicted logits to a hard or sampled token encoding and re-noising at each step within a left-to-right block loop [134]. TESS adapts simplex logit states to full-sequence text-to-text generation with a self-conditioning rule that averages current and previous token probabilities before re-embedding them [176], and TESS 2 scales this interface to a pretrained autoregressive backbone for instruction tuning [344]. Smoothie instead constructs a vocabulary-sized state from a fixed embedding table using negative half-squared Euclidean distances from the target token's embedding to all vocabulary embeddings. Its Gaussian reverse updates convert clean-embedding predictions trained with MSE back into distance coordinates, without intermediate token projection [319]. Because vocabulary-coordinate states represent token scores directly, external classifiers can take gradients with respect to them without an intermediate rounding step (Section 5.4).

Bayesian Flow Networks (BFN) parameterize the vocabulary-sized state as a categorical belief distribution over characters or tokens [124]. At each step, the network reads all position beliefs jointly to predict categorical distributions, and Gaussian evidence updates each belief multiplicatively via Bayes' rule before a terminal categorical draw emits the text [124]. Although beliefs, logits, simplex points, and distance vectors all have nominal width $|\mathcal{V}|$, passing a one-hot or simplex state into a Transformer through a linear input projection W maps the state into the model's hidden space with Gaussian noise covariance proportional to $W^\top W$, so the network effectively denoises in a learned embedding subspace under anisotropic noise [81].

Binary codes and spherical geometries Analog Bits compresses the channel width to $\lceil \log_2 |\mathcal{V}| \rceil$ by representing each token identifier in binary, diffusing the real-valued bits under Gaussian noise with squared-error regression, and thresholding the terminal prediction to look up tokens in parallel [64]. Because the binary code is fixed, no embedding table can collapse, and all lexical structure must be learned by the denoiser. Within the H-CDLM framework (distinct from HC-DLM in Section 5.2), Ollu and Komodakis augment CoBit's analog token bits with four auxiliary bits indexing 16 k-means clusters of pretrained token embeddings. H-CoBit corrupts the cluster and token channels under separate schedules and churn rates, denoises both jointly, and discards the cluster bits at readout, improving generative perplexity at dataset entropy and MAUVE at or above dataset entropy; a matched random-cluster ablation removes the generative-perplexity advantage [45, 275].

Spherical representations keep one vector per token and constrain states to lie on a hypersphere. The Hyperspherical Flow Language Model (S-FLM) normalizes learned token embeddings onto the unit sphere and learns a tangent velocity field from spherical noise [83], and Spherical Flows retract updated states onto the sphere at each step before a terminal classifier readout [57]. Intrinsic Brownian motion on the sphere supplies a dimension-free additive noise clock through the spherical heat semigroup and an analytic ancestral bridge; exact reverse sampling assumes the true token posterior, while the implementation introduces posterior and heat-table approximation errors (Section 5.1) [255]. Because manifold retraction preserves unit norm without snapping to a discrete token, spherical embedding models still face rounding ambiguity.[1] In SwaFlow (up to 3.741B parameters including embeddings), learned token embeddings become increasingly anisotropic, shifting the typical-case sufficient threshold for context-free recovery toward cleaner states; meanwhile, the model's low-loss recovery frontier moves toward noisier states [17]. LLaCO avoids embedding drift by freezing the 4096-dimensional input embeddings of Nemotron-Labs-Diffusion-8B and representing states by their unit directions, corrupting each direction along a great circle toward a mixture of a uniform direction and the `[MASK]` direction (Section 4.3) [106, 164]. Ranking candidate tokens by cosine similarity rather than Euclidean distance

[1] GeoFM addresses this in conditional generation by adding a token cross-entropy loss on a one-step reconstruction alongside its Riemannian flow objective [6].

prevents smaller-norm embeddings from overtaking the source token as signal decays.

A second spherical family maps categorical probability vectors onto the positive orthant of the sphere via the elementwise square root, where Fisher–Rao geodesics define the natural paths. Statistical Flow Matching (SFM) learns a tangent velocity field along these geodesics and samples tokens from the squared terminal state [68]. Fisher-Flow incorporates Riemannian optimal transport and mixes masked and uniform paths for language modeling [79], and RDLM learns a mixture of diffusion bridges toward the terminal token posterior on the square-root probability sphere [168]. Unlike learned-embedding spheres, which still require nearest-neighbor rounding or a classifier head, probability spheres maintain an explicit categorical distribution throughout generation.

3.2 Contextual and pretrained encodings

Unlike a static token lookup that assigns every occurrence of a word the same target vector, a pretrained text encoder produces position vectors that already reflect sentence-wide syntax and word sense. Generating in a pretrained contextual space lets the denoiser reuse the encoder's contextual representations, tokenizer, and scale. However, generated vectors can no longer be decoded by Euclidean proximity to a lookup table, so a learned or pretrained decoder is required to map contextual states back to tokens. The systems below maintain one vector per token position ($M = L$) and differ in whether the decoder is a separate module or shares weights with the denoiser.

TEncDM diffuses frozen contextual encodings at full sequence length ($M = L$), trains the denoiser by squared-error regression, and pairs it with a separately trained bidirectional decoder [317]. Embedded Language Flows (ELF) likewise generate pretrained contextual states but share a single Transformer backbone between clean-embedding prediction (converted to a flow velocity) and final token readout; in ELF's ablation, replacing the pretrained contextual targets with a jointly learned lookup table degrades generation quality [153]. Stronger encoder pretraining is not monotonically beneficial, however. Holding ELF's denoiser, objective, and sampler fixed, Zhang et al. find that swapping T5-small for a stronger T5Gemma-2 encoder lowers generative perplexity at matched entropy, but separates plausible word alternatives so widely that generated trajectories frequently land off-manifold [442]. Distilling the encoder into a student trained on the frozen teacher decoder's soft labels compacts the local embedding geometry, trading linear-probe classification separability for fewer off-manifold states, lower generative perplexity, and higher MAUVE [442]. Clean decodability alone is also insufficient for downstream reasoning: in CEDR, diffusing the top principal components of a single upper teacher layer yields near-perfect clean token reconstruction yet poor mathematical reasoning, whereas projecting across multiple intermediate layers under the teacher's decoding cross-entropy substantially boosts reasoning accuracy [72]. When a pretrained encoder comes from an encoder–decoder model, its pretrained decoder can be reused directly, as in the Latent Diffusion Paraphraser (LDP) over frozen BART states [472]. Separately, Diffusion-LEF pairs a frozen BART decoder with either learned 128-dimensional embeddings or frozen 768-dimensional BERT encodings [277].

Rather than freezing an off-the-shelf encoder, TextLDM trains a token-aligned variational autoencoder (TextVAE) from scratch with narrower channels ($M = L, d_z < d_{\text{model}}$) aligned to a frozen language model, then freezes the encoder to train a flow prior and decodes all tokens in parallel [166]. CoDAR also generates a token-aligned contextual sequence ($M = L$) but passes the completed states to a separate autoregressive decoder [327] (Sections 5.2 and 5.3 compare parallel, shared, and autoregressive readout).

3.3 Compressed text representations

When the generator operates on fewer positions than the text has tokens ($M < L$), or on narrower vectors at full length ($M = L, d_z < d_{\text{model}}$), a codec must first compress the text and later expand the generated code back into words. Position-count compression ($M < L$) shortens the sequence length M processed by the denoiser. Channel-width compression ($M = L, d_z < d_{\text{model}}$) reduces the dimension of the code at the state interface while leaving the per-layer Transformer denoiser cost at $O(LH^2 + L^2H)$ once codes are projected to the backbone's hidden width H. Both forms of compression face the trade-off between clean

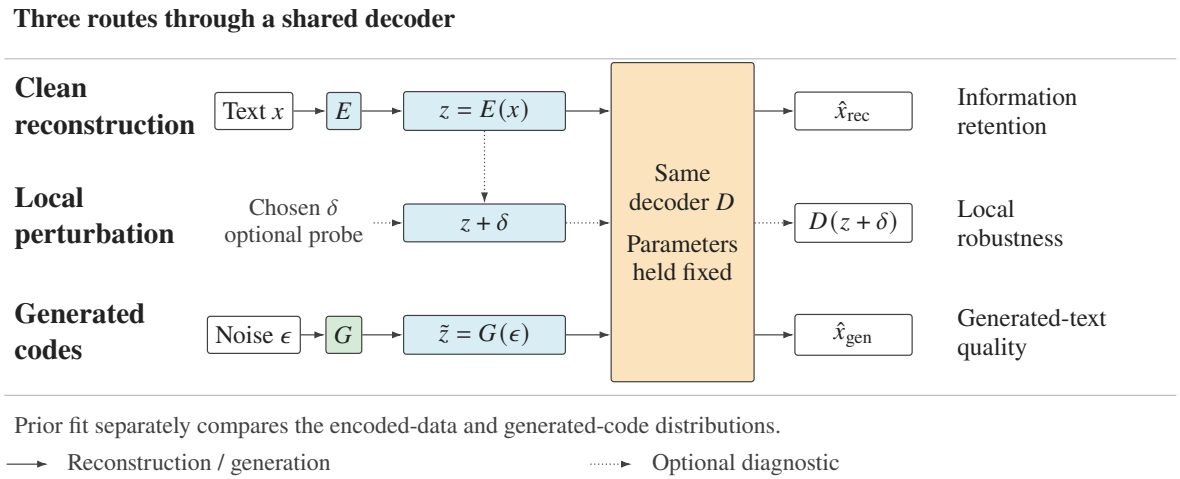


Figure 7. Evaluating a shared codec decoder on three input distributions: reconstructing clean encodings, probing local robustness under a perturbation δ, and decoding codes $\tilde{z}$ sampled from the learned latent prior. Because the decoder parameters are held fixed across all three settings, differences between reconstruction and generation performance reflect the distributional gap between encoded text and prior-generated codes. This schematic draws on diagnostics in PLANNER, LCM, SLD, TextLDM, LDLM, and Du and Ma [97, 166, 190, 253, 441, 462].

reconstruction fidelity and prior learnability introduced in Section 2.3 (Table 4): a latent space optimized solely for clean autoencoding can be difficult for the prior to match, whereas heavy regularization simplifies prior modeling at the expense of lexical detail. TextLDM illustrates this decoupling directly, as aligning its codec to a frozen language model leaves clean-text reconstruction almost unchanged while substantially improving continuation MAUVE [166]. Figure 7 compares decoder behavior on clean encodings, local perturbations, and prior-generated codes, with diagnostics detailed in Appendix B.

Position-compressed latent arrays LD4LG and PLANNER illustrate two designs for position-compressed arrays ($M < L$) with autoregressive decoding. LD4LG wraps a frozen pretrained language model in a deterministic codec: a Perceiver resampler compresses contextual states into a fixed-length code array, a reconstruction module expands generated codes back into the model's input space, and the language model's autoregressive decoder generates the surface text [239]. PLANNER instead trains a variational paragraph codec first, fits a diffusion prior to the frozen codec's posterior means, and decodes paragraphs autoregressively from sampled plans [441]. LLMAE constructs a compressed codec inside a single decoder-only language model by appending 256 learned query tokens, masking upper layers to reconstruct the document solely from those intermediate query activations via low-rank adaptation, and training an image-conditioned diffusion prior over the frozen LLMAE space for captioning (Appendix G.3) [283].

Position-compressed codes can also be decoded in parallel. Cosmos trains a deterministic Perceiver codec separately and decodes generated codes into all tokens simultaneously [252]. JPEG-DLM retains a Perceiver compressor over frozen T5-small features and a parallel decompressor, co-training the compressor alongside the flow network via an exponential-moving-average target code (Section 4.2). Compared with a retrained Cosmos baseline of matched depth, co-training improves generative perplexity and MAUVE across compression ratios, eliminates midpoint perplexity spikes along latent interpolations, and reaches lower perplexity than uncompressed ELF on longer sequences at a slight cost in diversity [426]. Manta-LM instead compresses sequences locally through a convolutional TextVAE whose codes preserve regional alignment, refines them with a context-aware flow Transformer, and decodes tokens through a parallel convolutional decoder [93].

The information a short latent array must retain depends on how the decoder consumes it. Latent Thought Flows compresses 256 tokens into 8 continuous latents and shows that codec designs must be ranked by prior-generated sample quality rather than clean reconstruction accuracy [288]. Predictive-State

Flows (PSF) instead restrict each of the $M = 8$ latents for a 1024-token document to encode only the prefix preceding its block. A rectified-flow prior generates all eight predictive states jointly, and a block decoder realizes the eight blocks in parallel, operating autoregressively within each block and coupling across boundaries only through the next block's initial token [140]. Supervising latents to represent predictive prefix states rather than within-block content enables PSF to outperform uncompressed token-aligned flows in MAUVE at a fraction of the per-document FLOPs [140].

A third interface pairs a continuous latent prior with a discrete-diffusion decoder. LaDiff compresses frozen BERT features via a Perceiver module trained under a masked-diffusion reconstruction loss, fits a continuous prior to the frozen codes, and unmasks tokens iteratively conditioned on the generated latent array; DiLaDiff distills the continuous prior into a few-step flow map (Section 5.1) [195]. CRoCoDiL applies a similar cascade to code generation, using an ordered bank of continuous summary registers to condition LLaDA [268] via prefix embeddings [357]. Cola2 couples a block-causal flow prior over dynamically segmented semantic spans (averaging roughly two tokens per 64-dimensional latent under a gradient-free rate controller) to a block-causal masked-diffusion decoder. In controlled scaling comparisons without compression, replacing Cola's causal decoder with the masked-diffusion decoder yields substantial gains in continuation accuracy [132].

Channel-width compression at full sequence length Several autoencoder systems narrow the channel dimension ($d_z < d_{\text{model}}$) while preserving one latent position per token ($M = L$). Like TextLDM, Cola's default TextVAE narrows channel width at full sequence length, treating patch compression ($M < L$) as a separate variant [131]. AURORA instead retains a full-width clean latent with $M = L$ and $d_z = 1024$, restricting only the denoiser's noisy-input pathway to 128 channels; length compression is a separate ablation (Table 22) [220]. In Cola's patch-compression experiments, degradation concentrates on prompts whose token lengths do not divide evenly into patches because of prefix boundary misalignment [131].

Reasoning steps as compressed units In multi-step reasoning, each compressed latent block can represent an intermediate reasoning step. LaDiR first trains a VAE to compress each rationale step into a block of thought vectors reconstructed by a frozen LLM decoder. It then trains a flow-matching model inside the LLM backbone to denoise thought blocks before emitting the final answer tokens autoregressively, adding a rollout stage that backpropagates answer supervision through generated latent trajectories [175]. LaDi-RL extends this architecture to reinforcement learning by sampling multiple answer realizations per latent trajectory to separate planning errors from realization errors [174]. By contrast, Coconut feeds a language model's last hidden state directly back as the next input embedding without a generative diffusion or flow prior, and is therefore covered in Appendix C [138].

3.4 Sentence and segment representations

At the coarsest granularity, an entire sentence or discourse segment is represented by a single vector or a small fixed-size slot array, delegating surface realization to a separate generative decoder. Sentence-level states capture high-level semantics and shorten the denoiser sequence dramatically, but leave exact entity names, numerals, and local syntax to the decoder. Sentence-space diffusion builds on earlier variational sentence autoencoders such as Optimus, which coupled a BERT encoder to a GPT-2 decoder [201]. Systems in this family vary in whether sentence vectors are generated jointly across a document or autoregressively one concept at a time, and whether decoding a sentence vector conditions on the surface words realized from preceding vectors (Section 5.2).

Joint arrays or next-vector prediction LaDiDa generates all sentence vectors of a document jointly via a self-conditioned DDPM over concatenated sentence representations, boundary markers, and padding, and reconstructs surface text with a BART-base Vec2Seq decoder [411]. Comparing its two decoder variants isolates the role of surface history: conditioning each sentence's realization on both its latent vector and the previously decoded sentences (LaDiDa-CV) lowers sentence-level reconstruction perplexity

relative to decoding each vector independently in parallel (LaDiDa-V). LaDiDa-CV exceeds fine-tuned autoregressive baselines in MAUVE on ROCStories and Scientific Papers, but remains below GPT-2 on WikiSection [411].

Diffusion LCM shifts next-unit autoregression from tokens to frozen SONAR sentence embeddings [98]: in its Two-Tower architecture, a causal Transformer encodes preceding sentence concepts to condition a diffusion denoiser that iteratively refines the next sentence vector before the frozen SONAR decoder converts it to text [190]. Mimir trains this Two-Tower recipe at 1.6B parameters over multilingual SONAR embeddings across 46 languages and instruction-tunes it on 35 languages, motivated by the imperfect cross-language alignment of SONAR embeddings [264]. By contrast, SONAR-LLM predicts the next SONAR vector in a single deterministic forward pass supervised by token cross-entropy through the frozen decoder [94]. Segment-Level Diffusion (SLD) instead encodes variable-length text segments into fixed-size latent slot arrays, denoises all output segment arrays jointly, and decodes segments in parallel with segment-local autoregression [462]. At the chunk level, CALM [323] primarily uses a single-step energy-score head (Appendix C) while also benchmarking multi-step diffusion and flow-matching heads over $K = 4$-token chunk vectors, feeding the decoded tokens of each chunk back into the outer autoregressive loop (Section 5.3).

Semantic proposals for an autoregressive decoder Rather than requiring the latent vector to specify every word, Diffusion-Guided Language Modeling (DGLM) treats a diffused sentence vector as a soft-prompt proposal for an adapted autoregressive language model, training the prompt interface on noisy proposals so that the decoder's adherence to the latent plan can be tuned continuously [240]. In multimodal generation, Versatile Diffusion generates Optimus sentence latents [201] from image or text conditions and decodes them through the pretrained GPT-2 decoder [396]; CoDi extends this latent-text approach to conditions combining text, images, audio, and video (Appendix G.3) [350].

4 The Transition: Learning the Denoiser

Once the state representation is fixed (Section 3), the denoiser or flow network must be trained to move noisy states toward text along a chosen time grid (Section 5.1). This section covers the prediction target and loss function (Section 4.1), the training schedule across encoder, denoiser, and decoder modules (Section 4.2), and noise calibration and self-conditioning (Section 4.3).

4.1 Prediction targets

Along a Gaussian path at nondegenerate noise levels ($\alpha(t) > 0, \sigma(t) > 0$), the standard continuous vector targets are algebraically equivalent: given $z_t = \alpha(t)z_0 + \sigma(t)\epsilon$, an estimate of the clean state $\hat{z}_0 \in \mathbb{R}^d$ uniquely determines the noise estimate $\hat{\epsilon}$, the plug-in score $\hat{s}_\theta$, and the v-parameterization via the invertible affine relations of Equation 2. By contrast, a predicted token posterior $p_\theta(\cdot \mid z_t, t, c) \in \Delta^{|\mathcal{V}|-1}$ determines the expected embedding $\hat{z}_0 = \sum_{v\in\mathcal{V}} p_\theta(v \mid z_t, t, c)e_v \in \mathbb{R}^d$, whereas the reverse mapping is underdetermined whenever $|\mathcal{V}| > d + 1$ because $E^\top$ has a nontrivial nullspace. The choice of prediction target and loss therefore governs whether the network outputs a vector in $\mathbb{R}^d$ or a distribution on $\Delta^{|\mathcal{V}|-1}$, how approximation errors are weighted across noise levels, and whether supervision is tied directly to discrete token identities.

Continuous vector and velocity targets Table 7 summarizes the main prediction targets. The most widely used vector objective regresses the clean state directly,

$$\mathcal{L}_{\text{clean}} = \mathbb{E}\left[\lambda(t)\,\|f_\theta(z_t, t, c) - z_0\|^2\right], \tag{5}$$

where $\lambda(t)$ weights noise levels and c denotes the condition. Diffusion-LM, SED, TEncDM, Analog Bits, and Diffusion LCM all train x_0-predictors under this loss [64, 190, 213, 317, 341]. Noise (ϵ) prediction, the standard parameterization in image diffusion, regresses ϵ instead and recovers the clean

Table 6. Summary of continuous state families by representation dimensions ($M \times d$), failure modes, and decoding requirements. Rows represent overlapping design axes illustrated by the exemplars of this section, as contextual encoding, position-count versus channel-width compression, state geometry, and generation order can be combined across designs. Hybrid discrete-canvas models that pair masked tokens with auxiliary continuous latents (e.g., CADD, CCDD, Co-LADD) are contrasted separately in Appendix C (Tables 19 and 20). Prediction targets and training stages are compared separately in Tables 7 and 8, and Appendix F lists the full configuration of each surveyed system (AR denotes autoregressive).

State	Generated unit and dimensions	Representation geometry, failure mode, and decoding requirement
Learned or fixed token table: Diffusion-LM; SED; CDCD; Plaid [87, 128, 213, 341]	One vector per token position ($M = L, d \ll \lvert\mathcal{V}\rvert$), from a learned or fixed input lookup	A learned table collapses under pure regression unless anchored by a fixed input lookup (SED) or by token-identity supervision (reconstruction cross-entropy, anchor loss, or uniformity regularization) paired with scale control; intermediate predictions between embeddings require clamping, rounding, or expected-embedding substitution.
Vocabulary coordinates: SSD-LM; TESS; Smoothie; BFN; SDM [84, 124, 134, 176, 319]	A vocabulary-sized vector per position ($M = L, d = \lvert\mathcal{V}\rvert$: logits, simplex point, distance vector, or belief)	Token identity and uncertainty are explicit in the coordinates and readout is an argmax or categorical draw; the state is as wide as the vocabulary, with dynamics set by logit noise, simplex/Dirichlet paths, belief updates, or distance vectors.
Bit codes and spherical states: Analog Bits; S-FLM; SFM [64, 68, 83]	Analog bits ($\lceil \log_2 \lvert\mathcal{V}\rvert \rceil$ per token) or one unit-norm point per position on a sphere ($M = L$)	Fixed bit codes narrow width without a learned table; spherical constraints fix vector norm, where a learned-embedding sphere still requires rounding or classification and uniformity control, whereas a square-root probability sphere represents an explicit categorical distribution.
Contextual and channel-compressed token states: TEncDM; ELF; TextLDM; Cola; CoDAR [131, 153, 166, 317, 327]	Token-aligned encoder outputs ($M = L$): full-width pretrained features, compact pretrained embeddings (64-dimensional Qwen3-Embedding states in CoDAR), or narrower TextVAE channels ($d_z < d_{\text{model}}$)	Pretraining or codec training supplies contextual structure; keeping $M = L$ leaves denoiser sequence length unchanged while requiring a trained or borrowed parallel, shared, or AR decoder.
Position-compressed code arrays: LD4LG; PLANNER; Cosmos; Manta-LM; LaDiR [93, 175, 239, 252, 441]	Fewer generated positions than tokens ($M < L$), summarizing a passage, span, or rationale step	Shortening $M < L$ reduces sequence-level denoiser cost and shifts surface realization to an AR, parallel, or in-backbone decoder; clean reconstruction accuracy does not guarantee that prior-generated codes decode faithfully (latent–prior mismatch).
Sentence or segment vectors: LaDiDa; LCM; SLD; DGLM [190, 240, 411, 462]	One vector (or fixed-size slot array) per sentence, segment, or chunk	Extreme position compression captures sentence-level semantics while leaving lexical fidelity, entities, and syntax to a generative decoder or soft-prompted LM.

state via $\hat{z}_0 = (z_t - \sigma(t)\hat{\epsilon})/\alpha(t)$. GENIE and CodeFusion pair ϵ-prediction with an endpoint embedding reconstruction loss [222, 334]. For language generation, however, x_0- and ϵ-prediction behave differently under finite model capacity: Diffusion-LM adopts x_0-prediction specifically so that every training update pulls the network output directly toward valid token embeddings [213]. Clean-state targets can also couple adjacent positions. SCLM retains Difformer's x_0 regression and anchor loss but trains the denoiser to predict B-spline control points whose evaluations at token indices yield the clean embeddings. This target couples predictions across token positions and improves English–German translation over Difformer on IWSLT14 and WMT14 when combined with sequence-level distillation [111, 144]. Salimans and Ho's v-parameterization, $v = \alpha(t)\epsilon - \sigma(t)z_0$, interpolates between x_0- and ϵ-prediction and avoids division by a

vanishing $\alpha(t)$ at high noise [311]. LD4LG's compressed latent prior, DGLM's sentence-vector denoiser, and RegDiff's latent denoiser all train with v-prediction and sample via DDPM or DDIM [239, 240, 455].

Distinct from diffusion's v-parameterization, flow matching regresses the conditional path velocity of Equation 3 (which simplifies to $z_0 - \epsilon$ along the linear path $z_s = (1-s)\epsilon + sz_0$) and integrates $z_{s+\Delta s} = z_s + \Delta s\, v_\theta(z_s, s, c)$ at inference, as in FlowSeq [154]. Alternatively, the denoiser can predict a categorical distribution over the vocabulary at each position, either supervised directly by token cross-entropy across noise levels (CDCD, TESS, FMLM, Masked-Diffuse LM) or used to compute a softmax-weighted expected embedding inside a variational likelihood bound (Plaid, RePlaid, Sigma).

Token-posterior targets and expected embeddings Predicting a categorical distribution over the vocabulary at each position allows a continuous denoiser to be supervised by cross-entropy while still evolving a real-valued state. CDCD predicts $p_\theta(x_i = v \mid z_t, t, c) \in \Delta^{|\mathcal{V}|-1}$ at each target position and computes the probability-weighted expected embedding:

$$\hat{z}_{0,i} = \sum_{v \in \mathcal{V}} p_\theta(x_i = v \mid z_t, t, c)\, e(v). \tag{6}$$

Under the Gaussian corruption of Equation 1, this expected embedding yields the plug-in score estimate:

$$\hat{s}_{\theta,i}(z_t, t, c) = \frac{\alpha(t)\hat{z}_{0,i} - z_{t,i}}{\sigma(t)^2}, \qquad \sigma(t) > 0. \tag{7}$$

With $\alpha(t) = 1$, $\sigma(t) = t$, and L_2-normalized token embeddings, CDCD trains p_θ via token cross-entropy and integrates the resulting probability-flow ODE $\mathrm{d}z/\mathrm{d}t = (z - \hat{z}_0)/t$ from high to low noise using 200 Euler steps, $z_{t-h} \approx z_t + (h/t)(\hat{z}_0 - z_t)$ [87]. Because the continuous state moves toward the expected embedding without snapping to a discrete token at intermediate steps, categorical supervision directly drives continuous transport. Moreover, when $|\mathcal{V}| > d + 1$, token cross-entropy supervises the full simplex distribution rather than only its d-dimensional projection $\hat{z}_{0,i}$ (Appendix A.2). Plaid uses the same categorical parameterization $\hat{z}_0 = W\,\mathrm{softmax}(f_\theta(z_t, t))$ to form the expected embedding, but supervises $\hat{z}_0$ inside the continuous-time diffusion integral via weighted squared error $-\frac{1}{2}\mathrm{SNR}'(t)\|z_0 - \hat{z}_0\|_2^2$, where $\mathrm{SNR}(t) = \alpha(t)^2/\sigma(t)^2$, paired with an endpoint token cross-entropy term at $t = 0$ [128]. TESS and Masked-Diffusion LM likewise apply token cross-entropy across noise levels over logit or hidden states [59, 176], and Chandra et al. separate context prediction from local corruption evidence with a hollow parameterization and use sufficient-statistic inputs to share one denoiser across discrete, Gaussian, and simplicial diffusion [55].

Endpoint token anchoring When a vector-regression denoiser learns its token embedding table jointly, an endpoint token-discrimination loss can counteract embedding collapse (Section 2.3) [111]. Difformer's anchor loss passes the predicted clean embedding $\hat{z}_0$ through the tied vocabulary projection and applies token cross-entropy [111]. FlowSeq applies the same cross-entropy to the linearly extrapolated endpoint $z_s + (1-s)v_\theta(z_s, s, c)$ [154], and YAN, which parameterizes velocities with a mixture of experts, supervises the terminal state reached by unrolling a multi-step training trajectory [196]. Rather than penalizing off-manifold endpoints only through the loss, ConvergeFlow builds the vocabulary constraint into the denoiser architecture by expressing the predicted endpoint as a learned positive-weight barycenter of vocabulary embeddings weighted by the Gaussian corruption kernel, guaranteeing convergence to a valid vocabulary embedding under its stated conditions [209]. Rectified Language Flow derives a related guarantee in closed form by separating singular logit terms that diverge as noise vanishes from a bounded neural residual, ensuring convergence to valid token embeddings under its stated assumptions and exact continuous-time dynamics [342].

Variational and probability-flow likelihood objectives Unlike heuristically weighted regression losses, which train a sampler without defining a calibrated data log-likelihood, Plaid optimizes a variational likelihood bound end to end, jointly learning the token embeddings, noise schedule, and endpoint token

Table 7. Denoiser prediction targets, training losses, and update roles. At nondegenerate noise levels ($\alpha(t) > 0, \sigma(t) > 0$) on a Gaussian path, the vector predictions ($\hat{z}_0, \hat{\epsilon}, \hat{v}$, and, when $\alpha(t)\dot{\sigma}(t) - \sigma(t)\dot{\alpha}(t) \neq 0$, the marginal path velocity) are invertible affine reparameterizations of one another (Equations 2 and 4), whereas a token posterior on $\Delta^{|\mathcal{V}|-1}$ determines the expected embedding $\hat{z}_0 \in \mathbb{R}^d$ without being uniquely determined by it when $|\mathcal{V}| > d + 1$. Endpoint anchor or reconstruction losses can supplement the vector-regression rows.

Denoiser prediction	Training loss supervising the denoiser	Role in a generation update
Clean state x_0 (Diffusion-LM, SED, TEncDM, Analog Bits, LCM)	Weighted squared error to the clean state, Equation 5 (in SED, readout CE is stopped before the denoiser)	Combine $\hat{z}_0$ with z_t and the schedule in a DDPM, DDIM, or ODE update (Section 5.1).
Noise ϵ (GENIE, CodeFusion)	Weighted squared error to the sampled noise ϵ, plus endpoint embedding reconstruction	Convert by $\hat{z}_0 = (z_t - \sigma(t)\hat{\epsilon})/\alpha(t)$ for $\alpha(t) > 0$, then update as above.
$v = \alpha(t)\epsilon - \sigma(t)z_0$ (LD4LG, DGLM, RegDiff)	Weighted squared error to v_t (LD4LG's uncompressed BART-Diffusion baseline instead uses x_0)	Convert by $\hat{z}_0 = \alpha(t)z_t - \sigma(t)\hat{v}$ when $\alpha^2 + \sigma^2 = 1$, then update as above.
Token posterior $p_\theta(\cdot \mid z_t) \in \Delta^{\lvert\mathcal{V}\rvert-1}$ (CDCD, TESS, FMLM, LangFlow)	Token cross-entropy against observed tokens x across noise levels	Average token embeddings (Equation 6) and form a plug-in score (Equation 7) or interval map to update the state.
Expected embedding via logits $\hat{z}_0 = W\,\mathrm{softmax}(f_\theta)$ (Plaid, RePlaid, Sigma)	Variational bound: weighted squared error $-\frac{1}{2}\mathrm{SNR}'(t)\lVert z_0 - \hat{z}_0\rVert_2^2$ on the diffusion path plus endpoint token cross-entropy at $t = 0$ (in Sigma, combined with joint AR token cross-entropy)	Use the expected embedding $\hat{z}_0$ in ancestral, DDIM, or ODE updates, and decode tokens from the terminal logits.
Conditional-path velocity u_s (FlowSeq, ELF, TextLDM)	Squared error to the conditional velocity $u_s(z_s \mid z_0, \epsilon)$, Equation 3	Integrate $z_{s+\Delta s} = z_s + \Delta s\, v_\theta(z_s, s, c)$ from noise toward data.

likelihood by combining the SNR-weighted expected-embedding diffusion loss with an endpoint token cross-entropy term at $t = 0$ and a prior Kullback–Leibler term at $t = 1$ [128]. In Plaid's human evaluation with fixed embeddings, annotators significantly preferred the likelihood-weighted model over variants trained with heuristic weight schedules [128], contrasting with Diffusion-LM's continuous-time one-hot likelihood experiments, whose combined modifications improved likelihood without improving generation quality [213]. Optimizing a valid bound also enables direct zero-shot perplexity comparisons against autoregressive models: Plaid 1B achieves lower held-out perplexity bounds than GPT-2 124M [128], albeit at a substantial training-compute multiple (Section 7.1). LangFlow connects token cross-entropy training to flow matching through a Bregman-divergence formulation, derives its continuous denoiser from predicted token probabilities, and separately constructs an ODE-based likelihood bound for evaluation [65], while RePlaid aligns Plaid's architecture and numerical precision with discrete diffusion baselines for compute-matched likelihood comparisons [406]. Sigma adapts Plaid's three-term variational bound to blockwise generation: following BD3-LM, the sequence likelihood factorizes across token blocks, and each block is supervised by a prior KL term, an endpoint cross-entropy term, and an SNR-derivative-weighted squared error between expected and clean embeddings [36, 405]. Appendix D.1 details these likelihood estimators and their sensitivity to self-conditioning.

4.2 Training stages and module adaptation

A continuous text generator comprises the token lookup or encoder that defines the state, the denoiser or flow network, and the readout head or decoder that maps states back to tokens (Section 5.2). Freezing a pretrained encoder gives the denoiser stationary targets and reuses existing representations. Staging codec pretraining before prior fitting preserves reconstruction fidelity at the risk of latent–prior mismatch,

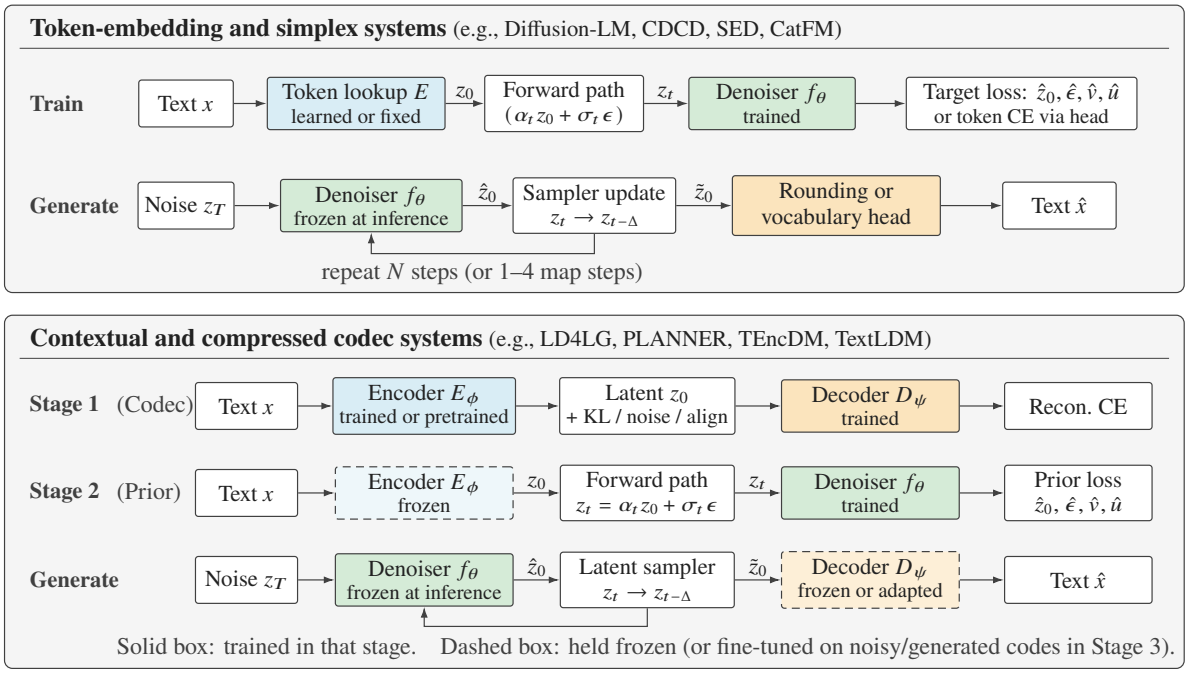


Figure 8. Training and generation pipelines for token-aligned and codec systems. Top: token-embedding and simplex systems (e.g., Diffusion-LM, CDCD, SED, CatFM), which map observed text x through a learned or fixed token lookup E to z_0 and decode generated states via nearest-embedding rounding or a vocabulary head. Bottom: the staged pipeline for contextual and compressed codec systems (e.g., LD4LG, PLANNER, TEncDM, TextLDM, AURORA), which train or align an encoder E_ϕ and decoder D_ψ in Stage 1, freeze E_ϕ while training the latent denoiser f_θ in Stage 2, and decode generated codes $\tilde{z}_0$ through D_ψ at inference; joint-adaptation recipes such as Cola and LDLM instead update the codec and latent prior together (Table 8) [166, 239, 341, 441].

whereas joint adaptation reshapes the latent space alongside the prior at the cost of non-stationary targets. Figure 8 illustrates these pipelines, and Table 8 summarizes which modules each recipe updates.

In token-aligned systems, the state representation can be either frozen in advance or learned jointly. SED freezes its stage-1 token-embedding lookup table, trains the denoiser by squared error, and trains a separate linear readout head by token cross-entropy on sampled clean embeddings; the reconstruction loss updates only the readout matrix [341]. TEncDM freezes a pretrained contextual encoder and trains a parallel decoder of its own [317]. Learning the table, as Diffusion-LM, CDCD, and Plaid do, lets the objective shape token geometry under noise but requires the anchoring or endpoint token losses of Section 4.1 [87, 128, 213].

Staged codecs In learned-codec systems, stage 1 typically trains the autoencoder on clean text and stage 2 freezes the encoder (or the entire codec) while fitting the latent prior. PLANNER freezes its variational paragraph codec before training the conditional encoder and diffusion prior [441]. Monolingual LD4LG learns its Perceiver compression and reconstruction modules around a frozen language model; for machine translation, it jointly fine-tunes the language model and autoencoding modules [239], and TextLDM and AURORA likewise freeze their stage-1 encoders during stage-2 flow-matching training [166, 220]. The strength of stage-1 regularization governs how well the frozen latent space supports prior generation: LaDiff trains its Perceiver compressor and masked-diffusion decoder under masking, Gaussian noise, and dropout, showing that heavier stage-1 regularization improves prior-generated sample quality even as clean reconstruction accuracy declines (Section 2.3) [195]. Stage 1 can also build control structure directly into the codec. RegDiff trains its BERT-initialized encoder and GPT-2-initialized decoder under joint reconstruction, KL, and attribute-classification losses before freezing both. It then supervises its v-parameterized denoiser with the frozen attribute classifier on pooled clean-latent predictions so that sampling requires no inference-time classifier gradients (Section 6.1.1) [455].

Table 8. Modules updated or frozen across training stages in representative recipes, ordered from frozen state representations to joint codec–prior adaptation (AR denotes autoregressive).

Training choice	Modules updated vs. frozen at each stage	Representative configuration
Fixed lookup or features	Stage-1 input lookup or pretrained encoder frozen; denoiser and separate readout head or decoder updated (in SED, readout CE is trained under a stop-gradient)	SED's bottleneck-BERT input lookup; TEncDM's frozen encoder
Learned lookup	Token embedding table, denoiser, and readout updated jointly under a loss with token-identity anchoring	Diffusion-LM; CDCD; Plaid
Staged text codec	Stage 1 trains encoder and decoder; stage 2 freezes the encoder (or full codec) while fitting the latent prior (a condition encoder may still update)	PLANNER; monolingual LD4LG; TextLDM ($M = L$, narrower channels); AURORA; LaDiff
Shared denoiser and vocabulary head	Pretrained target encoder frozen; one shared backbone updated for both continuous prediction and final token readout	ELF; LangFlow
Joint latent adaptation	Encoder, latent prior, and decoder updated together (upstream feature extractor may stay fixed)	LDEBM (Gaussian diffusion with energy-based reverse conditionals); LDLM; Cola (co-adapted after VAE pretraining); LDLM-s (jointly trained from random initialization)
Detached-prior co-training	Codec and latent prior trained together, but the prior is fitted to stop-gradient codes, so its loss does not reach the encoder	Cola2 (encoder co-adapted through masked-diffusion decoder losses); JPEG-DLM (compressor trained through a clean-prediction branch of the shared flow network)
Decoder-informed denoiser	Codec trained first and frozen; denoiser updated by gradients backpropagated through the frozen decoder	SLD diffusion stage
Adapted language model	Pretrained AR or masked backbone and continuous input/output interfaces updated by continued training	TESS 2; DSL-LLaDA; Sigma

Joint adaptation Rather than freezing the encoder after stage 1, joint adaptation updates the codec alongside the generative prior. LDEBM jointly trains a sentence encoder, an autoregressive decoder, and a Gaussian diffusion prior whose reverse conditionals are parameterized by an energy-based model sampled via short-run Langevin dynamics [421]. LDLM co-adapts an encoder, decoder, and denoiser on top of frozen pretrained contextual features using separate gradient pathways for reconstruction and diffusion [253]. Cola systematically compares frozen, periodically updated, weakly updated, and continuously co-adapted TextVAE codecs, finding the strongest scaling behavior when continuous co-adaptation follows an initial VAE reconstruction pretraining phase [131]. Unrestricted prior gradients can destabilize the encoder, however. Cola2 detaches the latents fed to its flow-matching loss, while retaining the codec's token-prediction, KL-regularization, and span-duration objectives [132], and JPEG-DLM (whose compressor collapsed when exposed to flow-matching gradients across all sampled times) updates its compressor only through the $t = 1$ clean-prediction branch [426]. Conversely, LDLM-s demonstrates that joint encoder–prior–decoder training from random initialization, without a separate stage-1 pretraining phase, can improve the OpenWebText GenPPL–entropy trade-off over the pretrained-feature baseline ELF-B in their overlapping entropy range [16].

Learning through a frozen decoder Even when the codec is held frozen during stage 2, gradients backpropagated through the frozen decoder can supervise the denoiser on token-level fidelity alongside latent-space distances. For example, SLD trains its codec with reconstruction, contrastive, and adversarial

objectives before freezing it and propagating token reconstruction losses back to the segment denoiser [462]. Reinforcement learning extends this supervision to sequence-level rewards: LaDi-RL updates its latent flow policy via group relative policy optimization (GRPO) [324] using squared-error step residuals [174], whereas x-Prediction Flow trains a GRPO policy to allocate per-position step sizes during sampling (Section 5.1) [371]. CFRL instead trains a critic that predicts the verifier outcome from intermediate ELF states and backpropagates its value through one stochastic denoiser transition, bypassing the non-differentiable readout and outperforming group-relative REINFORCE on synthetic reasoning puzzles [137].

Pretraining and adapting the denoiser Language knowledge can enter the denoiser either through denoising pretraining from scratch or by adapting a pretrained language model checkpoint. Among systems pretrained on denoising tasks, GENIE masks a contiguous paragraph span, encodes the surrounding document context, and trains a cross-attending denoiser to reconstruct the span embeddings via ϵ-prediction [222]. CodeFusion adapts this pretraining task to program synthesis with a separate contextual decoder (Section 6.1.5) [334], and TESS adapts a pretrained masked language model to continuous simplex logits [176]. At multi-billion-parameter scales, most token-level systems surveyed here adapt a pretrained autoregressive or masked checkpoint rather than pretraining from scratch. Following the discrete conversion recipe of DiffuGPT and DiffuLLaMA (DiffuGPT anneals causal masks toward bidirectional attention, while DiffuLLaMA-7B switches directly to bidirectional attention [121]), TESS 2 adapts Mistral-7B to simplex diffusion using UL2-style masking, label shifting, and bidirectional attention prior to instruction tuning [344]. L2D instead freezes the main causal backbone and attaches a low-rank-adapted diffusion branch initialized from the same blocks to refine the next token [53], while DSL-LLaDA continues pretraining the masked LLaDA-8B-Instruct checkpoint for 1,000 steps with continuous per-token Gaussian noise in place of binary masking [420].

Recent blockwise conversions pair continued pretraining with joint autoregressive and diffusion supervision. Sigma initializes 3B and 8B models from autoregressive checkpoints (with the 3B model pretrained on 1T tokens) and continues training for over 300B tokens, optimizing autoregressive cross-entropy on a clean token stream alongside a weighted blockwise diffusion bound on a noisy 16-dimensional embedding stream under block-causal attention [405]. Following NLD's use of autoregressive initialization and an auxiliary autoregressive objective to improve downstream accuracy [106], Sigma finds that autoregressive initialization accelerates convergence and its auxiliary autoregressive loss improves training stability [405]. Subsequent blockwise adaptations require far fewer tokens. Lafa adapts 0.5B–8B Qwen2.5 and Qwen3 checkpoints to blockwise flow matching using 9–27B instruction tokens, equipping each position with its own time via identity-initialized adaptive LayerNorm, maintaining causal attention over the clean prefix and bidirectional attention within each noisy block, and fixing each shared pretrained embedding's norm at its pretrained value while training its direction [56]. Retaining the auxiliary autoregressive loss and directional embedding updates improves mean accuracy across instruction, math, and coding benchmarks over frozen or unconstrained embeddings in the 0.5B ablation [56]. LLaCO adapts NLD-8B on roughly 1.7B instruction tokens by freezing its input embeddings and training a directional adapter that maps each unit-norm state to a softmax-weighted embedding mixture [106, 164], while SDAR provides a discrete counterpart that converts an autoregressive backbone to blockwise masked diffusion on 50B tokens (Appendix C) [71].

4.3 Noise schedules and self-conditioning

In continuous text generation, schedule design separates the forward corruption path, the training time weighting, and the inference discretization grid. The forward corruption path $(\alpha(t), \sigma(t))$ of Equation 1 defines the signal and noise scales at time t (where rescaling the embeddings alters the effective signal-to-noise ratio even if α and σ are held fixed). The training time distribution $p(t)$ and loss weight $\lambda(t)$ in Equation 5 govern how training compute is allocated across noise levels, with time resampling and loss reweighting acting equivalently in expectation. Finally, the inference discretization grid specifies where the sampler evaluates the denoiser at generation time (Section 5.1). Differmer's noise rescaling,

SeqDiffuSeq's adaptive per-token schedule, and AR-Diffusion's position-dependent clocks modify the forward path $(\alpha(t), \sigma(t))$, whereas DiNoiser's minimum noise threshold, CDCD's learned time warp, Plaid's SNR-based likelihood weighting, and AURORA's width-specific schedule sweeps reshape $p(t)$ or $\lambda(t)$ [220]. Self-conditioning, examined at the end of this subsection, complements schedule design by feeding the denoiser's previous clean-state prediction back as an auxiliary input.

Calibrating noise to the embedding geometry and reweighting time Because the signal-to-noise ratio at which a token ceases to be recoverable from its own perturbed vector depends on inter-embedding distances, schedules copied directly from vision can spend most of training on trivial local denoising. Difformer diagnoses this regime from single-token nearest-neighbor recoverability and rescales the forward noise so that isolated recovery fails [111]. DiNoiser derives a minimum noise threshold $\sigma_{\min}$ from embedding separability, restricts training time samples to $\sigma(t) \geq \sigma_{\min}$ so the denoiser must rely on context and source conditions, and feeds the denoiser an inflated noise level at inference while stepping with the nominal level [409]. SeqDiffuSeq learns position-specific forward schedules from observed denoising losses so that expected denoising loss increases approximately linearly with diffusion time [425], while CDCD stabilizes effective signal scale by L_2-normalizing its embeddings [87]. In AURORA's width ablations, the relative ranking of latent widths under a single fixed schedule changes once each width is granted a matched schedule sweep, showing that schedule calibration depends on representation width [220]. Rather than querying a trained denoiser, KIS estimates conditional token entropy under Gaussian corruption from unigram frequencies and fixed embedding geometry, using a full-vocabulary posterior for one-hot FLM and a top-K posterior approximation for other representations. It shifts the training log-SNR distribution while preserving its standard deviation and applies a positive affine correction to interior sampling log-SNR levels. On LM1B, calibrated FLM-B reaches GenPPL 94.73 at 256 steps versus 97.95 for the uncalibrated 1024-step baseline [24].

Even when the forward path $(\alpha(t), \sigma(t))$ is fixed, uniform time sampling allocates excessive effort to low-gradient regimes. CDCD learns a monotone time warp by fitting an unnormalized function $\tilde{F}$ to the cross-entropy loss across noise levels, normalizing it to a cumulative distribution function F, and sampling training times via inverse transform $t = F^{-1}(u)$ for $u \sim \mathcal{U}(0, 1)$, matching loss reweighting in expectation with lower variance [87]. Plaid sets $\lambda(t)$ directly from its learned variational bound, the weighting preferred in its human evaluation (Section 4.1) [128]. Sigma decouples schedule learning into two parts, optimizing the schedule endpoints under the variational bound while shaping the interior profile (to which the continuous-time diffusion integral is invariant in expectation) to minimize the Monte Carlo variance of the diffusion-loss estimator [405]. LLaCO instead warps time using embedding-space rank dynamics, fitting a logistic curve to the log-rank degradation of the source token along its spherical corruption path and sampling times uniformly in the resulting semantic time, which substantially outperforms top-1 or linear-rank schedules on mathematical reasoning [164].

Position- and content-dependent schedules Assigning different noise levels to different token positions or word classes induces a generation order inside a parallel denoiser without requiring an explicit autoregressive loop. AR-Diffusion assigns each target position n a local timestep $f(n, t)$ governed by a sentence-level clock t and a linear anchor rule, ensuring that earlier positions reach low noise first and guide subsequent positions under its 20-step skipping sampler (Section 5.3) [382]. Clock Diffusion generalizes this construction within a variational bound. All positions follow a shared local clock offset by position-specific phases, and positions whose local clock is paused hold their state fixed. Choosing appropriate phase offsets recovers either blockwise diffusion or a sliding window of at most S active positions (Section 5.3) [312]. Content-aware schedules instead order tokens by linguistic informativeness: InfoDiffusion preserves high-information words longer during forward corruption so they resolve earlier during generation [368], whereas Masked-Diffusion LM corrupts token groups ordered by relevance and entropy so that easier tokens are recovered first [59]. Adaptive variants infer the schedule from the input or intermediate state, using a meta-learned source scheduler in Meta-DiffuB [74] or predicted per-token progress in NeoDiff [199]. Similarly, Lafa applies CDCD's loss-based time warp independently to each

within-block position, causing earlier positions in the block to denoise first without an imposed left-to-right constraint (Appendix G) [56, 87].

Learned forward paths and depth-specialized denoisers Beyond reweighting a fixed corruption process, some designs parameterize the forward path itself or partition network capacity across noise intervals. In its language experiments, Neural Flow Diffusion Models learn the forward noising path jointly with the embeddings and denoiser, though without outperforming a fixed-schedule Diffusion-LM baseline in few-step generation [254]. Along the architectural axis, DiffusionBlocks partitions the noise schedule into equal-probability intervals assigned to distinct groups of Transformer layers that are trained independently under token cross-entropy [331], and BlockTrain distributes such depth-specialized blocks across workers for byte-level modeling with expected-embedding feedback at inference (Section 5.2) [355].

Self-conditioning Introduced in Analog Bits [64], self-conditioning feeds the denoiser's previous clean-state estimate (an embedding vector, simplex probability distribution, or logit array) back into the next denoiser evaluation alongside the noisy state z_t. During training, on a fraction p_{sc} of examples (typically $p_{\text{sc}} = 0.5$), the network first executes a stop-gradient forward pass on $(z_t, 0, t)$ with a zero placeholder and then passes the resulting estimate $\tilde{z}_0$ into the supervised call $f_\theta(z_t, \tilde{z}_0, t)$. At inference, $\tilde{z}_0$ is instead carried over from the preceding, noisier sampling step $z_{t+\Delta}$, allowing each update to refine a running draft rather than recomputing a hypothesis from scratch. SeqDiffuSeq applies this two-pass training rule to token embeddings [425]. SED instead optimizes both the initial unconditioned prediction and the self-conditioned prediction, detaching the initial estimate when feeding it into the second pass [341]. TEncDM injects $\tilde{z}_0$ layerwise on contextual states [317], TESS averages current and previous simplex probabilities before re-embedding them [176], and Plaid treats the recurrence as a fixed point with distinct unrolling rules for training, likelihood evaluation, and sampling [128]. At 3B–8B scale, Sigma lowers the training self-conditioning rate to $p_{\text{sc}} = 0.25$ (adding roughly 8% non-embedding training FLOPs) and feeds the previous unguided prediction only to the conditional branch during guided sampling [405].

Recent analyses link the effectiveness of self-conditioning to a train–inference distribution gap and examine its failure modes under prolonged feedback. Davis et al. observe that near the data endpoint, a standard one-hot flow is trained only on states close to a single simplex vertex, whereas inference trajectories traverse uncommitted interior states where the ideal denoiser's Jacobian diverges. Self-conditioning exposes training to such mixed states, and interpolating the simplex endpoint with a draw from a uniform continuous-simplex prior (DoubleCat) matches or exceeds the reasoning gains of two-pass self-conditioning without the extra forward pass [81]. To reduce over-reliance on imperfect drafts during sampling, FastDiSS generates the training bootstrap estimate from a more heavily corrupted state [267], and TREC adds a reinforcement reward for the BLEU improvement gained over a pass without self-conditioning [234]. LangDiT scales each position's bootstrapped training estimate by one minus its normalized token entropy, an uncertainty gate that delivers the largest adjacent GenPPL reduction in its cumulative LangFlow ablation [244]. Although self-conditioning improves sample quality in embedding and contextual ablations [317, 318, 341], heavily weighted feedback across many steps can lock trajectories into repetitive attractors, motivating inference-time feedback corrections such as Attractor-Contrast-Escape (ACE; Section 5.1) [436]. RePlaid likewise reports lower diversity at large sampling-step counts and attributes this to compounding confidence in the self-conditioning loop [406]. Untied Self-Conditioning instead addresses solver-induced redundancy of previous predictions in coarse-step sampling [197].

5 Sampling, the Text Interface, and Generation Order

Once a denoiser or velocity field has been trained, inference depends on the numerical sampler and few-step distillation strategy (Section 5.1) and on the readout rule that converts real-valued states into tokens

(Section 5.2). It also depends on the generation order across positions, blocks, or segments (Section 5.3) and on the conditioning, guidance, or candidate-selection mechanism applied during decoding (Section 5.4).

5.1 Samplers and few-step routes

As analyzed in Section 2.3, compressing a continuous text trajectory into a few updates exposes the conditional-mean mode-averaging problem: when a denoiser predicts $\hat{z}_0 \approx \mathbb{E}[z_0 \mid z_t]$ across multiple plausible continuations, taking a small number of large deterministic steps pulls the state toward high-frequency, repetitive tokens that score well on external generative perplexity (GenPPL) while collapsing unigram entropy. Few-step behavior depends on how the solver or interval map is constructed. Coarse deterministic Euler trajectories on fixed flow language model (FLM) checkpoints [191], the Eulerian variant of Discrete Flow Maps (DFM-ESD) and DFM's undistilled diagonal model [287], and the one-step map of Expanding Flow Maps (EFM; Section 5.3) [347] all suffer severe entropy collapse. The final published CatFM result instead reports one-step LM1B GenPPL 142.7 with entropy 4.03 [304]. Similarly, Wang documents repetitive few-step outputs behind low GenPPL in ELF and LangFlow [374], and evaluating released LangFlow, FLM, and ELF checkpoints in a single step without distillation yields deceptively low GenPPL alongside collapsed unigram entropy [228]. Conversely, stochastic endpoint bridges on fixed FLM checkpoints [38] and trajectory-consistent distilled one-step maps such as FMLM [191] and DFM's progressive semigroup variant (DFM-PSD) [287] preserve entropy near their many-step baselines. Few-step methods therefore rely either on reinjecting intermediate stochasticity or on training finite-interval maps and distilled generators to land directly on valid endpoints.

Baseline diffusion and flow updates Write the forward process as $z_t = \alpha(t)z_0 + \sigma(t)\epsilon$ (Eq. 1) and let the denoiser's output at (z_t, t) be converted into the pair $\hat{z}_0(z_t, t)$ and $\hat{\epsilon} = (z_t - \alpha(t)\hat{z}_0)/\sigma(t)$, whichever quantity it was trained to predict (Section 2). Standard diffusion and flow samplers combine this pair differently. A standard ancestral DDPM update with the posterior-variance choice draws from the tractable Gaussian forward posterior $q(z_{t-1} \mid z_t, z_0)$ evaluated at the plug-in estimate $z_0 = \hat{z}_0(z_t, t)$ [149]:

$$z_{t-1} = \frac{\alpha(t-1)\sigma^2_{t|t-1}}{\sigma(t)^2}\hat{z}_0 + \frac{\alpha_{t|t-1}\sigma(t-1)^2}{\sigma(t)^2}z_t + \tilde{\sigma}_t\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{8}$$

with $\alpha_{t|t-1} = \alpha(t)/\alpha(t-1)$, $\sigma^2_{t|t-1} = \sigma(t)^2 - \alpha^2_{t|t-1}\sigma(t-1)^2$, and $\tilde{\sigma}^2_t = \sigma^2_{t|t-1}\sigma(t-1)^2/\sigma(t)^2$. Although $q(z_{t-1} \mid z_t, \hat{z}_0)$ is Gaussian, the true marginal reverse conditional $p(z_{t-1} \mid z_t) = \int q(z_{t-1} \mid z_t, z_0)\, p(z_0 \mid z_t)\, \mathrm{d}z_0$ is a mixture over $p(z_0 \mid z_t)$ and need not be Gaussian at finite step sizes. The deterministic DDIM update ($\eta = 0$) defines an interpolation trajectory across $[t - \Delta, t]$ by holding the current step's predictions $(\hat{z}_0, \hat{\epsilon})$ fixed and re-forming the state at the next noise level [336]:

$$z_{t-1} = \alpha(t-1)\hat{z}_0 + \sigma(t-1)\hat{\epsilon}. \tag{9}$$

When $\eta > 0$, part of $\sigma(t-1)\hat{\epsilon}$ is replaced by fresh noise. The probability-flow ODE in the same parameterization, and its explicit Euler step, are

$$\frac{\mathrm{d}z_t}{\mathrm{d}t} = \dot{\alpha}(t)\hat{z}_0 + \dot{\sigma}(t)\hat{\epsilon}, \qquad z_{t-h} \approx z_t - h\big(\dot{\alpha}(t)\hat{z}_0 + \dot{\sigma}(t)\hat{\epsilon}\big), \tag{10}$$

which for CDCD's $\alpha(t) = 1$, $\sigma(t) = t$ reduces to $z_{t-h} \approx z_t + (h/t)(\hat{z}_0 - z_t)$ [87]. An exact finite-step solution of the learned ODE would integrate a time-varying $\hat{z}_0(z_t, t)$ across the interval; Eq. 9 instead integrates Eq. 10 under the local approximation that $(\hat{z}_0, \hat{\epsilon})$ remain constant over $[t - \Delta, t]$. For a velocity model trained with Eq. 3, the explicit Euler step in flow time s toward data reads

$$z_{s+\Delta s} = z_s + \Delta s\, v_\theta(z_s, s, c). \tag{11}$$

On the linear flow path $z_s = (1-s)\epsilon + sz_0$ ($\alpha(s) = s$, $\sigma(s) = 1 - s$) under the same $\hat{z}_0$ parameterization, $v_\theta = \hat{z}_0 - \hat{\epsilon}$ and the explicit Euler step in Eq. 11 coincides with the DDIM update in Eq. 9. Higher-order

solvers trade denoiser calls for accuracy. Heun re-evaluates the derivative at the provisional point and averages the two, at two calls per interval. DPM-Solver++ integrates the linear part of the ODE exactly in log signal-to-noise ratio and approximates $\hat{z}_0$ along the interval by a low-order polynomial through previous evaluations, so its multistep variant costs one call per step and its first-order case is DDIM [242]. Nominal update steps and denoiser NFE (Table 3) coincide for unguided Euler, DDIM, and multistep DPM-Solver++ updates, but predictor–corrector solvers such as Heun and two-pass self-conditioning evaluate the denoiser twice per step, and separate decoders or candidate rerankers add passes outside the denoiser loop (Section 5.4). Table 9 summarizes the main routes to few-step generation alongside their evaluation costs.

Solver, schedule, and bridge interventions The most direct acceleration route retains the trained denoiser weights and modifies only how the vector field is traversed at inference. Few-shot Temporal Pruning selects a subset of inference steps without retraining the denoiser; on DiffuSeq, its four-step sampler outperforms uniform subsampling but remains below the original 2000-step sampler [198]. NeoDiff trains a separate non-simultaneous diffusion model with a token-wise time predictor and additionally optimizes its extrinsic inference schedule after training [199]. On ELF, StepTune selects a frozen grid by minimizing terminal-latent error and decoder KL divergence relative to a fine trajectory from the same initial noise, allowing a 16-step grid to match or exceed the native 64-step grid on summarization and translation at a fraction of the batch-1 latency [450]. BFN-Solver modifies the numerical rule instead, deriving an SDE for the Gaussian logits underlying BFN's categorical beliefs to apply higher-order solvers to a pretrained BFN [398]. State geometry also shapes solver choice: S-FLM derives its schedule truncation from a spherical random-embedding model [83], Spherical Flows obtains its strongest spherical-path results with predictor–corrector sampling and entropy-constrained selection [57], and Logit-KL compares deterministic, re-noised, and mixed updates within a unified logit framework [316]. On ELF-based reasoning models, re-noising the clean prediction to the next time level (as in consistency-model sampling) rather than taking a deterministic Euler step improves mathematical reasoning accuracy at moderate step budgets, though the gain disappears at very small step counts [72, 218]. Training-free interventions also target few-step degradation on fixed flow checkpoints. Marginal-Conditioned Bridges (MCB) prevent mode-averaging collapse by sampling a one-hot endpoint per position from the predicted marginals at each step and drawing from an Ornstein–Uhlenbeck bridge conditioned on that endpoint and the current state [38]. Untied Self-Conditioning instead lowers the high perplexity of coarse trajectories on pretrained ELF (SDE sampling) and LangFlow checkpoints by combining a history-averaged solver correction with a damped self-conditioning input that filters out components redundant with the current noisy state (Section 6.3) [197].

Training for coarse updates and direct transport A second route modifies training so that the denoiser tolerates coarse inference steps. DiffuSeq-v2 pairs hybrid absorbing-Gaussian corruption with multistep DPM-Solver++ and reports its accelerated results without minimum-Bayes-risk (MBR) reranking [120]. Differmer combines an anchor loss and noise rescaling with shortened sampling grids and a preset early stop; its multi-candidate decoding selects among length–noise candidates by minimum expected risk (Section 5.4) [111]. BG-DiffuSeq and FastDiSS instead address train–inference mismatch along the trajectory, either post-training a converged denoiser for consistency between forward-noised and model-generated states under Adaptive Decay Sampling [349] or generating training self-conditioning drafts from noisier states [267]. Direct-transport methods learn a source-to-target map or a velocity field that permits coarse integration. FlowSeq supervises a conditional velocity alongside its linearly extrapolated token endpoint so that one Euler step reaches an embedding sequence; with MBR reranking over ten trajectories, it approaches 2,000-step DiffuSeq on question generation in BLEU (0.162 versus 0.173), slightly exceeds its ROUGE-L, but trails in BERTScore and on paraphrasing [154]. LanguageFlow learns source-to-target transport in a VAE latent space, using ten Euler updates in the reported experiments followed by autoregressive decoding [437]. Discrete Beckmann Transport Models (DBTM) eliminate the time coordinate altogether: averaging the flow-matching velocity between Gaussian noise and one-hot

Table 9. Routes to fewer denoising updates and their evaluation costs in denoiser NFE and auxiliary calls. Here a step denotes one interval on the sampling time grid, MBR minimum-Bayes-risk selection, MMD maximum mean discrepancy, DMD distribution-matching distillation, SiD Score identity Distillation, and PDD Parallel Decoding Distillation. Discrete and hybrid few-step boundary comparisons (e.g., IMDM, x-Prediction Flow) are covered in Appendix C (Table 20); Section 6.3 compares empirical operating points.

Route	Examples	Modification	NFE and extra calls
Reuse a trained field	BFN-Solver; MCB; StepTune; Untied Self-Conditioning [38, 197, 398, 450]	The numerical rule, the time grid, the endpoint sampling, or the use of prediction history; weights fixed	One denoiser call per step (two for Heun-type correctors); MCB adds a categorical endpoint draw per position and step; a calibrated grid (StepTune) adds only a one-time offline search; a separate vocabulary head adds a terminal call
Train for coarse updates	DiffuSeq-v2; Differmer; BG-DiffuSeq; Sigma (PDD) [111, 120, 349, 405]	Corruption, anchoring and noise calibration, consistency post-training, or multi-prediction block distillation (PDD)	One denoiser call per step of the shortened grid (in Sigma's PDD, each student backbone call emits multiple block predictions composed across intervals on the teacher grid), plus Sigma's final vocabulary-readout call per block; MBR multiplies calls by the pool size where used
Learn direct transport	FlowSeq; LanguageFlow; DBTM [154, 348, 437]	FlowSeq supervises a velocity and token endpoint for one-step transport; LanguageFlow learns a VAE-latent velocity field for few-step source-to-target transport; DBTM learns a time-independent map to simplex vertices from data without a teacher	FlowSeq uses one call per trajectory, with reported quality using MBR over several trajectories; LanguageFlow uses ten flow calls plus autoregressive decoding; DBTM spends one call per refinement round
Learn finite-interval maps	FMLM; CatFM; DFM; GSF; DiLaDiff [183, 191, 195, 287, 304]	A two-time map or an average velocity replaces many short updates; GSF takes its diagonal targets from an autoregressive teacher's Gumbel coupling	One call per interval; two with self-conditioning (DiLaDiff: ten calls for five latent steps) plus the discrete decoder's own trajectory; CatFM and GSF end with argmax or one categorical draw; DFM outputs a terminal vocabulary distribution, but its paper does not specify the discrete readout rule
Distill the output distribution	DLM-One; Simplex-DMD; Reinforce-DMD; ELF-MMD [63, 95, 358]	A one-pass student trained by Score identity Distillation (SiD) within the distribution-matching distillation (DMD) family; multi-step students trained by reverse-KL matching through simplex relaxations or categorical draws; or a generator trained by kernel MMD on features of the frozen model	One call plus the vocabulary head (DLM-One); one call per step with fresh re-noising (DMD students); K self-conditioned passes plus one decoding call (ELF-MMD); training needs the teacher with a student-score network or a density-ratio discriminator, or only the frozen model and reference samples (ELF-MMD)
Train a one-pass generator by drifting	Seq-Drifting; ReLaDrift [228, 276]	A generator regressed onto drifted targets (attraction to plausible tokens or reference latents, repulsion from other samples); no noise-to-data path	One call (Seq-Drifting); one call per recurrent round plus a parallel decoder pass (ReLaDrift); training needs a frozen support language model (Seq-Drifting) or task reference outputs, optionally augmented by execution-verified programs on GSM8K (ReLaDrift)

sequences over time yields an autonomous field whose stable equilibria are the simplex vertices, and DBTM learns the map to those vertices directly from data via a conservation-equation residual [348].

Seq-Drifting dispenses with noise-to-data trajectories entirely, training a one-pass generator from Gaussian noise to unit-norm token vectors by regressing onto targets pulled toward autoregressive next-token candidates and pushed away from other samples and other positions in the same sequence. Omitting the intra-sequence repulsion collapses LM1B outputs into repetition [228]. At 3B scale, Sigma pairs a spherical uniformity regularizer with Parallel Decoding Distillation (PDD), where a single student forward pass emits multiple clean-embedding block predictions composed across sub-intervals of the teacher's time grid, substantially improving four-step reasoning and coding accuracy over standard DDIM while remaining below many-step performance [405].

Finite-interval flow maps A flow map $X_{s,t}$ (Section 2.2) transports a state directly from time s to time t along a flow trajectory, reducing one-step generation to evaluating $X_{0,1}$. FMLM parameterizes this interval operator with a two-time denoiser evaluated over flexible interval partitions [191]. Categorical Flow Maps (CatFM) and Discrete Flow Maps (DFM) make the simplex endpoint explicit. With s and t denoting flow times increasing toward the one-hot data endpoint at $t = 1$ along the linear path $z_s = (1 - s)\epsilon + sx$, a simplex-valued endpoint prediction q_θ implies the noise estimate $\hat{\epsilon} = (z - sq_\theta)/(1 - s)$ and the interval transport rule:

$$X_{s,t}(z) = \frac{1-t}{1-s} z + \frac{t-s}{1-s} q_\theta(z, s, t), \qquad q_\theta(z, s, t) \in \Delta^{|\mathcal{V}|-1}, \quad 0 \le s < t \le 1. \tag{12}$$

This map blends the current state with the predicted endpoint and reduces to q_θ at $t = 1$ [287, 304]. Building on variational flow matching, CatFM supervises the diagonal ($s = t$) with token cross-entropy against ground-truth tokens. For off-diagonal maps ($s < t$), self-distillation uses a stop-gradient diagonal teacher evaluated at the transported state: CSD penalizes the Lagrangian derivative residual against the local denoising direction, while ECLD combines endpoint cross-entropy with a penalty on the target-time derivative of the partial denoiser [304]. DFM interprets q_θ as an interval mean denoiser and trains off-diagonal maps via KL divergence after diagonal pretraining, either matching one long interval to the composition of two shorter intervals under a progressive semigroup loss (PSD) or differentiating along source time under an Eulerian loss (ESD) [287]. Scaling CatFM applies progressive semigroup self-distillation after diagonal pretraining at 1.7B scale [80]. Gumbel Straight Flow (GSF) instead alters the noise–data coupling. Applying the Gumbel-max trick to a pretrained autoregressive teacher pairs each noise array with a deterministic sequence along non-intersecting straight paths. This coupling supervises the diagonal of a teacher-initialized two-time denoiser while a semigroup loss trains wider intervals, lowering single-step GenPPL relative to FMLM at comparable entropy [183].

Flow maps can also be parameterized via MeanFlow's average velocity [116] or distilled inside compressed and blockwise latent spaces. DiLaDiff uses MeanFlow to distill LaDiff's latent velocity across intervals while retaining its text codec and discrete-diffusion decoder. Because of two-pass self-conditioning, its five latent steps require ten denoiser evaluations before the discrete decoder runs [195]. Self-conditioned flow-map distillation incorporates prediction history directly through a fixed-point teacher [417]. In blockwise generation, Lafa distills its asynchronous block flow into a two-time map that substantially boosts two-step-per-block accuracy over the undistilled flow on instruction following, math, and code [56]. CoFlow distills a two-time flow map in a compressed latent diffusion space, briefly co-training the latent encoder with a tracking teacher and an encoder anchor; it improves few-step OpenWebText generation and GSM8K Python-solution accuracy relative to prior flow-map baselines [314].

Distribution-matching and score-identity distillation Distribution-matching distillation (DMD; Section 2.2) matches the student's noised outputs to the teacher's noised marginals across noise levels so that the student matches the teacher's marginal distributions without simulating multi-step trajectories. Within this family, DLM-One trains a one-step text generator via Score identity Distillation (SiD), matching continuous-space score identities between a fixed teacher and a learned student-score estimator, combined with real-data adversarial regularization and a second training stage. Its tests cover conditional DiffuSeq,

unconditional Plaid, and a TESS logit-space case, where the second stage improves reference overlap while reducing diversity and the TESS student loses overlap relative to its teacher [63]. Simplex-DMD and Reinforce-DMD extend the family to multi-step students of LangFlow, with pathwise gradients through simplex relaxations or REINFORCE with a discriminator-estimated density ratio. Simplex-DMD substantially lowers four-call generative perplexity at data-level entropy relative to its teacher while losing entropy at large step budgets, and Reinforce-DMD collapses without a KL anchor to the teacher [358]. ELF-MMD drops the score network, training a one-step ELF generator by a kernel maximum mean discrepancy on frozen-ELF features of generated and real latents [95]. Related discrete and hybrid few-step routes that retain categorical token outputs are contrasted in Appendix C (Table 20). Among them, IMDM [418] is a discrete masked sampler that distills a teacher's multi-step token predictions while adding an MLP-transformed auxiliary uniform random vector to `[MASK]` embeddings, and x-Prediction Flow [371] is a hybrid continuous decoder over a pretrained masked language model (LLaDA-8B), after x-prediction alignment of the backbone, with confidence-conditioned step sizes.

These few-step routes differ in their training dependencies. Solver and bridge modifications (MCB, Untied Self-Conditioning) operate on frozen checkpoints, whereas flow-map methods rely on self- or teacher-distilled interval targets. Distribution-matching methods require an auxiliary student-score network (DLM-One, Simplex-DMD), a density-ratio discriminator (Reinforce-DMD), or frozen-model feature kernels (ELF-MMD). By contrast, GSF distills an autoregressive teacher's Gumbel coupling, and DBTM and Seq-Drifting dispense with a diffusion teacher altogether [95, 183, 228, 348, 358].

5.2 From states to tokens

At the text interface, off-manifold estimation errors in the continuous state become lexical or semantic errors in the decoded sequence: a token-aligned vector lying between embeddings can round to the wrong word, and a perturbed sentence vector can decode into fluent text that drifts from the intended meaning. The decision can be made once at the end (by rounding, a vocabulary head, or a separate decoder) or repeatedly during sampling by replacing the clean estimate with a vector closer to a valid word. Because the decoder at inference time receives states generated by a finite-step sampler rather than clean encoder outputs, clean-text reconstruction accuracy tests only part of the text interface. Throughout, *clamping* denotes an intermediate replacement of $\hat{z}_0$ and *rounding* the terminal nearest-embedding decision.

Intermediate clamping, projection, and expected embeddings Because the baseline updates in Eqs. 8–10 are driven by the clean-state estimate $\hat{z}_0$, intermediate readout interventions modify $\hat{z}_0$ before the step is taken. Diffusion-LM clamps $\hat{z}_0$ to its nearest vocabulary embedding $\tilde{z}_0 = e_{\hat{w}}$ and evaluates the reverse posterior $q(z_{t-1} \mid z_t, \tilde{z}_0)$ of Eq. 8 at that discrete point [213]. SSD-LM and TESS project predicted logits before re-noising; TESS uses an argmax projection back to its $\{\pm k\}^{|\mathcal{V}|}$ token encoding [134, 176], and ParaGuide samples a token from the top-p logit distribution, re-embeds it, and re-noises [150]. Rather than projecting to a single candidate token at each step, CDCD and BlockTrain use soft token predictions by feeding the expected embedding $\sum_v p_\theta(v \mid z_t) e(v)$ into Eq. 10 and taking an argmax only at the final step. In CDCD's ablations, intermediate clamping degrades sample quality [87, 355]. Scaling logits by a temperature $T \to 0$ continuously interpolates between expected-embedding updates and hard top-1 projection. In posterior-induced flows such as S-FLM and Spherical Flows, however, dividing logits by T also tempers the intermediate bridge distribution via Bayes' rule while leaving the conditional velocities untempered [57, 83, 272]. Bayes-Consistent Tempering (BCT) tempers both the bridge and the velocity field after a 2,000-step continuation on tempered-bridge states, substantially improving low-temperature mathematical reasoning accuracy for S-FLM [272]. At the terminal step $t = 0$, GENIE clamps and rounds to the nearest embedding [222], whereas Differmer's anchor loss aligns the terminal embedding geometry directly with its tied vocabulary projection [111].

Intermediate token commitment versus remasking Intermediate projection allows a position's selected token to change at subsequent steps, whereas token commitment locks the position permanently. CANDI

combines continuous Gaussian corruption with progressively committed clean token positions [291], Posterior Refinement commits selected endpoint predictions between flow-map rounds while re-noising the remaining slots [1], and Masked Language Flow Models add confidence-based token promotion to continuous flow sampling [39]. GRAFT uses commitment with trajectory restarts to counteract rare-word averaging. When the vocabulary head of a frozen ELF or LangFlow model assigns non-trivial mass to a rare word mid-trajectory even as a common word ranks higher, GRAFT clamps that rare word as a fixed condition and restarts sampling from the same initial noise. This restart procedure restores rare-word frequencies to near-human rates, whereas weak mid-trajectory pulls are ineffective and hard token injection inflates non-word errors [401]. Committing a token provides stable context for undecided positions at the cost of locking in early mistakes, a trade-off that discrete remasking samplers such as ReMDM relax by allowing committed tokens to be re-masked [361]. Hard selection can also be restricted to the update direction: S-FLM's top-1 velocity follows the tangent direction of the most probable token without freezing the state [83], and Dirichlet Flow Matching transports distributions toward simplex vertices under posterior-weighted conditional fields, as demonstrated on DNA sequences [340].

Parallel, shared, and autoregressive decoders Even when states are aligned one-to-one with tokens ($M = L$), a contextual or autoregressive decoder reading the entire generated array can resolve local ambiguities that independent per-position rounding cannot. TEncDM pairs frozen contextual encoder targets with a separately trained bidirectional decoder [317], whereas ELF shares a single Transformer backbone between clean-embedding prediction (converted to a flow velocity) and terminal token readout (adding one final decoding pass) [153]. Both TEncDM's decoder comparisons on held-fixed generated states [317] and TextLDM's alignment ablations [166] show that decoder architectures must be evaluated on prior-generated states rather than clean encoder outputs alone. Among codec systems, TextLDM decodes all target positions in parallel [166], whereas LD4LG, PLANNER, and CoDAR decode completed latent arrays autoregressively [239, 327, 441]. CoDAR motivates autoregressive readout by decomposing the negative-log-likelihood gap of an optimal pointwise decoder into a cross-position conditional-dependence term and a locality term: a parallel contextual decoder addresses locality by attending across all generated latents, while an autoregressive decoder also conditions on previously realized surface tokens [327]. Hybrid and feedback architectures instead interleave discrete token decisions with continuous updates: CADD evolves a continuous noisy channel alongside a categorical token canvas [448], Soft-Masked DLM feeds expected embeddings back into unresolved masked positions [146], and DiffusionGemma revises categorical tokens across the canvas (Appendix C.2) [88]. HC-DLM inverts this coupling by maintaining only the continuous VAE latent between steps while using the uniform-noise-perturbed argmax of an auxiliary token predictor as a discrete conditioning scaffold for the next flow-matching update. Ablating this discrete scaffold sharply degrades combinatorial puzzle accuracy [299].

Decoding sentence and segment vectors Because a sentence or segment vector is not aligned with individual tokens, nearest-neighbor rounding is inapplicable and a generative decoder must reconstruct the surface wording from the continuous vector. The simplest approach reuses a frozen reconstruction decoder from a pretrained sentence autoencoder: Diffusion LCM decodes generated SONAR vectors with the frozen SONAR decoder [98, 190], while Versatile Diffusion and CoDi pass generated Optimus sentence latents to the VAE's GPT-2 decoder [201, 350, 396]. When a dedicated decoder is trained for the sentence generator, it can also incorporate previously realized text, as in LaDiDa's Vec2Seq decoder, whose context-and-vector configuration (LaDiDa-CV) conditions on preceding decoded sentences whereas its vector-only configuration (LaDiDa-V) decodes sentences independently in parallel [411]. Rather than requiring the vector to specify exact wording, DGLM and STAR-LDM map diffused sentence vectors into soft prompts for an autoregressive language model and train the adapter on noisy proposals to control how closely the backbone follows the latent plan [240, 241]. Alternatively, CoM-DAD realizes a diffused sentence plan through an absorbing discrete-diffusion decoder [397], SLD backpropagates token-level reconstruction losses through a frozen segment decoder during denoiser training [462], and Diffusion-GDP and CoDSA decode diffused GTR or SentenceBERT embeddings through iterative

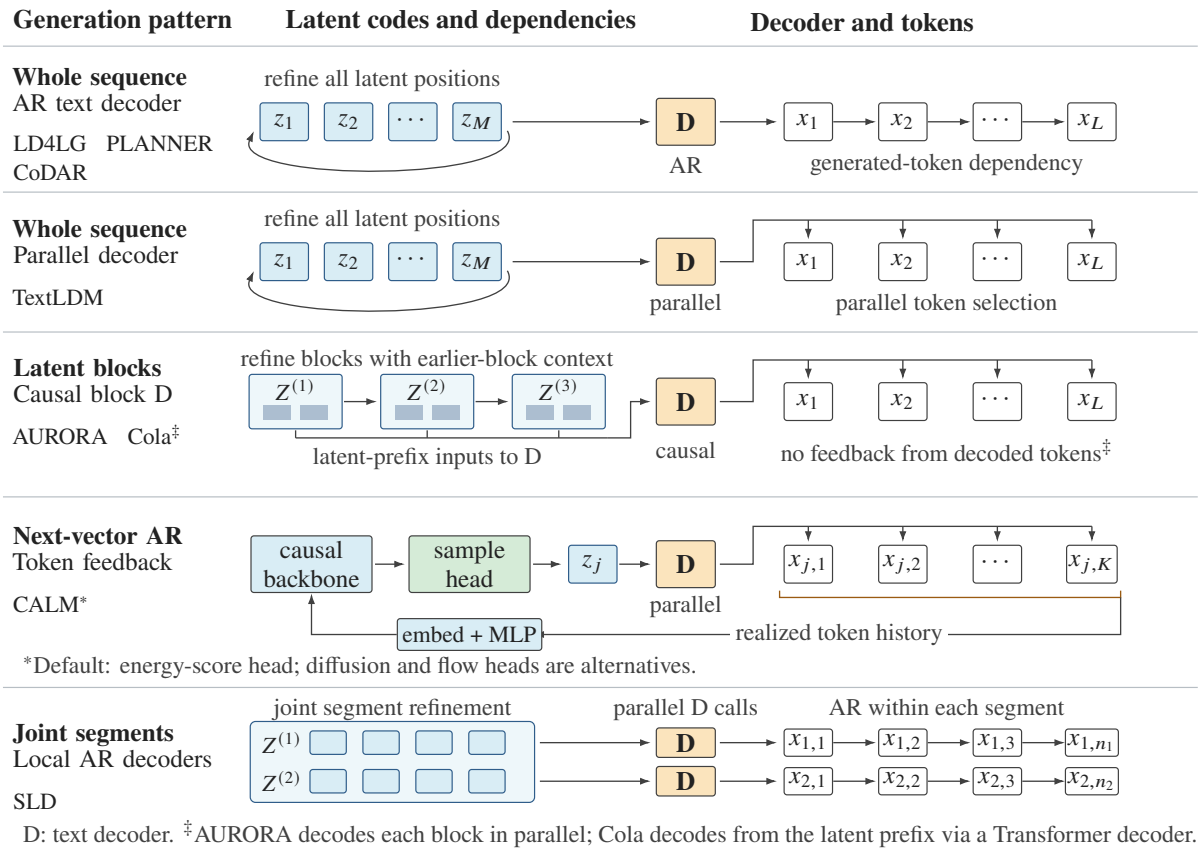


Figure 9. Representative generation orders and their conditioning dependencies: whole-sequence latent refinement followed by either an autoregressive decoder as in LD4LG, PLANNER, and CoDAR or a parallel decoder as in TextLDM; blockwise latent generation with causal dependence across latent blocks as in AURORA and Cola; next-vector prediction with decoded-token feedback as in CALM, which compares multi-step diffusion/flow chunk heads against a default 1-step energy-score head (Appendix C); and joint segment refinement with segment-local autoregressive decoders as in SLD [131, 166, 220, 239, 323, 327, 441, 462].

embedding inversion via Vec2Text [262, 353, 354]. Because sentence vectors compress an entire clause into a single point, off-manifold estimation errors manifest as semantic or lexical drift rather than isolated word flips. For instance, LCM measures declining reconstruction and semantic-similarity scores under increasing concept-vector perturbations, with substantial variation across sentences and decoders [190], and PLANNER finds that its composite codec-quality surrogate ceases to track generative perplexity when the input-token substitution rate used to train the paragraph embedder becomes excessive [441].

5.3 Generation order and parallelism

Removing token-by-token autoregression from the denoiser does not eliminate sequential dependencies across words. Instead, it relocates them to joint attention across a full-sequence state, to a separate decoder that reads the completed latent array, to an outer loop over blocks or sentence vectors, or to feedback of realized tokens between steps. The structure of these dependencies determines which positions remain revisable during generation, which stages execute in parallel, and how output length is controlled, whether through a pre-allocated window or a block- or concept-level stopping criterion. Figure 9 and Table 10 compare representative generation orders, illustrating how state compression, parallel latent refinement, and parallel token readout vary independently.

Full-sequence parallel refinement The token-aligned family refines a fixed-length state as a whole, so words depend on one another only through the joint refinement, and the terminal rounding decides all positions together (Diffusion-LM, DiffuSeq, TESS). AR-Diffusion keeps the joint update but gives

each position a time that depends on its index, so earlier positions become clean sooner and guide later ones, and its position-aware skipping sampler accelerates along that schedule [382]. Codec systems share a completed latent code and differ only in whether their decoder is autoregressive or parallel (Section 5.2; Table 10). Predictive-State Flows (PSF) combine the two: a rectified-flow prior generates eight states for a 1024-token document jointly, each encoding only the text before its block, and a block decoder realizes all blocks in parallel, autoregressively within each and coupled across blocks only by the first token of the next block [140]. LaDiff adds a cascade: continuous latent generation finishes first, then a conditional masked-diffusion decoder realizes the text over several unmasking rounds, under a conditional-independence argument that assumes the latent captures the clean-token dependencies [195]. CRoCoDiL lets the plan respond to emerging text: its evaluated configuration refreshes the sampled registers once, at the midpoint of demasking, from an encoding of the visible tokens, and its timing ablation favors this midpoint over earlier or later refreshes [357]. The cascade thus gains feedback without turning every token round into a new continuous trajectory.

Blockwise and next-token generation orders Blockwise generation reintroduces left-to-right causal dependence across blocks of B tokens while retaining bidirectional continuous refinement within each block. SSD-LM generates B-token logit blocks sequentially, conditioning each block on the fixed prefix of previously decoded blocks and applying projection and re-noising within the active block [134]. Its discrete counterpart, Block Diffusion (BD3-LM), likewise factorizes generation causally across blocks while running masked diffusion within each block over a key–value-cached prefix (Appendix C) [36]. Sigma combines BD3-LM's blockwise likelihood factorization and KV-cached prefix conditioning with continuous embedding diffusion over 32-token blocks, committing each block via vocabulary-head argmax after the final denoising step [405]. Clock Diffusion turns blockwise generation into a fine-grained sliding window: at most S positions are active simultaneously under staggered local clocks, and the leftmost position is decoded and appended to the KV cache as soon as its clock completes (Section 4.3) [312]. Lafa uses its shared backbone's greedy next-token head to verify one-step blockwise flow drafts, committing only the longest draft prefix that matches greedy autoregressive predictions (Section 6.3) [56].

Blockwise recurrence can also operate over compressed or contextual latent blocks rather than surface tokens. In AURORA and Cola, attention across latent blocks is causal, each latent block conditions on preceding latent blocks, and a causal block decoder emits tokens without feeding realized tokens back into the latent prior [131, 220]. Cola2 closes the feedback loop by re-encoding each decoded token block and replacing the proposed latents in the conditioning history [132], whereas BLD keeps the latent loop open, generating 64 block latents per 1024-token document across eight sequential groups before decoding all 64 spans in parallel with span-local autoregression [103]. LaDiR similarly generates latent thought blocks sequentially before its LLM backbone decodes the final answer tokens autoregressively [175]. At the single-token extreme, L2D and DiffusionBlocks run a multi-step continuous diffusion trajectory to refine only the next token embedding conditioned on a clean causal prefix, trading inter-token parallelism for iterative per-token refinement [53, 331].

Sentence- and segment-level recurrence LCM predicts the next sentence vector from earlier concept vectors; its recurrence never sees the realized words [190]. DGLM samples one continuation embedding conditioned on the text prefix, then maps it to a soft prompt for autoregressive decoding [240]. STAR-LDM similarly denoises a semantic plan for a prompted continuation and realizes it with an integrated autoregressive backbone; the continuation need not be a single sentence [241]. At the chunk level, CALM evaluates multi-step diffusion and flow-matching heads over $K = 4$-token chunks against a default 1-step energy-score boundary head (Appendix C), decoding each chunk in parallel and feeding those realized tokens back into the next step [323]. SLD generates the segment arrays jointly and keeps serial decoding only inside each segment [462]. The outer recurrence uses concept vectors in LCM and decoded-token history in CALM. DGLM and STAR-LDM instead plan a continuation before its autoregressive realization, while SLD refines its segment latents jointly.

Table 10. Sequential dependencies across representative continuous-state systems. Compressing the state, generating latents in parallel, and decoding tokens in parallel vary independently across the rows. Rows describe generation order rather than measured speed; discrete block diffusion (BD3-LM) and single-step continuous-AR predictors (e.g., CALM's default energy-score head) are contrasted in Appendix C. PAD and EOS are the padding and end-of-sequence tokens; AR means autoregressive.

Configuration	Generated unit and outer dependence	Token decoding and feedback
Diffusion-LM; DiffuSeq; TESS [119, 176, 213]	Fixed-length token-aligned state refined as a whole; no outer loop	Terminal rounding or argmax; generated PAD and EOS tokens set the length
AR-Diffusion [382]	Whole target with position-dependent times; earlier positions clean sooner	Parallel updates; terminal rounding
SSD-LM [134]	Blocks of B tokens left to right; each block's logit state refined with decoded blocks as context	Projection and re-noising inside the block; decoded blocks fixed
Sigma [405]	Blocks of 32 tokens left to right; each block's token embeddings refined bidirectionally on the cached clean prefix	Parallel vocabulary-head argmax per block; committed tokens extend the prefix
Clock Diffusion (ClockDLM) [312]	Sliding window of at most S token embeddings with staggered clocks; the leftmost position finishes first	Finished positions decoded by argmax and cached at every step; confident or AR-verified prefixes can be committed early
Lafa [56]	Blocks of B token embeddings with learned position-specific times on the cached clean prefix	Argmax per block, or one-step drafts committed only where the model's greedy AR head agrees
LD4LG; PLANNER; CoDAR [239, 327, 441]	Full latent code refined as a whole	Autoregressive decoder conditioned on the completed code
TextLDM [166]	Full target latent sequence refined as a whole	Parallel decoder over the completed latents
PSF [140]	Eight predictive states (each encoding only the preceding blocks) refined jointly; no outer loop	Block branches decoded in parallel, autoregressive within each; each branch also reads the next block's first token
LaDiff / DiLaDiff; CRoCoDiL [195, 357]	Continuous latent (or registers) first, then masked-token demasking	Unmasking conditioned on the completed latent; CRoCoDiL refreshes the registers once at midpoint from visible tokens
AURORA; Cola [131, 220]	Next latent block depends on earlier latent blocks	Causal block decoder conditioned on the latent prefix with no token feedback into the latent backbone (parallel per block in AURORA)
Cola2 [132]	Next latent block from the latent history	Block-causal masked-diffusion decoding of the block; the decoded block is re-encoded and its latents replace the proposal in the history
BLD [103]	Eight groups of eight block latents (64 per 1024 tokens), each group conditioned on the earlier latents	64 spans decoded in parallel, autoregressive within each; no token feedback into the latent prior
LaDiR [175]	Latent thought blocks generated block by block	Stage-1 VAE decoder reconstructs thought blocks during codec training; at inference the LLM backbone generates final answer tokens autoregressively
Diffusion LCM [190]	Next sentence vector from the concept history	Separate SONAR decoder per concept; recurrence uses vectors, not words
DGLM [240]	One continuation proposal sampled by multi-step Gaussian diffusion, conditioned on the text prefix	A separate prompt generator supplies the proposal as a soft prompt to an AR decoder; one plan per continuation
STAR-LDM [241]	A semantic continuation plan is denoised conditional on the token prefix	The integrated AR backbone realizes the continuation from its prefix and latent soft prompt
CALM [323]	Autoregressive across K-token chunks; default 1-step energy-score head, with multi-step diffusion/flow alternatives	Parallel chunk decoder; realized tokens are embedded and compressed as input to the next step
SLD [462]	Joint segment arrays refined together	Parallel across segments; autoregressive within each segment
L2D; DiffusionBlocks (AR) [53, 331]	Continuous refinement of the next token only; clean autoregressive prefix	The committed word extends the prefix; no parallelism across words

Sequence length control and stopping criteria Full-sequence token-aligned models typically pre-allocate a fixed canvas length and use special tokens such as EOS, SEP, or PAD to represent output boundaries or unused positions; padding supervision and final text postprocessing are model-specific [119, 176, 213, 425]. Plan Canvas adapts this mechanism inside an ELF window by reserving a fixed-capacity prefix region for a reasoning trace (training unused trace slots as padding) so that the final answer begins at a known index [270]. Rather than generating padding inside a maximal window, Differmer predicts a target-length distribution from the source encoder, initializes parallel canvases at the top candidate lengths, and uses a preset denoising cutoff (omitting the final five updates in the reported 20-step setting) [111]. Blockwise models instead decide when to stop appending blocks. SSD-2 pads incomplete training blocks with EOS tokens and halts generation once a decoded block contains EOS or reaches a maximum block count [135]. Similarly, AURORA sets the latent length $N = \mathrm{round}(cL)$ and block count $B = \lceil N/Q \rceil$ from a target token extent L, retention ratio c, and block size Q [220]. In compressed latent spaces, PACED uses a learned head on denoised latents to choose an initial output length, then extends it from cached layer states if EOS has not appeared. Its causal mask over output positions preserves parallel evaluation and makes shorter outputs consistent with full-window evaluation of the same masked decompressor. On Cosmos with a 128-token maximum, length-sorted batches of 16 or 64 achieve 1.17–1.79× decompression speedup, excluding diffusion time [10]. Sentence-level models decouple document length (in concepts) from sentence length (in tokens): Diffusion LCM halts concept generation when the predicted vector approaches an encoded end-of-text marker or repeats the preceding concept [190]. Finally, Expanding Generative Flows (EFlow) vary sequence length dynamically along the trajectory by inserting new token slots with local denoising clocks into a growing canvas (Appendix G; see Appendix C for DDOT's continuous token positions) [347].

5.4 Conditioning and guidance

A denoiser learns $p(z_0)$ or $p(z_0 \mid c)$, but a task also brings conditions that were training inputs and attributes that were not. Conditions enter as inputs: DiffuSeq concatenates source and target and corrupts only the target [119], Diffusion-LM's infilling re-noises the fixed prefix and suffix at each diffusion step [213], and SeqDiffuSeq encodes the source once and attends to it throughout target refinement [425]. The condition itself need not remain clean during training: Noise Your Prompt noises a flow language model's prompt to the response's noise level on a fraction of training examples while still sampling with a clean prompt, substantially improving combinatorial puzzle accuracy and summarization quality–diversity frontiers, though not open-domain dialogue [170]. Multimodal generators condition on non-text inputs through the same interfaces: ELF-S2T supplies projected speech features during refinement and decoding [215], CrossFlow and FlowTok initialize flow transport directly from encoded image latents [141, 230], and V-LCM prepends visual vectors to the clean concept history (Section 6.1.3; Appendix G.3) [295]. When target attributes are imposed at inference time rather than through direct input conditioning, systems either steer intermediate transitions through classifier and classifier-free guidance or select among completed trajectories via candidate reranking, both of which introduce additional evaluation overhead analyzed at the end of this subsection.

Classifier and classifier-free guidance By Bayes' rule, the exact conditional score satisfies $\nabla_z \log p_t(z \mid y) = \nabla_z \log p_t(z) + \nabla_z \log p_t(y \mid z)$. Classifier guidance forms a guided score estimate from the denoiser's plug-in score estimate $\hat{s}_\theta(z_t, t) = -\hat{\epsilon}_\theta(z_t, t)/\sigma(t)$ and a classifier $p_\phi(y \mid z_t, t)$ trained on noisy states:

$$\hat{s}^{(\lambda)}_{\theta,\phi}(z_t, t, y) = \hat{s}_\theta(z_t, t) + \lambda \nabla_{z_t} \log p_\phi(y \mid z_t, t), \qquad \hat{s}_\theta(z_t, t) = -\hat{\epsilon}_\theta(z_t, t)/\sigma(t), \tag{13}$$

so that $\hat{\epsilon}_\theta$ in Eqs. 8–10 is replaced by the guided noise estimate $\hat{\epsilon}_\theta - \lambda\sigma(t)\nabla_{z_t} \log p_\phi(y \mid z_t, t)$. Equality between $\hat{s}^{(\lambda)}_{\theta,\phi}$ and the true Bayes conditional score $\nabla_{z_t} \log p_t(z_t \mid y)$ holds only when $\lambda = 1$ and both score terms are exact; at general $\lambda \neq 1$, Eq. 13 defines a guided score or prediction estimate that sharpens updates toward y for $\lambda > 1$. Classifier-free guidance (CFG) needs no external classifier [148]: the denoiser

is trained with the condition randomly dropped, and at sampling time the conditional and unconditional plug-in estimates are extrapolated to form the guided prediction estimate

$$\hat{s}_w(z_t, t, c) = (1 + w)\hat{s}_\theta(z_t, t, c) - w\hat{s}_\theta(z_t, t, \varnothing), \tag{14}$$

where $\hat{s}_\theta$ may represent the score, $\hat{\epsilon}_\theta$, $\hat{z}_0$, or the velocity, since all four are related by affine transformations. Each guided step evaluates both the conditional and unconditional branches (two denoiser NFE per step). Batching the branches can reduce the wall-clock overhead, but the gain depends on the hardware, model, and batch size. LD4LG applies Eq. 14 to $\hat{z}_0$ for conditional generation [239], and RegDiff combines source-conditioned CFG under DDIM with training-time attribute supervision [455]. In blockwise generation, Sigma applies Eq. 14 to the predicted clean embedding using the clean prefix as the condition: because sequence packing already leaves the initial block of each training window without context, the unconditional branch simply drops the prefix without requiring explicit condition dropout during training. Packing both branches into one forward pass adds under 5% wall-clock overhead per call at batch size 1, and combining CFG with a score temperature $\tau \leq 1$ on the noise prediction controls the trade-off between single-sample accuracy and multi-sample pass@k coverage [405].

Unconditional flow models lack an external condition to drop and instead construct a weak reference branch from internal model states. ELF uses predictions with and without self-conditioning to define guidance and learns the extrapolation through training-time guidance, retaining a single denoiser pass per sampling step [153], while FLM extrapolates away from a dropout-perturbed forward pass [191]. To avoid the cost of a full second denoiser evaluation, Probe guidance obtains the weak reference prediction from a lightweight MLP attached to detached early-layer activations of the frozen backbone, lowering generative perplexity at matched entropy and improving zero-shot multiple-choice accuracy with negligible additional FLOPs [80, 89].

When steering toward external attributes or formal constraints at inference time, gradient-based methods differ in whether the guiding module requires noise-aware training. Diffusion-LM trains a dedicated noisy-state classifier $p_\phi(c \mid z_{t-1})$ for each target attribute and takes multiple gradient steps on z_{t-1} between sampler updates to maximize $\lambda \log p_\theta(z_{t-1} \mid z_t) + \log p_\phi(c \mid z_{t-1})$, using the model's reverse transition density as a fluency regularizer [213]. To reuse off-the-shelf classifiers or reward models trained only on clean text, SSD-LM, ParaGuide, TESS 2, and P3SUM pass the expected embedding $E^\top \mathrm{softmax}(\ell_t)$ of the current token logits into the frozen classifier and backpropagate gradients to ℓ_t [134, 150, 232, 344]. Because this soft-embedding relaxation evaluates the classifier on interior convex combinations outside its training distribution, overly large guidance weights can trigger reward hacking and degenerate text, as observed in TESS 2 [344]. An analogous failure occurs during RL training in CFRL if the value critic is not updated alongside the policy [137]. The point at which the guidance gradient enters also affects adherence. On three public ELF checkpoints, intervention evidence links weak state-only guidance to stale self-conditioning memory. Writing the decoder's reward gradient into that memory at each intermediate step, without a state push, improves held-out keyword coverage over state guidance at scale 0.25 at approximately equal wall-clock cost inside a preregistered quality gate [18]. For formal regular-expression constraints, Diffinity dispenses with neural classifiers altogether by compiling the regular expression into a token-aligned finite automaton and using dynamic programming to compute exact log-acceptance gradients under Plaid's factorized token posterior [180]. In style transfer, ParaGuide combines input conditioning with classifier guidance by holding an initial autoregressive paraphrase fixed as source context while a RoBERTa denoiser refines target embeddings under style gradients (Section 6.1.1) [150].

Candidate reranking and particle selection Minimum-Bayes-risk (MBR) selection draws a pool S of candidates and returns the one with the highest expected utility under the pool itself, $\hat{y} = \arg\max_{y \in S} \sum_{y' \in S} U(y, y')$, with U a task metric such as BLEU; it needs no reference and favors consensus. Diffusion-LM and DiffuSeq report MBR results, so their quality includes several trajectories and the selection [119, 213], whereas TESS and DiffuSeq-v2 report without it [120, 176]. Selecting candidates by their score against the ground-truth reference, as in GENIE's oracle candidate-pool evaluation, instead measures upper-bound pool coverage [222]. Feynman–Kac steering moves selection inside

the trajectory: a population of noisy states is scored by intermediate reward potentials, resampled, and propagated, with importance correction when the proposal changes. In its continuous configuration SSD-LM supplies the logit-state updates and tokens sampled from each intermediate logit vector let a text reward score the candidate [335]. Particle selection accepts non-differentiable rewards but multiplies the trajectories and the reward evaluations. The split of a fixed compute budget between candidate count and update steps interacts with the selection rule: under a fixed total NFE budget on mathematical reasoning, allocating compute to many few-step samples maximizes oracle pass@k coverage, whereas allocating more refinement steps to fewer samples favors majority-vote consensus [72].

Denoiser NFE versus end-to-end latency End-to-end generation cost includes condition encoding, denoiser evaluations, latent decoding, feedback, and candidate selection. Because denoiser NFE counts only denoiser passes and omits separate codec, decoder, or reranker stages, nominal NFE and wall-clock latency can rank systems differently. PLANNER's latency advantage over its autoregressive baseline reverses once that baseline uses length bucketing and batching [441]. Similarly, Du and Ma's early-exit criterion saves denoiser calls while a literal monitoring implementation increases elapsed time [97], and while LCM includes encoding and decoding in its FLOP accounting [190], SLD omits direct latency comparisons because the compared sampling strategies and output windows differ [462]. In speculative pipelines, autoregressive verification contributes to each round's computation and the net speedup depends on acceptance rate; Lafa fuses verification with redrafting (Section 5.3), while FlashMap uses a 16-token flow-map drafter for a separate Qwen3 target [56, 326]. Overall, solver choice governs denoiser call counts, state compression governs sequence length per call, and decoder topology governs sequential dependencies (Section 6.3 and Table 16 compare end-to-end latency across systems).

6 Tasks, Evidence, and Evaluation

This section reviews the empirical capabilities of continuous-space text generators, the evaluation metrics used to score them, and the resulting cross-paper comparisons. Section 6.1 examines downstream capabilities and links each to its primary design dimension (state representation, transition mechanism, or text interface). Section 6.2 analyzes the properties and failure modes of standard evaluation metrics, and Section 6.3 synthesizes the published quantitative comparisons.

6.1 Tasks and capabilities

Reported applications of continuous text generators span attribute-controlled generation, source-conditioned transformation, multimodal conditioning, discourse planning, formal code or structured synthesis, and reasoning. Table 11 summarizes these capability areas, representative system routes, and the evaluation criteria appropriate to each.

6.1.1 Controlled generation and editing

Because a continuous state remains real-valued across all positions until the final update, a classifier gradient can steer an entire candidate sequence jointly, and an edit can be formulated as a bounded displacement from a reference state (Table 12). Diffusion-LM established this baseline for gradient-guided control: it surpasses decoding-time steering and task-specific fine-tuning on fine-grained syntactic and span-level constraints, and matches task-trained autoregressive infillers when conditioned on bidirectional context [213]. Gradient control does not dominate prompted autoregressive models across all constraint types, however. When initialized from the same pretrained backbone and fine-tuned on the same target text, instruction-adapted autoregressive decoding achieves better external fluency, though Diffusion-LM retains stronger syntactic control [457]. Few-shot prompted ChatGPT also surpasses classifier-guided Diffusion-LM on the tested open-domain topic, sentiment, and keyword constraints, although the tested few-shot 7B open models still trail Diffusion-LM on topic control [343]. These comparisons show that the

Table 11. Downstream capabilities, representative system routes, and evaluation criteria. Most rows pair the primary task metric with a secondary check on retention, diversity, or observation alignment to capture trade-offs between competing objectives. The first column lists the corresponding subsection; AR denotes autoregressive.

Capability	Generated object and routes	Evidence that tests it
Controlled generation and editing (§6.1.1)	Diffusion-LM revises token states under classifier gradients; ParaGuide guides a source-conditioned target; RegDiff moves attribute supervision into codec and denoiser training; sentence-space methods compose attributes inside a latent code.	Constraint satisfaction together with content retention and fluency, scored by an evaluator withheld from the controller.
Source-conditioned transformation (§6.1.2)	DiffuSeq and SeqDiffuSeq refine target token states; ShortcutFlow and FMSeq learn finite-interval maps or flows; DSL-LLaDA adapts a masked backbone.	Target quality and diversity at a stated update budget, with the source encoder and the candidate-selection rule named.
Captioning and multimodal conditioning (§6.1.3)	CrossFlow and FlowTok transport an image code into a text code; LaDiC generates contextual BERT states from image patches; FlowBind averages per-source anchors; OmniFlow couples the requested output states; ELF-S2T conditions a contextual text state on speech; JUICE and MF-1 each denoise text-encoder states beside image features in one backbone trained from scratch.	Caption agreement and observation alignment read separately, and transcription error after the stated decoder.
Planning, dialogue, and discourse (§6.1.4)	PLANNER samples paragraph plans; DGLM samples sentence proposals for an AR decoder; LCM predicts the next sentence vector; SLD revises segment states jointly.	Repetition and judged quality over complete outputs, with the decoder and the context span held fixed.
Code, structured outputs, and synthetic data (§6.1.5)	CodeFusion generates program states behind a contextual decoder; CDA^2, DCA, and CoDSA generate labeled training text.	Execution or template match for programs; downstream accuracy under the actual label and allocation conditions for synthetic data.
Instruction following and reasoning (§6.1.6)	TESS 2 and SSD-2 adapt AR backbones to continuous logit states; Sigma continues 3B and 8B AR checkpoints as blockwise embedding diffusion; Diffusion of Thought revises rationale embeddings; LaDiR generates thought blocks; DiffuCOMET generates fact vectors; ELF-REG and CEDR train sub-1B flows over frozen-encoder or teacher-layer states.	Task accuracy with supervision, sample count, and answer realization specified.

advantage depends on the constraint type and the particular autoregressive baseline, rather than establishing uniform superiority across prompted autoregressive models [213, 343, 457].

In editing and style transfer, attribute control must also preserve source content. ParaGuide makes this tension explicit: strengthening inference-time guidance raises target style accuracy under a held-out evaluator at the expense of semantic retention and surface fluency, and system rankings can shift when scored outside the optimized guidance representation [150]. Moving attribute supervision from inference-time guidance into codec and denoiser training produces a similar trade-off. In RegDiff, supervising a latent-code classifier and regularizing the denoiser improves transfer accuracy up to an intermediate regularization weight, beyond which excessive control weight degrades semantic similarity and fluency [455]. When the attribute resides in a sentence vector rather than token states, control depends on how that latent space is organized: MacLaSa's aspect-discrepancy loss penalizes distances between

cross-aspect latent centers; its ablation improves multi-attribute correctness while slightly lowering external perplexity [90]. Appendix G.1 catalogs the remaining control interfaces (formal-constraint checking, corruption-based edit magnitude, and selection among completed candidates), and Appendix C.3 covers invertible editors that expose reversible content–style coordinates.

6.1.2 Source-conditioned transformation

In source-conditioned transformation, the source sentence constrains target semantics, shifting the main design questions to the transition and candidate selection: how many refinement updates the target state requires and how the final output is chosen from a candidate pool (Table 12). DiffuSeq serves as a reference point for source-conditioned continuous generation. Its multi-trajectory sampling yields a diverse candidate pool that minimum-Bayes-risk (MBR) selection can exploit to raise generation quality and narrow the gap with autoregressive baselines as the pool grows [119]. At the opposite end of the sampling budget, few-step flow and map models replace multi-step refinement with one or a few updates. ShortcutFlow trains its updates on finite intervals to improve single-step paraphrasing markedly over ordinary flow matching, with diminishing returns at larger step budgets [375]. The usable step reduction depends on the task's output ambiguity: FMSeq recovers much of its many-step baseline's reference-overlap quality in a few steps on structured transformations such as question generation, paraphrasing, and simplification, yet single-step generation degrades on open-ended dialogue [229].

Against autoregressive systems, outcomes depend on initialization, distillation, and task structure. Trained from random initialization on sequence-to-sequence translation, standard continuous diffusion variants remain below autoregressive and iterative non-autoregressive baselines even with sequence-level distillation, and degrade faster as source length grows [82]. By contrast, in sparse span infilling under known target lengths, fine-tuned flow models such as YAN beat comparably sized fine-tuned encoder–decoder baselines in just a few Euler steps [196]. In full-sentence translation, combining spline-curve targets (Section 4.1), autoregressive teacher distillation, and length-and-noise beam selection allows SCLM to surpass its base continuous baseline and match or exceed beam-search autoregressive Transformers on higher-resource benchmarks, falling slightly behind only on smaller setups [144]. Against discrete masked backbones, continuous-noise adaptation in DSL-LLaDA improves short-reference summarization (XSum and CNN/DM) across evaluation budgets, with its advantage on long-reference summarization (PubMed and arXiv) concentrated in the low-step regime [420].

6.1.3 Captioning and multimodal conditioning

In multimodal captioning, the trajectory can start from an encoded visual or audio observation instead of pure Gaussian noise, giving the state a semantic starting point and leaving the decoder to recover fluent text (Figure 10, Table 12). Direct image-to-text flow models FlowTok and CrossFlow reach reference-overlap scores close to the diffusion captioner SCD-Net in comparisons without CIDEr optimization [141], and training the image-to-text direction directly outperforms inverting a text-to-image flow [230]. LaDiC, which diffuses token-aligned contextual states conditioned on visual features, approaches pretrained autoregressive captioners on short captions and surpasses a fine-tuned baseline on longer paragraph captioning. In human evaluations, it receives higher ratings for factual accuracy and conciseness alongside lower ratings for surface fluency, with low-latency batched inference [373].

Unified multi-directional models trade single-task captioning accuracy for modality flexibility. OmniFlow couples text, image, and audio in a single rectified-flow backbone [211], whereas FlowBind maps each observed modality to a shared anchor before following a single text flow. Relative to OmniFlow, FlowBind improves CIDEr in both image and audio captioning; its CLIPScore also improves for image captioning, while its CLAPScore decreases for audio captioning [54]. Controlled comparisons within a shared backbone compare continuous and discrete text-generation pipelines. In JUICE, denoising frozen text-encoder states alongside image features within a shared flow-matching trunk falls short of a matched Transfusion-style autoregressive-text baseline on caption-reference overlap despite achieving nearly identical image–text semantic alignment [205]. Conversely, Multimodal Flow (MF-1), which

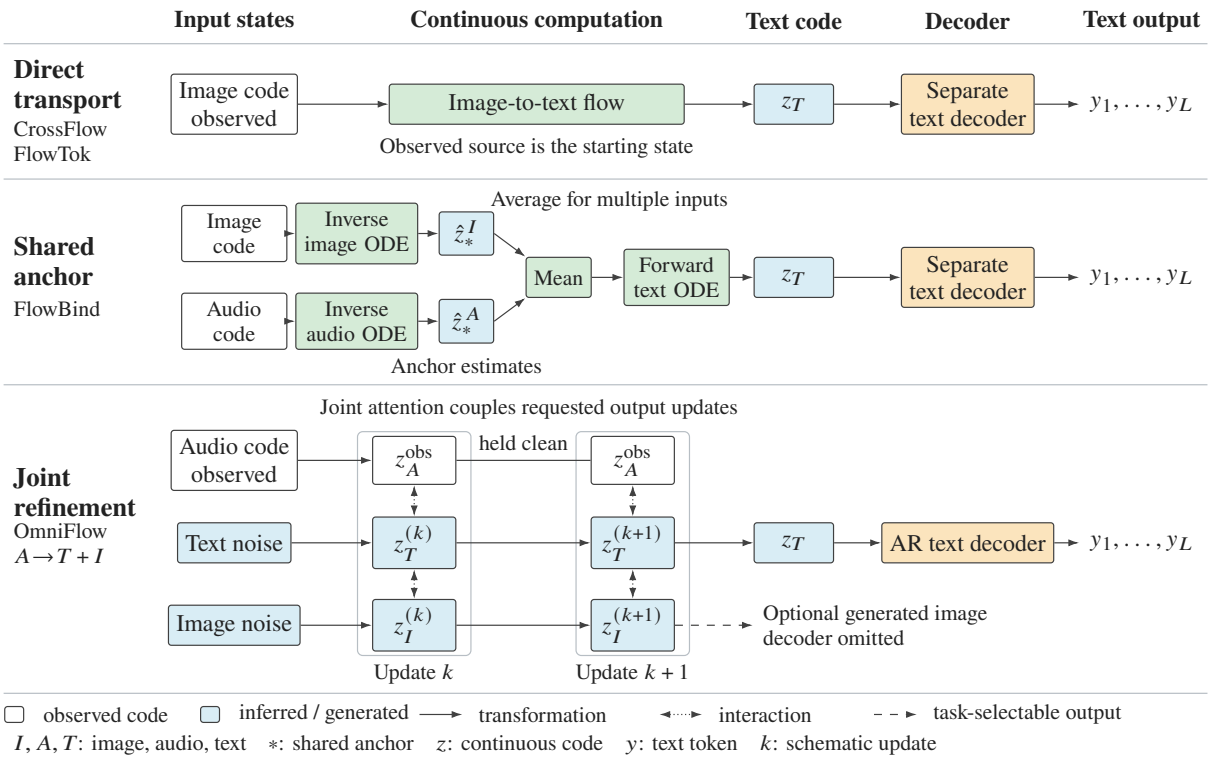


Figure 10. Multimodal conditioning routes for continuous text generation: direct transport from an observed image code into a text code (CrossFlow, FlowTok) [141, 230], shared-anchor mapping that averages per-source anchor estimates before following one text flow (FlowBind) [54], and joint refinement of text and image states through cross-modal attention while observed audio stays clean (OmniFlow) [211].

generates token-aligned, eight-token text blocks alongside image chunks in a matched chunk-causal flow backbone, beats its matched autoregressive-text counterpart across short-answer visual question answering benchmarks [351]. JUICE reports lower caption-reference overlap despite similar image–text alignment, whereas MF-1 improves short-answer VQA accuracy in its matched comparison [205, 351]. These results motivate separate evaluation of caption wording and visual question answering, though they do not by themselves establish a general separation between language realization and multimodal comprehension.

The placement of stochasticity along the multimodal trajectory determines output diversity, because a deterministic path from a fixed source code yields only a single output per observation. BIT uses a Brownian bridge between paired image codes and token embeddings, achieving lower image-to-text generative perplexity than its noise-started diffusion baseline while retaining comparable CLIP alignment to its deterministic-flow baseline [129]. In speech-to-text generation, ELF-S2T corrupts the text states used for decoder supervision (audio-forcing), reducing transcription error under matched sampling and guidance [215]. Appendix G.3 holds the per-modality configurations, and Appendix G.7 covers specialized interfaces for reports, neural-signal captioning, and domain descriptions.

6.1.4 Planning, dialogue, and discourse

In discourse planning, the state represents a sentence, segment, or paragraph vector while token realization remains autoregressive. The continuous prior models high-level variation, and a language model decoder supplies the surface wording. PLANNER illustrates how a continuous paragraph plan pairs with an autoregressive decoder: combining latent plan diffusion with greedy token decoding cuts surface repetition sharply compared with sampling directly from a larger autoregressive baseline, and human evaluators prefer its continuations for relevance, informativeness, and human-likeness [441]. Decoder adherence to the latent plan is also tunable: DGLM trains an autoregressive decoder on noisy sentence proposals,

showing that adjusting proposal noise smoothly trades plan adherence against surface perplexity and diversity [240].

At discourse scale, sentence- and segment-level diffusion models improve high-level structural coverage and summary overlap, though token-autoregressive models retain stronger local sentence fluency and open-ended narrative coherence (Table 12). In small-scale story generation, Two-Tower sentence-vector diffusion in LCM beats direct sentence-vector regression but falls behind token-autoregressive baselines on narrative coherence [190]. When scaled to billions of parameters and instruction-tuned, Two-Tower LCM surpasses comparably sized instruction-tuned autoregressive models on summarization overlap metrics at the cost of lower classifier-measured sentence fluency [190]. Extending this sentence-diffusion recipe to massively multilingual corpora in Mimir yields competitive multilingual summarization (XL-Sum) and extractive question answering (MLQA) on trained tasks, though zero-shot transfer without task-specific supervision remains prone to task drift and local repetition [264]. SLD jointly denoises compressed segments before segment-local autoregressive decoding. It improves dialogue compatibility over LD4LG and matches Flan-T5 Large on DeliData coherence and non-hallucination, while scoring lower than Flan-T5 on XSum ROUGE and ROCStories fluency and coherence [462].

6.1.5 Code, structured outputs, and synthetic data

Generating executable code or structured records depends critically on the text interface, because independent per-position rounding can violate formal syntax or schema constraints that couple tokens across positions. CodeFusion addresses this by pretraining on code-specific denoising, generating the program as a continuous state, and passing that state through a separate contextual decoder before parallel token classification so that neighboring positions are resolved jointly [334]. Despite using far fewer parameters than fine-tuned and few-shot autoregressive code models, CodeFusion matches their single-output accuracy across Python, Bash, and spreadsheet rule generation and surpasses them when selecting from multi-candidate sample pools, indicating that its primary advantage comes from trajectory-level candidate diversity (Table 12) [334]. CADD reaches executable-code benchmarks from the opposite direction: as a hybrid discrete–continuous model (Appendix C), CADD pairs a categorical token canvas with a continuous latent channel and adapts pretrained code backbones [122] and, for code generation, selects still-masked positions using top negative entropy, with the continuous latent channel providing semantic hints at unresolved positions [448]. Both CodeFusion and CADD replace independent per-position rounding with a mechanism that coordinates token choices across positions.

At multi-billion-parameter scale, comparisons between continuous and discrete diffusion differ between single-sample accuracy and multi-sample exploration (Table 13). Sigma generates code in blocks using embedding-space diffusion and achieves competitive single-sample pass rates relative to matched masked diffusion and its autoregressive initialization. In the reported MBPP comparison, Sigma overtakes NLD at larger candidate budgets; at the selected fixed sampler settings, it also produces fewer exact duplicate output strings [405]. For synthetic data generation, the evaluation metric is downstream classifier performance under domain shift or label transformation. In cross-domain sentiment transfer, CDA^2 combines continuous rewriting with sentence-level target-domain alignment to improve over word-substitution baselines; its filtered variant, CDA^2-F, matches UDALM's average accuracy across the twelve Amazon domain-transfer tasks [390]. For label-flipping data augmentation, DCA's antonymous continuous rewrites yield modest downstream gains over lexical substitution; its ablations show much larger losses from removing candidate-word selection or the dual sentiment classifier [389]. In structured tool-description synthesis with DiffLM, GPT-4 rates individual generated descriptions favorably but prefers real tool collections at the category level (64.9% versus 28.3%). A separate human evaluation of 100 individual tool pairs instead prefers DiffLM in 88% of pairs, with 6% ties and 6% preferring real tools [458].

6.1.6 Instruction following and reasoning

In reasoning and instruction following, iterative transition updates allocate test-time compute to revising an entire rationale or answer block in parallel, with outcomes governed by the continued-pretraining budget and task-specific supervision (Section 6.3, Tables 13 and 17). At multi-billion-parameter scale, adapting pretrained autoregressive backbones to simplex-logit or blockwise embedding diffusion requires an intermediate continued-pretraining stage before instruction tuning. TESS 2 leads on extractive question answering under matched continued pretraining and on symbolic mathematics after dedicated task fine-tuning, while trailing directly fine-tuned autoregressive models on broad untuned instruction suites [344]. After joint autoregressive and blockwise-diffusion pretraining on hundreds of billions of tokens, Sigma approaches both its autoregressive initialization and compute-matched masked diffusion (NLD) in average pass@1 over six mathematical and coding benchmarks, with task-dependent gains and deficits [106, 405]. SSD-2 combines general and domain-expert logit predictions at each refinement step. In the reported comparisons, this combination outperforms standalone generation and token-level autoregressive collaboration, although its single-model outputs remain less preferred than those of the instruction-tuned autoregressive baseline [135].

With denoisers below one billion parameters, reported reasoning capabilities rely on task-specific flow training over frozen pretrained representations rather than full-backbone adaptation [72, 218]. ELF-REG trains a flow denoiser over frozen contextual states and incorporates intermediate feature alignment and a global register token distilled from a larger teacher. This alignment improves mathematical reasoning over unaligned flows. The task-trained ELF-REG-L exceeds its untuned frozen encoder's GSM8K score, but its separately trained MATH and code variants have lower reported point estimates on MATH-500, HumanEval, and MBPP-378 [218]. CEDR extends this setup by generating multi-layer projections from an instruction-tuned teacher under group-specific denoising schedules and applying reinforcement learning with verifiable rewards, achieving further reasoning gains on its trained domains [72].

Reasoning systems also differ in the granularity of the continuous state. At the token level, Diffusion of Thought (DoT) denoises rationale and answer embeddings, using single-pass refinement on compact symbolic tasks and multi-pass block conditioning on multi-step math problems [407]. Coconut uses deterministic hidden-state recurrence for continuous reasoning [138]. Latent Thought Flows instead studies compressed latent-code flows for text generation, without establishing reasoning-task gains (Section 3.3) [288]. On Countdown, LaDiR's sentence-level thought blocks outperform autoregressive and LLaDA supervised fine-tuning on pass@1 but trail the task-specific masked diffusion model MGDM; among 100 samples, LaDiR achieves higher pass@100 and more unique valid solutions than these baselines [175]. Controlled studies on synthetic reasoning tasks further isolate how spatial layout and unrolled training govern multi-step accuracy [161, 270]. Plan Canvas shows that allocating a fixed-capacity planning canvas with separate plan and answer denoising clocks improves both answer accuracy and reasoning-trace validity [270], and Unrolled Flow Models show that optimizing an endpoint loss through unrolled latent updates, with truncated backpropagation for longer rollouts, enables test-time compute scaling where standard per-step flow matching plateaus [161]. Among hybrid masked–continuous models, EvoToken-DLM maintains a revisable soft token distribution prior to discrete commitment (Appendix C), improving average accuracy over LLaDA-Instruct-8B on Countdown, GSM8K, MATH500, and SVAMP, with a regression on Countdown at generation length 512 and 128 denoising calls [451]. For commonsense knowledge generation, DiffuCOMET jointly denoises sets of compressed fact vectors, generating more diverse and contextually relevant knowledge clusters than beam-search baselines as refinement steps trade relevance against diversity [109]. Section 6.3 synthesizes Cola's multi-task answer-generation benchmarks alongside the broader cross-paper comparisons.

6.2 Evaluation measures

Continuous language models are evaluated by held-out negative log-likelihood bounds or estimates, external generative perplexity paired with sample diversity, clean-text autoencoder reconstruction accuracy, and downstream task metrics. Because interventions that improve one metric can degrade another,

interpreting cross-paper comparisons requires matching each metric to the property it measures.

Held-out likelihood evaluation distinguishes exact autoregressive negative log-likelihood (NLL), a variational upper bound $\mathcal{B}_{\text{var}}(x) = -\log p_\theta(x) + \Delta(x)$ with non-negative gap $\Delta(x) \geq 0$, and an ODE probability-flow likelihood estimate (Appendix A). The variational gap $\Delta(x) \geq 0$ reflects the mismatch between the variational distribution and the model posterior, so a variational upper bound combines model fit with bound tightness rather than measuring $-\log p_\theta(x)$ directly. Finite Monte Carlo estimation of that bound introduces separate sampling error; an individual estimate need not upper-bound the NLL. For direct denoising-error integrals, a fixed log-SNR window with stale scale-dependent tail corrections can omit mass under representation drift, so the resulting truncated readout need not retain the full-line integral's entropy upper-bound guarantee; finite-window variational bounds with reconstruction and prior-KL endpoint terms evaluated on the live model retain their bound (Appendix D.1) [33]. Conversely, probability-flow ODE likelihood estimators must estimate the full divergence of the drift field actually integrated. In the evaluated LangFlow estimator, a self-conditioning bootstrap is recomputed from the current state, but held fixed in the divergence calculation; this omits its chain-rule Jacobian contribution and invalidates the claimed likelihood bound. RePlaid identifies this omission as a major source of LangFlow's low self-reported ODE perplexity, while its corrected ODE and variational-bound evaluations remain distinct estimators of the same checkpoint (Table 13, Appendix D.1) [65, 406]. For codec systems, a density at an encoded point is not the probability of the text because the decoder determines that probability. Diffusion-LM therefore reports its variational bound separately from external scores, and Plaid aligns segment boundaries and detokenization before comparing bounds with exact autoregressive NLL [128, 213]. For masked-token models, DUEL and TUBE show that a deterministic unmasking policy and a random-order marginal define different distributions with different gaps to autoregressive likelihood [160, 356]. In continuous models, perplexity comparisons across papers similarly require specifying the estimator, sampler, and decoder [134].

External generated-text perplexity (GenPPL) scores a sample $\hat{x}$ under a separate language model q:

$$\text{GenPPL}_q(\hat{x}) = \exp\left[-\frac{1}{|\hat{x}|}\sum_i \log q(\hat{x}_i \mid \hat{x}_{<i})\right]. \tag{15}$$

GenPPL rewards predictability of any kind, including repetition, and depends on the scorer q, tokenizer, sequence length, and aggregation rule, since averaging per-sample perplexities differs from exponentiating corpus-level cross-entropy. Used in isolation, GenPPL is vulnerable to two failure modes. Excessive decoder-input noise can lower GenPPL while collapsing both MAUVE and token entropy, as in LDLM's decoder-noise ablation (Table 4) [253]. Periodic token sequences can also achieve low GenPPL even at non-degenerate unigram entropy while failing distributional similarity checks [105]. Evaluating open-ended generation therefore requires pairing GenPPL with sample entropy and a distributional similarity metric such as MAUVE [286], as done throughout Section 6.3. Unigram entropy can still miss a deficit confined to rare words. Against a same-corpus human reference (included in ELF's training data but disjoint from the attestation lexicon), stock ELF and LangFlow samples under-produce rare words, a tail deficit that a word-frequency profile exposes even when GenPPL and unigram entropy appear healthy. In addition, ELF-B's 16–64-step sweep lowers GenPPL and raises four-gram repetition, although the 64-step setting also changes SDE churn [401].

Because lowering sampling temperature, sharpening the decoder, or reducing deterministic step counts moves a fixed checkpoint along a trade-off curve between GenPPL and sample entropy, single-point comparisons at unaligned temperatures can reverse when evaluated along a full frontier. In discrete diffusion, Generative Frontiers shows that small temperature-induced entropy differences can reverse single-point rankings, and recommends comparing perplexity along temperature-sweep frontiers at matched entropy [292]. A Tale of Two Temperatures shows that rankings depend on whether temperature scales token sampling or position unmasking and whether budgets fix sample count or network evaluations [274], and Zheng et al. trace apparent perplexity gains in SEDD and MDLM samplers to floating-point truncation in categorical draws [449]. Continuous generators exhibit the same frontier behavior: MCB's stochastic endpoint bridges trade GenPPL against entropy through endpoint temperature on fixed checkpoints

[38]. Similarly, S-FLM with top-1 velocity matches masked diffusion on mathematical reasoning at unit temperature but falls behind under tuned low-temperature decoding [83], and Untied Self-Conditioning lowers few-step GenPPL at the cost of higher n-gram repetition [197].

The locus of temperature scaling and the choice of evaluation metric also shift these rankings. In BCT's reproduction of S-FLM's reasoning protocol, standard logit tempering leaves S-FLM well behind low-temperature discrete diffusion, whereas tempering the endpoint, bridge, and velocity consistently after tempered-bridge continuation narrows the gap [272]. Sampler-side logit shaping can move entire frontiers: with nucleus truncation, temperature annealing, and frequency penalties, multi-step simplex, masked, and uniform-state diffusion models converge to nearly identical GenPPL-at-entropy frontiers that remain slightly above a same-architecture autoregressive model, so this frontier alone does not separate small diffusion variants [84]. Within the DistillScope benchmark for flow-map distillation [9], semigroup distillation reaches lower external perplexity than Lagrangian distillation even as MAUVE reverses their order. Counting wall-clock training time likewise reverses an advantage observed at equal update counts, and a given student can pass degeneration checks under a larger inference budget that it fails under a tighter one. Meaningful comparisons therefore either align the entropy or diversity level across methods or report the full operating configuration alongside both metrics.

Downstream task metrics depend on the evaluation protocol: BLEU and ROUGE measure reference overlap, answer F1 and exact match depend on answer normalization, and pass@k requires an execution harness. Their comparability depends on the candidate-selection rule applied before scoring. DiffuSeq's minimum-Bayes-risk choice among ten candidates and GENIE's best-of-ten selection against the reference are different procedures, and only the first is available without ground-truth references [119, 222]. Similarly, in their original arXiv version, Zhang et al. show with ACE that scoring a low-repetition accepted subset instead of the full sample pool reverses the ELF-B/ELF-M ranking [436]. Multi-stage systems require component-level baselines and explicit compute accounting: SpecRef reports drafter-only, refiner-only, and pipeline scores, showing that refinement can improve over the refiner while remaining below the drafter [133]. Human evaluations directly assess perceived quality, relevance, and fluency, and are distinguished from automated LLM-as-judge preference scores across Tables 12, 13, and 17.

Autoencoder reconstruction accuracy evaluates the encoder–decoder path on clean text rather than the learned prior on generated states. A codec that reconstructs clean text more accurately can yield worse samples when paired with a continuous prior, because generated codes do not follow the distribution of encoded clean text, as LaDiff's codec ablation and TextLDM's alignment variants illustrate (Sections 2.3 and 3.3) [166, 195]. Reconstruction remains useful as a diagnostic of what a codec can carry and of input familiarity. Wang et al. use reconstruction error under fixed corruption to detect out-of-domain inputs to an adapted RoBERTa denoiser; in their separate SST2 generation experiment, longer training lowers reconstruction loss while raising generated-text perplexity [363]. DiffuTruth turns the stability of a claim's embeddings under noise and reconstruction into a factuality score through an external natural-language-inference model [113]. Finally, the metric implementation itself must be held fixed. In a reproduction study of controlled text generation that refers to PriorControl as PriorCTG, recomputing Distinct-2 with a different script on the exact same saved outputs shifts absolute scores and reverses the two variants' ranking, whereas rerunning inference under a fixed evaluator preserves the pairwise orderings [237].

6.3 Cross-paper empirical comparisons

Tables 12, 13, 14, and 17 assemble the published empirical comparisons across autoregressive baselines, few-step operating regimes, and model scale, and Table 18 summarizes the controlled ablations behind them.

Continuous versus autoregressive baselines Tables 12 and 13 collect the comparisons reported in this survey in which a continuous system is scored against an autoregressive model, and Tables 14 and 17 cover comparisons against masked models or a system's own teacher. Across these tables, no row demonstrates a general advantage over a same-size autoregressive model on open-ended text under

matched training compute. On prompt completion, CDCD slightly edges out a matched-architecture autoregressive baseline in distributional similarity only under asymmetric training sequence lengths and separate sampling-hyperparameter sweeps [87], and in SED's pairwise human evaluation at matched parameter scale, raters still prefer the autoregressive baseline on the majority of unconditional and infilling samples [341]. On held-out likelihoods at matched architecture and step budget, continuous variational bounds and corrected ODE evaluations remain above exact autoregressive perplexity [65, 406], and matching exact autoregressive negative log-likelihood in zero-shot bounds requires much larger parameter counts and training compute multiples, as quantified by Plaid [128]. On open-ended story generation and machine translation from random initialization, autoregressive baselines similarly lead, though distilling targets from an autoregressive teacher enables SCLM to match or exceed beam-search autoregressive translation on higher-resource benchmarks [144]. At multi-billion-parameter scale, outcomes depend on the state granularity and evaluation protocol. Sentence-level Two-Tower LCM achieves higher summarization overlap than comparably sized instruction-tuned autoregressive models at the cost of lower sentence-level grammatical fluency [190]. Standalone simplex-logit diffusion in SSD-2 falls behind its instruction-tuned autoregressive counterpart in LLM-as-judge pairwise preference [135], whereas blockwise embedding diffusion in Sigma closely matches its autoregressive starting checkpoints on multi-task mathematical and coding pass@1 averages after extensive continued pretraining, lagging slightly on likelihood-ranked multiple-choice benchmarks [405].

Favorable head-to-head results against autoregressive baselines instead cluster into three specific settings. The first setting involves inference-time continuous steering or model combination, including gradient-guided syntactic control (Diffusion-LM) and logit-level collaborative decoding with a domain expert (SSD-2). The second leverages plan-level or multi-trajectory diversity, such as paragraph-plan diffusion paired with greedy token decoding (PLANNER), paragraph captioning and higher human accuracy and conciseness ratings for short captions (LaDiC), and multi-candidate sample pools for code or puzzle generation (CodeFusion, Sigma pass@k, and LaDiR). The third relies on task-aligned representations or targeted post-training, including adapted simplex diffusion after matched continued pretraining or symbolic-math fine-tuning (TESS 2), short-answer visual question answering on a matched chunk-causal backbone (MF-1), pathwise reinforcement learning on fixed-horizon constraint puzzles such as Sudoku and Game24 (CFRL), and end-to-end prior–codec co-adaptation on multi-task answer generation (Cola) [131, 137, 351]. Conversely, few-shot ChatGPT leads on the tested open-domain topic, sentiment, and keyword constraints; autoregressive baselines also lead on caption-reference overlap and human fluency ratings, single-output code generation without extensive continued pretraining, and broad untuned instruction suites. Within controlled studies, autoregressive baselines also retain higher caption reference overlap on a shared backbone (JUICE), lead on variable-branching search puzzles such as Countdown and Knights-and-Knaves (CFRL), and have higher reported point estimates than task-trained ELF-REG on MATH-500, HumanEval, and MBPP-378 [137, 205, 218]. To make protocol differences explicit, the "Matched" column in Tables 12 and 13 records what each comparison holds fixed alongside any differences in scale, training data, or decoding budget.

Table 12. Continuous versus autoregressive head-to-heads (I): controlled generation, conditional transformation, captioning, planning, and code. "Matched" names what the comparison holds fixed and notes key protocol differences; "Dimension" names the primary design dimension stressed (Section 6.1). FlowTok is included as a diffusion-captioner reference row and marked accordingly; comparisons against masked models (DSL-LLaDA) appear in Table 17. AR means autoregressive, IT instruction-tuned, CNN/DM CNN/DailyMail, MOS mean opinion score, and CoLA a classifier's linguistic-acceptability score.

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
Syntax control (E2E)	Diffusion-LM, classifier guidance	FUDGE; task fine-tuned AR	data	syntax-tree F1 86.0 and span match 93.8% above both baselines	state	§6.1.1; [213]

Table 12 (continued)

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
Infilling (aNLG; ROCStories training)	Diffusion-LM	task-trained AR infiller	data	automatic scores competitive; no significant human preference	state	§6.1.1; [213]
Constraint following	Diffusion-LM initialized from T0-11B and fine-tuned on target text	InstructCTG (T0-11B with instructions and demonstrations)	T0-11B backbone, target data	T0-11B Diffusion-LM higher syntax score; InstructCTG lower OPT-30B perplexity	state	§6.1.1; [457]
Topic, sentiment, keyword control	Diffusion-LM (BERT-large, guided)	ChatGPT, zero- and few-shot	evaluation prompts only	zero-shot ChatGPT trails on topic, leads on sentiment and keywords; few-shot ChatGPT ahead on all three	state	§6.1.1; [343]
Translation (CoVoST De–En, reduced)	Diffusion-LM and DiffuSeq variants, random init	AR Transformer; Levenshtein Transformer	data	below both; sequence-level distillation helps DiffuSeq but does not close the gap	transition	§6.1.2; [82]
Translation (WMT14, IWSLT14; En–De, De–En)	SCLM (Difformer with spline-curve targets), 20 updates	AR Transformer, beam 5 (authors' runs)	data; SCLM uses distilled targets and length-and-noise beam selection	WMT14 SacreBLEU 28.19 / 33.49 vs 27.71 / 33.00; IWSLT14 28.10 / 32.56 vs 28.24 / 33.83	transition	§6.1.2; [144]
Sparse infilling (NarrativeQA)	YAN, 200M, four Euler updates, oracle lengths	fine-tuned BART, 139M	data, task fine-tuning, oracle sequence lengths; differs in model size and context length	ROUGE-L 93.9 vs 80.6	transition	§6.1.2; [196]
Paragraph completion (TripAdvisor)	PLANNER, latent plan + greedy GPT-2 medium decoder	fine-tuned GPT-2 large, sampled	data; differs in decoder size (GPT-2 medium vs GPT-2 large)	repeated 4-grams 1.55% vs 4.49%; human judges prefer PLANNER over GPT-2 on relevance, informativeness, human-likeness; human text preferred over PLANNER (no direct human–GPT-2 pair evaluated)	state	§6.1.4; [441]
Story generation (Cosmopedia, small models)	LCM, Two-Tower diffusion	token-level AR	data, small scale	ROUGE-L 33.64 vs sentence regression 23.69; token-level AR baseline leads both on ROUGE-L and coherence	state	§6.1.4; [190]
Summarization (CNN/DM, XSum)	Diffusion LCM, Two-Tower-7B-IT	Llama-3.1-8B-IT, Mistral-7B-v0.3-IT, Gemma-7B-IT	≈7B scale, instruction tuning; differs in pretraining data and compute	ROUGE-L above all three (CNN/DM 36.47 vs 34.97 for Llama-3.1-8B-IT; XSum 23.71 vs 20.35); CoLA fluency below all three (CNN/DM 0.767 vs 0.973); dataset-fine-tuned T5-3B higher on CNN/DM ROUGE-L (37.56)	state	§6.1.4; [190]

Table 12 (continued)

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
Summarization (XSum); dialogue (DeliData)	SLD	Flan-T5 Large	evaluation tasks only (instruction-tuned baseline)	Flan-T5 Large stronger on short XSum summaries; tied on DeliData coherence and non-hallucination	state	§6.1.4; [462]
Image captioning (COCO)	CoDi, sentence code	listed AR captioners	evaluation split only	above diffusion captioners compared in study; some AR systems stronger on particular measures	state	App. G.3; [350]
Image captioning (MS-COCO)	Analog Bits, image-conditioned	study's AR captioning baseline	data	caption measures improve from 5 to 20 updates but do not uniformly exceed the AR baseline	state	App. G.3; [64]
Image captioning (COCO Karpathy test)	LaDiC, 30 steps, BERT contextual state	published BLIP CapFilt-L	evaluation split; differs in pretraining data and architecture	BLEU-4 / CIDEr 38.2 / 126.2 vs BLIP CapFilt-L 39.7 / 133.3; A100 inference time (batch size unspecified) 0.020 s/img (5 steps) and 0.105 s/img (30 steps); A40 batch-256 latency 0.049 s/img	state	§6.1.3; [373]
Paragraph captioning (Stanford); short-caption human MOS (COCO)	LaDiC	fine-tuned BLIP-base (Stanford); BLIP (MOS, 10 COCO images, 20 raters)	task split and human rating protocol	Stanford paragraph BLEU-4 7.3 vs 6.1 (fine-tuned BLIP-base); 1–5 MOS accuracy 4.4 vs 4.2, conciseness 4.7 vs 4.4, fluency 4.5 vs 4.9	state	§6.1.3; [373]
Image captioning (COCO Karpathy)	FlowTok, direct transport	SCD-Net (diffusion captioner trained with AR teacher); CrossFlow	data; no CIDEr optimization	CIDEr 117.0 (FlowTok), 118.0 (SCD-Net), 116.2 (CrossFlow): numerically close among non-autoregressive captioners	state	§6.1.3; [141]
Image captioning (5,000 held-out pairs)	JUICE, 668M, Flan-T5 text states denoised with SigLIP2 image features	Transfusion-style AR text, same trunk	trunk, data, task ratios, schedule	CIDEr 88.80 vs 160.52; CLIPScore 80.20 vs 80.37; GenEval 48.41 vs 44.23	state	§6.1.3; [205]
Visual QA (GQA, VQAv2, MMBench, SEED-Bench)	MF-1, 1.6B, chunk-causal flow over T5 text-block states	Transfusion-style; Chameleon-style (fully discrete)	data, schedule, 1.6B trainable parameters	GQA 55.60 vs 52.81 / 45.83; VQAv2 69.03 vs 68.49 / 56.37; MMBench 46.74 vs 33.68 / 38.40; SEED-Bench 51.85 vs 31.31 / 42.40	state	§6.1.3; [351]
NL to code, single-output top-1 (Python, Bash, Excel)	CodeFusion, 75M	fine-tuned CodeT5, T5, CodeGen; 5-shot GPT-3, ChatGPT, StarCoder, CodeT5+	task training data for fine-tuned baselines; differs in size (75M vs 350M–175B)	top-1 close to CodeT5 and GPT-3: Python CodeBERTScore 80.7 vs 82.5 (GPT-3) / 80.5 (CodeT5); Bash template match 66.7 vs 67.6 (CodeT5); Excel execution match 72.8 vs 72.7 (CodeT5)	text interface	§6.1.5; [334]
NL to code, multi-candidate top-3 / top-5	CodeFusion, 75M (best of 3 or 5 samples)	same fine-tuned and 5-shot AR baselines (best of 3 or 5 samples)	candidate count ($k \in \{3, 5\}$)	exceeds all baselines on top-3 and top-5 candidate pools (top-5 Python CodeBERTScore 90.3 vs 85.8 for GPT-3 and 85.0 for CodeT5)	text interface	§6.1.5; [334]

Table 13. Continuous versus autoregressive head-to-heads (II): open-ended text, instruction following, reasoning and code, and likelihood. Each protocol variant (such as TESS 2's three baseline regimes, SSD-2's collaborative human evaluation versus standalone 13B GPT-4 judge evaluation, and LangFlow's ODE estimate versus RePlaid's variational bound) is listed in its own row. Conventions follow Table 12; likelihood comparisons are analyzed in Section 7.1. SFT is supervised fine-tuning, NLL negative log-likelihood, PPL perplexity, DDPM the ancestral update of Eq. 8, and NLD Nemotron-Labs-Diffusion.

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
Prompt completion (C4)	CDCD, 1.3B, seq. len. 256	matched-architecture AR Transformer, 1.3B, seq. len. 2048	parameter count (1.3B), 600k steps at 524,288 tokens/step; differs in seq. len. and separate sampling sweeps	MAUVE 0.953 vs 0.940	transition	§6.3; [87]
Unconditional generation & suffix infilling (human eval)	SED-L (420M; 135M SED-S also trained)	AR-L (420M)	parameter scale (420M), six human raters, each judging 20 sample pairs per comparison	SED-L human preference: 43.8% unconditional and 37.7% suffix infilling (AR-L preferred on the remainder)	state	§6.2; [341]
Open-ended continuation (OpenWebText)	SSD-LM, block generation	study's GPT-2 baselines	within-study evaluation protocol	favorable MAUVE and repetition combinations across decoding settings	transition (order)	App. G.4; [134]
Unconditional generation (OpenWebText)	BLD, 594M, 64 block latents per 1024 tokens	AR, 730M	data, tokenizer, ≈10.5B tokens	GenPPL / entropy / MAUVE 26.6 / 5.17 / 0.82 vs 26.5 / 5.37 / 0.92; 9.4 vs 1.5 documents/s at batch 32 (one H100, cached AR)	transition (order)	§5.3; [103]
Unconditional generation (OpenWebText)	PSF ($K = 8$, 197M); PSF-L (677M)	same-data AR, 197M	data, tokenizer; 4-gram ban on both for GenPPL	States-only GenPPL at entropy 5.16: 19.0 (PSF) and 16.4 (PSF-L) vs 15.2; separately measured MAUVE (first 512 tokens, Table 1 samplers) 0.85–0.93 vs 0.95	state	§3.3; [140]
Unconditional generation (OpenWebText)	ELF-M on distilled T5Gemma-2 states	GPT-2-M	best GenPPL at real-text entropy; not compute-matched	GenPPL 17.8 vs 20.8 at entropy 5.45; MAUVE 0.89 vs 0.91	state	§3.2; [442]
Collaborative instruction following (DOLLY)	SSD-2 collaboration (13B core + 0.1B user, simplex-logit models)	matched OPT collaboration pair	collaboration setup and base scale	collaboration improves automatic relevance and factuality over standalone SSD-2, unlike matched OPT collaboration, and wins human pairwise preference against OPT collaboration (45.7% win, 26.6% loss, 27.7% tie)	state	§6.1.6; [135]

Table 13 (continued)

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
Standalone instruction following (DOLLY, Vicuna)	SSD-2 13B (OPT-13B continued on ≈38B C4 tokens + DOLLY SFT)	instruction-tuned OPT-DOLLY-13B	13B backbone and DOLLY SFT data; evaluated by GPT-4 pairwise judge	GPT-4 prefers tuned OPT-DOLLY-13B over standalone SSD-2 13B on 58.6% of DOLLY prompts and 62.9% of Vicuna prompts	transition	§6.1.6; [135]
Countdown reasoning	LaDiR, sentence-level thought blocks	AR SFT baseline (masked MGDM also compared)	task data; pass@1 and multi-sample pool reported separately	single-sample pass@1 above AR SFT and below MGDM; with a large sample pool, higher success and more distinct valid solutions	transition	§6.1.6; [175]
Math and code, zero-shot (GSM8K, MATH-500, HumanEval, MBPP)	ELF-REG-L (652M + 157M decoder; frozen 0.6B encoder separate), task-trained; MATH initialized from GSM8K	Qwen3-0.6B-Base, untuned	same zero-shot template, 16 seeds; ELF early-stop at 64 NFE (GSM8K), 128 (others); task-training and response budgets differ	GSM8K 55.96 vs 39.53; MATH-500 13.39 vs 26.30; HumanEval 22.56 vs 22.98; MBPP-378 28.92 vs 40.62	transition	§6.1.6; [218]
Synthetic puzzles (Countdown, Sudoku, Knights-and-Knaves, Game24)	ELF-B (105M), SFT + pathwise RL (CFRL)	GPT-2 (124M), SFT + token-level GRPO/PPO	task data, SFT-then-RL pipeline, similar size	flow leads on Sudoku (68.7 vs 0.8) and Game24 (58.2 vs 46.7); AR leads on Countdown (60.9 vs 49.7) and Knights-and-Knaves (92.8 vs 77.5)	transition	§4.2; [137]
Instruction QA & suite, vs. directly fine-tuned AR (Tulu 2 SFT)	TESS 2 v0.1 (Mistral-7B-v0.1 adapted for 200k steps / ≈45B Dolma tokens + Tulu 2 SFT, 100 updates)	Mistral-7B-v0.1 with direct Tulu 2 AR SFT (no Dolma stage, autoregressive decoding)	initial Mistral-7B-v0.1 backbone and Tulu 2 SFT data; differs in intermediate Dolma adaptation and inference NFE	close on SQuAD F1 (85.4 vs 86.0) and TriviaQA (49.3 vs 50.4); directly fine-tuned AR leads on AlpacaEval (63.1 vs 77.1), IFEval (30.5 vs 36.8), BBH (8.4 vs 43.3), and untuned GSM8K (14.5 vs 52.5)	transition	§6.1.6; [344]
Instruction QA, vs. Dolma-adapted AR	TESS 2 v0.1 (Mistral-7B-v0.1, 200k-step / ≈45B-token Dolma adaptation + Tulu 2 SFT)	Mistral-7B-v0.1 AR baseline sharing the 200k-step, ≈45B-token Dolma continued-pretraining budget before SFT	backbone, Dolma step and token budget, SFT data	TESS 2 leads on SQuAD F1 (85.4 vs 71.4) and TriviaQA (49.3 vs 20.5)	transition	§6.1.6; [344]

Table 13 (continued)

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
Mathematics after GSM8K fine-tuning	TESS 2 v0.1 + dedicated symbolic-math GSM8K fine-tuning	Mistral-7B-v0.1 AR + same GSM8K fine-tuning	GSM8K fine-tuning set and initial Mistral-7B-v0.1 backbone; differs in pre-math adaptation history and inference updates	GSM8K accuracy 66.6% vs 51.2%	transition	§6.1.6; [344]
Math and code, few-shot (GSM8K, Minerva Math, HumanEval, MBPP, and plus variants)	Sigma-3B, 256 DDPM updates per 32-token block	its AR initialization (pretrained on 1T tokens)	architecture and parameter count; differs in Sigma's >300B further training tokens and 256-step block sampling	six-task pass@1 average 64.34 vs 64.73; ahead on GSM8K (84.50 vs 83.70) and HumanEval (59.45 vs 51.22); behind on Minerva Math (64.60 vs 66.28) and MBPP (52.32 vs 60.60)	transition	§6.1.6; [405]
Math and code, few-shot (same six tasks)	Sigma-8B, 256 DDPM updates per 32-token block	its AR initialization	architecture and parameter count; differs in >300B further training tokens and sampling budget	six-task average 69.67 vs 69.80; GSM8K 89.07 vs 85.90; HumanEval 61.40 vs 59.15; Minerva Math 67.47 vs 69.18; MBPP 64.78 vs 68.20	transition	§6.1.6; [405]
Instruction, math, code (IFEval, GSM8K, HumanEval)	Lafa 7B (Qwen2.5-7B-Instruct adapted), flow decoding, 4-token blocks	AR SFT of the same checkpoint on the same data	initialization, data, budget; flow adds a guidance call per step	three-task average 75.2 vs 73.9 at 32 Euler steps per block (71.2 at 8 steps; 66.5 with 16-token blocks)	transition (order)	§5.3; [56]
Same three tasks, end-to-end throughput	Lafa 7B, self-speculative decoding, 16-token blocks	AR SFT 7B, HuggingFace greedy with key–value cache	as above; one H100, batch size 1	average 74.5 vs 73.9 at 100.2 vs 39.5 tokens/s (2.54×; HumanEval 73.8 vs 78.7)	transition (order)	§6.3; [56]
Math (GSM8K after TinyGSM training)	ClockDLM, window $S = 32$, jointly trained AR + diffusion checkpoint, Cache Grab-SSD sampler	AR retrained on TinyGSM	training data; NFE counts every model call	pass@1 67.6 vs 64.3 at 35.5 vs 158.4 NFEs per sequence (diffusion-only standard model 59.6 at 188.1)	transition (order)	§5.3; [312]

Table 13 (continued)

Task (data)	Continuous system	AR baseline	Matched	Outcome	Dimension	Where
LM benchmarks, generated answers	Cola, ≈2.3B total (1.8B prior + 0.5B codec)	AR (and LLaDA), ≈1.8B backbones	training corpus, training steps, prior backbone size; Cola adds 0.5B codec stage	highest final eight-task average; best or near-best on MMLU, RACE, Story Cloze, OpenBookQA; LAMBADA close to AR; SQuAD overtakes AR late in training, below LLaDA	state	Table 17; [131]
Likelihood (E2E, small models)	Diffusion-LM	small AR model	architecture class	variational NLL upper bound 2.28 vs exact AR NLL 1.77 nats/token	transition	[213]
Likelihood (OpenWebText, 1M steps)	RePlaid ≈0.1B	AR Transformer (GPT-2 scale), same architecture	architecture, data, 1M training steps, batch 512	variational PPL upper bound 22.1 vs exact AR PPL 17.5 (SlimPajama IsoFLOP scaling fit estimates 20× compute to match exact AR NLL, or 27× without self-conditioning, vs 14× for MDLM)	transition	§7.1; [406]
Likelihood (OpenWebText, 1M steps)	ClockDLM, sliding window $S = 4$	AR Transformer	data, 1M steps; small windows approach AR factorization	variational PPL bound 19.5 vs exact 17.5 ($S = 16$: 20.6; BD3-LM, $S = 4$: 20.7; retrained RePlaid 21.3)	transition (order)	§6.3; [312]
Likelihood & zero-shot transfer (OpenWebText, LM1B, 7 transfer sets)	LangFlow 130M	AR Transformer, same architecture	architecture, data, 1M training steps, batch 512	self-reported OWT ODE PPL estimate 24.6 and RePlaid variational-bound re-evaluation 32.2 vs exact AR PPL 17.5; self-reported LM1B ODE PPL 30.0 vs 22.8; original uncorrected ODE estimator reports lower zero-shot PPL on 4 of 7 sets; transfer ranking not revalidated	transition	§7.1; [65, 406]
Zero-shot likelihood (six benchmarks)	Plaid 1B (1.3B parameters, 314B OWT2 tokens)	GPT-2 124M	evaluation benchmarks only (≈10× parameters; OWT2 scaling fit estimates ≈64× compute to match AR NLL)	lower variational PPL/BPC upper bounds than GPT-2 124M exact NLL on all six (WikiText103 PPL bound 28.28 vs exact 35.92)	transition	§7.1; [128]
Likelihood-ranked multiple choice (ARC-E/C, HellaSwag, PIQA, WinoGrande, MMLU)	Sigma-3B and Sigma-8B, candidates ranked by blockwise bound	their AR initializations (ranked by exact AR likelihood)	architecture and size; differs in Sigma's >300B further tokens and bound vs exact scoring	average accuracy 68.40 vs 71.52 (3B) and 72.28 vs 75.95 (8B); masked NLD scores 68.03 (3B) and 73.02 (8B)	transition	§7.1; [405]

Few-step operating points Table 14 compiles the reported few-step operating points alongside their network evaluation counts, pairing each generative perplexity with sample entropy or distributional similarity and noting how stochasticity enters. Figure 11 plots four representative within-study interventions. Table 14 separates few-step regimes that suffer mode-averaging entropy collapse from those that preserve sample diversity. Undistilled coarse Euler trajectories on fixed flow checkpoints, undistilled diagonal maps, and several one-step learned flow maps all collapse sample entropy when compressed to a single step [38, 287, 347]. CatFM uses Lagrangian self-distillation and reports one-step LM1B entropy 4.03 in its final published evaluation [304]. By contrast, flow maps distilled with progressive or trajectory-consistent semigroup targets (such as FMLM and DFM-PSD) maintain sample entropy close to their many-step teachers even in a single step [191, 287]. Without retraining, MCB preserves entropy on coarse grids by sampling categorical endpoints at each bridge step [38], and Untied Self-Conditioning repairs LangFlow's uncollapsed few-step deterministic ODE trajectory by correcting solver feedback, sharply lowering generative perplexity while keeping repetition below the many-step baseline [197]. In TTCD's prefix-conditioned continuation, shortcut consistency training likewise lowers few-step generative perplexity while preserving sample entropy (Table 14) [42]. Distribution matching, teacher couplings, and drifting shift these operating points without removing the trade-off, and the objectives that avoid collapse rely on an explicit anti-collapse mechanism that holds samples apart or anchors them to a teacher (Table 18) [95, 228, 358]. For example, Simplex-DMD beats its teacher at few-step budgets when evaluated at matched entropy but loses diversity if unrolled to many steps, and Reinforce-DMD collapses without its teacher KL anchor [358]. Similarly, ELF-MMD preserves data-level entropy across few-step budgets only when its repulsive kernel terms are active [95]. In single-step generation, GSF transfers an autoregressive teacher's Gumbel coupling, DBTM learns an autonomous map directly from data, and Seq-Drifting uses intra-sequence repulsion. DBTM already produces non-collapsed one-step samples; its anchor-time ablation links overly early target supervision to repetitive collapse, while additional refinement changes its quality–diversity trade-off (Table 14) [183, 228, 348]. For a discrete masked-diffusion contrast that uses noise-augmented masks and distilled two-time categorical draws rather than a continuous trajectory, see IMDM [418] in Appendix C.

Fewer denoiser evaluations do not automatically yield an end-to-end wall-clock latency advantage over an optimized autoregressive baseline. LD4LG is the only system in Table 14 reporting end-to-end wall-clock time per batch of outputs, and its runtime includes autoregressive decoding (Figure 12). DiLaDiff compresses its latent-prior runtime through distillation only to become bottlenecked by its multi-step discrete decoder [195], and PLANNER's latency advantage reverses once its autoregressive baseline uses length sorting and batching [441]. Similarly, BiMol-Diff's higher-quality sampling settings remain slower than its autoregressive baselines in batched timing while reducing refinement steps removes its quality margin [320], and Du and Ma show that a literal early-exit monitoring implementation increases elapsed time despite saving denoiser calls [97]. Standalone continuous generators have not yet surpassed optimized autoregressive baselines at matched quality. Instead, the two systems reporting matched-quality wall-clock speedups both pair continuous drafting with autoregressive verification. Lafa commits one-step flow drafts only where they agree with the same model's greedy next-token head, achieving multi-fold single-stream throughput gains at comparable average accuracy across instruction following, math, and coding over key–value-cached greedy decoding [56]. FlashMap conditions a flow-map drafter on the hidden states of an autoregressive target to propose multi-token blocks in one or two steps, delivering multi-fold wall-clock speedups at concurrency one while preserving the target's exact output distribution [326]. Clock Diffusion similarly reduces total network evaluations per sequence via self-speculative window sampling and exceeds a retrained autoregressive baseline on mathematical reasoning [312]. Compressed-latent systems with parallel token decoders (BLD, JPEG-DLM, and PSF) achieve higher batched document throughput than key–value-cached autoregressive baselines, trading this throughput for lower distributional similarity (MAUVE) or slightly higher generative perplexity at matched entropy [103, 140, 426]. Table 16 records the extra network calls and decoding stages that nominal step labels omit.

Table 14. Few-step operating points. GenPPL uses each paper's external scorer (GPT-2 Large unless stated) and is paired with unigram entropy (data entropy 4.31 on LM1B and 5.44 on OWT in FMLM's evaluation); n/a means the source reports no value. The second column separates denoiser steps, auxiliary network predictions, and decoder passes from non-network readout rules such as vocabulary argmax, and the third column states where randomness enters. NFE counts network function evaluations, OWT is OpenWebText, NLL is negative log-likelihood, Rep-4 the four-gram repetition rate, and OU Ornstein–Uhlenbeck; ECLD, CSD, PSD, and ESD are defined in Section 5.1.

Method (data)	NFE and extra calls	Randomness; final decision	GenPPL / entropy, or task metric	Time	Status and note
LangFlow / INCO (OWT)	1024 Euler updates; terminal prediction excluded from update count (total NFE unspecified)	deterministic ODE; argmax	36.7 / 5.25; with INCO 28.1 / 5.18	n/a	many-step reference [298]
Untied Self-Conditioning on LangFlow (OWT)	8, incl. terminal prediction	deterministic ODE; argmax	$531.2 \pm 3.4 \rightarrow 61.6 \pm 0.2$ / $5.81 \rightarrow 5.33$; Rep-4 $0.11 \rightarrow 1.82$ ($\times 10^{-3}$)	n/a	inference-only; baseline is not collapsed [197]
Simplex-DMD; Reinforce-DMD, distilled from LangFlow (OWT)	2; 4; 8; 16 (Simplex); 128; 256 (Reinforce)	fresh Gaussian re-noising at each step; categorical decoder	Simplex-DMD 178 / 5.65 (2), 45.6 / 5.44 (4), 26.7 / 5.38 (8), 17.0 / 5.20 (16), 3.60 / 3.82 (128), 2.51 / 3.31 (256); Reinforce-DMD 26.9 / 5.27 (128), 14.9 / 5.00 (256); LangFlow teacher at matched entropy 317 (4), 21.3 (256)	n/a	single seed; temperature-selected points; baselines interpolated to each budget's target entropy [358]
TTCD, without and with shortcut (OWT, 128-token continuation, temperature $T = 1$)	4 or 8 nominal token-time updates + initial rank-assignment pass and terminal prediction (followed by argmax readout)	initial Gaussian noise and uniform ranks; deterministic updates, vocabulary argmax	$1033.8 \rightarrow 437.82$ / $4.39 \rightarrow 4.38$ (4 nominal); $576.86 \rightarrow 371.18$ / $4.42 \rightarrow 4.38$ (8 nominal)	n/a	prefix-conditioned continuation [42]
CatFM, one map step (Text8 ECLD; LM1B CSD)	1	deterministic map; terminal argmax or draw	external NLL 5.33 (Text8, GPT-J-6B); GenPPL 142.7 / 4.03 (LM1B, GPT-2 Large)	n/a	self-distilled; final ICML results [304]
DFM, one map step (LM1B; OWT)	1	deterministic map; terminal readout	PSD 94.08 / 4.06 (LM1B); 180.29 / 4.91 (OWT). ESD 68.11 / 3.79 (LM1B); 5.33 / 0.26 (OWT). Undistilled diagonal 2.05 / 0.76 (LM1B); 29.79 / 1.55 (OWT)	n/a	self-distilled; ESD and diagonal collapse, PSD does not [287]
Self-conditioned FMLM, one map step (OWT)	1	deterministic map; shared decoder head	$176.31 \rightarrow 112.52$ / $5.47 \rightarrow 5.37$ across online and two-stage offline distillation (arXiv v1; Table 15)	n/a	distilled from teacher refinements [417]
FMLM, distilled one-hot map (OWT; LM1B)	1; 2; 4 map steps	deterministic map; argmax	OWT 168.3 / 5.17 (1), 133.29 / 5.25 (2), 111.31 / 5.26 (4); LM1B 119.34 / 4.16 (1); FLM teacher, 1024 steps: 62.23 / 5.33 (OWT)	n/a	distilled from the FLM teacher [191]

Table 14 (continued)

Method (data)	NFE and extra calls	Randomness; final decision	GenPPL / entropy, or task metric	Time	Status and note
GSF, distilled from an AR teacher (OWT; LM1B)	1; 2; 4 map steps	Gaussianized Gumbel noise; deterministic map; argmax	OWT 114.9 / 5.07 (1), 105.6 / 5.30 (2), 99.7 / 5.32 (4); LM1B 118.6 / 4.05 (1); AR teacher, 1,024 steps: 46.5 / 5.59 (OWT)	n/a	167.8M OWT student initialized from the teacher; LM1B uses a smaller vocabulary [183]
DBTM, refinement-trained (LM1B; OWT)	1; 2; 4 rounds, one call each	re-noising of uncommitted positions; argmax	LM1B 85.50 / 3.92 (1), 81.00 / 4.05 (2), 73.54 / 4.04 (4); OWT 65.2 / 4.95 (1), 62.2 / 5.38 (2), 57.5 / 5.52 (4); with FMLM's attention backbone 133.0 / 4.94 (1)	n/a	teacher-free; OWT rows use a linear-attention backbone of matched size; entropy can exceed the data's 5.44 [348]
Seq-Drifting (LM1B; OWT)	1 (no sampler)	Gaussian input vector; nearest token by cosine	corpus-trained support model: 93.81 / 4.21 (LM1B), 183.98 / 4.95 (OWT); public GPT-2 support: 36.01 / 4.42, 128.33 / 5.08	latency 10–11× below four-step FMLM	no diffusion teacher; public support model shares the scorer's GPT-2 family [228]
Lafa 7B flow map, 4-token blocks (IFEval, GSM8K, HumanEval)	1; 2; 4 map steps per block	deterministic map; argmax	average accuracy 49.0 / 63.3 / 69.4 vs 43.7 / 46.2 / 64.3 for the undistilled flow (flow at 64 steps 75.9; autoregressive fine-tuning 73.9)	n/a	distilled; no self-conditioning or prefix guidance [56]
ELF-B, 105M (OWT)	32 + final decode call	SDE-type sampler; shared decoder call	24.08 / 5.15	n/a	six evaluation seeds [153]
StepTune grid on ELF-B (OWT)	8; 16; 32	SDE-type sampler on a frozen calibrated grid	69.17 → 41.26 / 5.23 → 5.23 (8); 32.56 → 26.06 / 5.17 → 5.11 (16); 24.02 → 20.70 / 5.16 → 5.11 (32)	n/a	inference-only; native grid as baseline; six runs [450]
ELF-MMD on ELF-B (OWT, T5 latents)	2; 4; 8 generator passes + decode call	one noise draw; self-conditioned passes at a fixed time	192.28 / 5.53 (2), 110.07 / 5.49 (4), 56.63 / 5.45 (8); with refinement distillation 47.33 / 5.39 (8); same-study ELF 91.77 / 5.38 (8)	n/a	feature-space MMD post-training; five seeds; compare within study [95]
JPEG-DLM, $N/L = 0.25$ (LM1B)	8; 16; 32 Euler steps + a $t = 1$ pass and a decoder pass	deterministic ODE; greedy argmax	225.24 / 4.162; 137.86 / 4.270; 96.76 / 4.263 (truncated MAUVE 0.432; 0.896; 0.951)	9.13× AR without key–value cache (32 steps)	compressed latent, 32 positions per 128 tokens; five seeds [426]
EFlow / EFM (LM1B)	1024; 64; 4; 2; 1 nominal	noise-injected flow or map; argmax	EFlow 103.63 / 4.16 (1024), 133.62 / 4.14 (64); EFM 99.85 / 3.93 (4), 125.95 / 3.95 (2), 98.35 / 3.56 (1)	n/a	EFM distilled from FLM; one-step collapse stated by source [347]
MCB on fixed FLM checkpoints (LM1B, OWT)	coarse grids	Euler ODE vs. OU bridges with endpoint draws	coarse Euler: low GenPPL at collapsed entropy; bridges retain entropy; endpoint temperature trades both	n/a	curves only [38]
LaDiff teacher (OWT)	200 latent updates + 1024 decoder steps	latent ODE; masked-decoder draws	62.9 / MAUVE 0.82 vs masked baseline 90.5 / 0.74	n/a	quality setting [195]

Table 14 (continued)

Method (data)	NFE and extra calls	Randomness; final decision	GenPPL / entropy, or task metric	Time	Status and note
DiLaDiff student (OWT)	5 latent steps (10 calls) + 64 decoder steps	latent map; masked-decoder draws	student below teacher	latent-to-decoder time ratio 1.116 → 0.050	timing setting, batch 32; decoder-bound [195]
LD4LG, BART codec (ROCStories)	50; 250; 1000 + AR decoding	latent sampler; AR decoder	52.6 / MAUVE 0.684; 30.6 / 0.716; 25.9 / 0.667	87; 200; 630 s per 1,000 outputs	time includes AR decoding (Figure 12) [239]
CRoCoDiL prior in LLaDA-8B (Python, length 512)	40 demasker-equivalent vs 512	continuous registers; masked demasking	MAUVE 0.60 / GenPPL 14.3 vs baseline 0.62 / 19.4	n/a	parameter-weighted proxy [357]

Table 15. One-step OpenWebText generation in self-conditioned FMLM, using the results reported in arXiv v1, as a function of the number of fixed-point refinements used to build distillation targets [417]. External scorer GPT-2 Large. More refinements lower perplexity at nearly constant entropy and then plateau; training a separate fixed-point teacher first improves further.

Distillation route	Training refinements	GenPPL ↓	Token entropy
Online	1	176.31	5.47
Online	9	116.74	5.39
Online	17	118.26	5.38
Offline, two stages	Separate fixed-point teacher	112.52	5.37

Model scale and training regimes Table 17 lists the parameter counts, training data, and baselines of the continuous systems that report them. The largest continuous model trained from scratch is the sentence-level Two-Tower LCM, pretrained on multi-trillion-token corpora encoded as SONAR sentence embeddings [190]. Token-level continuous models trained from scratch reach up to roughly 4B parameters (including Sigma's from-scratch training-loss ablation [405]): representative systems include Scaling CatFM [80], CDCD, Plaid, SED, the compressed-code generator Cola, and the unified multimodal flows MF-1 and JUICE [205, 351]. In a controlled design-space study at 1.7B parameters, Davis et al. train a one-hot flow from scratch and compare it with published masked (MDLM) and uniform-state (Duo) baselines of the same size, corpus, and token budget. When scoring answers by continuation cross-entropy rather than a likelihood bound, the pure one-hot flow falls behind both discrete counterparts across multiple-choice benchmarks, whereas its hybrid SemiCat variant (Appendix C), which reveals clean tokens at random times as CANDI does, leads or ties on the majority of those tasks [81, 291]. Hyperspherical SwaFlow models trained from scratch up to 3.741B parameters improve in held-out loss and unconditional generation with scale. Under the paper's clean-prompt scoring protocol, their average multiple-choice accuracy remains near or below chance until clean-prefix examples enter late training [17].

Other token-level systems at 3B–13B scales adapt existing autoregressive or discrete masked backbones. SSD-2 continues pretraining OPT checkpoints before instruction tuning [135], TESS 2 adapts Mistral-7B to simplex diffusion [344], and DSL-LLaDA adapts LLaDA-8B-Instruct with continuous noise across few- to medium-step budgets [420]. Sigma continues 3B and 8B autoregressive checkpoints with joint autoregressive and blockwise-diffusion training. With DDPM sampling at 256 steps per block, its six-task mathematical and coding pass@1 averages closely match compute-matched masked diffusion (NLD) and the corresponding autoregressive initializations, and exceed those of LLaDA-8B [268] and

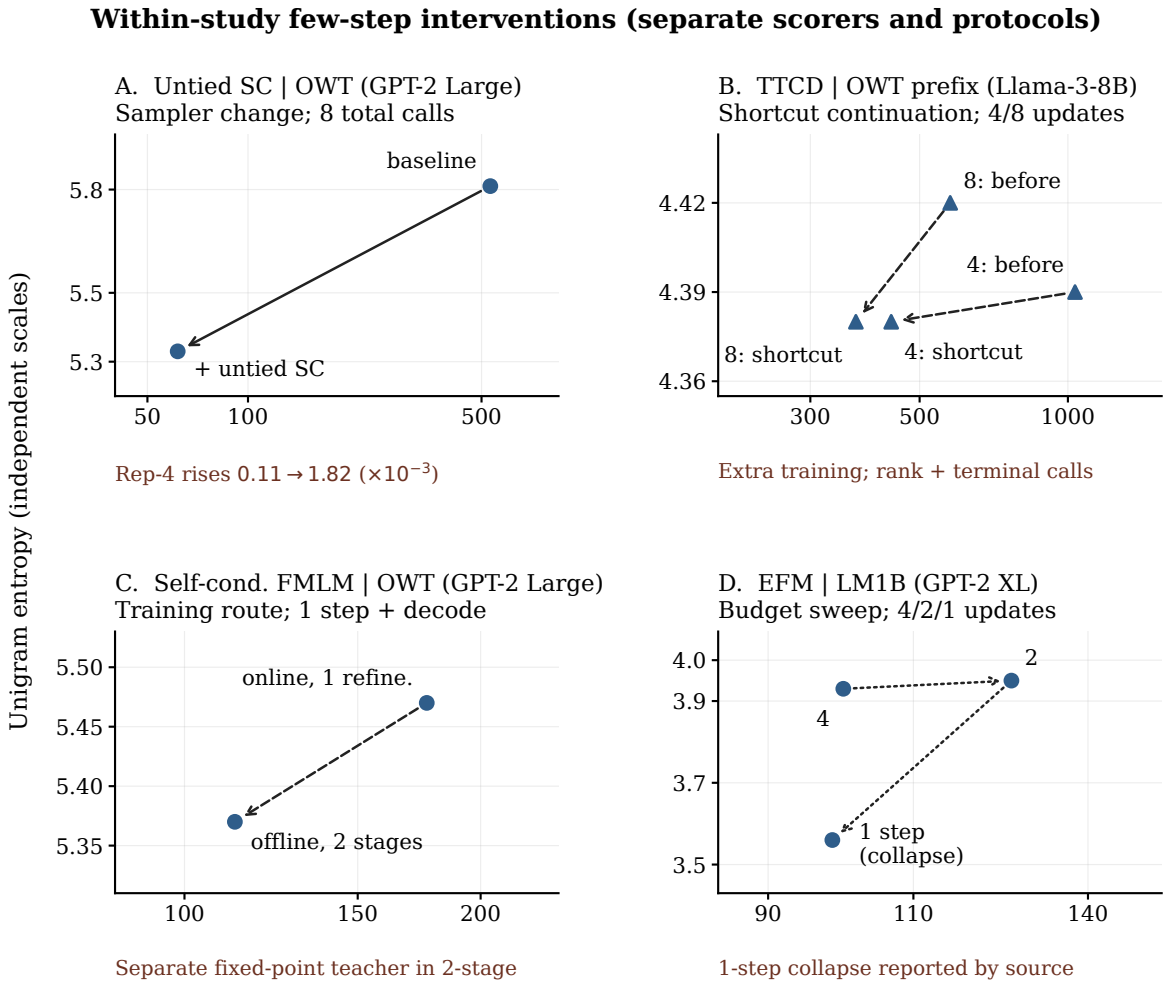


Figure 11. Within-study few-step interventions from Table 14: inference-only Untied Self-Conditioning on LangFlow at 8 NFE (A) [197], TTCD shortcut continuation at 4 and 8 nominal token-time updates on OpenWebText prefix continuation (B) [42], one-step self-conditioned FMLM across online refinement and two-stage distillation from arXiv v1, reproduced in Table 15 (C) [417], and the EFlow / EFM step sweep on LM1B showing one-step entropy collapse (D) [347].

Dream-7B [408] in the reported evaluation [405]. BD3-LM provides a related blockwise discrete formulation [36]. Subsequent blockwise conversions require smaller adaptation budgets. Lafa adapts pretrained 7B–8B instruction and base checkpoints with modest instruction token budgets and surpasses matched autoregressive fine-tuning when evaluated through its retained next-token head [56]. Similarly, LLaCO adapts an 8B autoregressive–diffusion hybrid backbone on instruction data to beat earlier adapted continuous baselines across knowledge, math, and coding suites [164], and 7B autoregressive checkpoints can be converted to blockwise semi-continuous generation under a lightweight token budget [15].

At the 0.1B scale where held-out likelihoods are compared under matched training steps, continuous variational bounds and corrected ODE estimates narrow the gap to discrete masked and uniform-state diffusion while remaining above exact autoregressive Transformer perplexity (Appendix D.1) [65, 406]. Semi-autoregressive factorizations tighten the continuous bound further: under the same training protocol, Clock Diffusion's sliding-window model achieves a lower variational perplexity bound than both full-sequence continuous diffusion and blockwise masked diffusion as the window size shrinks toward an

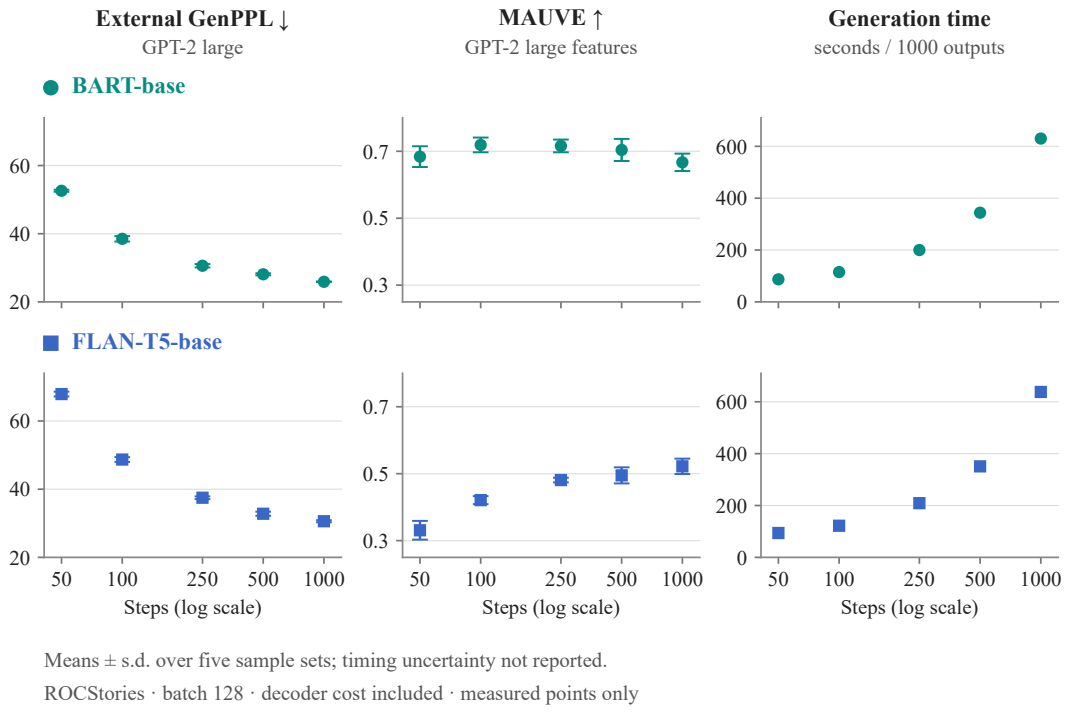


Figure 12. LD4LG's ROCStories sampling-budget sweep for two codecs [239]. External perplexity (GPT-2 Large) keeps improving with more updates, but BART-codec MAUVE peaks near 100 updates while time per 1,000 outputs, which includes autoregressive decoding, grows sevenfold. Error bars: standard deviations over five sample sets.

autoregressive factorization [312]. Across multi-scale IsoFLOP and scaling-law fits, Plaid and RePlaid show that continuous diffusion likelihood bounds improve at a slope comparable to autoregressive models while requiring a large compute multiple to match exact autoregressive negative log-likelihood, with self-conditioning narrowing the gap relative to discrete masked diffusion at larger compute budgets [128, 406]. Section 7.1 examines the mechanisms behind this likelihood and compute gap.

Table 16. Nominal sampling budgets and additional network or decoding stages across representative end-to-end pipelines (NFE denotes network function evaluations).

Configuration	Budget label	Calls and stages outside the label
TTCD	4 and 8 global token-time updates	the algorithm also runs an initial forward pass for entropy-based rank assignment, local-time updates, and a terminal network prediction before vocabulary argmax readout [42]
DSL hybrid	continuous and discrete stage counts	the Heun stage, the projection to sampled tokens, and targeted remasking are separate stages; the projection call is not in the step sum [384]
DSL-LLaDA	NFE 8, 16, 32, and 64	the label counts the paired Heun evaluations; terminal decoding is an additional call [420]
Sigma	updates per 32-token block	each block's final readout call is not counted; with guidance, each call packs a conditional and an unconditional branch (4.2% more time per call at batch size 1 on one H100); in its four-NFE setting, the Sigma-3B PDD distilled sampler uses 4 student backbone calls, each emitting multiple block predictions composed along the teacher grid, and folds guidance into one call [405]
Lafa	tokens per forward pass; tokens per second	each fused verify-and-redraft pass carries the B-token draft and B noisy successor blocks, one per possible acceptance length ($B(B+1)$ positions); tokens per second are measured end to end on one H100 at batch size 1 against HuggingFace greedy autoregressive generation; flow decoding without verification instead adds one prefix-free guidance call per Euler step [56]
Clock Diffusion (Cache Grab)	NFEs per sequence	the count already includes prompt ingestion, key–value cache updates, final readout calls, and the autoregressive verification call that each Cache Grab-SSD iteration adds to its denoising call; per-call cost differs from an autoregressive step, and no wall-clock time is reported [312]
FlashMap	speedup over the target's own autoregressive decoding, with 1 or 2 flow-map steps per 16-token draft	each round adds one verification pass of the target, and the drafter reads hidden states of five target layers; timing is end to end in SGLang at concurrency 1 on one H200 GPU, batched serving is not evaluated, and the output distribution equals the target's [326]
CRoCoDiL	parameter-weighted demasker-equivalent NFE	the 400M continuous denoiser is converted to 8B demasker activations; refresh-encoder and prefix-attention work are not timed [357]
DiLaDiff	latent-to-decoder wall-time ratio	batch 32 against a fixed 64-step discrete decoder; each latent update makes two network calls (velocity and self-conditioning) [195]
LD4LG	seconds per 1,000 outputs	time includes reconstruction and autoregressive decoding [239]
PLANNER	generation time against a fine-tuned GPT-2	the comparison reverses once the autoregressive baseline is batched with length sorting [441]
JPEG-DLM	32 Euler steps	33 denoiser evaluations (the extra call is the $t = 1$ clean prediction passed to the decoder) plus one decompressor and token-projection pass; throughput is the median FP32 rate at batch 64 (LM1B) or 16 (OpenWebText) relative to an autoregressive model without key–value cache, which the cache speeds up 1.62× and 7.14× [426]
BLD	50 SDE-style steps per latent group	eight sequential groups, each with one block-causal backbone pass and 50 flow-head passes (400 head passes in all), then about 16 parallel branch-decoding steps with per-branch key–value caches; runtimes compare fp32 BLD with bf16 ELF and an fp32 key–value-cached autoregressive model on one H100 [103]
PSF ($K = 8$, plain/code-augmented)	50 prior Euler steps	plus $L/K = 128$ parallel block-decoding steps: 178 serial network evaluations per 1024-token document against 33 for 32-step ELF-B and 1024 for autoregressive decoding; 27.3 ms per document at batch 32 against 20.9 ms for 32-step ELF-B on one H100, with CUDA graphs for PSF and the same-data autoregressive model [140]
Du and Ma early exit	denoising evaluations saved	the literal monitoring implementation increases elapsed time [97]

Table 17. Model scale and training regimes for continuous-space diffusion and flow language models. Every row is a continuous-space diffusion or flow language model. The largest continuous model trained from scratch is the 7B sentence-level Two-Tower LCM. Token-level continuous models trained from scratch include Scaling CatFM (1.7B), CDCD and Plaid (1.3B), Cola (2.3B with codec), Sigma's nominal 3B training-loss ablation, and SwaFlow (3.741B including embeddings), while the other 3B–13B token-level rows (Sigma, Lafa, LLaCO, TESS 2, DSL-LLaDA, SSD-2) adapt existing autoregressive or masked backbones. NLD is Nemotron-Labs-Diffusion, a blockwise masked model, and DDPM the ancestral update of Eq. 8. External discrete masked-diffusion references compared in the Baseline and Result columns include MDLM [308], LLaDA-8B-Base [268] (published Table 1: GSM8K 70.3, HumanEval 35.4, MBPP 40.0), and Dream-7B-Base [408] (GSM8K 77.2, HumanEval 57.9, MBPP 56.2). Parameter counts and data are those stated by each paper; n/a means the paper does not state the value. PPL is perplexity, BPC bits per character, NFE the number of network evaluations, OWT OpenWebText, AR autoregressive, IT instruction-tuned, MC QA multiple-choice question answering, CNN/DM CNN/DailyMail, and CoLA a classifier's linguistic-acceptability score.

System	Parameters	Training data	Baseline	Metric	Result	Source
Plaid 1B	1.3B	314B tokens, OpenWebText2 (2.5×10^{21} FLOPs)	GPT-2 124M	zero-shot variational PPL/BPC upper bounds vs exact AR NLL, six benchmarks	lower PPL/BPC upper bounds than GPT-2 124M's exact values on all six; OWT2 scaling fit estimates ≈64× compute for bound to match exact AR NLL	[128]
Scaling CatFM	1.7B	Vocab. simplex flow map (2.1T tokens; 4-step generation after self-distillation)	1.7B discrete / AR refs.	Zero-shot MC QA via MC ELBO	ARC-E, BoolQ, OBQA, PIQA, RACE, SIQA; below MDLM and Duo on all six tasks	[80]
CDCD	1.3B	C4, 600k steps at 524,288 tokens/step (seq. len. 256)	matched-architecture AR 1.3B, 600k steps (seq. len. 2048)	C4 prompt-completion MAUVE	0.953 vs 0.940 (separate sampling sweeps)	[87]
SED	135M (S); 420M (L)	C4	AR-L (420M)	six raters, 20 sample pairs each per comparison (unconditional; suffix infilling)	SED-L preferred on 43.8% (unconditional) and 37.7% (suffix infilling) vs AR-L	[341]
RePlaid	≈0.1B (MDLM / Duo architecture)	OWT, 1M steps, batch 512 (scaling fit on SlimPajama)	AR Transformer 17.5; MDLM 23.1; Duo 25.2	OWT variational PPL upper bound; SlimPajama IsoFLOP scaling multiple	OWT PPL bound 22.1; SlimPajama IsoFLOP fit estimates 20× (27× without self-conditioning, vs 14× MDLM) compute to match exact AR NLL; lower loss than MDLM past compute-optimal budgets	[406]

Table 17 (continued)

System	Parameters	Training data	Baseline	Metric	Result	Source
LangFlow	130M	LM1B and OWT, 1M steps, batch 512	AR 22.8 / 17.5; MDLM 31.0 / 23.2	LM1B / OWT PPL (ODE estimate vs variational bound); GenPPL	self-reported ODE PPL estimate 30.0 / 24.6; RePlaid variational-bound re-evaluation on OWT 32.2; GenPPL 92.2 / 36.5; original uncorrected ODE estimator reports lower zero-shot PPL on 4 of 7; transfer ranking not revalidated	[65, 406]
ELF	105M (B); 342M (M); 652M (L)	OWT (≈9B tokens): 5/4/3 epochs for B/M/L; MT: 100 epochs	MDLM, Duo, FLM, LangFlow (≈170M); AR 99M (MT)	GenPPL / entropy at 32 updates; De–En BLEU	24.08 / 5.15 (ELF-B); 26.4 vs 25.2	[153]
CodeFusion	75M	code denoising pretraining + task fine-tuning (Python, Bash, Excel)	fine-tuned CodeT5, T5, CodeGen; 5-shot GPT-3, ChatGPT, StarCoder, CodeT5+ (350M–175B)	Python CodeBERTScore, Bash template match, Excel execution match (top-1 vs top-3/top-5)	top-1 close to CodeT5 / GPT-3 (Python 80.7 vs 80.5 CodeT5 / 82.5 GPT-3; Bash 66.7 vs 67.6 CodeT5; Excel 72.8 vs 72.7 CodeT5); top-3/top-5 pools exceed all baselines (top-5 Python 90.3 vs 85.8 GPT-3 / 85.0 CodeT5)	[334]
S-FLM	n/a (12 layers, width 768)	TinyGSM; OWT	MDLM, Duo; AR	GSM8K accuracy	18.0 (top-1 velocity) = MDLM 18.0 at $T = 1$; at $T = 0.1$ MDLM 33, Duo 36, S-FLM ≈18; AR 53.9 sampled, 63.3 greedy; BCT's tempered-bridge continuation with bridge-consistent tempering 27.0 / 26.0 at $T = 0.1$ (matched continuation with standard tempering 17.5 / 17.3)	[83, 272]
SwaFlow	0.175–3.741B (hyper-spherical; incl. embeddings)	Dolma 3 Mix: 2T-token ladder; 4T final run	AR twins (0.161–1.435B)	held-out CE; GenPPL; multiple-choice accuracy	at 2T, mean MC accuracy 0.354–0.369 under uniform corruption, 0.403–0.434 after 100B clean-prefix tokens (chance 0.375)	[17]
Diffusion LCM, Two-Tower-7B-IT	7B (sentence-level Two-Tower diffusion)	2.7T tokens (2.3B documents) as SONAR sentence embeddings; instruction tuning	Llama-3.1-8B-IT, Mistral-7B-v0.3-IT, Gemma-7B-IT	ROUGE-L; CoLA fluency (CNN/DM, XSum)	ROUGE-L above all three (CNN/DM 36.47 vs 34.97 for Llama-3.1-8B-IT; XSum 23.71 vs 20.35); CoLA below all three (CNN/DM 0.767 vs 0.973)	[190]

Table 17 (continued)

System	Parameters	Training data	Baseline	Metric	Result	Source
SSD-2	0.1B; 2.7B; 13B (OPT-13B adaptation)	≈38B C4 tokens (13B model) continued pretraining + DOLLY SFT	matched OPT collaboration; standalone tuned OPT-DOLLY-13B	human preference (collaboration); GPT-4 pairwise judge (standalone 13B on DOLLY, Vicuna)	collaboration beats OPT collaboration in human preference (45.7% win, 26.6% loss, 27.7% tie); standalone 13B trails tuned OPT-DOLLY-13B in GPT-4 preference (58.6% DOLLY, 62.9% Vicuna)	[135]
TESS 2	7B (Mistral-7B-v0.1 adaptation)	200k-step, ≈45B-token Dolma adaptation + Tulu 2 SFT (+ GSM8K math SFT)	directly fine-tuned, Dolma-adapted, and GSM8K-tuned Mistral-7B-v0.1	SQuAD F1, TriviaQA, AlpacaEval, IFEval, BBH, GSM8K	vs directly fine-tuned AR: SQuAD 85.4 vs 86.0, TriviaQA 49.3 vs 50.4, trails on AlpacaEval (63.1 vs 77.1), IFEval (30.5 vs 36.8), BBH (8.4 vs 43.3), and untuned GSM8K (14.5 vs 52.5); vs Dolma-adapted AR: leads SQuAD 85.4 vs 71.4, TriviaQA 49.3 vs 20.5; GSM8K math-tuned accuracy: 66.6% vs 51.2%	[344]
DSL-LLaDA	8B (LLaDA-8B-Instruct adaptation)	continued pretraining with continuous noise (tokens n/a)	LLaDA-8B-Instruct	ROUGE-1 at 8, 16, 32, and 64 NFE	ahead on XSum and CNN/DM at 8, 16, 32, and 64 NFE; ahead on PubMed and arXiv at 8 and 16 NFE; split at 32 NFE (PubMed 36.9 vs 36.5, arXiv 32.9 vs 33.8); behind on both at 64 NFE	[420]
Sigma-3B	3.8B (3.0B non-embedding)	AR initialization pretrained on 1T tokens, then >300B tokens of joint AR and diffusion training	NLD-3B (same data, backbone, size); LLaDA-8B; Dream-7B; AR initialization	pass@1, six math and code tasks; 256 DDPM updates per 32-token block	average 64.34 vs NLD 64.74 and AR initialization 64.73; GSM8K 84.50 vs 83.40, 70.96, 77.18; MBPP 52.32 vs 57.60, 40.80, 56.80	[405]
Sigma-8B	8.5B (7.4B non-embedding)	AR initialization (tokens n/a), then >300B tokens of joint training	NLD-8B; AR initialization	as Sigma-3B	average 69.67 vs NLD 70.22 and AR initialization 69.80; GSM8K 89.07 vs 87.26 and 85.90	[405]
Lafa	0.5B; 7B; 8B (Qwen adaptations)	9–27B instruction tokens (Dolci-Instruct-SFT)	AR SFT on the same data	six-task accuracy	next-token-head average 31.6, 60.9, 63.1 vs 28.7, 58.4, 56.8 (8B gap mostly IFEval); throughput in Tables 13 and 16	[56]

Table 17 (continued)

System	Parameters	Training data	Baseline	Metric	Result	Source
LLaCO	8B (NLD-8B adaptation)	≈1.69B instruction tokens	TESS 2, DSL-LLaDA, Dream-7B; not its NLD-8B initialization	nine knowledge, math, and code tasks (authors' evaluation)	above both continuous baselines on all nine (GSM8K 80.74 vs 25.40 for DSL-LLaDA); vs Dream-7B: MATH 62.12 vs 51.20, HumanEval 48.17 vs 54.27	[164]
Cola	≈2.3B (1.8B prior, 0.5B codec)	from scratch; same corpus and tokens as its baselines (size n/a)	AR and LLaDA, ≈1.8B backbones	generated-answer accuracy, eight benchmarks	best final Task Average; SQuAD overtakes AR late, below LLaDA	[131]
One-hot flow	1.7B	from scratch; 2.1T Nemotron tokens	MDLM, Duo, hybrid SemiCat (same data and tokens); AR	six multiple-choice tasks (continuation cross-entropy, no bound)	one-hot flow below Duo, MDLM, and hybrid SemiCat (Appendix C) on all six; AR higher on all six	[81]
AURORA-S	130M denoiser (268M system)	OWT	AR, MDLM, SEDD, Duo, ELF-B	GenPPL / MAUVE, 64 SDE-DPM++ steps/block	24.33 / 0.9016 at SC-CFG 3	[220]
Cola2	≈4B prior (from scratch); Qwen3-4B-Base-initialized causal decoder	346.03B source tokens	Llama-3-8B, Qwen2.5-7B, Qwen3-8B-Base, LLaDA-8B-Base (not compute-matched)	generated answers, eight tasks	average 60.10 vs 66.25, 75.08, 75.87, and 72.11	[132]
MF-1	1.6B backbone	from scratch; 150B + 5B tokens (70% text-only)	Transfusion- and Chameleon-style 1.6B models (matched)	visual QA; GenEval	matched comparison in Table 12; main model VQAv2 72.6, GenEval 0.82	[351]
JUICE	2.1B backbone	from scratch; image–caption pairs + Cambrian-7M	Transfusion-, D-DiT-, MMaDA-style on a matched 668M trunk	captioning CIDEr; GenEval; VQAv2	matched comparison in Table 12; 2.1B: GenEval 0.87–0.88, VQAv2 73.1	[205]
Mimir	1.6B (sentence-level)	from scratch; SONAR embeddings of 38.9B sentences, 46 languages	Qwen3-1.7B	ROUGE-L (XL-Sum; MLQA)	20.73 vs 10.97; 36.12 vs 36.71; Mimir's instruction data include both training splits	[264]

Controlled ablations and mechanism-level evidence Table 18 summarizes controlled within-study ablations across the three design dimensions, recording for each intervention the experimental modification, the measured effect, and the observed outcome. On the state and text-interface side, TEncDM isolates decoder robustness on held-fixed generated states: retraining only the bidirectional decoder with corrupted contextual encodings raises distributional similarity markedly [317]. Together with TextLDM's REPA alignment [166] and LDLM's decoder-noise ablations [253], these results show that generation quality depends on how the codec and decoder handle prior-generated states rather than clean-text reconstruction accuracy alone. On the transition side, training-free sampler modifications shift few-step quality and diversity on fixed checkpoints either by injecting endpoint stochasticity (MCB) or by correcting deterministic self-conditioning feedback (Untied Self-Conditioning), while trajectory-consistent distillation

(self-conditioned FMLM, DFM-PSD) supports one-step generation without entropy collapse.

Table 18. Summary of controlled mechanism-level ablations, grouped by the design dimension they target (Section 1.3). GenPPL is external generative perplexity, NFE the number of network evaluations, OWT OpenWebText, FK Feynman–Kac, FLM a flow language model, NLL negative log-likelihood, REPA representation alignment, and NLD Nemotron-Labs-Diffusion; ACE and ESD are defined in Sections 4.3 and 5.1.

Intervention	Experimental comparison	Measured effect	Observation
State: representation and geometry			
Probability geometry	α-Flow's Text8 sweep: the spherical path gives the lowest external GPT-J NLL, while mixture and exponential paths keep entropy nearer the data; intermediate geometries need numerical time reparameterization during training [69].	qualitative ordering across paths; reimplemented discrete baselines remain lower in external NLL	Information geometry trades external predictability against sample entropy
REPA alignment during codec training	TextLDM aligns its VAE encoder to a frozen language model during stage 1, then freezes the encoder for flow training; alignment improves continuation MAUVE while clean-text reconstruction stays nearly unchanged [166].	continuation MAUVE improves at nearly constant clean reconstruction	REPA shapes a more useful latent space without joint encoder–prior training
Predictive versus content latents	PSF replaces each predictive state (an encoding of the preceding blocks only) with a content state that also encodes its own block, repeating decoder fine-tuning and prior training at the same budget [140].	GenPPL at entropy 5.16 higher by 8–16% across samplers, 12% with the prior bypassed	A short code can leave block-level token choices to the decoder
Encoder scaling and soft-label distillation	ELF-B/M with only the frozen encoder changed: T5-small, T5Gemma-1, T5Gemma-2, and a T5Gemma-2 student distilled on the teacher decoder's soft labels, with one-hot cross-entropy and embedding-MSE students as controls [442].	GenPPL ≈40% lower for T5Gemma-2 than T5-small at matched entropy; invalid generated positions 109 → 12 per sample; ELF-M GenPPL 19.3 → 17.8 at real-text entropy	Diffusibility rises with encoder pretraining until plausible alternatives are pushed apart; soft-label distillation pulls them back together
Transition: update dynamics and order			
Depth-specialized local training	DiffusionBlocks trains one layer group per noise range and samples in four steps with greedy output, compared against an autoregressive baseline sampled with top-p = 0.95 [331].	MAUVE 0.71 vs 0.50 (LM1B), 0.82 vs 0.85 (OWT); greedy vs top-p	Noise-specialized depth blocks can be trained modularly (comparison varies decoding rule)
Selection at inference	FK steering resamples intermediate SSD-LM states and beats best-of-n at equal particle count on the targeted reward; Zhang et al. (ACE, arXiv v1) score a low-repetition accepted subset and reverse the ELF-B/ELF-M ranking [335, 436]. Reward calls and rejected samples add cost.	text-quality gains at $k = 4$ and toxicity gains at $k = 4, 8$ (reported tables)	Intermediate resampling and post-hoc filtering substantially shift reported quality
Inference-only sampler repair	Untied Self-Conditioning keeps LangFlow's weights, noise, grid, and decoder and applies its self-conditioning transform and history-based solver correction without retraining; the repaired eight-step samples increase four-gram repetition relative to the high-PPL baseline [197].	GenPPL 531.2 → 61.6; entropy 5.81 → 5.33 at 8 NFE	Part of the coarse-step quality drop in flow models stems from solver and feedback error
Stochastic endpoints	MCB replaces coarse Euler updates with stochastic endpoint bridges on fixed FLM checkpoints; an endpoint temperature trades perplexity against entropy; repeated draws complicate deterministic distillation [38].	plotted trade-off curves	Stochastic endpoint bridges prevent mode-averaging entropy collapse on fixed checkpoints

Table 18 (continued)

Intervention	Experimental comparison	Measured effect	Observation
Denoise-and-renoise sampling for reasoning	On fixed ELF-REG-L checkpoints, the comparison jointly replaces the logit-normal time grid with a power grid ($p = 0.5$) and Euler updates with clean-prediction re-noising ($\eta = 1.2$), retaining the same early-stop exit and NFE; results average four seeds [218]; CEDR-L compares ODE, EDM-style, and DMD2-style samplers at 32 steps on fixed checkpoints [72].	ELF-REG-L GSM8K pass@1 55.91 → 60.37 (64 NFE) but 32.30 → 30.50 (4 NFE); CEDR-L GSM8K 59.78 → 66.68, MATH500 19.70 → 27.90 (ODE → DMD2-style)	Re-noising helps reasoning accuracy at moderate budgets but not at the smallest
Learned maps and distillation	Self-conditioned FMLM builds one-step targets by repeated teacher refinement (Table 15); Scaling CatFM's progressive self-distillation recovers conditional MAUVE at four updates; DFM's one-step ESD map collapses while its PSD map, like FMLM's distilled map, keeps entropy (Table 14) [80, 287, 417].	arXiv v1: one-step GenPPL 176.31 → 112.52 at entropy 5.47 → 5.37; DFM one step (OWT): PSD 180.29 / 4.91, ESD 5.33 / 0.26	Trajectory-consistent and semigroup distillation prevent one-step entropy collapse
Anti-collapse terms in one-step and distribution-matching training	Reinforce-DMD trained without its KL anchor to the LangFlow teacher and a Simplex-DMD student initialized from scratch [358]; ELF-MMD with attraction-only kernel terms [95]; Seq-Drifting without its intra-sequence repulsion [228]; DBTM anchored to its target before the derived vertex-commitment time [348].	Reinforce-DMD entropy → 0; scratch Simplex-DMD: entropy ≈6.9, GenPPL $>6 \times 10^4$; ELF-MMD: collapse (plotted only); Seq-Drifting (LM1B, one step): 36.01 / 4.42 → 10.01 / 1.82; DBTM: collapse to repeated tokens	One-step and distribution-matching objectives use distinct anti-collapse mechanisms, including sample repulsion, teacher anchors or initialization, and appropriately timed data-target anchoring
Shortcut continuation	TTCD continues training its token-time model with shortcut targets; the improvement is in coarse sampling after representation learning (Table 14) [42].	GenPPL 1033.8 → 437.82 (4 nominal updates), 576.86 → 371.18 (8); entropy ≈4.4 unchanged	Shortcut consistency continuation improves coarse-grid token-time sampling
Teacher alignment of denoiser features	ELF-REG aligns intermediate ELF-B denoiser features with a frozen Qwen3-1.7B-Base teacher (REPA) and adds a jointly denoised teacher-derived global token (REG); a stripped variant removes the token at inference, and REPA with one learnable token controls for the extra position [218].	GSM8K pass@1 at epoch 6: 21.74 (baseline), 24.56 (REPA), 28.92 (REPA+REG), 28.66 (token removed at inference), 25.60 (learnable token)	Teacher supervision of the denoiser speeds task training, and the REG gain survives removing the token at inference
Backbone adaptation	TESS 2 (v0.1) adapts Mistral-7B-v0.1 via a 200k-step, ≈45B-token Dolma stage before Tulu 2 SFT, compared against directly fine-tuned Mistral-7B-v0.1, Dolma-adapted Mistral-7B-v0.1, and GSM8K symbolic-math fine-tuning [344]; DSL-LLaDA adds continuous-noise pretraining to LLaDA-8B-Instruct across 8, 16, 32, and 64 NFE [420].	TESS 2: SQuAD 85.4 vs 86.0 (directly fine-tuned AR) and vs 71.4 (Dolma-adapted AR); TriviaQA 49.3 vs 50.4 (direct) and vs 20.5 (Dolma-adapted); trails directly fine-tuned AR on AlpacaEval (63.1 vs 77.1), IFEval (30.5 vs 36.8), BBH (8.4 vs 43.3), and untuned GSM8K (14.5 vs 52.5); math-tuned GSM8K accuracy 66.6% vs 51.2%. DSL-LLaDA: ahead on XSum, CNN/DM at 8/16/32/64 NFE; ahead on PubMed, arXiv at 8/16 NFE, split at 32 NFE (PubMed 36.9 vs 36.5, arXiv 32.9 vs 33.8), behind at 64 NFE	Continuous backbone adaptation is competitive on extractive QA, targeted math SFT, and low-NFE summarization

Table 18 (continued)

Intervention	Experimental comparison	Measured effect	Observation
Blockwise continuous diffusion at 3–8B	Sigma continues 3B and 8B autoregressive checkpoints for >300B tokens as blockwise embedding-diffusion models on NLD's pretraining data and backbone, sampled with embedding-space guidance and score temperature [405]. At high step counts, decoding remains slower per token than an optimized autoregressive model [405].	six-task pass@1 64.34 vs NLD 64.74 and AR initialization 64.73 (3B), 69.67 vs 70.22 and 69.80 (8B); 3B MBPP pass@256 91.8 vs 87.6 (NLD; selected high-coverage samplers)	Blockwise embedding diffusion closely matches same-scale masked diffusion in the six-task math/code average at 3–8B
Latent group size	BLD varies only the number G of block latents denoised jointly per sequential group [103].	GenPPL / entropy 31.7 / 5.22 ($G = 1$), 24.7 / 5.21 (4), 26.6 / 5.17 (8), 27.7 / 5.03 (16); batch-1 latency 4.93 → 1.21 s (H200)	Small groups expose generation to its own errors over many sequential commitments; large groups lower entropy
Prior-gradient routing into the codec	Cola2 retains token supervision and codec regularizers while comparing full stop-gradient on the flow-matching loss with injecting that loss into the encoder every ten steps; a separate update-policy ablation includes a frozen codec [132].	LAMBADA: injection ahead by 2.2 points at 20k steps, behind by 1.0 at 140k; separate frozen-codec comparison: 28.7% vs 32.8%	Late in training, codec co-adaptation with detached prior gradients beats co-adaptation that also receives the prior's loss
Text interface: readout and decoding			
Prior–decoder allocation	DiLaDiff distills the latent prior with a fixed 64-step discrete decoder and becomes decoder-bound; LD4LG's fixed-codec sweep lowers perplexity with more updates while MAUVE peaks early and time grows [195, 239].	time ratio 1.116 → 0.050; LD4LG PPL 52.6 → 25.9, MAUVE 0.684 → 0.667 (peak 0.719), 87 → 630 s per 1,000	End-to-end latency becomes decoder-bound once the latent prior is distilled
Decoder corruption training on fixed generated states	TEncDM holds generated contextual states fixed and retrains only the separate bidirectional decoder with corrupted contextual encodings [317].	Wikipedia MAUVE 0.526 → 0.874 on the same generated states	Corruption-robust decoder training improves generation quality on fixed generated states
Decoder adaptation to prior outputs	BLD evaluates a fixed $G = 8$ prior with the branching decoder adapted on latent predictions from the fully parallel $G = K$ prior [103].	GenPPL / entropy 127 / 5.34 → 26.6 / 5.17; MAUVE 0.67 → 0.82	For BLD's fixed $G = 8$ prior, adapting the decoder to prior-generated latents improves MAUVE from 0.67 to 0.82
Rare-word commitment with restarts	GRAFT commits rare words read part-way through frozen ELF and LangFlow trajectories and regenerates from the same noise around them; an argmax control shares positions, mechanics, dose, and compute, and stock best-of-4 sampling uses a larger trajectory budget [401].	rare-word share (Zipf < 4) on ELF-B 0.42× → 0.82× human (argmax control 0.45×, best-of-4 0.50×); LangFlow 0.77× → 1.03× with a held-out-tuned dose; GPT-2 Large perplexity of accepted ELF-B samples 26.47 → 33.77 for commitment alone	Conditional-mean updates suppress rare candidates; clamping a mid-trajectory rare word and restarting restores rare-word frequency, whereas committing the local argmax does not
Joint adaptation with decoder-input noise & decoder post-training	LDLM injects noise into the decoder input during joint codec–denoiser training ($\sigma_{\text{dec}} = 3 \rightarrow 4$), lowering GenPPL while collapsing MAUVE and entropy (Table 4), and separately evaluates post-training the decoder alone with the encoder and prior held frozen [253].	joint-training $\sigma_{\text{dec}} = 3 \rightarrow 4$: GenPPL 66.3 → 38.1, MAUVE 89.1 → 0.4, entropy 4.25 → 2.19	Excessive decoder noise can lower GenPPL while collapsing sample diversity and MAUVE

7 Open Problems

The preceding sections point to open problems in likelihood and compute scaling, discrete readout from generated states, end-to-end serving latency, representation reuse over long contexts, and mechanistic analysis of multi-step refinement.

7.1 The likelihood and compute gap

Gaussian corruption of discrete token sequences produces a multimodal density as noise decreases. Learned denoising errors and time discretization can push DDPM trajectories into low-density regions between modes, yielding out-of-distribution inputs; in the studied toy setting, DDPM with the optimal conditional-mean predictor recovers valid samples [318]. Modern parameterizations, self-conditioning, and blockwise adaptation have narrowed the variational-bound gap to discrete masked diffusion and enabled competitive sample quality at scale (Section 6.3; Tables 13 and 17) [65, 87, 128, 344, 405, 406]. Nevertheless, continuous models still require a substantial training-compute multiple to match exact autoregressive negative log-likelihood, and no continuous counterpart has yet demonstrated the data-constrained scaling crossover observed for masked diffusion under repeated epochs [289].

The primary theoretical rationale for paying this training-compute premium is parallel distillability: a continuous trajectory can be compressed into a flow map that updates all positions jointly in one or a few steps, whereas a single-step masked model samples positions independently [86, 191]. Chen and Wang compare the parallel sampling complexity of Gaussian, uniform-state, and masked diffusion in a score-oracle model [62]. With accurate scores, Gaussian diffusion on one-hot encodings (like uniform-state diffusion) samples within KL error ε in $\widetilde{O}(\mathrm{DTC}/\varepsilon)$ passes, where the dual total correlation DTC can be far below sequence length. For a family of random empirical measures on the d-dimensional Boolean hypercube and fixed target error, $\widetilde{\Theta}(\sqrt{d})$ passes are necessary and sufficient for Gaussian and uniform-state diffusion against $\widetilde{\Omega}(d)$ for an adversarial approximate masked-diffusion score oracle, because masking concentrates posterior resolution into a narrower $O(\log(d)/d)$ noise band than the $O(\sqrt{\log(d)/d})$ band of Gaussian or uniform corruption [62]. Empirically, FlashMap confirms that shared block noise in a distilled flow map preserves cross-position dependencies that independent per-slot noise destroys, though aggressive few-step distillation can still cause entropy collapse or generation-quality degradation [195, 326, 405].

Whether these parallel complexity bounds extend from their discrete-domain score-oracle settings [62] to low-dimensional embeddings, Riemannian manifolds, and compressed latent spaces remains open, particularly once representation geometry shapes trajectory curvature and distillation error. A related modeling question is whether non-isotropic or discrete-aware transport couplings can concentrate probability mass along valid lexical bridges rather than uniform Euclidean balls, reducing capacity spent on inter-token regions. On the empirical side, existing scaling fits use different tokenizers, backbones, and data regimes across papers [128, 406]. Comparing continuous, masked, and autoregressive models on a shared tokenizer, architecture, and corpus across both unique-token and multi-epoch regimes (while amortizing pretraining plus distillation FLOPs against the inference compute needed for matched few-step sample quality) would directly test when parallel distillability offsets the pretraining compute gap.

7.2 Recovering precise text from generated states

Every continuous generator must eventually map real-valued states back to discrete tokens, exposing an interface mismatch: decoders trained on clean encodings face off-manifold states at inference, compressed codes can lose exact surface tokens, and token-aligned trajectories remain sensitive to small directional residuals near decision boundaries. Across existing systems, clean autoencoder reconstruction does not by itself establish prior-driven sample quality (Sections 2.3 and 3.3) [166, 195, 253, 317]. Decoder sensitivity varies strongly across matched-norm perturbation directions [97]; in ELF and LangFlow, realized deterministic sampler residuals flip far more tokens than norm-matched isotropic perturbations [374]. In Sigma, DDIM with 4–8 steps loses accuracy relative to DDPM-256; tested higher-order solvers can leave terminal states off-manifold, and the authors identify readout sensitivity as a possible bottleneck for code generation [405]. In JUICE's 668M comparison, 16-step continuous caption generation achieves lower caption-reference overlap than the Transfusion-style autoregressive baseline despite similar image–text alignment, even though its parallel decoder recovers 99.2% of tokens from clean encoder states [205]. At sentence level, Two-Tower-7B-IT has lower CoLA fluency scores than the compared instruction-tuned autoregressive baselines [190]; SLD scores below Flan-T5 Large on ROCStories fluency and coherence despite higher title compatibility [462].

To separate prior errors from decoder errors, candidate decoders can be benchmarked against a frozen bank of prior-generated states, scoring entity, numeral, ordering, and syntactic recovery separately while comparing priors inside a shared coordinate space under a fixed decoder. Because Euclidean distance in state space poorly reflects token-flip probability near vocabulary decision boundaries, a controlled test on such benchmarks is whether weighting continuous regression or distillation residuals by the local Fisher information or Jacobian of the decoder distribution reduces word flips relative to isotropic squared error [374, 450]. For tasks demanding exact identifiers, numerals, or formal syntax, a complementary alternative to pure parallel rounding is to apply a lightweight local autoregressive or discrete-diffusion refiner over low-margin positions.

7.3 Few-step quality and end-to-end latency

In the comparisons of Section 6.3, trajectory-consistent distillation and stochastic bridges prevent few-step entropy collapse (Table 14) [38, 183, 191, 287, 358]. Separately, solver feedback corrections improve high-perplexity few-step trajectories whose entropy has not collapsed [197]. The standalone-generator results in Table 16 do not constitute a uniform matched-quality latency benchmark against key–value-cached autoregressive decoding: decoder passes, monitoring overhead, and compression-induced quality changes must be included, and comparison baselines vary across studies [97, 103, 140, 195, 405, 426, 441]. CALM reports inference FLOPs rather than an end-to-end wall-clock comparison [323]. Verified wall-clock speedups at matched quality have so far been demonstrated by pairing one- or two-step continuous block drafters with autoregressive verification at batch size one [56, 326].

The immediate systems question is whether the gains reported at batch size one survive concurrent serving. This can be tested by integrating flow-map block drafters into paged-KV-cache serving engines with tree- or block-verification kernels and measuring both latency and throughput across batch sizes. Adaptive step allocation is a separate question: allocating single-step transport to predictable spans while reserving multi-step updates for high-entropy positions only lowers wall-clock time if the reduction in denoiser work exceeds the cost of deciding when to stop [97]. Evaluating these designs requires end-to-end wall-clock Pareto curves that include encoder, denoiser, decoder, and verifier passes against speculatively decoded autoregressive baselines.

7.4 Reusing representations and extending length

Reusing pretrained representations allows a continuous generator to inherit lexical and semantic structure from an existing encoder, yet off-the-shelf encoders were not shaped for Gaussian or flow transport. Scaling a frozen encoder can push plausible token alternatives so far apart that imperfect sampling produces off-manifold states, motivating distillation to compact the local geometry [442], whereas learning or co-adapting a codec introduces additional optimization choices, including how to calibrate the training-noise distribution to the latent width [131, 132, 220]. In latent flow-map distillation, LDLM-FM freezes its encoder and teacher; jointly adapting both in LDLM-FM-A can improve multi-sample coverage without improving single-sample accuracy [14]. Extending generation to long documents exposes a parallel tension between fixed-window parallel arrays and sequential block, segment, or sentence hierarchies, where sub-token patch compression degrades at unaligned prompt boundaries and coarse sentence vectors trade exact entity and numeric detail for abstraction [131, 134, 190]. Growing token arrays offer another direction, but EFlow/EFM language experiments remain capped at 128 tokens and do not establish document-scale behavior [347].

One open question is whether encoder pretraining objectives can regularize local tangent structure and interpolation smoothness directly, avoiding the post-hoc distillation used to improve on-manifold generation over frozen encoders [442]. Boundary-aligned variable-rate codes are one candidate alternative to fixed-ratio patching for sequence compression [132]. A complementary direction is to learn token-slot insertion and denoising jointly, as in EFlow/EFM [347]. A useful first test would hold the total latent budget fixed and vary how slots are allocated across spans, measuring lexical recovery and document consistency together with the cost of predicting boundaries and code lengths. At document scale, coupling

a slow-moving discourse state with block-causal continuous refinement raises a similar empirical question: whether a coarse global state can preserve rare entities, numerical commitments, and cross-paragraph constraints over thousands of tokens under bounded memory.

7.5 Interpreting and optimizing revisable computation

Because a continuous generator updates an entire state array over multiple steps, intermediate trajectories can represent competing candidate hypotheses, revision timing, and cross-position coordination before discrete readout. Observational studies link sampler residuals to low-margin decoder directions [374]; a separate discrete-data analysis finds DDPM failures when trajectories enter low-density regions between modes [318]. Controlled interventions help distinguish the roles of residual direction, decoder geometry, and cross-step information. In GRAFT's reference-text reconstruction probe, replacing ELF's prediction error relative to the clean embedding with a norm-matched isotropic or permuted error preserves almost every reference word, whereas the original prediction error disproportionately changes rare words into common ones [401]. StepTune shows that a decoder-aware quadratic form predicts readout KL divergence where Euclidean distance fails [450], and freezing surrounding prediction memory alters local revision dynamics [8]. In DiffusionGemma, the self-conditioning channel passes a soft embedding of the previous step's token distribution, allowing several token hypotheses to pass between steps; preserving the top-eight probabilities while distributing the remaining mass uniformly matches baseline benchmark performance [100]. In continuous Sudoku FLMs, the state decomposes into residual initial noise plus a weighted history of clean predictions: removing non-top-1 historical support can change prediction margins while preserving the current hard-token sequence, whereas a separate intervention that restricts locally feasible digits to the top one or two reduces subsequent recovery [40].

Applying subspace projection and top-k hypothesis-pruning interventions to open-ended language, code, and mathematical reasoning can test whether multi-step continuous refinement performs multi-hypothesis search and global constraint propagation or primarily smooths local token commitments. Where intermediate trajectories retain competing hypotheses, differentiable objectives can be optimized through continuous refinement transitions, with the differentiated path depending on the method [137, 161]. LaDi-RL instead optimizes latent transition likelihoods with GRPO using rewards averaged over multiple text realizations of each latent trajectory [174]. The applicability of this pathwise route depends on the reward or value surrogate being differentiated, and comparisons with discrete post-training methods should include both score-function estimators and the continuous relaxations discussed in Appendix C.2.

8 Conclusion

Continuous-space diffusion and flow language models decouple the refinement of a real-valued state from the discrete readout of surface tokens. Evaluating these systems requires considering state representation, transition dynamics, and decoder architecture together, because gains in an isolated component (such as clean autoencoder reconstruction or nominal denoiser step count) often fail to transfer to end-to-end sample quality or serving speed.

In the comparisons reviewed in Section 6.3, calibrated token-level continuous models match equal-scale autoregressive baselines in prompt-continuation distributional similarity [87], and self-conditioned RePlaid matches or improves upon discrete masked diffusion in held-out likelihood bounds past compute-optimal budgets [406]. Even so, variational-bound scaling studies still indicate a large training-compute multiple for matching exact autoregressive negative log-likelihood (Table 17; Section 7.1) [128, 406]. Separately, Yang et al.'s re-evaluation of the released LangFlow checkpoint shows that omitting the self-conditioning chain-rule term understates its ODE-based perplexity estimate [65, 406]. On downstream tasks, continuous generators show their clearest gains over autoregressive decoding when the task exploits full-sequence differentiability, latent planning, or multi-sample diversity. Representative examples include gradient-guided syntactic control, paragraph planning with an autoregressive decoder, multi-candidate code synthesis, and mathematical reasoning after targeted adaptation or pathwise reinforcement learning

[72, 137, 218, 344]. On the COCO Karpathy split, FlowTok-XL achieves captioning scores comparable to the evaluated non-autoregressive methods trained without CIDEr optimization [141]. For general-purpose instruction following and single-pass open-ended fluency at the same scale, autoregressive decoding remains stronger (Tables 12 and 13). Under few-step budgets, trajectory-consistent flow-map distillation and stochastic bridges prevent the entropy collapse of coarse deterministic solvers (Table 14) [38, 191, 287]. Once condition encoding, latent decoding, and monitoring costs are counted, however, matched-quality serving speedups over key–value-cached autoregressive baselines have been demonstrated by continuous drafting with autoregressive verification at batch size one rather than by standalone continuous generators (Table 16) [56, 326]. Whether those speedups extend to concurrent serving remains open.

These trade-offs also shape how continuous models scale. Training from scratch has reached 7B parameters in sentence-embedding space [190] and roughly 1–4B parameters in token-aligned or compressed spaces [17, 80, 128, 131, 405]. At 3B–13B token-level scales, pretrained autoregressive or masked backbones can be adapted into blockwise or full-sequence continuous generators [56, 135, 164, 344, 405]. Their continued-training budgets and task-level comparisons vary substantially (Section 6.3). In particular, Sigma uses autoregressive initialization to accelerate convergence, then trains on more than 300B additional tokens; with 256 DDPM steps per block, its 3B and 8B models approach the respective same-scale masked-diffusion baselines on the six-task mathematical-reasoning and coding average [405]. Consequently, continuous language models are currently best suited to settings where joint revision, gradient-based control, candidate-pool diversity, or speculative block drafting justify the interface and training overhead. Progress on the remaining bottlenecks will require evaluating sample quality, diversity, likelihood calibration, and end-to-end latency under matched compute budgets (Section 7).

Contributors

Haocheng Mei, Yuchuan Tian, Haojun Jiang, Wei He,
Ye Yuan, Chao Xu, Yunhe Wang, Allen Chen

A Mathematical Details

This appendix states, with their scope, the four results that Section 2 uses: the reverse-time diffusion and its probability flow, the posterior-mean and score identities for token-aligned states, the relation between a continuous density and a discrete text probability, and the flow-map form of the categorical map used in Section 5.1.

A.1 Reverse-time diffusion and probability flow

Let the forward corruption be the linear Markov stochastic differential equation

$$\mathrm{d}z = f(z,t)\,\mathrm{d}t + g(t)\,\mathrm{d}w, \qquad t \in [0,T], \tag{16}$$

with linear drift $f(z,t) = f(t)z$, a scalar state-independent diffusion coefficient $g(t)$, and a standard Wiener process w. For a smooth nondegenerate Gaussian path $z_t = \alpha(t)z_0 + \sigma(t)\epsilon$ ($\alpha(t) > 0, \sigma(t) > 0$, with diffusion time t increasing toward noise and flow time $s = 1 - t$ increasing toward data), the marginals of Equation 1 are realized by the Markov forward SDE of Equation 16 if and only if

$$f(t) = \frac{\dot\alpha(t)}{\alpha(t)}, \qquad g(t)^2 = 2\sigma(t)\left(\dot\sigma(t) - \frac{\sigma(t)\dot\alpha(t)}{\alpha(t)}\right) = \alpha(t)^2\frac{\mathrm{d}}{\mathrm{d}t}\left(\frac{\sigma(t)^2}{\alpha(t)^2}\right) \geq 0,$$

which requires a monotonically nondecreasing noise-to-signal ratio $\sigma(t)/\alpha(t)$; the variance-preserving family has $f(z,t) = -\frac{1}{2}\beta(t)z$ and $g^2 = \beta(t)$, and the variance-exploding family has $f = 0$ and $g^2 = \mathrm{d}\sigma^2/\mathrm{d}t$. Writing p_t for the marginal density at time t and $s(z,t) = \nabla_z \log p_t(z)$ for the true marginal score, the process run backward from $t = T$ to $t = 0$ satisfies the reverse-time equation

$$\mathrm{d}z = \left[f(z,t) - g(t)^2\nabla_z \log p_t(z)\right]\mathrm{d}t + g(t)\,\mathrm{d}\bar w, \tag{17}$$

where $\bar w$ is a Wiener process running backward in time and $\mathrm{d}t$ is a negative increment [5, 337]. The probability-flow ordinary differential equation

$$\mathrm{d}z = \left[f(z,t) - \frac{1}{2}g(t)^2\nabla_z \log p_t(z)\right]\mathrm{d}t \tag{18}$$

has the same marginals p_t for every t [337], and substituting $f(t)$ and $g(t)^2$ recovers the Gaussian marginal velocity of Equation 4 even for smooth nondegenerate Gaussian paths that are not generated by a Markov forward SDE.

Three qualifications fix the scope of Equations 17–18. First, the equality of marginals requires the exact marginal score $s(z_t,t) = \nabla_{z_t}\log p_t(z_t)$, the same terminal distribution p_T at $t = T$ for both processes, and standard regularity; it is an equality of time-marginal densities rather than individual trajectories, so two samplers started from the same noise follow different paths. Second, in practice the true score $s(z_t,t)$ is replaced by the network's plug-in estimate $\hat s_\theta(z_t,t) = -\hat\epsilon_\theta(z_t,t)/\sigma(t) = (\alpha(t)\hat z_{0,\theta}(z_t,t) - z_t)/\sigma(t)^2$ for $\sigma(t) > 0$ via Equation 2, which equals $s(z_t,t)$ only when $\hat z_{0,\theta}(z_t,t) = \mathbb{E}[z_0 \mid z_t]$ (or equivalently $\hat\epsilon_\theta(z_t,t) = \mathbb{E}[\epsilon \mid z_t]$). With a learned score and a finite number of updates, neither process is simulated exactly, and the stochastic DDPM discretization of Equation 17 and the deterministic DDIM or higher-order ODE discretizations of Equation 18 written out in Section 5.1 need not yield the same samples even in distribution [149, 242, 336]. Third, the scalar formulas assume a state-independent diffusion coefficient; manifold-valued and hybrid processes carry additional terms and cannot be treated with Equations 17–18 unchanged.

A.2 Posterior mean, Tweedie's formula, and the token-posterior score

Consider a token-aligned state with fixed embeddings, $z_{0,i} = e(x_i)$, corrupted by Equation 1 with a common $\alpha(t) > 0$ and $\sigma(t) > 0$ at every position. The exact posterior over the token at position i given

the whole noisy state and the condition is

$$p(x_i = v \mid z_t, c) = \frac{\sum_{x:\, x_i = v} p(x \mid c)\, \mathcal{N}\big(z_t; \alpha(t) e(x), \sigma(t)^2 I\big)}{p_t(z_t \mid c)},$$
$$\mathbb{E}[z_{0,i} \mid z_t, c] = \sum_{v \in \mathcal{V}} p(x_i = v \mid z_t, c)\, e(v). \tag{19}$$

The conditional mean at one position is therefore a probability-weighted sum of embeddings, and it depends on the entire state z_t because the prior $p(x \mid c)$ couples positions. With $\sigma(t) > 0$, Tweedie's formula gives the true marginal score $s_i(z_t, t, c) = \nabla_{z_{t,i}} \log p_t(z_t \mid c)$ at position i in terms of that conditional mean:

$$s_i(z_t, t, c) = \nabla_{z_{t,i}} \log p_t(z_t \mid c) = \frac{\alpha(t)\, \mathbb{E}[e(x_i) \mid z_t, c] - z_{t,i}}{\sigma(t)^2}. \tag{20}$$

The identity holds blockwise because the Gaussian kernel factorizes over positions with a common variance: differentiating $\log p_t(z_t \mid c) = \log \sum_x p(x \mid c) \prod_j \mathcal{N}(z_{t,j}; \alpha e(x_j), \sigma^2 I)$ with respect to $z_{t,i}$ touches only the factor at position i, and the resulting weights are the posterior $p(x \mid z_t, c)$, so the derivative equals $\mathbb{E}[(\alpha e(x_i) - z_{t,i})/\sigma^2 \mid z_t, c]$ [99].

Equations 19–20 are statements about the data distribution; a model enters only as an approximation. A network $q_\theta(x_i \mid z_t, c) \in \Delta^{|\mathcal{V}|-1}$ trained by cross-entropy on the clean tokens estimates $p(x_i \mid z_t, c)$, and substituting it gives the model's expected-embedding prediction $\hat{z}_{0,i} = \sum_v q_\theta(x_i = v \mid z_t, c)\, e(v) \in \mathbb{R}^d$ and, through Equation 20, its plug-in score estimate $\hat{s}_{\theta,i}(z_t, t, c) = (\alpha(t)\hat{z}_{0,i} - z_{t,i})/\sigma(t)^2$, which equals the true score $s_i(z_t, t, c)$ only when $\hat{z}_{0,i} = \mathbb{E}[e(x_i) \mid z_t, c]$. CDCD uses this substitution, which its authors call score interpolation [87], and the token-posterior objectives of Section 4.1 share it. At nondegenerate noise levels ($\alpha(t) > 0, \sigma(t) > 0$), $\hat{z}_0$, $\hat{\epsilon}_\theta$, $\hat{s}_\theta$, and $\hat{v}_\theta$ are connected by the invertible affine relations of Equation 2. By contrast, the mapping from a token posterior $q_\theta(\cdot \mid z_t, c) \in \Delta^{|\mathcal{V}|-1}$ to the expected embedding $\hat{z}_{0,i} = E^\top q_\theta(\cdot \mid z_t, c) \in \mathbb{R}^d$ is many-to-one whenever $|\mathcal{V}| > d + 1$, because $E^\top \in \mathbb{R}^{d \times |\mathcal{V}|}$ has a nontrivial nullspace and distinct token posteriors can induce the same expected embedding and plug-in score. A network trained instead by squared error on z_0 estimates the conditional mean $\mathbb{E}[z_{0,i} \mid z_t, c]$ in $\mathbb{R}^d$ directly without specifying a unique point on $\Delta^{|\mathcal{V}|-1}$ (Section 4.1). When the embeddings are learned jointly, the identity still holds for the current table, but the table itself can move, producing the embedding-collapse failure of Section 2.3.

A.3 Continuous density and discrete text probability

For a separate probabilistic decoder, a latent-variable construction gives

$$p_{\theta,\psi}(x \mid c) = \int p_\theta(z \mid c)\, p_\psi(x \mid z, c)\, \mathrm{d}z. \tag{21}$$

For a deterministic decoder D, the induced text probability is instead the prior mass of the decoder's preimage, $\int_{D^{-1}(x)} p_\theta(z \mid c)\, \mathrm{d}z$, when a density exists. A density evaluated at an encoded point is not that discrete probability mass. An autoregressive decoder factorizes the conditional text probability through preceding output tokens; a parallel decoder may define a different factorization. Replacing the decoder can therefore change the likelihood object even if the continuous trajectory is unchanged, and evaluating these text probabilities requires integration over codes or decoder regions, which is why Section 6.2 treats reported likelihoods of codec systems as bounds or estimates.

A.4 Flow-map form of the categorical map

Section 5.1 writes the categorical flow map of Equation 12 with two flow times $0 \le s < t \le 1$ running toward data, so that $t = 1$ is the one-hot endpoint; both arguments there are flow times, not the diffusion time of Equation 1. The map follows from the linear path alone. Along $z_\tau = (1 - \tau)\epsilon + \tau y$, with ϵ

the noise endpoint and y the clean endpoint, the state at time s determines the noise endpoint given y, $\epsilon = (z_s - sy)/(1-s)$, and substituting it into the state at a later time t gives

$$z_t = (1-t)\frac{z_s - sy}{1-s} + ty = \frac{1-t}{1-s}z_s + \frac{t-s}{1-s}y. \tag{22}$$

Replacing the unknown endpoint y by a prediction gives a one-interval map, and which prediction is used matters. Substituting the local posterior mean $\mathbb{E}[y \mid z_s]$ gives the one-step DDIM-type update: for a Gaussian endpoint, $y = \hat{z}_0$ yields the linear-path identities of Equation 2, $\hat{z}_0 = z_s + (1-s)\hat{u}$ with $\hat{u} = \hat{z}_0 - \hat{\epsilon}$, and the map $z_{s'} = s'\hat{z}_0 + (1-s')\hat{\epsilon}$, which is the deterministic DDIM step. Unless the conditional mean remains constant along the interval $[s,t]$, that finite-step update is in general an approximation to the integrated flow trajectory that becomes exact as the interval shrinks ($t \to s$), because the single-time posterior mean at s averages over endpoints that the remainder of the trajectory would separate, producing the few-step mode averaging analyzed in Section 5.1.

A flow map instead targets an ideal interval-dependent two-time denoiser $q^*(z,s,t)$: by construction, $q^*(z,s,s)$ equals the true posterior mean $\mathbb{E}[y \mid z_s = z]$ on the diagonal $t = s$ and for $s < t$ equals the trajectory-weighted average of single-time posterior means along $[s,t]$ that makes the interval step exact [191], which DFM terms the mean denoiser [287]. In practice, a learned network $q_\theta(z,s,t) \in \Delta^{|\mathcal{V}|-1}$ is trained to approximate $q^*(z,s,t)$ (or a finite-step teacher). Substituting q_θ into Equation 22 yields Equation 12, and at $t = 1$ the map returns q_θ itself. For the exact solution operator $X_{s,t}$ of the flow ODE, the diagonal identity $X_{s,s}(z) = z$ and the semigroup composition property $X_{t,u}(X_{s,t}(z)) = X_{s,u}(z)$ hold identically. For a learned network $X^\theta_{s,t}(z) = z + (t-s)\bar{u}_\theta(z,s,t)$ (or Equation 12 with q_θ), the state-map boundary identity $X^\theta_{s,s}(z) = z$ holds algebraically at $t = s$ because the coefficient $(t-s)/(1-s)$ vanishes, whereas $q_\theta(z,s,s) \approx \mathbb{E}[y \mid z_s = z]$ and the semigroup composition property $X^\theta_{t,u}(X^\theta_{s,t}(z)) \approx X^\theta_{s,u}(z)$ hold only approximately and are what diagonal cross-entropy supervision together with consistency and self-distillation losses train the network toward, either by matching the output of one large interval step to the composition of two shorter ones or through the average-velocity identities of Section 2.2 [48, 116, 338]. The categorical maps of Section 5.1 differ from this derivation only in what they constrain: the predictor q_θ lies on the simplex, and the intermediate state z may lie anywhere in the ambient vocabulary-dimensional space [287, 304].

B Reconstruction and Generated-State Diagnostics

A text codec can be evaluated along three distinct input routes through its decoder (Figure 7 in Section 3.3): reconstructing encodings of observed clean text, decoding encodings subjected to controlled local perturbations, and decoding codes sampled from the learned latent prior. Whereas the first two routes test information retention and local smoothness around the data manifold [190, 430, 441, 462], only the third evaluates the decoder on the distribution produced by the prior and sampler at inference time. Across both compressed and full-length contextual codecs, clean-text reconstruction and prior-driven generation frequently rank architectures in opposite orders: stronger latent regularization or position pooling can lower clean reconstruction accuracy while substantially improving generation MAUVE and generative perplexity, and adapting a decoder to prior-denoised latents or aligning the encoder to a frozen language model improves generated text, with clean-reconstruction changes that need not predict the generation gains [103, 166, 195, 253, 317]. A related discrete-codec case study isolates reconstruction loss from code-space diffusion and finds that improved local geometry diagnostics need not improve decoded-text quality [114]. Comparing prior distributional quality across representations therefore requires holding a shared decoder and coordinate space fixed.

B.1 Local decoding geometry and its limits

Isotropic squared error in the continuous state does not directly predict token stability because decoder decision boundaries are anisotropic: perturbation direction, magnitude, and local logit margins jointly

govern whether a token flips [97, 374]. If the clean winning-token margin is $m(z) > 0$ and competing logit differences are L-Lipschitz throughout the perturbation neighborhood, perturbations in that neighborhood satisfying $\|\delta\| < m(z)/L$ preserve the decoded label [97]. Realized deterministic generator residuals can nevertheless flip far more tokens than norm-matched isotropic perturbations, demonstrating why a norm-only error criterion misses decoding failures [374].

C Boundary with Discrete Diffusion and Hybrid States

This appendix summarizes closely related discrete diffusion, hybrid discrete–continuous, and pre-diffusion latent-variable systems, noting the continuous design questions they inform. While core models in this survey iteratively refine a real-valued representation across the generated unit (Section 1.2), this appendix collects categorical and autoregressive backbones, direct baselines such as NLD for Sigma [106, 405], and mechanistic contrasts. Duo relates the argmax marginals of Gaussian-corrupted one-hot vectors to uniform-state discrete diffusion under a transformed noise schedule [309]; Chapter II develops predictor–corrector samplers and a memory-efficient curriculum [85], while Haxholli extends the marginal correspondence to convex-interpolant flows under explicit source and coupling assumptions [139]. BD3-LM introduces blockwise diffusion with causal autoregression across blocks [36]; EDLM augments a frozen factorized denoiser with a residual sequence-level energy and importance-resamples candidates drawn from that denoiser [393]; and Coconut defines continuous hidden-state recurrence without a generative diffusion prior [138]. The broader discrete diffusion and flow family—including D3PM, SEDD, MDLM, DiffuLLaMA, LLaDA, Dream, Discrete Flow Matching, and ReMDM—supplies backbones and evaluation references for continuous generators [37, 112, 121, 238, 268, 308, 361, 408]. Industrial systems also provide reported throughput reference points: Mercury is a commercial diffusion-model family [159], and Seed Diffusion Preview is an experimental research preview [339]. Gemini Diffusion is an experimental text-diffusion model whose public description does not disclose its internal state space [123].

C.1 Hybrid states: a continuous channel beside a categorical one

Hybrid processes pair a categorical token canvas with a real-valued channel, differing in what the continuous channel represents and when discrete choices become permanent (Table 19). Product-state and jump formulations include Sticky Jump Diffusions and CCDD [171, 453], continuous word-position models (DDOT [429]), alternating Gaussian–one-hot updates (TDD [214]), and continuous-to-discrete reveal schedules (CANDI, MLFM [39, 291]). In latent-coupled hybrids, Co-LADD and Di-LADD pair token masking with continuous or quantized latent diffusion [325], CADD aligns noisy vectors with categorical tokens [448], VADD redraws a step-local Gaussian latent [388], IMDM attaches a continuous random vector to unresolved masks for few-step distillation [418], and x-Prediction Flow runs bounded Euler embedding steps as a continuous decoding wrapper over a pretrained masked model after x-prediction alignment [371].

C.2 Connections to categorical feedback, commitment, and evaluation studies

Although categorical diffusion models keep their generated canvas discrete, recent studies directly inform continuous design choices regarding inter-step feedback, dependency coupling, reward guidance, and sampler evaluation. Table 20 synthesizes these studies by mechanism.

C.3 Adjacent continuous-variable models and internal-state diffusion

These related methods differ from the survey's core setting in their training objective, refined state, or output interface. First, latent-variable generators use normalizing flows, Wasserstein autoencoders, or energy-based priors over character, token, or sentence latents, including TextFlow [469], FlowSeq (2019) [248], LaNMT and its iterative score/energy refinement [193, 332], RNF-WAE [367], short-run Langevin EBM priors [281], and symbol–vector EBM priors trained with persistent Langevin chains [280]. Flow-augmented

Table 19. Hybrid and continuous-feedback categorical systems grouped by the role of their real-valued channel.

Method(s)	State role	Information carried between calls	Where categorical choices enter
BFN [124]	Generation belief	Updated categorical belief parameters	Noisy evidence updates; terminal category draw
Simplax [310]	Categorical state + simplex auxiliary	Discrete denoiser input with retained Dirichlet bridge state	Categorical and Dirichlet updates at every reverse step
HC-DLM [299]; CADD [448]	Coupled latent & token state	Continuous latent/vector paired with uniform-noised or absorbing tokens	Stepwise argmax scaffold (HC-DLM) or absorbing unmasking (CADD)
EvoToken-DLM [451]; LRD [460]; DMax [67]	Revisable soft token mixture	Soft posterior- or confidence-weighted token–mask mixtures	Committed when a block closes, KL-monitored refinement stabilizes, or a confidence threshold is met
Soft-Masked / S-SM [146, 269]; RCD [155]	Prediction feedback	Committed tokens plus prediction-derived embeddings at unresolved positions	Categorical sampling and unmasking each round until canvas closes
DiffusionGemma [88]	Canvas-wide prediction feedback	Revisable categorical tokens plus expected-embedding feedback at every canvas position	Uniform-noise categorical resampling; final argmax after entropy/stability stopping
LDDM [169]; Relay [307]; MetaState [386]; VADD [388]	Hidden memory or step-local latent	Backbone hidden states or recurrent slot bank; VADD's Algorithm 2 instead redraws a Gaussian latent each step	Vocabulary projection and categorical sampling at each round

summarization and translation posteriors include FlowSUM [404], planar/IAF/Sylvester/coupling flows [290, 315], TGVAE [372], and flow-based neural topic summarization [266]. Related latent style/attribute editors and prefix controllers include StyleFlow [459], spline/coupling INNs [440, 443], PriorControl and Distributional Lens [125, 126, 237], and the variational prototype editor CEV-LM [261]. Prescribed Brownian-bridge methods support discourse planning (Time Control [369], COLOR [364]), contrastive narrative generation [263], and bridge-augmented dialogue [245].

Second, inference-time energy and gradient controllers steer generation without a trained text denoiser, using continuous relaxations, discrete gradient-informed proposals, or latent-space dynamics. Examples include simplex exponentiated gradients (MuCoCo [188]), logit Langevin dynamics (COLD [294]), embedding-space updates (MuCoLa [189]), Metropolis–Hastings-corrected discrete proposals (p-NCG, GwL [96]), and latent ODE attribute composition (LATENTOPS, MacLaSa [90, 225]).

Third, methods operating within causal language models have distinct mechanisms: activation flow transport (UniSteer, SchröMind [328, 329]); internal conditional diffusion that reconstructs prefix hidden states before each next-token prediction by a retained causal suffix (DiHAL [184]); hierarchical computation with adaptive concept pooling (DLCM [296]); input-embedding feedback during autoregressive decoding (Mixture of Inputs [468]); and recurrent-depth diffusion-forcing sampling with shared KV caches, applied without additional model training [115].

Other boundary cases differ in their output interface or refined modality. DiffuSum and TermDiffuSum learn diffusion over sentence representations but return summaries by selecting source sentences [92, 431]. MindEye2 predicts visual embeddings for a separate GIT caption decoder [313]; BLIP–SD reranks autoregressive captions using image reconstruction [203]; and FM-Refiner transports distorted acoustic latents toward clean latents before CTC decoding [400]. Audio-Native ASR instead connects a frozen acoustic encoder to a discrete-diffusion text decoder, whose generated state is categorical [178].

Continuous Interaction Diffusion is a further conceptual boundary case: it proposes an asynchronous tool runtime coupling a locally diffused continuous thought tensor to a discrete display canvas, specifies architecture and an evaluation protocol, and reports no empirical performance results [51].

Table 20. Related categorical systems and evaluation studies grouped by mechanism and the continuous design question each informs.

Mechanism category	Representative systems and studies	Continuous design question informed
Prediction-embedding & soft-mixture feedback	Soft-Masked DLM [146], S-SM [269], SCMDM [52], RCD [155], LRD [460], ReMix [410], dInfer [249], SSM [73], DMax [67], EvoToken-DLM [451], DiffusionGemma [88]	Parallels continuous self-conditioning (Section 4.3): propagating prediction-derived embeddings or pre-logit mixtures across steps improves few-step quality, KV-cached serving, and rollout robustness, while entropy-based early exit risks repetitive collapse.
Recurrent hidden memory & self-prompting	DFlow [438], LDDM [169], Relay [307], MetaState [386], ADJUST [41], Diffusion-NAT [456]	Distinguishes hidden-memory and verifier-feature feedback from discrete self-prompting; Diffusion-NAT prefixes its predicted clean token sequence to the source condition for repeated refinement.
Commitment, dependency coupling, & block conversion	Mask-Predict [117], CoDD [204], CoDiLA [145], DEMASK [302], Path Planning / PAPL [284, 285], DCDM [463], SDAR [71], SimSD [76], SchED [257], Ada-DLM [202], and HDLM [452]	Contrasts discrete confidence remasking, progress-aware early exit, semantic-convergence detection, hierarchical semantic refinement, circuit/dependency-grouped sampling, soft-conditioned local AR decoding, AR-to-block-diffusion conversion, and speculative draft verification with continuous per-position noise schedules and block codecs.
Training-time self-distillation	SelFusion [221]	Relates bidirectional distillation between high- and low-mask views, directed by token correctness and confidence, to continuous denoising objectives and self-conditioning.
Reward optimization, selection, & reasoning scores	APS [305], EntRGi [352], DRAKES (fine-tuning) [360], SVDD rewriting [279], Diff4TST [247], Diffuse Thinking [322], Detect–Remask–Repair [471], Reasoning-as-infilling (prompting and intermediate scoring) [151]	Contrasts Gumbel–Softmax [162] or expected-embedding reward relaxations, soft-value candidate selection, and autoregressive-LLM evaluation of discrete thought proposals with direct pathwise gradient guidance in continuous spaces (Section 5.4). Reasoning-as-infilling instead uses a fixed reasoning/answer template for posterior trace sampling and intermediate answer-token log-probability scores given a reference answer.
Sampler bias, likelihood bounds, & task baselines	HMM oracle & order studies [346, 435], DUEL [356] and TUBE [160], Gumbel precision [449], A Tale of Two Temperatures [274], Generative Frontiers [292], SpecRef [133], DLLM-VSR [412], TreeDiff [427], DiffusionCLS [66]	Shows that sampler policy, numerical draw precision, and temperature sweeps shift likelihood bounds and GenPPL–entropy frontiers independently of denoiser quality, and provides masked counterparts for lip reading, AST-aware code generation, and label-conditioned data augmentation for sentiment classification.

D Experimental Protocols and Evaluation Settings

Table 21 summarizes the evaluation protocols, scoring conventions, and control variables behind the numerical comparisons in Section 6.3, grouped by evaluation regime.

D.1 Likelihood estimators and endpoint rules

Reported likelihoods depend critically on the estimator graph and integration domain. In probability-flow ODE evaluation of self-conditioned models, holding the bootstrap prediction fixed when computing the drift divergence omits its chain-rule Jacobian contribution; differentiating through the self-conditioning pass restores this term and raises the ODE perplexity estimate of the released LangFlow checkpoint from 24.94 to 33.29 on the same fixed 1,024-sequence OpenWebText validation subset. These subset ODE estimates are distinct from the full-set results: the RePlaid evaluation gives a variational PPL bound of 32.2 for LangFlow, versus LangFlow's self-reported ODE PPL of 24.6; RePlaid's own variational PPL bound is 22.1 and the autoregressive baseline's exact PPL is 17.5 [65, 406]. Furthermore, a direct denoising-error integral can lose mass when embedding scale grows while the log-SNR window and tail corrections stay

Table 21. Summary of experimental protocols, scoring conventions, and control variables across evaluation regimes.

Evaluation regime	Standard protocol, scoring conventions, and key control variables	Representative studies
Unconditional & prefix continuation	OpenWebText, LM1B, ROCStories, C4, and related corpora; protocols are study-specific. LD4LG uses a 64-token monolingual maximum and five sets of 1,000 samples; CDCD uses 5,000 C4 prompts. DGLM uses 5,000 C4/OWT examples, a 32-token prefix and 32-token continuation, OLMo-1B perplexity, and 2–4-gram diversity; its MAUVE standard error uses five seeds. BLD uses 1,000 sequences without establishing a universal seed count; TTCD reports three seeds but no text-generation sample count; PSF includes 500 prompted continuations and single-run cells. External perplexity is paired, depending on the study, with unigram entropy, n-gram diversity, and/or MAUVE; scorer tokenization, conditioning context, decoding rules, and sampler choice must be controlled.	[42, 87, 95, 103, 140, 195, 220, 239, 240, 253, 298, 347, 426, 441, 442]
Likelihood bounds & multi-scale compute	Held-out OpenWebText/OpenWebText2 (0.1B–1.3B, model-specific training budgets) and OpenWebText2/SlimPajama IsoFLOP scaling fits (14×–64× compute multiples relative to AR); variational NELBO bounds vs. probability-flow ODE divergence estimates; matched detokenization and non-embedding FLOP accounting.	[65, 128, 312, 406]
Few-step solvers, flow maps, DMD, & drifting	One- to eight-step generation on LM1B, OpenWebText, Text8, and TinyStories; BFN-Solver instead evaluates text8 at 10–1000 NFEs using spelling accuracy; NFE accounting must specify whether self-conditioning bootstrap calls, commit-quality heads, and terminal readout passes are counted; temperature sweeps or log-linear interpolation align GenPPL at matched unigram entropy.	[80, 183, 197, 228, 304, 348, 358, 398, 417, 418]
Speculative drafting & block verification	Batch-1 wall-clock latency and tokens/s on single H100/H200 GPUs; continuous block drafters verified by causal target heads (preserving exact target greedy/sampling quality) vs. standalone unverified block/compressed generators measured at study-specific batch sizes (up to 256 for PSF) with or without KV caching and CUDA graphs.	[56, 103, 140, 326, 426]
Controllable generation, editing, & transfer	E2E syntactic/semantic control, GYAFC/Yelp style transfer, NarrativeQA sparse infilling, and sentiment/toxicity steering; classifier accuracy, Sentence-BERT similarity, CoLA acceptability, and Distinct-n (sensitive to saved-output rescoring vs. fresh inference).	[180, 196, 213, 237, 261, 335, 389, 390, 455]
Sequence-to-sequence & discourse tasks	Paraphrase (QQP), question generation (Quasar-T), summarization (XSum, CNN/DM), table-to-text (ToTTo), multilingual generation (SONAR/Mimir), dialogue (DialogSum), and instruction following (DOLLY); candidate-pool selection (unsupervised MBR vs. oracle reference selection) and pool size strongly shift BLEU/ROUGE.	[43, 119, 135, 187, 190, 207, 264, 382, 420, 454, 462]
Multimodal, speech, & code/reasoning tasks	Image/audio/video/motion captioning (COCO Karpathy without CIDEr optimization, CapArena-Auto, GPIC, AudioCaps, Clotho, MSR-VTT, HumanML3D), ASR (LibriSpeech WER), medical reports (MIMIC-CXR), unconditional Python synthesis (MAUVE and generative perplexity), and math/code reasoning (GSM8K, MATH, HumanEval, MBPP pass@k), plus language modeling (LAMBADA).	[49, 54, 61, 75, 106, 129, 131, 132, 141, 152, 167, 172, 178, 194, 211, 215, 230, 259, 295, 344, 357, 366, 370, 381, 383, 392, 405, 414, 428, 451, 465]

fixed. Here s denotes the per-dimension second moment: when embedding geometry is approximately preserved, the predicted peak shift is $-\ln(s/s_0)$ relative to reference scale s_0, or $-2\ln k$ for amplitude multiplier k. Safeguards include convergence-verified windows, current-scale tail corrections, and live variational endpoint terms; Plaid additionally normalizes embeddings each forward pass [33, 128]. In discrete and semi-discrete settings, greedy deterministic unmasking policies (DUEL) and random-order marginals (TUBE) define distinct distributions with different gaps to autoregressive NLL [160, 356], and semi-discrete flow-map bounds require stochastic terminal categorical draws rather than deployed argmax readout [80].

D.2 Implementation and reporting notes

Cross-section and code-release conventions occasionally differ across studies: Diffusion-LM's official clamping implementation evaluates the reverse posterior $q(z_{t-1} \mid z_t, \tilde{z}_0)$ rather than drawing fresh forward noise from the clamped vector $\tilde{z}_0$ [213]; ConvergeFlow and Simplex-DMD initialize from frozen or continued LangFlow checkpoints [209, 358]; and NFE accounting varies in whether self-conditioning calls, shared-network readout passes, or parameter-scaled demasker equivalents are included [42, 153, 197, 357, 420].

E Literature Selection and Scope

The main literature corpus was assembled through 24 October 2026 and focuses on generative systems that iteratively refine a real-valued representation across the generated unit at some stage of natural-language or code generation (Section 1.2). Citation corrections and version updates in this revision use public sources available by 28 October 2026. Literature was gathered through keyword searches on arXiv (`cs.CL`, `cs.LG`, `stat.ML`), OpenReview (including public ICLR 2027 submissions and NeurIPS 2026 workshops), and the ACL Anthology (including EMNLP 2026) combining representation terms (*continuous*, *latent*, *embedding*, *simplex*, *spherical*) with process terms (*diffusion*, *flow matching*, *flow map*, *bridge*) and task terms (*language model*, *text generation*), supplemented by forward and backward citation tracing from prior surveys and community trackers (Table 1) [86, 461]. Because papers were added continuously through the cutoff date rather than drawn from a single frozen query snapshot, this survey is a curated technical synthesis rather than a PRISMA-style systematic review with stagewise deduplication counts. Operationally, a system is placed in the core lookup table (Appendix F, Table 22) when it specifies a complete continuous-state generative pipeline (state encoding, continuous noise-to-data transition, and text decoder) and serves as a primary architectural or empirical reference in Sections 3–6. Task-specific adaptations, domain applications, and follow-up solver or schedule variants are grouped in Appendix G, while discrete diffusion models, hybrid discrete-canvas models, and deterministic latent-recurrence methods are contrasted in Appendix C. Patent filings, unreviewed code repositories without technical reports, and withdrawn preprints are excluded. All quantitative comparisons use published results aligned by protocol (Appendix D).

F Method and Configuration Lookup

Table 22 lists the core continuous-space systems once each, grouped by the family of the generated state, and classifies each method along four standardized attributes. The *state* column records the real-valued object revised by the sampler: a learned or fixed token lookup, vocabulary logits or simplex coordinates, belief vectors, analog bits, a full-length contextual encoding, a compressed code with fewer positions or narrower channels, or a sentence or segment vector. The *target* column indicates what the network is trained to predict, whether a clean state x_0, noise ϵ, velocity v, the score, a token-posterior cross-entropy, probabilities under a weighted squared error, a flow-matching velocity, or a two-time flow map. The *transition and order* column pairs the generative dynamics, such as Gaussian or Dirichlet diffusion, flow matching, flow maps, belief updates, energy-based refinement, or drifting, with the unit refined at each step: the full sequence, latent blocks, the next vector, the next token, or a growing array. Finally, the *decoder* column specifies how terminal states become tokens, distinguishing direct rounding or a vocabulary head, a shared contextual readout, a separate parallel or autoregressive decoder, and a discrete-diffusion decoder. Small text under a value records the configuration-specific qualifier, and the last column points to where the survey discusses the method: a main-text section that cites it, prioritizing Sections 3–5 followed by Section 6, or its Appendix G entry when it appears only in the appendix. Related discrete, hybrid, and pre-diffusion systems are summarized in Appendix C (Tables 19–20), and additional task-specific configurations are indexed in Appendix G.

Table 22. Standardized lookup of core continuous-space text generation systems, grouped by state family. Year follows the cited bibliography and may reflect formal publication or first preprint release rather than the date of the linked revision. Columns record the refined continuous state, training target, transition dynamics and generation order, decoding interface, primary task domain, and survey section ('n/a' indicates unspecified in source; CE = cross-entropy, FM = flow matching, MSE = mean squared error, AR = autoregressive).

Method	Year	State	Target	Transition; order	Decoder	Task(s)	§
Learned token lookup							
CDCD [87]	2022	learned lookup	token CE	Gaussian diffusion; full sequence	rounding / vocab. head	unconditional LM; machine translation	§3.1
DiffuSeq [119]	2023	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	seq2seq: paraphrase, QG, dialogue, simplification; MBR selection	§3.1
Diffusion-LM [213]	2022	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	controllable generation; infilling	§3.1
AR-Diffusion [382]	2023	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	summarization; MT	§5.3
CodeFusion [334]	2023	learned lookup	ϵ	Gaussian diffusion; full sequence	separate parallel decoder	natural language to code	§6.1.5
DiffCap [142]	2023	learned lookup	x_0 + token CE	Gaussian diffusion; full sequence	rounding / vocab. head	image captioning	App. G.3
DiffuSeq-v2 [120]	2023	learned lookup	x_0	Gaussian + absorbing noise; full sequence	rounding / vocab. head	seq2seq few-step without MBR	§5.1
GENIE [222]	2023	learned lookup	ϵ	Gaussian diffusion; full sequence	rounding / vocab. head	summarization; CommonGen	§4.2
InfoDiffusion [368]	2023	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	seq2seq	§4.3
Masked-Diffusion LM [59]	2023	learned lookup	token CE	Gaussian soft masking; importance-ordered	vocab. projection	controllable generation	§4.1
Plaid [128]	2023	learned lookup	$\hat{z}_0$ MSE + tok. CE	Gaussian diffusion; full sequence	rounding / vocab. head	likelihood LM; zero-shot likelihood	§3.1
Boundary-cond. diff. [127]	2024	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	MT; summarization	App. G.8
DDLM (early halting) [236]	2023	learned lookup	sampler only	Gaussian diffusion; full sequence	rounding / vocab. head	unconditional LM; quality–step tradeoff	App. G.9
Difformer [111]	2024	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	MT; summarization	§4.1
DiNoiser [409]	2024	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	MT; paraphrase	§4.3
DoT (Plaid) [407]	2024	learned lookup	$\hat{z}_0$ MSE + tok. CE	Gaussian diffusion; full sequence / rationale blocks	rounding / vocab. head	math reasoning	§6.1.6
FlowSeq (2024) [154]	2024	learned lookup	FM velocity	flow matching; full sequence	rounding / vocab. head	seq2seq few-step; MBR	§5.1
FMSeq [229]	2024	learned lookup	x_0	flow matching; full sequence	rounding / vocab. head	seq2seq few-step	§6.1.2
Meta-DiffuB [74]	2024	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	seq2seq	§4.3
ParaGuide [150]	2024	learned lookup	token CE	Gaussian diffusion; full sequence	rounding / vocab. head	plug-and-play textual style transfer	§5.4
SeqDiffuSeq [425]	2024	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	seq2seq	§4.3
Temporal Pruning (DiffuSeq) [198]	2024	learned lookup	sampler only	Gaussian diffusion; full sequence	rounding / vocab. head	seq2seq few-step	§5.1
DLM-One [63]	2025	learned lookup	SiD	Gaussian diffusion; full sequence	rounding / vocab. head	one-step seq2seq and LM	§5.1
L2D [53]	2025	learned lookup	token CE	Gaussian diffusion; next token	rounding / vocab. head	instruction / reasoning benchmarks	§5.3
NeoDiff [199]	2025	learned lookup	z_0 MSE + time KL + tok. CE	Gaussian noise; Poisson token clocks; full sequence	rounding / vocab. head	MT and other seq2seq	§4.3
BCT [272]	2026	learned lookup	token CE	flow matching; full sequence	rounding / vocab. head	GSM8K after TinyGSM training; Sudoku-Hard	§5.1
ClockDLM [312]	2026	learned lookup	$\hat{z}_0$ MSE + tok. CE	Gaussian diffusion; sliding window	rounding / vocab. head	likelihood LM; GSM8K after TinyGSM training	§5.3

Continued on next page

Table 22 (continued)

Method	Year	State	Target	Transition; order	Decoder	Task(s)	§
Diffinity [180]	2026	learned lookup	inherited Plaid; guidance only	Gaussian diffusion; full sequence	rounding / vocab. head	formal-syntax-constrained generation	§5.4
DiffusionBlocks (AR) [331]	2026	learned lookup	token CE	Gaussian diffusion; next token	rounding / vocab. head	unconditional LM	§5.3
DSL [384]	2026	normalized token lookup	token CE	continuous diffusion; optional discrete refinement	vocab. head	unconditional LM	§6.3
DSL-LLaDA [420]	2026	learned lookup	token CE	Gaussian diffusion; full sequence	rounding / vocab. head	summarization and instruction tasks vs LLaDA-8B	§4.2
FastDiSS [267]	2026	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	seq2seq few-step	§5.1
Lafa [56]	2026	learned lookup	$\hat{z}_0$ MSE + tok./AR CE	flow matching; latent blocks	rounding / vocab. head	instruction following, math, code	§5.3
LangDiT [244]	2026	learned lookup	token CE	flow matching; full sequence	rounding / vocab. head	unconditional LM	§4.3
LangFlow [65]	2026	learned lookup	token CE	flow matching; full sequence	rounding / vocab. head	unconditional / likelihood LM	§4.1
RePlaid [406]	2026	learned lookup	$\hat{z}_0$ MSE + tok. CE	Gaussian diffusion; full sequence	rounding / vocab. head	likelihood LM vs discrete baselines	§4.1
S-FLM [83]	2026	learned lookup	token CE	flow matching; full sequence	rounding / vocab. head	unconditional LM (OWT); Sudoku; GSM8K after TinyGSM training	§3.1
SCLM [144]	2026	learned lookup	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	MT	§4.1
ShortcutFlow [375]	2026	learned lookup	flow map	flow map; full sequence	rounding / vocab. head	seq2seq one-step	§6.1.2
Sigma [405]	2026	learned lookup	$\hat{z}_0$ MSE + tok./AR CE	Gaussian diffusion; latent blocks	rounding / vocab. head	math and code; instruction tasks after tuning	§6.1.6
Simplex/Reinf.-DMD [358]	2026	learned lookup	reverse-KL DMD	Gaussian diffusion; full sequence	rounding / vocab. head	few-step unconditional LM	§5.1
TTCD [42]	2026	learned lookup	token CE	flow matching; full sequence	rounding / vocab. head	unconditional / prefix-conditioned LM; Sudoku	§6.3
INCO [298]	2026	learned lookup (frozen at inference)	sampler only	flow-matching ODE; full sequence	rounding / vocab. head	unconditional LM	§6.3
Fixed or pretrained token lookup							
SED [341]	2022	fixed lookup	$\hat{z}_0$; emb. MSE + readout CE	Gaussian diffusion; full sequence	rounding / vocab. head	unconditional and conditional LM; infilling	§3.1
BIT [129]	2026	fixed lookup	bridge score	Brownian bridge SDE; full sequence	rounding / vocab. head	image captioning and text-to-image	§6.1.3
ConvergeFlow [209]	2026	fixed lookup	x_0	flow matching; full sequence	rounding / vocab. head	unconditional LM (OWT)	§4.1
LLaCO [164]	2026	fixed lookup	token CE	spherical denoising; latent blocks	rounding / vocab. head	knowledge, reasoning, math, code	§3.1
Seq-Drifting [228]	2026	fixed lookup	drift field	drifting; full sequence	rounding / vocab. head	one-step unconditional and prefix LM; MT	§5.1
Vocabulary logits, simplex, and one-hot endpoints							
SSD-LM [134]	2023	vocab. logits/simplex	token CE	Gaussian diffusion; latent blocks	rounding / vocab. head	unconstrained continuation; controlled generation	§3.1
Fisher-Flow (LM) [79]	2024	vocab. logits/simplex	token CE	flow matching; full sequence	n/a	LM1B	§3.1
SFM [68]	2024	$\sqrt{p}$ sphere	FM velocity	flow matching; full sequence	categorical draw	text8 character LM; DNA	§3.1
TESS [176]	2024	vocab. logits/simplex	token CE	Gaussian diffusion; full sequence	vocab. head; argmax	summarization, paraphrase, QG, simplification; GLUE	§3.1
FK steering (SSD-LM) [335]	2025	vocab. logits/simplex	inherited SSD-LM; sampler only	Gaussian diffusion; latent blocks	rounding / vocab. head	controlled generation	§5.4

Continued on next page

Table 22 (continued)

Method	Year	State	Target	Transition; order	Decoder	Task(s)	§
RDLM [168]	2025	probability sphere	token CE	Riemannian diffusion; full sequence	terminal-state argmax	Text8; LM1B	§3.1
Smoothie [319]	2026	vocab. logits/simplex	clean emb.; MSE	Gaussian diffusion; full sequence	rounding / vocab. head	unconditional; seq2seq	§3.1
TESS 2 [344]	2025	vocab. logits/simplex	token CE	Gaussian diffusion; full sequence	rounding / vocab. head	instruction following; QA	§6.1.6
CatFM [304]	2026	vocab. logits/simplex	flow map	flow map; full sequence	rounding / vocab. head	unconditional LM few-step	§5.1
DBTM [348]	2026	vocab. logits/simplex	flow map	flow map; full sequence	rounding / vocab. head	few-step unconditional LM; Sudoku	§5.1
DFM [287]	2026	vocab. logits/simplex	flow map	flow map; full sequence	n/a	unconditional LM few-step	§5.1
DoubleCat / SemiCat [81]	2026	vocab. logits/simplex	token CE	flow matching; full sequence; SemiCat adds discrete reveals	vocab. head; intermediate commitment in SemiCat	GSM8K after TinyGSM training; unconditional LM	§4.3
EFlow / EFM [347]	2026	vocab. logits/simplex	mixed	flow matching; growing array	rounding / vocab. head	variable-length unconditional LM	§5.3
FlashMap [326]	2026	vocab. logits/simplex	flow map	flow map; latent blocks	rounding / vocab. head	speculative drafting for Qwen3-4B/8B	§5.4
FMLM [191]	2026	noised one-hot; simplex head	flow map	flow map; full sequence	vocabulary argmax	one-step LM; prefix LM; Sudoku	§5.1
GSF [183]	2026	vocab. logits/simplex	flow map	flow map; full sequence	rounding / vocab. head	few-step unconditional LM; prefix completion	§5.1
MCB [38]	2026	vocab. logits/simplex	sampler only	flow matching; full sequence	rounding / vocab. head	few-step unconditional LM	§5.1
Noise Your Prompt [170]	2026	vocab. logits/simplex	token CE	flow matching; full sequence	rounding / vocab. head	Sudoku and N-Queens; summarization	§5.4
Posterior Refine. [1]	2026	vocab. logits/simplex	flow map	flow map; full sequence	rounding / vocab. head	unconditional LM few-step	§5.2
RCFG [235]	2026	vocab. logits/simplex	inherited CatFM; guidance only	flow map; full sequence	rounding / vocab. head	attribute- and target-word-controlled generation	App. G.9
Scaling CatFM (PSD) [80]	2026	vocab. logits/simplex	flow map	flow map; full sequence	rounding / vocab. head	unconditional LM few-step at scale	§5.1
SDM [84]	2026	vocab. logits/simplex	token CE	Dirichlet diffusion; full sequence	rounding / vocab. head	GSM8K after TinyGSM training; Sudoku	§3.1
Belief vectors							
BFN [124]	2023	beliefs	prob. MSE	belief update; full sequence	rounding / vocab. head	character-level LM	§3.1
BFN-Solver [398]	2024	beliefs	sampler only	belief update; full sequence	rounding / vocab. head	text8 few-step	§5.1
Analog bits and visual encodings							
Analog Bits [64]	2023	analog bits	x_0	Gaussian diffusion; full sequence	rounding / vocab. head	image captioning	§3.1
SCD-Net [243]	2023	analog bits	x_0 bit MSE + tok. CE	Gaussian diffusion; full sequence	rounding / vocab. head	image captioning	App. G.3
BitLM [467]	2026	analog bits	x_0	Gaussian diffusion; latent blocks	rounding / vocab. head	multi-token LM generation	App. G.8
CoBit [45]	2026	analog bits	bit-posterior MSE	Gaussian diffusion; full sequence	rounding / vocab. head	unconditional LM	App. G.8
H-CDLM (H-CoBit) [275]	2026	analog bits	prob. MSE	Gaussian diffusion; full sequence	rounding / vocab. head	unconditional LM; GSM8K after TinyGSM training	§3.1
Contextual encodings, full length (frozen or jointly learned)							
LaDiC [373]	2024	contextual, full length	x_0	Gaussian diffusion; full sequence	separate parallel decoder	image captioning	§6.1.3
TEncDM [317]	2025	contextual, full length	x_0	Gaussian diffusion; full sequence	separate parallel decoder	unconditional; seq2seq	§3.2
DAC [392]	2025	contextual, full length	x_0 MSE + tok. CE + validity	Gaussian diffusion; full sequence	separate parallel decoder	diverse and efficient audio captioning	App. G.3
LDP [472]	2026	contextual, full length	x_0	Gaussian diffusion; full sequence	separate AR decoder	controllable paraphrase generation	§3.2
AURORA [220]	2026	contextual, full length	x_0	flow matching; latent blocks	separate parallel decoder	LM benchmarks	§3.3

Continued on next page

Table 22 (continued)

Method	Year	State	Target	Transition; order	Decoder	Task(s)	§
CEDR [72]	2026	contextual, full length	x_0 (FM loss)	flow matching; full sequence	shared contextual readout	math reasoning; code	§6.1.6
CoDAR [327]	2026	contextual, full length	v; v-MSE	Gaussian diffusion; full sequence	separate AR decoder	LM	§3.2
DeltaFlow [130]	2026	contextual, full length	$\hat{z}_0$; vel. MSE + tok. CE	flow matching; full sequence	shared contextual readout	unconditional LM; MT; summarization	App. G.8
ELF [153]	2026	contextual, full length	$\hat{z}_0$; velocity-MSE + terminal CE	flow matching; full sequence	shared contextual readout	unconditional LM; MT; summarization	§3.2
ELF (T5Gemma-2) [442]	2026	contextual, full length	FM velocity	flow matching; full sequence	shared contextual readout	unconditional LM	§3.2
ELF-MMD [95]	2026	contextual, full length	feature MMD	fixed-time latent refinement; full sequence	shared contextual readout	few-step unconditional LM; GSM8K after TinyGSM training	§5.1
ELF-REG [218]	2026	contextual, full length	FM velocity	flow matching; full sequence	shared contextual readout	math reasoning; code	§6.1.6
ELF-S2T [215]	2026	contextual, full length	FM velocity	flow matching; full sequence	shared contextual readout	speech recognition and translation	§6.1.3
GRAFT [401]	2026	contextual, full length	sampler only	flow matching; full sequence	shared contextual readout	unconditional LM	§5.2
HC-DLM [299]	2026	contextual; full length for LM, one vector for puzzles	x_0 + tok. CE	flow matching; full sequence	separate parallel decoder	unconditional LM; Sudoku	§5.2
JUICE [205]	2026	contextual, full length	FM velocity (x_0-param.)	flow matching; full sequence	separate parallel decoder	image captioning; visual QA	§6.1.3
LDLM [253]	2026	contextual, full length	x_0	Gaussian diffusion; full sequence	separate parallel decoder	unconditional LM	§4.2
Multimodal Flow (MF-1) [351]	2026	contextual, full length	FM velocity (x_0-param.)	flow matching; latent blocks	separate parallel decoder	unified text–image pretraining; visual QA	§6.1.3
Probe guidance (ELF) [89]	2026	contextual, full length	inherited ELF; guidance only	flow matching; full sequence	shared contextual readout	unconditional LM	§5.4
Self-cond. FMLM [417]	2026	contextual, full length	flow map	flow map; full sequence	shared contextual readout	unconditional LM few-step	§5.1
StepTune [450]	2026	contextual, full length	sampler only	flow matching; full sequence	shared contextual readout	summarization; MT	§5.1
TextLDM [166]	2026	contextual, full length	FM velocity	flow matching; full sequence	separate parallel decoder	text continuation (four benchmarks)	§3.2
Untied Self-Cond. (ELF) [197]	2026	contextual, full length	sampler only	flow matching; full sequence inference-only feedback and solver correction; offline calibration	shared contextual readout	few-step unconditional LM	§5.1
Compressed codes with fewer positions							
LD4LG [239]	2023	code, fewer positions	$\hat{v}$; L2 (uncond./class), L1 (seq2seq)	Gaussian diffusion; full sequence BART-Diffusion baseline uses $\hat{z}_0$; self-conditioning; CFG for seq2seq; DDPM 250 steps	separate AR decoder	unconditional; seq2seq	§3.3
PLANNER [441]	2023	code, fewer positions	x_0	Gaussian diffusion; full sequence	separate AR decoder	paragraph generation; completion	§3.3
DiffSLT / DiffSLT-P [260]	2025	code, fewer positions	$\hat{z}_0$ L1	Gaussian diffusion; full sequence	separate AR decoder	sign language translation	App. G.3
LanguageFlow [437]	2024	code, fewer positions	FM velocity	flow matching; full sequence	separate AR decoder	sentiment transfer; structural control	§5.1
Cosmos [252]	2025	code, fewer positions	z_0; latent MSE	Gaussian diffusion; full sequence	separate parallel decoder	unconditional and conditional generation	§3.3
EdiText [192]	2025	code, fewer positions	x_0	Gaussian diffusion; full sequence	separate AR decoder	controllable editing	App. G.1
OmniFlow [211]	2025	code, fewer positions	FM velocity	flow matching; full sequence	separate AR decoder	any-to-any including captioning	§6.1.3
BLD [103]	2026	code, fewer positions	x_0 induced velocity loss	flow matching; latent blocks	separate AR decoder	unconditional LM	§5.3

Continued on next page

Table 22 (continued)

Method	Year	State	Target	Transition; order	Decoder	Task(s)	§
Cola2 [132]	2026	code, fewer positions dynamic semantic spans, ≈2 tokens per latent, $d_z = 64$; codec co-adapted with the decoder	FM velocity	flow matching; latent blocks	discrete-diffusion decoder	LM benchmarks	§4.2
CRoCoDiL [357]	2026	code, fewer positions	x_0; MSE	Gaussian diffusion; full sequence	discrete-diffusion decoder	unconditional code generation	§3.3
JPEG-DLM [426]	2026	code, fewer positions	x_0	flow matching; full sequence	separate parallel decoder	unconditional LM	§3.3
LaDiff / DiLaDiff [195]	2026	code, fewer positions	z_0 MSE / MeanFlow	Gaussian diffusion / flow map; full sequence	discrete-diffusion decoder	unconditional LM	§3.3
LaDiR [175]	2026	code, fewer positions	FM velocity	flow matching; latent blocks	VAE (thought) + in-model AR (ans.)	text reasoning	§3.3
LLMAE [283]	2026	code, fewer positions	x_0	Gaussian diffusion; full sequence	separate AR decoder	detailed image captioning	§3.3
Manta-LM [93]	2026	code, fewer positions	FM velocity	flow matching; full sequence	separate parallel decoder	infilling; conditional generation	§3.3
PSF [140]	2026	code, fewer positions	x_0; velocity loss	flow matching; full sequence	separate AR decoder	unconditional LM; prompted continuation	§3.3
Compressed codes with narrower channels							
UniDiffuser [44]	2023	code, narrower channels	ϵ	Gaussian diffusion; full sequence	separate AR decoder	image captioning and text-to-image	App. G.3
FlowTok [141]	2025	code, narrower channels	FM velocity	flow matching; full sequence	separate parallel decoder	image captioning	§6.1.3
Cola [131]	2026	code, narrower channels	FM velocity	flow matching; latent blocks	causal block decoder	LM benchmarks	§3.3
Remapped cross-modal variational codes							
CrossFlow [230]	2025	2D variational code	FM velocity	flow matching; full sequence	separate text decoder	image captioning	§6.1.3
Sentence and segment vectors							
LDEBM [421]	2022	sentence/segment vector	variational ELBO + IB + clustering	Gauss. + EBM rev.; full sequence	separate AR decoder	interpretable text modeling	§4.2
CoDi (text branch) [350]	2023	sentence/segment vector	ϵ	Gaussian diffusion; full sequence	separate AR decoder	any-to-any generation including captioning	§3.4
Dior-CVAE [403]	2023	sentence/segment vector	x_0	Gaussian diffusion; full sequence	separate AR decoder	variational dialogue generation	App. G.4
LaDiDa [411]	2023	sentence/segment vector	n/a	Gaussian diffusion; full sequence	separate AR decoder	document generation	§3.4
DGLM [240]	2024	sentence/segment vector	v	Gaussian diffusion; one continuation proposal	separate AR decoder	controlled continuation with fluency	§3.4
DiffuCOMET [109]	2024	sentence/segment vector	z_0 MSE + anchor	Gaussian diffusion; full sequence	separate AR decoder	contextual commonsense knowledge inference	§6.1.6
Diffusion LCM [190]	2024	sentence/segment vector	x_0	Gaussian diffusion; next vector	separate AR decoder	summarization and summary expansion	§3.4
DiffusionDialog [387]	2024	sentence/segment vector	x_0	Gaussian diffusion; full sequence	separate AR decoder	diverse dialogue response generation	App. G.4
EDDPMs [226]	2024	sentence/segment vector	ϵ + align. + tok. CE	Gaussian diffusion; full sequence	separate AR decoder	generation, reconstruction, and representation	App. G.1
SLD [462]	2025	sentence/segment vector	v (released config.)	Gaussian diffusion; full sequence	separate AR decoder	summarization; story and dialogue generation	§3.4
DiffLM [458]	2025	sentence/segment vector	ϵ	Gaussian diffusion; full sequence	separate AR decoder	controllable synthetic data generation	§6.1.5
FlowBind [54]	2026	sentence/segment vector	FM velocity	flow matching; full sequence	separate AR decoder	any-to-any captioning via a shared anchor	§6.1.3
Mimir [264]	2026	sentence/segment vector	x_0	Gaussian diffusion; next vector	separate AR decoder	multilingual QA and summarization	§3.4

G Extended Catalog of Systems and Applications

Whereas Sections 3–6.1 analyze canonical exemplars of each design dimension and downstream capability, this appendix indexes the broader body of task-specific, multimodal, and architectural extensions by capability (Appendices G.1–G.7) and by representation or sampler mechanism (Appendices G.8–G.9).

G.1 Controlled generation, editing, and attribute structure

Constraint and control interfaces inject supervision into the corruption schedule, denoiser objective, starting state, or inference-time guidance. Quantized-embedding guidance refines token-level gradient steering in QE-CDLM [173]. DLMDC uses a DistillClassifier to reduce guidance-gradient noise during non-autoregressive continuous diffusion denoising [210]. Conditioned denoisers and classifier-guided sampling target structural, prosodic, or emotion–topic constraints (SynText/STDiff [434], PoetryDiffusion [156], PromptDiffusion [256], knowledge-guided emotion–topic diffusion [359]). Diffinity applies training-free automaton guidance during sampling [180]. Importance-ordered corruption supports controlled generation (Masked-Diffusion LM [59]). Manta-LM supports infilling through locality-preserving latent compression and boundary-conditioned flow matching [93]. Other proposed control interfaces include compact-code residual control [7]; ELF extends contextual-embedding flows to conditional generation [153]. Seeded parallel batches enable linguistic steganography (GTSD [385]).

In source-preserving editing, systems condition or guide denoising, vary noise depth, or modify latent transport and token-local schedules, or mix conditional and unconditional denoising, for sentiment transfer, detoxification, paraphrasing, and fine-grained stylistic transformation (EdiText [192], LanguageFlow [437], LDP [472], fine-grained and gender style transfer [246, 251], DiffuDetox [104], StyleShield [447], and gradient-steered paraphrase editors DiffusionAttacker and Babel [110, 362]). In controllable generation, TTA-Diffusion assigns token-specific timesteps to preserve classifier-induced updates across denoising steps [182]. Sentence-space attribute models structure global latents via energy-based priors, joint encoder–denoiser alignment, or topic priors (LDEBM [421], EDDPMs [226], DeTiME [395]). Adversarially trained classifiers in latent space also guide a rectified-flow language model for debiased text generation [282]. Provenance and red-teaming studies likewise steer or optimize continuous latent states for watermarking and adversarial prompt search (COMPASS on Cola and ELF-L [12, 131, 153], Cola-DLM attacks [3]).

G.2 Source representations and conditional generation

Conditional sequence-to-sequence extensions either refine target token embeddings directly or generate compact latent conditions for downstream modules. Token-level target denoisers extend source-conditioned generation across table-to-text generation (a GENIE adaptation [187]), knowledge-graph and molecular verbalization (DLM4G [321], with Diffusion-LM as an embedding-diffusion antecedent [213] and SeqDiffuSeq as a comparison baseline [425]), molecular generation and captioning (BiMol-Diff [320]), and query-focused summarization (QFS-DLM [158]). Related variants introduce reinforced self-conditioning with time-aware variance scaling (TREC [234]) and reciprocal source–target attention (DiffuSIA [345]), training–sampling alignment (BG-DiffuSeq [349], FastDiSS [267]), and M2Emb text embeddings with adaptive clustering and multi-scale residual fusion (EmbDiffounder [157]). Conditional question generation also directly denoises text vectors before rounding [377]. A second family generates compact conditions or pseudo-documents for downstream decoders and retrievers (DiffuD2T [118], DiFRa [219], DQG-DM [391], DiffusionRet [293]), while factuality- and stance-preserving summarizers apply classifier or source-alignment guidance between denoising updates (P3SUM [232], factuality-guided AR-Diffusion [330]).

IDEATION uses latent diffusion to support diverse sampling for advertising text generation [376].

G.3 Conditional and joint multimodal configurations

Multimodal captioning and any-to-any generators fall into five architectural groups depending on their text state and cross-modal coupling. First, token-, bit-, and simplex-level image captioners refine embeddings, analog bits, or one-hot states under visual features, syntactic skeletons, user traces, or visual regeneration losses (Analog Bits [64], CLIP-Diffusion-LM, DiffCap, and Prefix-diffusion [142, 227, 394], SCD-Net [243], cascaded caption diffusion [61], MirrorDiff [365], POSCD-Net [224], trace-conditioned captioning [147], LaDiC [373], the analog-bit suggestion variant of Show, Suggest and Tell (whose main model uses discrete diffusion with an autoregressive captioner) [152], TDD [214]). Second, visual-storytelling, video, audio, speech, and sign-language systems adapt continuous diffusion or flow matching to temporal streams (MM-Diff-Net with analog-bit caption states [172], DiffVC [366], DiffuVST [381], LipsFlow [60], DAC, DACRLD, ZS-DDAC, and FAC [107, 392, 464, 465], DiffSLT and gloss-to-text DiffuSeq [260, 454]).

Third, cross-modal bridges, unified multimodal generators, and their applications connect text with visual/audio representations (BIT [129], Diffusion Bridge and Diffusion-Link [194, 265], EasyGen with its BiDiffuser-to-LLM projection [445], Cola text–image, GenFirst, CoUPLe, unified latent diffusion, and CoRe [20, 25, 28, 131, 446], Versatile Diffusion, CoDi, C3Net, CoDi-based LAM-MSC, and Modaliff [165, 179, 350, 396, 432]). Fourth, joint multi-output generators co-denoise text alongside images, 3D human motion, or speech bitstreams (UniDiffuser [44], MLD [49], MoTe and PackDiT [167, 383], Multimodal CoBit [45, 46]). Fifth, shared-posterior, shared-latent, and sentence-latent generators pair multimodal VAEs, SONAR vectors, GTR/SentenceBERT embeddings, or frozen LLM autoencoders with text decoders (UniTMGE [370], Diff-MVAE [378], MUNI and MUNITE [413, 414], ShaLa and IDMVAE [75, 439], V-LCM [295], Diffusion-GDP [354], LLMAE [283]).

G.4 Planning, dialogue, and discourse

Planning, discourse, and dialogue systems span blockwise logit sequences with optional multi-model collaboration (SSD-LM [134], SSD-2 [135]), direct response-token denoising, and utterance- or plan-level latent vectors that condition autoregressive realization. Beyond blockwise logit models, examples include backbone-integrated continuation planners (STAR-LDM [241], Reason Before You Speak [21]), hierarchical and background-factor dialogue priors (Dior-CVAE [403], DiffusionDialog [387], DEDM [77]), and empathy- and intent-guided response generators (DiffusEmp [47], LisenDiff [207]). Diffuse-CG instead edits token-aligned prototype hidden states with structured Gaussian soft masking and utterance shuffling before vocabulary projection [58].

G.5 Code, structured outputs, and synthetic data

Beyond direct program token generation, continuous diffusion models synthesize soft prompts for frozen code models (DDPT [206]) and compressed register plans for masked Python decoders (CRoCoDiL [357]). For structured and tabular data, specialized priors generate structured poll questions and options (DiffusPoll [70]), symbolic-regression expressions (DiffuSR [136]), structured synthetic data (DiffLM [458]), and label-conditioned or differentially private synthetic training corpora (CoDSA [353], DP-SGD diffusion text [273], abuse-detection augmentation [300]).

G.6 Knowledge, reasoning, and instruction interfaces

Knowledge-intensive and reasoning extensions span several directions. Concept-to-text, retrieval-augmented, and skill-generation systems condition diffusion or flow generation on concept names and ontology types, retrieved evidence, or task queries (BioNGen [78], RaDi4GCR [101], GRADi4GCR with continuous, latent, and discrete diffusion backbones [102], ASSIGNER [43], query-conditioned one-step agent skill generation SkillFM [444]). L2D trains a LoRA diffusion branch with token cross-entropy while freezing the main language-model path, enabling iterative next-token refinement [53]. Post-training and latent-reasoning methods use reinforcement learning, rationale distillation, self-distillation,

intermediate-step supervision, semi-continuous block conversion, or latent flow matching with verified-rollout training (LaDi-RL [174], SC-dLLM [15], LaDi-OPSD [26], FLaRe [278], which evaluates LaDiR as a latent-reasoning baseline [175], RDD with teacher-rationale distribution distillation [181], ATF [216], ReasonFlow with intermediate CoT-anchor supervision [399], critic-guided CFRL [137], Tilted Generator Matching [35], ReLPO [29]). Related progression architectures include Evo, which balances autoregressive refinement and diffusion-style planning through token-level progression times [380]. Finally, combinatorial puzzle flows and theoretical expressivity analyses examine multi-step search and test-time computation: maze/Sudoku flows [40]; Unrolled Flow Models [161] and their FLM/S-FLM baselines [83, 191]; and circuit-complexity comparisons of exact-score sampling with more general drift fields [233].

G.7 Domain observations, reports, and neural interfaces

Scientific and clinical applications condition token- or sentence-level denoisers on specialized domain encoders. Representative tasks include remote-sensing change captioning (Diffusion-RSCC [423]), difficulty-scheduled molecular captioning (BiMol-Diff [320]), multi-view 3D shape captioning (Diff-3DCap [333]), cross-modal radiology report generation (MedM2G [428], CoDiXR and XGeM [258, 259]), and fMRI-to-text neural decoding (UniBrain [250]).

G.8 Representation and training variants

Additional state parameterizations include time-free stochastic localization (DSL and DSL-LLaDA [384, 420]), geometry-aware Euclidean Gaussian diffusion (LanGeo [466]), simplex, logit, and information-geometric flows (Logit-KL FM [316], SimpLM [402], Simplex-to-Euclidean ILR flows [379], α-Flow [69]), binary and hierarchical cluster-bit codecs (BitLM, CoBit, and H-CoBit [45, 275, 467]), and visual-glyph rendering (GlyphDiffusion [208]). Contextual and tokenizer-free latent spaces further encompass sequence-to-sequence latent adapters, hybrid recurrent–attention flows, spectral decompositions, and character-level bottlenecks (LaDiffuSeq [200], LDSeq [91], Diffusion-LEF [277], CoM-DAD [397], DeltaFlow's hybrid bidirectional gated-delta backbone for ELF [130, 153], frequency-aware spectral diffusion [13], Zonkey [306]). On the training and schedule side, related studies investigate unified diffusion parameterizations, cross-channel conservation laws, auxiliary word-difference prediction targets, boundary-rescaled or entropy-profiled schedules, denoiser architectures, and self-regularized training (SSP [55], GEXIT [2], WDR [143], boundary-conditional diffusion on a Differmer backbone [111, 127], DiffuER's auxiliary embedding and reconstruction losses [108], Meta-DiffuB [74], InfoNoise [297], NeoDiff [199], TTCD [42], OFE+STDL [271], LangDiT applied to LangFlow [65, 244], SERF self-regularization applied to S-FLM [83, 186]).

G.9 Sampling, decoding, and conditioning variants

Inference-time control and adaptive solver extensions encompass step-size control, early halting, and latent-value pruning or guidance (INCO [298], PI step-size control [45, 177], DDLM early halting [236], FlowScope [27] on LangFlow and ELF [65, 153], Reading Latent States [30], LQG [34], GuideDiffuSeq [50], RCFG on CatFM [235, 304]). Distillation and trajectory-straightening extensions include source-clean target flows, recurrent drifting, spherical reflow, and implicit-midpoint or score-distilled maps (FMSeq [229], FlowLM [433], ShortcutFlow [375], ReLaDrift [276], ReSFlow's extension of SFM [68, 163], score distillation [19], asynchronous flow maps [22], AIM-FLM [32]). Finally, sequence-progression and evaluation studies explore token-wise progression and length extrapolation (Evo [380], EFlow/EFM [347], RangeSWA [11]), alongside density scoring without explicit divergence computation, sliced-Wasserstein sample evaluation, and standalone-generation comparisons with drafter-style models (Yu et al.'s denoiser-based estimator [419], SOL [185] evaluated on spherical-flow and Soft-Masked models [57, 146], LangFlow vs. standalone DSpark adaptations [23]).

References

[1] Manan Agarwal, Sheel Shah, Chanhyuk Lee, Jaehoon Yoo, Jerry Huang, Seunghoon Hong, Aditi Raghunathan, Jinwoo Kim, and Nicholas M. Boffi. Posterior Refinement: Fast Language Generation via Any-Order Flow Maps. arXiv preprint arXiv:2606.24773, 2026. URL https://arxiv.org/abs/2606.24773.

[2] Ziv Aharoni and Henry D. Pfister. Conservation Laws for Diffusion Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=RxdFtImvkv.

[3] Ali Al-Lawati, Elsayed Eshra, Dongwon Lee, and Suhang Wang. Diffusion Language Models for Perplexity-Aware Black-Box Attacks. arXiv preprint arXiv:2610.31558, 2026. URL https://arxiv.org/abs/2610.31558.

[4] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic Interpolants: A Unifying Framework for Flows and Diffusions. *Journal of Machine Learning Research*, 26(209): 1–80, 2025. URL https://jmlr.org/papers/v26/23-1605.html.

[5] Brian D. O. Anderson. Reverse-Time Diffusion Equation Models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.

[6] Anonymous. Riemannian Flow Matching on the Fisher–Rao Sphere for Non-Autoregressive Conditional Text Generation. Anonymous submission to ACL Rolling Review, May 2025. URL https://openreview.net/forum?id=NfRX3JKSTG.

[7] Anonymous. Recovering Control from Compact Codes: Residual Control in Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=0VaeYBYxot.

[8] Anonymous. Explore, Revisit, or Commit: Prediction Dynamics in Continuous Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=1dysL8xBDt.

[9] Anonymous. DistillScope: Benchmarking Flow-Map Distillation for Language Generation. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=2edihBesnJ.

[10] Anonymous. Length from the Latents: Dynamic-Length Compressed Latent Diffusion Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=4Gc7zLa7R7.

[11] Anonymous. RangeSWA: Training-Free Length Extrapolation for Continuous Diffusion Language Models. Anonymous submission to ACL Rolling Review, October 2026. URL https://openreview.net/forum?id=614wZmgrF5.

[12] Anonymous. Watermarking Continuous Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=6SzEZJld4O.

[13] Anonymous. A Spectral Perspective on Continuous Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=9D0MMjtW7l.

[14] Anonymous. How to Train Your Latent Flow Map for Few-Step Text Generation. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=bPRgkZ2qJC.

[15] Anonymous. Into the Continuum: Efficient Semi-Continuous Diffusion LLMs. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=C0Kdy12R9J.

[16] Anonymous. Training Latent Language Diffusion Models Requires Less Than You Think. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=DsDsfb70CC.

[17] Anonymous. Inside the Scaling of Riemannian Flow Language Models: Geometry, Generation, and Conditioning. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=dZHUgHhxKz.

[18] Anonymous. Control Enters Through the Memory: Self-Conditioning as a Control Interface in Continuous Language Flows. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=E69E9arCnW.

[19] Anonymous. Understanding and Enabling Score Distillation in Continuous Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=EKOJO3jGHl.

[20] Anonymous. CoUPLe: Continuous Flow Unifying Pixels and Language. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=GDxcnGBaWc.

[21] Anonymous. Reason Before You Speak with Latent Diffusion. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=GedeRvDLsJ.

[22] Anonymous. Asynchronous Flow Maps for Flow Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=GpVgI7Kpu4.

[23] Anonymous. The Generative Gap Between Speculative Drafter and Continuous Diffusion Language Model. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=mgQmqz3bom.

[24] Anonymous. Calibrating Noise Schedules for Continuous Language Diffusion with Token Uncertainty. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=mIGmmZK0O6.

[25] Anonymous. Exploring Unified Latent Diffusion for Multimodal Modeling. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=o0oOtuIREn.

[26] Anonymous. Reinforcing LLM Latent Reasoning with Contextual Loops through On-Policy Flow-Map Self-Distillation. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=okdvJC8ox7.

[27] Anonymous. FlowScope: Interpreting Generation Dynamics in Continuous Flow Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=OXASG2dO4Z.

[28] Anonymous. CoRe: Jointly Learning Contextual Representations for Understanding and Generation. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=PcY9mhQZT2.

[29] Anonymous. ReLPO: Resolution-Lifted Policy Optimization for Autoregressive–Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=qFLgJL8LRX.

[30] Anonymous. Reading Latent States: Adaptive Halting in Continuous Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=qX3oE4zmC8.

[31] Anonymous. Understanding Encoding in Continuous Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=rbALk4dQLc.

[32] Anonymous. The Denoising Estimate Is a State Variable: Joint Refinement for Few-Step Flow Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=sXgBvlYEjD.

[33] Anonymous. The Truncation Trap: Silent Failure of Fixed-Window Diffusion Likelihood and Information Estimators under Representation Drift. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=tEE9SNLR4F.

[34] Anonymous. Choose Before You Decode: Latent Q-Guided Generation for Diffusion Language Models. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=USTceERA7Z.

[35] Anonymous. Tilted Generator Matching: One RL Objective For Every Diffusion and Flow Model. Anonymous submission to ICLR 2027, 2026. URL https://openreview.net/forum?id=wgAXxOgnEE.

[36] Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/7ede97c3e082c6df10a8d6103a2eebd2-Abstract-Conference.html.

[37] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured Denoising Diffusion Models in Discrete State-Spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 17981–17993, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/958c530554f78bcd8e97125b70e6973d-Abstract.html.

[38] Iskander Azangulov and Leo Zhang. Sampling from Flow Language Models via Marginal-Conditioned Bridges. arXiv preprint arXiv:2605.13681, 2026. URL https://arxiv.org/abs/2605.13681.

[39] Iskander Azangulov, Kianoosh Ashouritaklimi, Leo Zhang, Simon Vary, and Patrick Rebeschini. Masked Language Flow Models. arXiv preprint arXiv:2606.27617, 2026. URL https://arxiv.org/abs/2606.27617.

[40] Hanru Bai, Faissal Izermine, Oscar Davis, and T. Konstantin Rusch. Efficient Reasoning with Flow Language Models. arXiv preprint arXiv:2610.09416, 2026. URL https://arxiv.org/abs/2610.09416.

[41] Parikshit Bansal and Sujay Sanghavi. Enabling Approximate Joint Sampling in Diffusion LMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=qFl5qT3qAP.

[42] Parikshit Bansal and Sujay Sanghavi. Token Time Continuous Diffusion for Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=HSVpmk76UW.

[43] Shubhi Bansal, Seerla Parimala, and Nagendra Kumar. Retrieval Augmented Encoder-Decoder with Diffusion for Sequential Hashtag Recommendation in Disaster Events. *Proceedings of the International AAAI Conference on Web and Social Media*, 19:206–225, 2025. doi: 10.1609/icwsm.v19i1.35812. URL https://ojs.aaai.org/index.php/ICWSM/article/view/35812.

[44] Fan Bao, Shen Nie, Kaiwen Xue, Chongxuan Li, Shi Pu, Yaole Wang, Gang Yue, Yue Cao, Hang Su, and Jun Zhu. One Transformer Fits All Distributions in Multi-Modal Diffusion at Scale. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 1692–1717. PMLR, 2023. URL https://proceedings.mlr.press/v202/bao23a.html.

[45] Georgios Batzolis, Mark Girolami, and Luca Ambrogioni. Towards Closing the Autoregressive Gap in Language Modeling via Entropy-Gated Continuous Bitstream Diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=WvVrxnsb7c. arXiv v2 title: CoBit: Language Modeling with Bitstream Diffusion.

[46] Georgios Batzolis, Georgios Kontos, Bruno Trentini, Pranam Chatterjee, Micha Livne, Mark Girolami, and Luca Ambrogioni. Multimodal Co-Generation With Continuous Bitstream Diffusion. arXiv preprint arXiv:2610.14349, 2026. URL https://arxiv.org/abs/2610.14349.

[47] Guanqun Bi, Lei Shen, Yanan Cao, Meng Chen, Yuqiang Xie, Zheng Lin, and Xiaodong He. DiffusEmp: A Diffusion Model-Based Framework with Multi-Grained Control for Empathetic Response Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2812–2831. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.158. URL https://aclanthology.org/2023.acl-long.158/.

[48] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. Flow Map Matching with Stochastic Interpolants: A Mathematical Framework for Consistency Models. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=cqDH0e6ak2.

[49] Mustapha Bounoua, Giulio Franzese, and Pietro Michiardi. Multi-Modal Latent Diffusion. *Entropy*, 26(4):320, 2024. doi: 10.3390/e26040320. URL https://arxiv.org/abs/2306.04445.

[50] Zak Buzzard. Understanding the Quality-Diversity Trade-off in Diffusion Language Models. arXiv preprint arXiv:2503.10683, 2025. URL https://arxiv.org/abs/2503.10683.

[51] Yuhang Cao. Continuous Interaction Diffusion: A Diffusion-Native Runtime for Asynchronous Tool-Augmented Reasoning. arXiv preprint arXiv:2608.10438, 2026. URL https://arxiv.org/abs/2608.10438.

[52] Michael Cardei, Huu Binh Ta, and Ferdinando Fioretto. Simple Self-Conditioning Adaptation for Masked Diffusion Models. arXiv preprint arXiv:2604.26985, 2026. URL https://arxiv.org/abs/2604.26985.

[53] Edoardo Cetin, Tianyu Zhao, and Yujin Tang. Large Language Models to Diffusion Finetuning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 7035–7057. PMLR, 2025. URL https://proceedings.mlr.press/v267/cetin25a.html.

[54] Yeonwoo Cha, Semin Kim, Jinhyeon Kwon, and Seunghoon Hong. FlowBind: Efficient Any-to-Any Generation with Bidirectional Flows. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/623e5a86fcedca573d33390dd1173e6b-Abstract-Conference.html.

[55] Nuria Alina Chandra, Yucen Lily Li, Alan N. Amin, Alex Ali, Joshua Rollins, Sebastian W. Ober, Aniruddh Raghu, and Andrew Gordon Wilson. A Unification of Discrete, Gaussian, and Simplicial Diffusion. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/45d5823950cab9a88e10c6e8dbff351c-Abstract-Conference.html.

[56] Arnav Chavan, Chanhyuk Lee, Udbhav Bamba, Jaehoon Yoo, Seunghoon Hong, Aditi Raghunathan, and Jinwoo Kim. Large Flow Language Models. arXiv preprint arXiv:2610.28559, 2026. URL https://arxiv.org/abs/2610.28559.

[57] Jannis Chemseddine, Gregor Kornhardt, and Gabriele Steidl. Spherical Flows for Sampling Categorical Data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=uVMZotQcJ7.

[58] Jiaao Chen and Diyi Yang. Controllable Conversation Generation with Conversation Structures via Diffusion Models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 7238–7251, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.454. URL https://aclanthology.org/2023.findings-acl.454/.

[59] Jiaao Chen, Aston Zhang, Mu Li, Alex Smola, and Diyi Yang. A Cheaper and Better Diffusion Language Model with Soft-Masked Noise. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4765–4775, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.289. URL https://aclanthology.org/2023.emnlp-main.289/.

[60] Lin Chen, Jingping Fang, Hairui Liu, Chenyang Xu, Junhao Chen, Xiaorui Li, Weidong Cai, and Xiaoming Chen. A First Exploration of Neuromorphic OT-CFM for Multi-Speaker VSR. In *Proceedings of the European Conference on Computer Vision (ECCV)*, Lecture Notes in Computer Science, pages 133–151. Springer, 2026. doi: 10.1007/978-3-032-37281-9_8. URL https://arxiv.org/abs/2606.31225.

[61] Qiaohong Chen, Menghao Guo, Xian Fang, and Qi Sun. Image Captioning Based on Cross-Modal Cascaded Diffusion Model. *Journal of Zhejiang University (Engineering Science)*, 59(4):787–794, April 2025. doi: 10.3785/j.issn.1008-973X.2025.04.014. URL https://www.zjujournals.com/eng/article/2025/1008-973X/20250414.shtml.

[62] Sitan Chen and Liye Wang. Parallelism, critical windows, and separations among diffusion language models. arXiv preprint arXiv:2609.20539, 2026. URL https://arxiv.org/abs/2609.20539.

[63] Tianqi Chen, Shujian Zhang, and Mingyuan Zhou. DLM-One: Diffusion Language Models for One-Step Sequence Generation. arXiv preprint arXiv:2506.00290, 2025. URL https://arxiv.org/abs/2506.00290.

[64] Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog Bits: Generating Discrete Data Using Diffusion Models with Self-Conditioning. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=3itjR9QxFw.

[65] Yuxin Chen, Chumeng Liang, Hangke Sui, Ruihan Guo, Chaoran Cheng, Jiaxuan You, and Ge Liu. LangFlow: Continuous Diffusion Rivals Discrete in Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=fUzRdnKBK4.

[66] Zhuowei Chen, Lianxi Wang, Yuben Wu, Xinfeng Liao, Yujia Tian, and Junyang Zhong. An Effective Deployment of Diffusion LM for Data Augmentation in Low-Resource Sentiment Classification. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1844–1856. Association for Computational Linguistics, November 2024. doi: 10.18653/v1/2024.emnlp-main.109. URL https://aclanthology.org/2024.emnlp-main.109/.

[67] Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. DMax: Aggressive Parallel Decoding for dLLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=RSM3ZqRlL2.

[68] Chaoran Cheng, Jiahan Li, Jian Peng, and Ge Liu. Categorical Flow Matching on Statistical Manifolds. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 54787–54819, 2024. doi: 10.52202/079017-1738. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/62a58f2130894e44e8a272c563a2c6f1-Abstract-Conference.html.

[69] Chaoran Cheng, Jiahan Li, Jiajun Fan, and Ge Liu. α-Flow: A Unified Framework for Continuous-State Discrete Flow Matching Models. arXiv preprint arXiv:2504.10283, 2025. URL https://arxiv.org/abs/2504.10283.

[70] Le Cheng and Shuangyin Li. DiffusPoll: Conditional Text Diffusion Model for Poll Generation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 925–935, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.54. URL https://aclanthology.org/2024.findings-acl.54/.

[71] Shuang Cheng, Yihan Bian, Dawei Liu, Yuhua Jiang, Yihao Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. SDAR: A Synergistic Diffusion-AutoRegression Paradigm for Scalable Sequence Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 22058–22075. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1110. URL https://aclanthology.org/2026.findings-acl.1110/.

[72] Xiang Cheng. Reasoning with Continuous Latent Diffusion. arXiv preprint arXiv:2609.35694, 2026. URL https://arxiv.org/abs/2609.35694.

[73] Zifeng Cheng, Keda Li, Zhiwei Jiang, Cong Wang, Fei Shen, and Qing Gu. Accelerating Diffusion Language Models via Structured Suffix Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=SkQxO613nL.

[74] Yun-Yen Chuang, Hung-Min Hsu, Kevin Lin, Chen-Sheng Gu, Ling Zhen Li, Ray-I Chang, and Hung-yi Lee. Meta-DiffuB: A Contextualized Sequence-to-Sequence Text Diffusion Model with Meta-Exploration. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 79843–79867. Curran Associates, Inc., 2024. doi: 10.52202/079017-2536. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/91d193b65d0b120d29503590827de1ea-Abstract-Conference.html.

[75] Jiali Cui, Yan-Ying Chen, Yanxia Zhang, and Matthew Klenk. ShaLa: Multimodal Shared Latent Generative Modelling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 20658–20666, 2026. doi: 10.1609/aaai.v40i25.39203. URL https://doi.org/10.1609/aaai.v40i25.39203.

[76] Junxia Cui, Haotian Ye, Runchu Tian, Hongcan Guo, Jinya Jiang, Haoru Li, Chaojie Ren, Yiming Huang, Kaijie Zhu, Zhongkai Yu, Kun Zhou, and Jingbo Shang. SimSD: Simple Speculative Decoding in Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=NYqEuoKbxZ.

[77] Zishun Cui, Xiao Sun, and Meng Wang. Modeling Latent Background Information in Multiturn Dialogues Using Diffusion Model. *IEEE Transactions on Artificial Intelligence*, pages 1–14, 2026. doi: 10.1109/TAI.2026.3671250. URL https://doi.org/10.1109/TAI.2026.3671250. Early access.

[78] Shailesh Dahal, Ratri Mukherjee, Nicholas Mathews, Bijaya Adhikari, and Kishlay Jha. Semantic Knowledge Guided Diffusion Model for Biomedical Ontology Definition Generation. *Journal of*

Healthcare Informatics Research, 10(3):710–730, 2026. doi: 10.1007/s41666-026-00240-0. URL https://link.springer.com/article/10.1007/s41666-026-00240-0.

[79] Oscar Davis, Samuel Kessler, Mircea Petrache, İsmail İlkan Ceylan, Michael Bronstein, and Avishek Joey Bose. Fisher Flow Matching for Generative Modeling over Discrete Data. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 139054–139084, 2024. doi: 10.52202/079017-4413. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/fadec8f2e65f181d777507d1df69b92f-Abstract-Conference.html.

[80] Oscar Davis, Anastasiia Filippova, Pierre Ablin, Victor Turrisi, Amitis Shidani, Marco Cuturi, and Louis Béthune. Scaling Categorical Flow Maps. arXiv preprint arXiv:2605.07820, 2026. URL https://arxiv.org/abs/2605.07820.

[81] Oscar Davis, Daan Roos, Alice Bizeul, Amitis Shidani, Victor Turrisi, Louis Béthune, Pierre Ablin, and Marco Cuturi. On the Design Space of Continuous Diffusion Language Models. arXiv preprint arXiv:2610.13834, 2026. URL https://arxiv.org/abs/2610.13834.

[82] Yunus Demirag, Danni Liu, and Jan Niehues. Benchmarking Diffusion Models for Machine Translation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, pages 313–324. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.eacl-srw.25. URL https://aclanthology.org/2024.eacl-srw.25/.

[83] Justin Deschenaux and Caglar Gulcehre. Language Modeling with Hyperspherical Flows. *Transactions on Machine Learning Research*, 2026. URL https://openreview.net/forum?id=jBh2WNssnL.

[84] Justin Deschenaux, Alexandre Galashov, Andrew Campbell, Li Kevin Wenliang, James Thornton, Arnaud Doucet, and Valentin De Bortoli. Simplex Diffusion Models. arXiv preprint arXiv:2609.35553, 2026. URL https://arxiv.org/abs/2609.35553.

[85] Justin Deschenaux, Caglar Gulcehre, and Subham Sekhar Sahoo. The Diffusion Duality, Chapter II: Ψ-Samplers. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/9adf58ca1ed848228377d61692d66541-Abstract-Conference.html.

[86] Sander Dieleman. Continuous diffusion language models. Blog post, 24 August 2026, https://sander.ai/2026/08/24/continuous-dlms.html, 2026. Accessed 4 October 2026.

[87] Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H. Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, Curtis Hawthorne, Rémi Leblond, Will Grathwohl, and Jonas Adler. Continuous diffusion for categorical data. arXiv preprint arXiv:2211.15089, 2022. URL https://arxiv.org/abs/2211.15089.

[88] DiffusionGemma Team. DiffusionGemma Technical Report. arXiv preprint arXiv:2608.00146, 2026. URL https://arxiv.org/abs/2608.00146.

[89] Rohit Dilip, Tianrong Chen, Yuyang Wang, David Van Valen, Joshua Susskind, and Miguel Angel Bautista. How to Guide Your Language Flow. arXiv preprint arXiv:2609.19356, 2026. URL https://arxiv.org/abs/2609.19356.

[90] Hanxing Ding, Liang Pang, Zihao Wei, Huawei Shen, Xueqi Cheng, and Tat-Seng Chua. MacLaSa: Multi-Aspect Controllable Text Generation via Efficient Sampling from Compact Latent Space. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4424–4436. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.292. URL https://aclanthology.org/2023.findings-emnlp.292/.

[91] Yizhou Ding, Jidong Tian, Shanxing Mei, Yifan Zhou, Yuwu Dong, Hao He, and Weisheng Hu. LDSeq: Latent Diffusion Models for Sequence to Sequence Text Generation. In *Proceedings of the 2023 7th International Conference on Computer Science and Artificial Intelligence (CSAI)*, pages 546–552. ACM, 2023. doi: 10.1145/3638584.3638617. URL https://doi.org/10.1145/3638584.3638617.

[92] Xiangyun Dong, Wei Li, Yuquan Le, Zhangyue Jiang, Junxi Zhong, and Zhong Wang. TermDiffuSum: A Term-guided Diffusion Model for Extractive Summarization of Legal Documents. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 3222–3235, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.coling-main.216/.

[93] ZiYi Dong, Yuliang Huang, Weijian Deng, Xiangyang Ji, Liang Lin, and Pengxu Wei. Language Generation as Optimal Control: Closed-Loop Diffusion in Latent Control Space. arXiv preprint arXiv:2605.14531, 2026. URL https://arxiv.org/abs/2605.14531.

[94] Nikita Dragunov, Temurbek Rahmatullaev, Elizaveta Goncharova, Nikita Kurdiukov, Aysel Mirzoeva, Anna Borisiuk, Andrey Kuznetsov, and Anton Razzhigaev. SONAR-LLM: Autoregressive Transformer that Thinks in Sentence Embeddings and Speaks in Tokens. arXiv preprint arXiv:2508.05305, 2025. URL https://arxiv.org/abs/2508.05305.

[95] Ilya Drobyshevskiy, Ilia Sudakov, Maksim Semenov, Denis Kuznedelev, Maksim Ignatov, Pavel Temirchev, Nikita Balagansky, Viacheslav Meshchaninov, Nikita Gushchin, and Dmitry Baranchuk. Representation-Space MMD for Diffusion Language Models. arXiv preprint arXiv:2610.06648, 2026. URL https://arxiv.org/abs/2610.06648.

[96] Li Du, Afra Amini, Lucas Torroba Hennigen, Xinyan Velocity Yu, Holden Lee, Jason Eisner, and Ryan Cotterell. Principled Gradient-Based MCMC for Conditional Sampling of Text. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 11663–11685. PMLR, 2024. URL https://proceedings.mlr.press/v235/du24a.html.

[97] Zhicheng Du and Lan Ma. Continuous Language Diffusion as a Decoder-Interface Problem. arXiv preprint arXiv:2606.08810, 2026. URL https://arxiv.org/abs/2606.08810.

[98] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. SONAR: Sentence-Level Multimodal and Language-Agnostic Representations. arXiv preprint arXiv:2308.11466, 2023. URL https://arxiv.org/abs/2308.11466.

[99] Bradley Efron. Tweedie's Formula and Selection Bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011. doi: 10.1198/jasa.2011.tm11181.

[100] Joshua Engels, Callum McDougall, Bilal Chughtai, Janos Kramar, Senthooran Rajamanoharan, Cindy Wu, Arthur Conmy, Asic Q Chen, Jean Tarbouriech, Min Ma, Brendan O'Donoghue, João Gabriel Lopes de Oliveira, Rohin Shah, and Neel Nanda. How Transparent is DiffusionGemma? arXiv preprint arXiv:2606.20560, 2026. URL https://arxiv.org/abs/2606.20560.

[101] Yubo Fang, Hai-Tao Yu, Hideo Joho, and Sumio Fujita. Retrieval-Augmented Diffusion Language Model for Generative Commonsense Reasoning. In *Database Systems for Advanced Applications (DASFAA 2026)*, volume 16537 of *Lecture Notes in Computer Science*, pages 341–356. Springer, 2026. doi: 10.1007/978-981-92-0369-7_22. URL https://doi.org/10.1007/978-981-92-0369-7_22.

[102] Yubo Fang, Hai-Tao Yu, Hideo Joho, Sumio Fujita, Yan Ge, and Chao Lei. An Empirical Study of Retrieval-Augmented Diffusion Language Models for Generative Commonsense Reasoning. *Information Sciences*, 761:124147, 2027. doi: 10.1016/j.ins.2026.124147. URL https://doi.org/10.1016/j.ins.2026.124147.

[103] Xinsong Feng, Peng Du, Zhizhuo Yang, Daniel M. Bikel, Jiayun Wang, and Haipeng Chen. Denoising Blocks, Not Tokens: Efficient Compressed Continuous Diffusion with Branching Token Realization. arXiv preprint arXiv:2610.09311, 2026. URL https://arxiv.org/abs/2610.09311.

[104] Griffin Floto, Mohammad Mahdi Abdollah Pour, Parsa Farinneya, Zhenwei Tang, Ali Pesaranghader, Manasa Bharadwaj, and Scott Sanner. DiffuDetox: A Mixed Diffusion Model for Text Detoxification. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 7566–7574, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.478. URL https://aclanthology.org/2023.findings-acl.478/.

[105] Antonio Franca and Alexander Tong. Hacking Generative Perplexity: Why Unconditional Text Evaluation Needs Distributional Metrics. In *4th Workshop on Structured Probabilistic Inference & Generative Modeling (SPIGM) at ICML 2026*, 2026. URL https://arxiv.org/abs/2606.08417v2.

[106] Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, Sharath Turuvekere Sreenivas, Shizhe Diao, Chenhan D. Yu, Ye Yu, Weijia Chen, Sajad Norouzi, Jingyu Liu, Shiyi Lan, Ligeng Zhu, Jin Wang, Jindong Jiang, Morteza Mardani, Mehran Maghoumi, Song Han, Ante Jukić, Nima Tajbakhsh, Jan Kautz, and Pavlo Molchanov. Nemotron-Labs-Diffusion: A Tri-Mode Language Model Unifying Autoregressive, Diffusion, and Self-Speculation Decoding. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 54868–54887, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.emnlp-main.2416/.

[107] Naoaki Fujita, Hiroki Nakamura, and Kosuke Itakura. Diverse and Few-Step Audio Captioning via Flow Matching. In *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 15492–15496, 2026. doi: 10.1109/ICASSP55912.2026.11462589. URL https://doi.org/10.1109/ICASSP55912.2026.11462589.

[108] Miao Gao, Chuanqi Shi, Baixuan Li, Yikemaiti Sataer, and Youyong Kong. DiffuER: Auxiliary Regularized Diffusion Model for Text Generation with Semantic Consistency. In *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2025. doi: 10.1109/IJCNN64981.2025.11228459. URL https://doi.org/10.1109/ijcnn64981.2025.11228459.

[109] Silin Gao, Mete Ismayilzada, Mengjie Zhao, Hiromi Wakaki, Yuki Mitsufuji, and Antoine Bosselut. DiffuCOMET: Contextual Commonsense Knowledge Diffusion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4809–4831. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.264. URL https://aclanthology.org/2024.acl-long.264/.

[110] Xuanqi Gao, Weipeng Jiang, Juan Zhai, Shiqing Ma, Siyi Xie, Xinyang Yin, and Chao Shen. Mitigating Stylistic Biases of Machine Translation Systems via Monolingual Corpora Only. arXiv preprint arXiv:2507.13395, 2025. URL https://arxiv.org/abs/2507.13395.

[111] Zhujin Gao, Junliang Guo, Xu Tan, Yongxin Zhu, Fang Zhang, Jiang Bian, and Linli Xu. Empowering Diffusion Models on the Embedding Space for Text Generation. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1:*

Long Papers), pages 4664–4683, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.261. URL https://aclanthology.org/2024.naacl-long.261/.

[112] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete Flow Matching. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 133345–133385, 2024. doi: 10.52202/079017-4239. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/f0d629a734b56a642701bba7bc8bb3ed-Abstract-Conference.html.

[113] Arpit Singh Gautam, Kailash Talreja, and Saurabh Jha. The Energy of Falsehood: Detecting Hallucinations via Diffusion Model Likelihoods. In *Proceedings of the Ninth Fact Extraction and VERification Workshop (FEVER)*, pages 47–58. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.fever-1.4. URL https://aclanthology.org/2026.fever-1.4/.

[114] Alexey Gavrilov, Alan-Barsag Gazzaev, and Sergey Muravyov. Where Quality Breaks in Compressed Short-Text Generation: Staged Bottleneck Localization. In *Proceedings of the 39th Conference of the Open Innovations Association FRUCT*, pages 69–76, 2026. doi: 10.23919/FRUCT70069.2026.11506553. URL https://arxiv.org/abs/2607.24176.

[115] Jonas Geiping, Xinyu Yang, and Guinan Su. Efficient Parallel Samplers for Recurrent-Depth Models and Their Connection to Diffusion Language Models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 34442–34462. PMLR, 2026. URL https://proceedings.mlr.press/v306/geiping26a.html.

[116] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean Flows for One-step Generative Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 84032–84054, 2025. doi: 10.52202/085713-2534. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/6d13e085b79d454da5910e4ca82a3d9d-Abstract-Conference.html.

[117] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-Predict: Parallel Decoding of Conditional Masked Language Models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6112–6121. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1633. URL https://aclanthology.org/D19-1633/.

[118] Heng Gong, Xiaocheng Feng, and Bing Qin. DiffuD2T: Empowering Data-to-Text Generation with Diffusion. *Electronics*, 12(9):2136, 2023. doi: 10.3390/electronics12092136. URL https://www.mdpi.com/2079-9292/12/9/2136.

[119] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. DiffuSeq: Sequence to Sequence Text Generation with Diffusion Models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=jQj-_rLVXsj.

[120] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. DiffuSeq-v2: Bridging Discrete and Continuous Text Spaces for Accelerated Seq2Seq Diffusion Models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9868–9875, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.660. URL https://aclanthology.org/2023.findings-emnlp.660/.

[121] Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Hao Peng, Jiawei Han, and Lingpeng Kong. Scaling Diffusion Language

Models via Adaptation from Autoregressive Models. In *International Conference on Learning Representations (ICLR)*, volume 2025, pages 5046–5073, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/0fa81c3f0d57f95b8776de3a248ef0ed-Abstract-Conference.html.

[122] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. DiffuCoder: Understanding and Improving Masked Diffusion Models for Code Generation. In *International Conference on Learning Representations (ICLR)*, volume 2026, pages 15723–15750, 2026. URL https://openreview.net/forum?id=58NA3unZj5.

[123] Google DeepMind. Gemini Diffusion. https://deepmind.google/models/gemini-diffusion/, 2025. Model page of the experimental text diffusion model. Accessed 4 October 2026.

[124] Alex Graves, Rupesh Kumar Srivastava, Timothy Atkinson, and Faustino Gomez. Bayesian Flow Networks. arXiv preprint arXiv:2308.07037, 2023. URL https://arxiv.org/abs/2308.07037.

[125] Yuxuan Gu, Xiaocheng Feng, Sicheng Ma, Lingyuan Zhang, Heng Gong, and Bing Qin. A Distributional Lens for Multi-Aspect Controllable Text Generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1023–1043. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.67. URL https://aclanthology.org/2022.emnlp-main.67/.

[126] Yuxuan Gu, Xiaocheng Feng, Sicheng Ma, Lingyuan Zhang, Heng Gong, Weihong Zhong, and Bing Qin. Controllable Text Generation via Probability Density Estimation in the Latent Space. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12590–12616. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.704. URL https://aclanthology.org/2023.acl-long.704/.

[127] Yuxuan Gu, Xiaocheng Feng, Lei Huang, Yingsheng Wu, Zekun Zhou, Weihong Zhong, Kun Zhu, and Bing Qin. Discrete Modeling via Boundary Conditional Diffusion Processes. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 121204–121235, 2024. doi: 10.52202/079017-3852. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/dbae915128892556134f1c5375855590-Abstract-Conference.html.

[128] Ishaan Gulrajani and Tatsunori B. Hashimoto. Likelihood-Based Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 16693–16715, 2023. doi: 10.52202/075280-0730. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/35b5c175e139bff5f22a5361270fce87-Abstract-Conference.html.

[129] Gabe Guo, Elon Litman, Thanawat Sornwanee, Jose Blanchet, and Stefano Ermon. There and Back Again: Bidirectional Diffusion Bridges for Multimodality Translation. arXiv preprint arXiv:2608.27885, 2026. URL https://arxiv.org/abs/2608.27885.

[130] Guangfu Guo, Xiaoqian Lu, Linsey Pang, Weiran Yao, Haolin Chen, Kunpeng Liu, and Long Cheng. DeltaFlow: Noise-Adaptive Bidirectional Gated Delta Networks for Embedded Language Flows. arXiv preprint arXiv:2608.01240, 2026. URL https://arxiv.org/abs/2608.01240.

[131] Hongcan Guo, Qinyu Zhao, Yian Zhao, Shen Nie, Rui Zhu, Qiushan Guo, Feng Wang, Tao Yang, Hengshuang Zhao, Guoqiang Wei, and Yan Zeng. Continuous Latent Diffusion Language Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=BXGQBSGCtv.

[132] Hongcan Guo, Yian Zhao, Feng Wang, Shen Nie, Guoqiang Wei, Yan Zeng, and Hengshuang Zhao. Cola2: Co-Adaptive Hierarchical Language Modeling with Continuous Semantic Compression and Balanced Discrete Generation. arXiv preprint arXiv:2610.31305, 2026. URL https://arxiv.org/abs/2610.31305.

[133] Aditi Gupta, Neel Mishra, Kushagra Trivedi, and Pawan Kumar. Speculative Refinement: A Hybrid Autoregressive Diffusion Decoding Strategy and Its Behavior Across Benchmarks. In *Proceedings of the Fifth Workshop on Generation, Evaluation and Metrics (GEM)*, pages 355–363. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.gem-main.33. URL https://aclanthology.org/2026.gem-main.33/.

[134] Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. SSD-LM: Semi-autoregressive Simplex-based Diffusion Language Model for Text Generation and Modular Control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11575–11596. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.647. URL https://aclanthology.org/2023.acl-long.647/.

[135] Xiaochuang Han, Sachin Kumar, Yulia Tsvetkov, and Marjan Ghazvininejad. David helps Goliath: Inference-Time Collaboration Between Small Specialized and Large General Diffusion LMs. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8385–8400. Association for Computational Linguistics, June 2024. doi: 10.18653/v1/2024.naacl-long.464. URL https://aclanthology.org/2024.naacl-long.464/.

[136] Xiaoxu Han, Chengzhen Ning, Jinghui Zhong, Fubiao Yang, Yu Wang, and Xin Mu. Discovering Mathematical Equations with Diffusion Language Model. arXiv preprint arXiv:2509.13136, 2025. URL https://arxiv.org/abs/2509.13136.

[137] Xu Han, Jiajing Hu, and Li-Ping Liu. Backpropagating Verifier Rewards Through Continuous Flow Language Models. arXiv preprint arXiv:2610.19080, 2026. URL https://arxiv.org/abs/2610.19080.

[138] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*, 2025. URL https://arxiv.org/abs/2412.06769.

[139] Etrit Haxholli. Flow Duality and Source Geometry for Categorical Generation. arXiv preprint arXiv:2609.10863, 2026. URL https://arxiv.org/abs/2609.10863.

[140] Jinjin He, Xinsong Feng, Lucheng Fu, Sinan Wang, Bo Zhu, and Jiayun Wang. Rethinking the Roles of Diffusion and Decoding: Predictive-State Flows for Parallel Language Generation. arXiv preprint arXiv:2610.12898, 2026. URL https://arxiv.org/abs/2610.12898.

[141] Ju He, Qihang Yu, Qihao Liu, and Liang-Chieh Chen. FlowTok: Flowing Seamlessly Across Text and Image Tokens. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16629–16640, 2025. URL https://openaccess.thecvf.com/content/ICCV2025/html/He_FlowTok_Flowing_Seamlessly_Across_Text_and_Image_Tokens_ICCV_2025_paper.html.

[142] Yufeng He, Zefan Cai, Xu Gan, and Baobao Chang. DiffCap: Exploring Continuous Diffusion on Image Captioning. arXiv preprint arXiv:2305.12144, 2023. URL https://arxiv.org/abs/2305.12144.

[143] DongNyeong Heo, Daniela Noemi Rim, and Heeyoul Choi. Diverse Target Representations for Language Models with Word Difference Representations. In Chu-Ren Huang, Yasunari Harada,

Jong-Bok Kim, Nguyen T.M. Huyen, Le Thanh Huong, Pham Hien, Emmanuele Chersoni, Le Minh Nguyen, Rachel Edita Oñate Roxas, and Sherly Dita, editors, *Proceedings of the 39th Pacific Asia Conference on Language, Information and Computation*, pages 742–754, Hanoi, Vietnam, dec 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.paclic-1.68/.

[144] DongNyeong Heo, Taehwan Kim, and Heeyoul Choi. Sentence Curve Language Models. arXiv preprint arXiv:2602.01807, 2026. URL https://arxiv.org/abs/2602.01807.

[145] Michael Hersche, Nicolas Menet, Ronan Tanios, and Abbas Rahimi. Locally Coherent Parallel Decoding in Diffusion Language Models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 42855–42879. PMLR, 2026. URL https://proceedings.mlr.press/v306/hersche26a.html.

[146] Michael Hersche, Samuel Moor-Smith, Thomas Hofmann, and Abbas Rahimi. Soft-Masked Diffusion Language Models. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=Gba02UMvrG.

[147] Satoko Hirano and Ichiro Kobayashi. Interactive Image Caption Generation Reflecting User Intent from Trace Using a Diffusion Language Model. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 29(6):1417–1426, 2025. doi: 10.20965/jaciii.2025.p1417. URL https://doi.org/10.20965/jaciii.2025.p1417.

[148] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance. arXiv preprint arXiv:2207.12598, 2022. URL https://arxiv.org/abs/2207.12598.

[149] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html.

[150] Zachary Horvitz, Ajay Patel, Chris Callison-Burch, Zhou Yu, and Kathleen McKeown. ParaGuide: Guided Diffusion Paraphrasers for Plug-and-Play Textual Style Transfer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18216–18224, 2024. doi: 10.1609/aaai.v38i16.29780. URL https://ojs.aaai.org/index.php/AAAI/article/view/29780.

[151] Zachary Horvitz, Raghav Singhal, Hao Zou, Carles Domingo-Enrich, Zhou Yu, Rajesh Ranganath, and Kathleen McKeown. Rethinking Reasoning with MDLMs: Early Exits, Post-hoc Reasoning, and Beyond. In *Conference on Language Modeling (COLM)*, 2026. URL https://proceedings.colm.cc/paper_files/paper/2026/hash/e2a1ebfc9d91cfab030c503e4ed8a284-Abstract-Conference.html.

[152] Jia Cheng Hu, Roberto Cavicchioli, and Alessandro Capotondi. Diffusion Is Your Friend in Show, Suggest and Tell. In *2025 IEEE International Conference on Big Data (BigData)*, pages 5142–5151, 2025. doi: 10.1109/BigData66926.2025.11400981. URL https://doi.org/10.1109/BigData66926.2025.11400981.

[153] Keya Hu, Linlu Qiu, Yiyang Lu, Hanhong Zhao, Tianhong Li, Yoon Kim, Jacob Andreas, and Kaiming He. ELF: Embedded Language Flows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=z3I6Aj563f.

[154] Vincent Tao Hu, Di Wu, Yuki M. Asano, Pascal Mettes, Basura Fernando, Björn Ommer, and Cees G. M. Snoek. Flow Matching for Conditional Text Generation in a Few Sampling Steps. In Yvette Graham and Matthew Purver, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*,

pages 380–392, St. Julian's, Malta, mar 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.eacl-short.33. URL https://aclanthology.org/2024.eacl-short.33/.

[155] Yuezhou Hu, Harman Singh, Monishwaran Maheswaran, Haocheng Xi, Coleman Hooper, Jintao Zhang, Aditya Tomar, Michael W. Mahoney, Sewon Min, Mehrdad Farajtabar, Kurt Keutzer, Amir Gholami, and Chenfeng Xu. Residual Context Diffusion Language Models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 45657–45672. PMLR, 2026. URL https://proceedings.mlr.press/v306/hu26as.html.

[156] Zhiyuan Hu, Chumin Liu, Yue Feng, Anh Tuan Luu, and Bryan Hooi. PoetryDiffusion: Towards Joint Semantic and Metrical Manipulation in Poetry Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18279–18288, 2024. doi: 10.1609/aaai.v38i16.29787. URL https://ojs.aaai.org/index.php/AAAI/article/view/29787.

[157] Jingchi Huang, Dashun Zheng, Jiaxuan Li, and Patrick Cheong-Iao Pang. EmbDiffounder: Semantic-Transfer Enhanced Sequence Diffusion Model for Text Generation. *IEEE Access*, 13: 205521–205531, 2025. doi: 10.1109/ACCESS.2025.3638810. URL https://doi.org/10.1109/ACCESS.2025.3638810.

[158] Shaoyao Huang, Luozheng Qin, and Ziqiang Cao. Diffusion Language Model with Query-Document Relevance for Query-Focused Summarization. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 11020–11030, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.735. URL https://aclanthology.org/2023.findings-emnlp.735/.

[159] Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, Stefano Ermon, Aditya Grover, and Volodymyr Kuleshov. Mercury: Ultra-Fast Language Models Based on Diffusion. arXiv preprint arXiv:2506.17298, 2025. URL https://arxiv.org/abs/2506.17298.

[160] Arseny Ivanov, Sergei Kholkin, Vladislav Gromadskii, Grigoriy Ksenofontov, Ivan Oseledets, and Alexander Korotin. TUBE: Tangent Upper Bound on Evidence for Discrete Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=1P9BKutXbt.

[161] Faissal Izermine, Hanru Bai, Oscar Davis, and T. Konstantin Rusch. Unrolled Flow Models for Reasoning. arXiv preprint arXiv:2610.09759, 2026. URL https://arxiv.org/abs/2610.09759.

[162] Eric Jang, Shixiang Gu, and Ben Poole. Categorical Reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01144.

[163] Xiaohuan Jia, Renzhe Xu, Xiao Wang, Jiayun Wu, and Shaohua Fan. Rectifying Categorical Flows on Statistical Manifolds for One-Step Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=WQqyE4RiCg.

[164] Zijian Jia, Peixuan Zhang, Genying Li, Di Wu, Yujie Fu, Si Li, Boxin Shi, and Shuchen Weng. LLaCO: Continuous Diffusion for Large Language Models with Competition-Aware Semantic Scheduling. arXiv preprint arXiv:2610.32604, 2026. URL https://arxiv.org/abs/2610.32604.

[165] Feibo Jiang, Li Dong, Yubo Peng, Kezhi Wang, Kun Yang, Cunhua Pan, and Xiaohu You. Large AI Model Empowered Multimodal Semantic Communications. *IEEE Communications Magazine*, 63(1): 76–82, 2025. doi: 10.1109/MCOM.001.2300575. URL https://arxiv.org/abs/2309.01249.

[166] Jiaxiu Jiang, Jingjing Ren, Wenbo Li, Bo Wang, Haoze Sun, Yijun Yang, Jianhui Liu, Yanbing Zhang, Shenghe Zheng, Yuan Zhang, Haoyang Huang, Nan Duan, and Wangmeng Zuo. TextLDM: Language Modeling with Continuous Latent Diffusion. arXiv preprint arXiv:2605.07748, 2026. URL https://arxiv.org/abs/2605.07748.

[167] Zhongyu Jiang, Wenhao Chai, Zhuoran Zhou, Cheng-Yen Yang, Hsiang-Wei Huang, and Jenq-Neng Hwang. PackDiT: Joint Human Motion and Text Generation via Mutual Prompting. arXiv preprint arXiv:2501.16551, 2025. URL https://arxiv.org/abs/2501.16551.

[168] Jaehyeong Jo and Sung Ju Hwang. Continuous Diffusion Model for Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 97394–97430, 2025. doi: 10.52202/085713-3256. URL https://openreview.net/forum?id=VGv5y60sXC.

[169] Mingyu Jo, Jaesik Yoon, Justin Deschenaux, Caglar Gulcehre, and Sungjin Ahn. Loopholing Discrete Diffusion: Deterministic Bypass of the Sampling Wall. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/975d11c51406cd10be48e47b36fb8698-Abstract-Conference.html.

[170] Justin Jung. Noise Your Prompt: Noising Conditioning Tokens in Continuous Diffusion Language Models. *Transactions on Machine Learning Research*, 2026. URL https://openreview.net/forum?id=carxDjFbtO.

[171] Pascal Jutras-Dubé, Patrick Pynadath, Jeremy Lu, Yuan Gao, and Ruqi Zhang. Sticky Jump Diffusions: A Unifying Framework for Discrete, Continuous, and Hybrid Diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=TmV4qk3gOS.

[172] Jaakko Kainulainen, Zixin Guo, and Jorma Laaksonen. Diffusion-based Multimodal Video Captioning. In *Proceedings of the Asian Conference on Computer Vision (ACCV)*, pages 2820–2837, December 2024. URL https://openaccess.thecvf.com/content/ACCV2024/html/Kainulainen_Diffusion-based_Multimodal_Video_Captioning_ACCV_2024_paper.html.

[173] Cheng Kang, Xinye Chen, Yong Hu, and Daniel Novak. Quantized Embedding Vectors for Controllable Diffusion Language Models. arXiv preprint arXiv:2402.10107, 2024. URL https://arxiv.org/abs/2402.10107.

[174] Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Yi-An Ma, and Lianhui Qin. LaDi-RL: Latent Diffusion Reasoning Prevents Entropy Collapse in Reinforcement Learning. arXiv preprint arXiv:2602.01705, 2026. URL https://arxiv.org/abs/2602.01705.

[175] Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Nicklas Majamaki, Navdeep Jaitly, Yi-An Ma, and Lianhui Qin. LaDiR: Latent Diffusion Enhances LLMs for Text Reasoning. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/843e1d8ab4819c8c8a4221fce2c8d455-Abstract-Conference.html.

[176] Rabeeh Karimi Mahabadi, Hamish Ivison, Jaesung Tae, James Henderson, Iz Beltagy, Matthew E. Peters, and Arman Cohan. TESS: Text-to-Text Self-Conditioned Simplex Diffusion. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2347–2361. Association for Computational Linguistics, March 2024. doi: 10.18653/v1/2024.eacl-long.144. URL https://aclanthology.org/2024.eacl-long.144/.

[177] Ella Kemperman and Luca Ambrogioni. Adaptive Second-Order Solvers for Fast Stochastic Diffusion Sampling. arXiv preprint arXiv:2610.03034, 2026. URL https://arxiv.org/abs/2610.03034.

[178] Harsha Vardhan Khurdula, Abhinav Kumar Singh, Yoeven D Khemlani, and Vineet Agarwal. Audio-Native Speech Recognition with a Frozen Discrete-Diffusion Language Model. arXiv preprint arXiv:2607.13013, 2026. URL https://arxiv.org/abs/2607.13013.

[179] Duc Kieu, Kien Do, Minh Khoa Le, Tung Kieu, Dang Nguyen, and Thin Nguyen. Modaliff: A Unified Token Diffusion Model for Any-to-Any Generation. arXiv preprint arXiv:2610.25533, 2026. URL https://arxiv.org/abs/2610.25533.

[180] Jinwoo Kim, Taylor Berg-Kirkpatrick, and Loris D'Antoni. Continuous Diffusion Models Can Obey Formal Syntax. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 57136–57151. PMLR, 2026. URL https://proceedings.mlr.press/v306/kim26b.html.

[181] Mingyeong Kim, Gyusang Cho, Juho Lee, and Giung Nam. Rationale Distribution Distillation via Flow Matching for Distributional Continuous Latent Reasoning. arXiv preprint arXiv:2610.19871, 2026. URL https://arxiv.org/abs/2610.19871.

[182] Woojin Kim and Jaeyoung Do. Don't Let It Fade: Preserving Edits in Diffusion Language Models via Token Timestep Allocation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 148863–148898, 2025. doi: 10.52202/085713-4979. URL https://openreview.net/forum?id=LOVuux2kvs.

[183] Yeongmin Kim, Arnaud Doucet, Andrew Campbell, Valentin De Bortoli, Thomas Mensink, and David Ruhe. Gumbel Straight Flow: Distilling Autoregressive Models into One-step Flow Maps. arXiv preprint arXiv:2610.00497, 2026. URL https://arxiv.org/abs/2610.00497.

[184] Injin Kong, Hyoungjoon Lee, and Yohan Jo. Where Should Diffusion Enter a Language Model? Geometry-Guided Hidden-State Replacement. arXiv preprint arXiv:2605.14368, 2026. URL https://arxiv.org/abs/2605.14368.

[185] Gregor Kornhardt, Moritz Piening, Jannis Chemseddine, and Gabriele Steidl. SOL: Measuring Gaps between Text Distributions by Double Sliced Wasserstein Metrics. arXiv preprint arXiv:2610.06513, 2026. URL https://arxiv.org/abs/2610.06513.

[186] Theodoros Kouzelis, Spyros Gidaris, and Nikos Komodakis. Self-Regularized Flow Language Models. arXiv preprint arXiv:2610.37115, 2026. URL https://arxiv.org/abs/2610.37115.

[187] Aleksei S. Krylov and Oleg D. Somov. Table-to-Text Generation with Pretrained Diffusion Models. *IEEE Access*, 12:110517–110525, 2024. doi: 10.1109/ACCESS.2024.3440006. URL https://arxiv.org/abs/2409.13739.

[188] Sachin Kumar, Eric Malmi, Aliaksei Severyn, and Yulia Tsvetkov. Controlled Text Generation as Continuous Optimization with Multiple Constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 14542–14554. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/79ec2a4246feb2126ecf43c4a4418002-Abstract.html.

[189] Sachin Kumar, Biswajit Paria, and Yulia Tsvetkov. Gradient-based Constrained Sampling from Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2251–2277. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.144. URL https://aclanthology.org/2022.emnlp-main.144/.

[190] LCM team, Loïc Barrault, Paul-Ambroise Duquenne, Maha Elbayad, Artyom Kozhevnikov, Belen Alastruey, Pierre Andrews, Mariano Coria, Guillaume Couairon, Marta R. Costa-jussà, David Dale, Hady Elsahar, Kevin Heffernan, João Maria Janeiro, Tuan Tran, Christophe Ropers, Eduardo

Sánchez, Robin San Roman, Alexandre Mourachko, Safiyyah Saleem, and Holger Schwenk. Large Concept Models: Language Modeling in a Sentence Representation Space. arXiv preprint arXiv:2412.08821, 2024. URL https://arxiv.org/abs/2412.08821.

[191] Chanhyuk Lee, Jaehoon Yoo, Manan Agarwal, Sheel Shah, Jerry Huang, Aditi Raghunathan, Seunghoon Hong, Nicholas M. Boffi, and Jinwoo Kim. Flow Map Language Models: One-step Language Modeling via Continuous Denoising. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=VqiKJfJ98m.

[192] Che Hyun Lee, Heeseung Kim, Jiheum Yeom, and Sungroh Yoon. EdiText: Controllable Coarse-to-Fine Text Editing with Diffusion Language Models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22798–22815, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1111. URL https://aclanthology.org/2025.acl-long.1111/.

[193] Jason Lee, Raphael Shu, and Kyunghyun Cho. Iterative Refinement in the Continuous Space for Non-Autoregressive Neural Machine Translation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1006–1015. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.73. URL https://aclanthology.org/2020.emnlp-main.73/.

[194] Jeong Ryong Lee, Yejee Shin, Geonhui Son, and Dosik Hwang. Diffusion Bridge: Leveraging Diffusion Model to Reduce the Modality Gap Between Text and Vision for Zero-Shot Image Captioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4050–4059, 2025. doi: 10.1109/cvpr52734.2025.00383. URL https://openaccess.thecvf.com/content/CVPR2025/html/Lee_Diffusion_Bridge_Leveraging_Diffusion_Model_to_Reduce_the_Modality_Gap_CVPR_2025_paper.html.

[195] Jean-Marie Lemercier, Tomas Geffner, Morteza Mardani, Karsten Kreis, Arash Vahdat, and Ante Jukić. DiLaDiff: Distilled Latent-Augmented Diffusion for Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=l1EueDpKBW.

[196] Aihua Li. Towards Faster Language Model Inference Using Mixture-of-Experts Flow Matching. arXiv preprint arXiv:2604.15009, 2026. URL https://arxiv.org/abs/2604.15009.

[197] Bocheng Li and Linli Xu. Improving Few-Step Language Flows with Untied Self-Conditioning. arXiv preprint arXiv:2608.22244, 2026. URL https://arxiv.org/abs/2608.22244.

[198] Bocheng Li, Zhujin Gao, Yongxin Zhu, Kun Yin, Haoyu Cao, Deqiang Jiang, and Linli Xu. Few-shot Temporal Pruning Accelerates Diffusion Models for Text Generation. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 7259–7269, Torino, Italia, May 2024. ELRA and ICCL. URL https://aclanthology.org/2024.lrec-main.637/.

[199] Bocheng Li, Zhujin Gao, and Linli Xu. Unifying Continuous and Discrete Text Diffusion with Non-simultaneous Diffusion Processes. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11530–11551, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.565. URL https://aclanthology.org/2025.acl-long.565/.

[200] Chenyang Li, Long Zhang, and Qiusheng Zheng. Utilizing Latent Diffusion Model to Accelerate Sampling Speed and Enhance Text Generation Quality. *Electronics*, 13(6):1093, 2024. doi: 10.3390/electronics13061093. URL https://www.mdpi.com/2079-9292/13/6/1093.

[201] Chunyuan Li, Xiang Gao, Yuan Li, Baolin Peng, Xiujun Li, Yizhe Zhang, and Jianfeng Gao. Optimus: Organizing Sentences via Pre-trained Modeling of a Latent Space. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4678–4699. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.378. URL https://aclanthology.org/2020.emnlp-main.378/.

[202] Fan Li, Yu Gu, Zhigang Wang, Fangling Leng, Zhenghao Liu, and Ge Yu. Towards Efficient and Effective Diffusion Language Model Inference via Semantic-Aware Adaptive Denoising. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17988–18002, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.819. URL https://aclanthology.org/2026.acl-long.819/.

[203] Hang Li, Jindong Gu, Rajat Koner, Sahand Sharifzadeh, and Volker Tresp. Do DALL-E and Flamingo Understand Each Other? In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1999–2010, 2023. doi: 10.1109/ICCV51070.2023.00191. URL https://openaccess.thecvf.com/content/ICCV2023/html/Li_Do_DALL-E_and_Flamingo_Understand_Each_Other_ICCV_2023_paper.html.

[204] Ian Li, Zilei Shao, Benjie Wang, Rose Yu, Guy Van den Broeck, and Anji Liu. Breaking the Factorization Barrier in Diffusion Language Models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 69716–69732. PMLR, 2026. URL https://proceedings.mlr.press/v306/li26fd.html.

[205] Jiahan Li, Huangyuan Su, Shengbang Tong, Shusheng Yang, Ruihan Guo, Suozhi Huang, Hexu Zhao, Pratyusha Sharma, Sherry Yang, Jiatao Gu, Rob Fergus, and Kyunghyun Cho. JUICE: Unified Multimodal Modeling in Pretrained Representation Spaces. arXiv preprint arXiv:2610.28801, 2026. URL https://arxiv.org/abs/2610.28801.

[206] Jinyang Li, Sangwon Hyun, and M. Ali Babar. DDPT: Diffusion-Driven Prompt Tuning for Large Language Model Code Generation. In *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, pages 190–200, 2025. doi: 10.1109/CAIN66642.2025.00030. URL https://arxiv.org/abs/2504.04351.

[207] Junlin Li, Bo Peng, and Yu-Yin Hsu. Facilitating Cross-lingual Transfer of Empathy through Language-independent Latent Diffusion: A Case Study in Chinese. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24177–24192, Suzhou, China, nov 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.1313. URL https://aclanthology.org/2025.findings-emnlp.1313/.

[208] Junyi Li, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. GlyphDiffusion: Text Generation Is Also Image Generation. arXiv preprint arXiv:2304.12519, 2023. URL https://arxiv.org/abs/2304.12519v2.

[209] Na Li, Yuchen Jiao, Changxiao Cai, and Gen Li. ConvergeFlow: Language Flow with Provable Convergence to Token Embeddings. arXiv preprint arXiv:2608.23551, 2026. URL https://arxiv.org/abs/2608.23551.

[210] Qiang Li, RuiZhou Zhang, JinCheng Kang, YanNi Tang, ZiYe Deng, Wu Chen, and MingYue Zhang. DLMDC: A Method for Controllable Text Generation. In *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2626–2630, 2026. doi: 10.1109/ICASSP55912.2026.11463107. URL https://doi.org/10.1109/ICASSP55912.2026.11463107.

[211] Shufan Li, Konstantinos Kallidromitis, Akash Gokul, Zichun Liao, Yusuke Kato, Kazuki Kozuka, and Aditya Grover. OmniFlow: Any-to-Any Generation with Multi-Modal Rectified Flows. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13178–13188, June 2025. URL https://openaccess.thecvf.com/content/CVPR2025/html/Li_OmniFlow_Any-to-Any_Generation_with_Multi-Modal_Rectified_Flows_CVPR_2025_paper.html.

[212] Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A Survey on Diffusion Language Models. arXiv preprint arXiv:2508.10875, 2025. URL https://arxiv.org/abs/2508.10875.

[213] Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. Diffusion-LM Improves Controllable Text Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 4328–4343, 2022. doi: 10.52202/068431-0313. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/1be5bc25d50895ee656b8c2d9eb89d6a-Abstract-Conference.html.

[214] Xiao Li, Jiaqi Zhang, Shuxiang Zhang, Tianshui Chen, Liang Lin, and Guangrun Wang. In-Situ Tweedie Discrete Diffusion Models. arXiv preprint arXiv:2510.01047, 2025. URL https://arxiv.org/abs/2510.01047.

[215] Xuanchen Li, Tianrui Wang, Yuheng Lu, Zikang Huang, Yu Jiang, Chenghan Lin, Chenrui Cui, Ziyang Ma, Xingyu Ma, Chunyu Qiang, Guochen Yu, Xie Chen, Longbiao Wang, and Jianwu Dang. Speech Meets ELF: Audio Conditional Continuous-Target Diffusion for Speech Recognition and Translation. arXiv preprint arXiv:2606.10368, 2026. URL https://arxiv.org/abs/2606.10368.

[216] Yang Li, Yi Wang, Shiyuan Huang, Yang Liu, Hao Wang, and Chengzhi Mao. Next Thoughts Are Distributions: Generative Autoregressive Reasoning in the Latent Space. arXiv preprint arXiv:2609.33271, 2026. URL https://arxiv.org/abs/2609.33271.

[217] Yifan Li, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. Diffusion Models for Non-autoregressive Text Generation: A Survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23), Survey Track*, pages 6692–6701, 2023. doi: 10.24963/ijcai.2023/750. URL https://arxiv.org/abs/2303.06574.

[218] Zeyu Michael Li, William Xingxu Chen, Bingshuo Qian, Jiayin Liu, and Xiang Cheng. ELF-REG: Scaling Continuous Diffusion Language Models to Reasoning Tasks. arXiv preprint arXiv:2609.29102, 2026. URL https://arxiv.org/abs/2609.29102.

[219] Zhenqin Li, Shengyong Ding, and Shuangyin Li. DiFRa: A Unified Framework for Harmonizing Semantic Diversity and Factual Consistency in Question-Answer Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 29857–29875. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1493. URL https://aclanthology.org/2026.findings-acl.1493/.

[220] Jiajun Liang, Yucheng Liao, Yukang Cao, Jiazhe Wei, Ken Li, Wende Tan, Jiankun Zhang, ZY Cui, Jingkang Yang, Liucheng Guo, Shiqi Yang, B. Yang, Caifeng Shan, Ziwei Liu, and Chenyang Si. AURORA-LM: Autoencoding Unified Representation for Continuous-Latent Diffusion Language Modeling. arXiv preprint arXiv:2608.02602, 2026. URL https://arxiv.org/abs/2608.02602.

[221] Hyeong Soo Lim, Jin Young Kim, Eun Seo Seo, Min Ho Jang, and Ji Won Yoon. SelFusion: Self-distillation for Diffusion Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22077–22089, San Diego, California, United States, jul 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.1008. URL https://aclanthology.org/2026.acl-long.1008/.

[222] Zhenghao Lin, Yeyun Gong, Yelong Shen, Tong Wu, Zhihao Fan, Chen Lin, Nan Duan, and Weizhu Chen. Text Generation with Diffusion Language Models: A Pre-training Approach with Continuous Paragraph Denoise. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 21051–21064. PMLR, 2023. URL https://proceedings.mlr.press/v202/lin23d.html.

[223] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=PqvMRDCJT9t.

[224] Bing Liu, Wenjie Yang, Mingming Liu, Hao Liu, Yong Zhou, and Peng Liu. Syntactic-Conditional Diffusion Networks for Controllable Image Captioning. *ACM Transactions on Multimedia Computing, Communications, and Applications*, 21(9):1–25, 2025. doi: 10.1145/3748653. URL https://doi.org/10.1145/3748653.

[225] Guangyi Liu, Zeyu Feng, Yuan Gao, Zichao Yang, Xiaodan Liang, Junwei Bao, Xiaodong He, Shuguang Cui, Zhen Li, and Zhiting Hu. Composable Text Controls in Latent Space with ODEs. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 16543–16570, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.1030. URL https://aclanthology.org/2023.emnlp-main.1030/.

[226] Guangyi Liu, Yu Wang, Zeyu Feng, Qiyu Wu, Liping Tang, Yuan Gao, Zhen Li, Shuguang Cui, Julian McAuley, Zichao Yang, Eric P. Xing, and Zhiting Hu. Unified Generation, Reconstruction, and Representation: Generalized Diffusion with Adaptive Latent Encoding-Decoding. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 31964–31993. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/liu24bh.html.

[227] Guisheng Liu, Yi Li, Zhengcong Fei, Haiyan Fu, Xiangyang Luo, and Yanqing Guo. Prefix-diffusion: A Lightweight Diffusion Model for Diverse Image Captioning. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 12954–12965, Torino, Italia, May 2024. ELRA and ICCL. URL https://aclanthology.org/2024.lrec-main.1134/.

[228] Jiayu Liu, Ruizhi Zhao, Tianyun Ji, Yee Whye Teh, Zhenya Huang, and Ning Miao. One-Step Text Generation by Seq-Drifting. arXiv preprint arXiv:2610.12879, 2026. URL https://arxiv.org/abs/2610.12879.

[229] Pan Liu, Xiaohua Tian, and Zhouhan Lin. Enable Fast Sampling for Seq2Seq Text Diffusion. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8495–8505. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.497. URL https://aclanthology.org/2024.findings-emnlp.497/.

[230] Qihao Liu, Xi Yin, Alan Yuille, Andrew Brown, and Mannat Singh. Flowing from Words to Pixels: A Noise-Free Framework for Cross-Modality Evolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2755–2765, June 2025. URL

https://openaccess.thecvf.com/content/CVPR2025/html/Liu_Flowing_from_Words_to_Pixels_A_Noise-Free_Framework_for_Cross-Modality_CVPR_2025_paper.html.

[231] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=XVjTT1nw5z.

[232] Yuhan Liu, Shangbin Feng, Xiaochuang Han, Vidhisha Balachandran, Chan Young Park, Sachin Kumar, and Yulia Tsvetkov. P^3Sum: Preserving Author's Perspective in News Summarization with Diffusion Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2154–2173. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.naacl-long.119. URL https://aclanthology.org/2024.naacl-long.119/.

[233] Yuxi Liu. Perfect diffusion is TC^0 – Bad diffusion is Turing-complete. arXiv preprint arXiv:2507.12469, 2025. URL https://arxiv.org/abs/2507.12469.

[234] Yuxuan Liu, Tianchi Yang, Shaohan Huang, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. Text Diffusion with Reinforced Conditioning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 14069–14077, 2024. doi: 10.1609/aaai.v38i12.29316. URL https://ojs.aaai.org/index.php/AAAI/article/view/29316.

[235] Zihao Liu, Hongchen Wu, Xiaochang Fang, Hongxuan Li, Huaxiang Zhang, and Jiande Sun. Reliability-Calibrated Guidance for Few-Step Categorical Flow-Map Text Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, pages 42517–42546, Budapest, Hungary, 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.findings-emnlp.1995/.

[236] Sofia Maria Lo Cicero Vaina, Nikita Balagansky, and Daniil Gavrilov. Diffusion Language Models Generation Can Be Halted Early. arXiv preprint arXiv:2305.10818, 2023. URL https://arxiv.org/abs/2305.10818v4. Version 4, revised February 2024; first arXiv release May 2023.

[237] Michela Lorandi and Anya Belz. Reproducing the Metric-Based Evaluation of a Set of Controllable Text Generation Techniques. In *Proceedings of the Fourth Workshop on Human Evaluation of NLP Systems (HumEval) @ LREC-COLING 2024*, pages 125–131. ELRA and ICCL, 2024. URL https://aclanthology.org/2024.humeval-1.12/.

[238] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 32819–32848. PMLR, 2024. URL https://proceedings.mlr.press/v235/lou24a.html.

[239] Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q. Weinberger. Latent Diffusion for Language Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 56998–57025, 2023. doi: 10.52202/075280-2492. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/b2a2bd5d5051ff6af52e1ef60aefd255-Abstract-Conference.html.

[240] Justin Lovelace, Varsha Kishore, Yiwei Chen, and Kilian Q. Weinberger. Diffusion Guided Language Modeling. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14936–14952. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.887. URL https://aclanthology.org/2024.findings-acl.887/.

[241] Justin Lovelace, Christian Belardi, Sofian Zalouk, Adhitya Polavaram, Srivatsa Kundurthy, and Kilian Q. Weinberger. Stop-Think-AutoRegress: Language Modeling with Latent Diffusion Planning. In *Conference on Language Modeling (COLM)*, 2025. URL https://arxiv.org/abs/2602.20528.

[242] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models. *Machine Intelligence Research*, 22 (4):730–751, 2025. doi: 10.1007/s11633-025-1562-4. URL https://arxiv.org/abs/2211.01095.

[243] Jianjie Luo, Yehao Li, Yingwei Pan, Ting Yao, Jianlin Feng, Hongyang Chao, and Tao Mei. Semantic-Conditional Diffusion Networks for Image Captioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 23359–23368, June 2023. doi: 10.1109/CVPR52729.2023.02237. URL https://openaccess.thecvf.com/content/CVPR2023/html/Luo_Semantic-Conditional_Diffusion_Networks_for_Image_Captioning_CVPR_2023_paper.html.

[244] Weijian Luo, Yujian Chen, Hao Jiang, Guang Cheng, and Tianyang Hu. Designing Transformers for Continuous Diffusion Language Models. arXiv preprint arXiv:2610.22330, 2026. URL https://arxiv.org/abs/2610.22330.

[245] Ang Lv, Jinpeng Li, Yuhan Chen, Xing Gao, Ji Zhang, and Rui Yan. DialoGPS: Dialogue Path Sampling in Continuous Semantic Space for Data Augmentation in Multi-Turn Conversations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1267–1280, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.70. URL https://aclanthology.org/2023.acl-long.70/.

[246] Yiwei Lyu, Tiange Luo, Jiacheng Shi, Todd C. Hollon, and Honglak Lee. Fine-grained Text Style Transfer with Diffusion-Based Language Models. In *Proceedings of the 8th Workshop on Representation Learning for NLP (RepL4NLP 2023)*, pages 65–74. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.repl4nlp-1.6. URL https://aclanthology.org/2023.repl4nlp-1.6/.

[247] Xinchen Ma, Gaole He, Yunshi Lan, and Weining Qian. Diff4TST: Masked Diffusion Language Model for Text Style Transfer. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6736–6748. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.acl-long.306. URL https://aclanthology.org/2026.acl-long.306/.

[248] Xuezhe Ma, Chunting Zhou, Xian Li, Graham Neubig, and Eduard Hovy. FlowSeq: Non-Autoregressive Conditional Sequence Generation with Generative Flow. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4282–4292. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1437. URL https://aclanthology.org/D19-1437/.

[249] Yuxin Ma, Lun Du, Lanning Wei, Kun Chen, Qian Xu, Kangyu Wang, Guofeng Feng, Guoshan Lu, Lin Liu, Xiaojing Qi, Xinyuan Zhang, Zhen Tao, Haibo Feng, Zhiyun Jiang, Ying Xu, Zenan Huang, Yihong Zhuang, Haokai Xu, Jiaqi Hu, Zhenzhong Lan, Junbo Zhao, Jianguo Li, and Da Zheng. dInfer: An Efficient Inference Framework for Diffusion Language Models. arXiv preprint arXiv:2510.08666, 2025. URL https://arxiv.org/abs/2510.08666.

[250] Weijian Mai and Zhijun Zhang. UniBrain: Unify Image Reconstruction and Captioning All in One Diffusion Model from Human Brain Activity. arXiv preprint arXiv:2308.07428, 2023. URL https://arxiv.org/abs/2308.07428.

[251] Saad Mankarious and Aya Zirikly. Style Transfer as Bias Mitigation: Diffusion Models for Synthetic Mental Health Text for Arabic. arXiv preprint arXiv:2601.14124, 2026. URL https://arxiv.org/abs/2601.14124.

[252] Viacheslav Meshchaninov, Egor Chimbulatov, Alexander Shabalin, Aleksandr Abramov, and Dmitry Vetrov. Cosmos: Compressed and Smooth Latent Space for Text Diffusion Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 15938–15966, 2025. doi: 10.52202/085713-0479. URL https://arxiv.org/abs/2506.21170.

[253] Viacheslav Meshchaninov, Alexander Shabalin, Egor Chimbulatov, Nikita Gushchin, Ilya Koziev, Alexander Korotin, and Dmitry Vetrov. How to Train Your Latent Diffusion Language Model Jointly With the Latent Space. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=7KCjc4rgB6.

[254] Nesta Midavaine, Christian A. Naesseth, and Grigory Bartosh. Towards Latent Diffusion Suitable For Text. arXiv preprint arXiv:2601.16220, 2026. URL https://arxiv.org/abs/2601.16220.

[255] Andrea Miele and Ilija Bogunovic. Spherical Brownian Diffusion for Language Modeling. arXiv preprint arXiv:2610.35101, 2026. URL https://arxiv.org/abs/2610.35101.

[256] Mohaddeseh Mirbeygi and Hamid Beigy. Prompt Guided Diffusion for Controllable Text Generation. In JinYeong Bak, Rob van der Goot, Hyeju Jang, Weerayut Buaphet, Alan Ramponi, Wei Xu, and Alan Ritter, editors, *Proceedings of the Tenth Workshop on Noisy and User-generated Text*, pages 78–84, Albuquerque, New Mexico, USA, May 2025. Association for Computational Linguistics. ISBN 979-8-89176-232-9. doi: 10.18653/v1/2025.wnut-1.9. URL https://aclanthology.org/2025.wnut-1.9/.

[257] Amr Mohamed, Yang Zhang, Michalis Vazirgiannis, and Guokan Shang. Fast-Decoding Diffusion Language Models via Progress-Aware Confidence Schedules. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 35793–35807, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.1782. URL https://aclanthology.org/2026.findings-acl.1782/.

[258] Daniele Molino, Francesco di Feola, Linlin Shen, Paolo Soda, and Valerio Guarrasi. Any-to-Any Vision-Language Model for Multimodal X-ray Imaging and Radiological Report Generation. In *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2025. doi: 10.1109/IJCNN64981.2025.11227399. URL https://arxiv.org/abs/2505.01091.

[259] Daniele Molino, Francesco Di Feola, Eliodoro Faiella, Deborah Fazzini, Domiziana Santucci, Linlin Shen, Valerio Guarrasi, and Paolo Soda. XGeM: A Multi-Prompt Foundation Model for Multimodal Medical Data Generation. *Computerized Medical Imaging and Graphics*, 128:102718, 2026. doi: 10.1016/j.compmedimag.2026.102718. URL https://arxiv.org/abs/2501.04614.

[260] JiHwan Moon, Jihoon Park, Jungeun Kim, Jongseong Bae, Hyeongwoo Jeon, and Ha Young Kim. DiffSLT: Enhancing Diversity in Sign Language Translation via Diffusion Model. *Pattern Recognition Letters*, 196:117–125, 2025. doi: 10.1016/j.patrec.2025.06.008. URL https://arxiv.org/abs/2411.17248.

[261] Samraj Moorjani, Adit Krishnan, and Hari Sundaram. CEV-LM: Controlled Edit Vector Language Model for Shaping Natural Language Generations. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1325–1340. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.eacl-long.80. URL https://aclanthology.org/2024.eacl-long.80/.

[262] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. Text Embeddings Reveal (Almost) As Much As Text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12448–12460. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.765. URL https://aclanthology.org/2023.emnlp-main.765/.

[263] Feiteng Mu and Wenjie Li. Generating Contrastive Narratives Using the Brownian Bridge Process for Narrative Coherence Learning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6538–6555. Association for Computational Linguistics, August 2024. doi: 10.18653/v1/2024.acl-long.353. URL https://aclanthology.org/2024.acl-long.353/.

[264] Elio Musacchio, Lucia Siciliani, and Pierpaolo Basile. Mimir: Large-Scale Multilingual Concept Modeling. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 13051–13073, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.emnlp-main.569/.

[265] KiHyun Nam, Jongmin Choi, Hyeongkeun Lee, Jungwoo Heo, and Joon Son Chung. Diffusion-Link: Diffusion Probabilistic Model for Bridging the Audio-Text Modality Gap. In *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 19112–19116. IEEE, 2026. doi: 10.1109/ICASSP55912.2026.11460544. URL https://arxiv.org/abs/2510.11330.

[266] Thong Nguyen, Anh Tuan Luu, Truc Lu, and Tho Quan. Enriching and Controlling Global Semantics for Text Summarization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9443–9456. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.744. URL https://aclanthology.org/2021.emnlp-main.744/.

[267] Dat Nguyen-Cong, Tung Kieu, and Hoang Thanh-Tung. FastDiSS: Few-step Match Many-step Diffusion Language Model on Sequence-to-Sequence Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 17572–17592. Association for Computational Linguistics, jul 2026. doi: 10.18653/v1/2026.findings-acl.870. URL https://aclanthology.org/2026.findings-acl.870/.

[268] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large Language Diffusion Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 50608–50646, 2025. doi: 10.52202/085713-1689. URL https://arxiv.org/abs/2502.09992.

[269] Lavanya Nigam, Ishaan Bansal, Aryan Sood, Vidit Aggarwal, and Gaurav Kumar Nayak. Lost in Interpolation: Why Predictive Feedback Fails in Diffusion Language Models. arXiv preprint arXiv:2608.06529, 2026. URL https://arxiv.org/abs/2608.06529.

[270] Miaohe Niu, Pengxiang Li, Jingbo Zhu, and Tong Xiao. Plan Canvas: Fixed Reasoning Regions for Continuous Language Flows. arXiv preprint arXiv:2610.05815, 2026. URL https://arxiv.org/abs/2610.05815.

[271] Ryota Nosaka and Takuya Matsuzaki. Timestep Embeddings Trigger Collapse in Diffusion Text Generation. In *Proceedings of the 29th Conference on Computational Natural Language Learning*, pages 397–406. Association for Computational Linguistics, July 2025. doi: 10.18653/v1/2025.conll-1.26. URL https://aclanthology.org/2025.conll-1.26/.

[272] Daisuke Oba, Hiroki Furuta, and Naoaki Okazaki. BCT: Bayes-Consistent Tempering for Flow Language Models. arXiv preprint arXiv:2610.27641, 2026. URL https://arxiv.org/abs/2610.27641.

[273] Sebastian Ochs and Ivan Habernal. Private Synthetic Text Generation with Diffusion Models. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10612–10626, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.532. URL https://aclanthology.org/2025.naacl-long.532/.

[274] Theo X. Olausson, Metod Jazbec, Xi Wang, Armando Solar-Lezama, Christian A. Naesseth, Stephan Mandt, and Eric Nalisnick. A Tale of Two Temperatures: Simple, Efficient, and Diverse Sampling from Diffusion Language Models. In *Conference on Language Modeling (COLM)*, 2026. URL https://proceedings.colm.cc/paper_files/paper/2026/hash/7b102c908e9404dd040599c65db4ce3e-Abstract-Conference.html.

[275] Mathias Ollu and Nikos Komodakis. Denoising Hierarchical Representations: Joint Continuous Diffusion for Language Modeling. arXiv preprint arXiv:2610.08738, 2026. URL https://arxiv.org/abs/2610.08738.

[276] Yuki Otsuka, Daisuke Oba, and Naoaki Okazaki. ReLaDrift: Recurrent Language Generation via Drifting for Test-Time Scaling. arXiv preprint arXiv:2610.23225, 2026. URL https://arxiv.org/abs/2610.23225.

[277] Yimin Ou and Ping Jian. Effective Integration of Text Diffusion and Pre-Trained Language Models with Linguistic Easy-First Schedule. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 5551–5561, Torino, Italia, May 2024. ELRA and ICCL. URL https://aclanthology.org/2024.lrec-main.493/.

[278] Yassine Ouali, Adrian Bulat, and Georgios Tzimiropoulos. What Matters for Latent Reasoning with Flow Matching. arXiv preprint arXiv:2610.06666, 2026. URL https://arxiv.org/abs/2610.06666.

[279] Tejomay Kishor Padole, Suyash P. Awate, and Pushpak Bhattacharyya. Improving Text Style Transfer Using Masked Diffusion Language Models with Inference-Time Scaling. In *Proceedings of the European Conference on Artificial Intelligence (ECAI 2025)*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 4476–4483. IOS Press, 2025. doi: 10.3233/FAIA251347. URL https://arxiv.org/abs/2508.10995.

[280] Bo Pang and Ying Nian Wu. Latent Space Energy-Based Model of Symbol-Vector Coupling for Text Generation and Classification. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8359–8370. PMLR, 2021. URL https://proceedings.mlr.press/v139/pang21a.html.

[281] Bo Pang, Tian Han, Erik Nijkamp, Song-Chun Zhu, and Ying Nian Wu. Learning Latent Space Energy-Based Prior Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 21994–22008, 2020. URL https://papers.nips.cc/paper_files/paper/2020/hash/fa3060edb66e6ff4507886f9912e1ab9-Abstract.html.

[282] Jiwon Park, Sooyeon Kim, and Weonyoung Joo. Debiased controllable text generation with classifier-guided rectified flow language model. *Journal of the Korean Statistical Society*, 2026. doi: 10.1007/s42952-026-00383-1. URL https://doi.org/10.1007/s42952-026-00383-1. Published online 5 August 2026.

[283] Arkanath Pathak, Unnat Jain, and Alexander C. Berg. Repurposing Pre-trained LLMs as High Fidelity Continuous Text Autoencoders. arXiv preprint arXiv:2609.27248, 2026. URL https://arxiv.org/abs/2609.27248.

[284] Fred Zhangzhi Peng, Zachary Bezemek, Sawan Patel, Jarrid Rector-Brooks, Sherwood Yao, Avishek Joey Bose, Alexander Tong, and Pranam Chatterjee. Path Planning for Masked Diffusion Model Sampling. arXiv preprint arXiv:2502.03540, 2025. URL https://arxiv.org/abs/2502.03540.

[285] Fred Zhangzhi Peng, Zachary Bezemek, Jarrid Rector-Brooks, Shuibai Zhang, Anru R. Zhang, Michael Bronstein, Alexander Tong, and Avishek Joey Bose. Planner Aware Path Learning in Diffusion Language Models Training. In *International Conference on Learning Representations (ICLR)*, volume 2026, pages 126799–126844, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/cdc1d08ee82d4818758d229abb7f1ce8-Abstract-Conference.html.

[286] Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaid Harchaoui. MAUVE: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 4816–4828. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/260c2432a0eecc28ce03c10dadc078a4-Abstract.html.

[287] Peter Potaptchik, Jason Yim, Adhi Saravanan, Peter Holderrieth, Eric Vanden-Eijnden, and Michael S. Albergo. Discrete Flow Maps. arXiv preprint arXiv:2604.09784, 2026. URL https://arxiv.org/abs/2604.09784.

[288] Mihir Prabhudesai and Zhengyang Geng. Latent Thought Flows with Text Compression. Project page and blog post, 9 June 2026, https://latent-thought-flows.vercel.app/, 2026. Accessed 4 October 2026.

[289] Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion Beats Autoregressive in Data-Constrained Settings. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 10581–10606. Curran Associates, Inc., 2025. doi: 10.52202/085713-0354. URL https://openreview.net/forum?id=W5Ht05jF4c.

[290] Michael Przystupa, Mark Schmidt, and Muhammad Abdul-Mageed. Investigating the Impact of Normalizing Flows on Latent Variable Machine Translation. In *First Workshop on Invertible Neural Networks and Normalizing Flows (INNF) at ICML 2019*, 2019. URL https://invertibleworkshop.github.io/INNF_2019/accepted_papers/pdfs/INNF_2019_paper_29.pdf.

[291] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. CANDI: Hybrid Discrete-Continuous Diffusion Models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 100176–100208. PMLR, 2026. URL https://proceedings.mlr.press/v306/pynadath26a.html.

[292] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Generative Frontiers: Why Evaluation Matters for Diffusion Language Models. arXiv preprint arXiv:2604.02718, 2026. URL https://arxiv.org/abs/2604.02718.

[293] Shanbao Qiao, Xuebing Liu, and Seung-Hoon Na. DiffusionRet: Diffusion-Enhanced Generative Retriever using Constrained Decoding. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9515–9529. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.638. URL https://aclanthology.org/2023.findings-emnlp.638/.

[294] Lianhui Qin, Sean Welleck, Daniel Khashabi, and Yejin Choi. COLD Decoding: Energy-based Constrained Text Generation with Langevin Dynamics. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 9538–9551. Curran Associates, Inc., 2022. doi: 10.52202/068431-0693. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/3e25d1aff47964c8409fd5c8dc0438d7-Abstract-Conference.html.

[295] Yifu Qiu, Paul-Ambroise Duquenne, and Holger Schwenk. Unified Vision–Language Modeling via Concept Space Alignment. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/22d72e9f55bc29cafcca6814a7feac8c-Abstract-Conference.html.

[296] Xingwei Qu, Shaowen Wang, Zihao Huang, Kai Hua, Fan Yin, Rui-Jie Zhu, Jundong Zhou, Qiyang Min, Zihao Wang, Yizhi Li, Tianyu Zhang, He Xing, Zheng Zhang, Yuxuan Song, Tianyu Zheng, Zhiyuan Zeng, Chenghua Lin, Ge Zhang, and Wenhao Huang. Dynamic Large Concept Models: Latent Reasoning in an Adaptive Semantic Space. arXiv preprint arXiv:2512.24617, 2025. URL https://arxiv.org/abs/2512.24617.

[297] Gabriel Raya, Bac Nguyen, Georgios Batzolis, Yuhta Takida, Dejan Stancevic, Naoki Murata, Chieh-Hsin Lai, Yuki Mitsufuji, and Luca Ambrogioni. Noise Scheduling as Information-Guided Allocation in Diffusion Training. arXiv preprint arXiv:2602.18647, 2026. URL https://arxiv.org/abs/2602.18647.

[298] Andrei Rekesh, Jarrid Rector-Brooks, and Cheng-Hao Liu. Inverse-Confidence Sampling for Continuous Diffusion Language Models. In *Workshop on Foundations of Deep Generative Models: Understanding Memorization, Generalization, and Reasoning (FoGen) at ICML 2026*, 2026. URL https://openreview.net/forum?id=TzA1wJpzHB.

[299] Hui Ren, Zihan Li, Chang Liu, Huidong Liu, and Alexander Schwing. Hierarchical Continuous Diffusion Language Models. arXiv preprint arXiv:2610.02193, 2026. URL https://arxiv.org/abs/2610.02193.

[300] Antonio D. Reyes-Ramírez, Mario Ezra Aragón, Fernando Sánchez-Vega, and A. Pastor López-Monroy. Improving aggressiveness detection using a data augmentation technique based on a Diffusion Language Model. In *Proceedings of the 8th Workshop on Online Abuse and Harms (WOAH 2024)*, pages 171–177. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.woah-1.13. URL https://aclanthology.org/2024.woah-1.13/.

[301] Pierre H. Richemond, Sander Dieleman, and Arnaud Doucet. Categorical SDEs with Simplex Diffusion. arXiv preprint arXiv:2210.14784, 2022. URL https://arxiv.org/abs/2210.14784.

[302] Liran Ringel, Ameen Ali, and Yaniv Romano. Dependency-Guided Parallel Decoding in Discrete Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=aGLo39lNPa.

[303] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html.

[304] Daan Roos, Oscar Davis, Floor Eijkelboom, Michael Bronstein, Max Welling, İsmail İlkan Ceylan, Luca Ambrogioni, and Jan-Willem van de Meent. Categorical Flow Maps. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 105618–105642. PMLR, 2026. URL https://proceedings.mlr.press/v306/roos26a.html.

[305] Litu Rout, Andreas Lugmayr, Yasamin Jafarian, Srivatsan Varadharajan, Constantine Caramanis, Sanjay Shakkottai, and Ira Kemelmacher-Shlizerman. Test-Time Anchoring for Discrete Diffusion Posterior Sampling. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 105808–105848. PMLR, 2026. URL https://proceedings.mlr.press/v306/rout26a.html.

[306] Alon Rozental. Zonkey: A Hierarchical Diffusion Language Model with Differentiable Tokenization and Probabilistic Attention. arXiv preprint arXiv:2601.21768, 2026. URL https://arxiv.org/abs/2601.21768.

[307] Benjamin Rozonoyer, Jacopo Minniti, Dhruvesh Patel, Neil Band, Avishek Joey Bose, Tim G. J. Rudner, and Andrew McCallum. Learned Relay Representations for Forward-Thinking Discrete Diffusion Models. In *4th Workshop on Structured Probabilistic Inference & Generative Modeling (SPIGM) at ICML 2026*, 2026. URL https://arxiv.org/abs/2605.22967.

[308] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and Effective Masked Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 130136–130184, 2024. doi: 10.52202/079017-4135. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb0b13cc515724ab8015bc978fdde0ad-Abstract-Conference.html.

[309] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin T. Chiu, and Volodymyr Kuleshov. The Diffusion Duality. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 52584–52619. PMLR, 2025. URL https://proceedings.mlr.press/v267/sahoo25a.html.

[310] Jinya Sakurai, Patrick Pynadath, Satoshi Hayakawa, Jaehong Yoon, Xulei Yang, Nancy F. Chen, and Xun Xu. Simplex Relaxation for Discrete Diffusion. arXiv preprint arXiv:2608.10615, August 2026. URL https://arxiv.org/abs/2608.10615.

[311] Tim Salimans and Jonathan Ho. Progressive Distillation for Fast Sampling of Diffusion Models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=TIdIXIpzhoI.

[312] Yair Schiff, Omer Belhasin, Roy Uziel, Matan Rusanovsky, Ran Zilberstein, Marianne Arriola, Gilad Turok, Guanghan Wang, Volodymyr Kuleshov, and Michael Elad. Clock Diffusion: Efficient Semi-Autoregressive Continuous Diffusion Language Models. arXiv preprint arXiv:2610.00894, 2026. URL https://arxiv.org/abs/2610.00894.

[313] Paul S. Scotti, Mihir Tripathy, Cesar Kadir Torrico Villanueva, Reese Kneeland, Tong Chen, Ashutosh Narang, Charan Santhirasegaran, Jonathan Xu, Thomas Naselaris, Kenneth A. Norman, and Tanishq Mathew Abraham. MindEye2: Shared-Subject Models Enable fMRI-To-Image With 1 Hour of Data. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 44038–44059. PMLR, 2024. URL https://proceedings.mlr.press/v235/scotti24a.html.

[314] Maksim Semenov, Nikita Balagansky, Ilya Drobyshevskiy, Dmitry Baranchuk, Nikita Gushchin, Daniil Gavrilov, Aibek Alanov, and Viacheslav Meshchaninov. Latent Space Co-training for Few-Step Text Generation. In *Workshop on Beyond Next-Token Prediction: Diffusion and Flow Models for Next-Generation Decoding (BeNTo) at NeurIPS 2026*, 2026. URL https://openreview.net/forum?id=vqMoJMqAbO.

[315] Hendra Setiawan, Matthias Sperber, Udhyakumar Nallasamy, and Matthias Paulik. Variational Neural Machine Translation with Normalizing Flows. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7771–7777. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.694. URL https://aclanthology.org/2020.acl-main.694/.

[316] Egor Sevriugov, Nikita Dragunov, Anton Razzhigaev, Andrey Kuznetsov, and Ivan Oseledets. Logit-KL Flow Matching: Non-Autoregressive Text Generation via Sampling-Hybrid Inference. In

International Conference on Learning Representations (ICLR), 2026. URL https://openreview.net/forum?id=scgtQSpROE.

[317] Alexander Shabalin, Viacheslav Meshchaninov, Egor Chimbulatov, Vladislav Lapikov, Roman Kim, Grigory Bartosh, Dmitry Molchanov, Sergey Markov, and Dmitry Vetrov. TEncDM: Understanding the Properties of the Diffusion Model in the Space of Language Model Encodings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25110–25118, 2025. doi: 10.1609/aaai.v39i23.34696. URL https://arxiv.org/abs/2402.19097.

[318] Alexander Shabalin, Simon Elistratov, Viacheslav Meshchaninov, Ildus Sadrtdinov, and Dmitry Vetrov. Why Gaussian Diffusion Models Fail on Discrete Data and How to Prevent It? In *Conference on Language Modeling (COLM)*, 2026. URL https://proceedings.colm.cc/paper_files/paper/2026/hash/20d50ff26f7ab9c8c2578a3eec69c922-Abstract-Conference.html.

[319] Alexander Shabalin, Viacheslav Meshchaninov, and Dmitry Vetrov. Smoothie: Smoothing Diffusion on Token Embeddings for Text Generation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 109429–109446. PMLR, 2026. URL https://proceedings.mlr.press/v306/shabalin26a.html.

[320] Aditya Hemant Shahane, Anuj Kumar Sirohi, Devansh Arora, Nitin Kumar, Prathosh AP, and Sandeep Kumar. BiMol-Diff: A Unified Diffusion Framework for Molecular Generation and Captioning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23104–23117. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.acl-long.1059. URL https://aclanthology.org/2026.acl-long.1059/.

[321] Aditya Hemant Shahane, Anuj Kumar Sirohi, Tanmoy Chakraborty, Prathosh A P, and Sandeep Kumar. Factual and Edit-Sensitive Graph-to-Sequence Generation via Graph-Aware Adaptive Noising. arXiv preprint arXiv:2604.24104, 2026. URL https://arxiv.org/abs/2604.24104.

[322] Chenyang Shao, Sijian Ren, Fengli Xu, and Yong Li. Diffuse Thinking: Exploring Diffusion Language Models as Efficient Thought Proposers for Reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26739–26757, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.1231. URL https://aclanthology.org/2026.acl-long.1231/.

[323] Chenze Shao, Darren Li, Fandong Meng, and Jie Zhou. Continuous Autoregressive Language Models. arXiv preprint arXiv:2510.27688, 2025. URL https://arxiv.org/abs/2510.27688.

[324] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300, 2024. URL https://arxiv.org/abs/2402.03300.

[325] Dario Shariatian, Alain Durmus, Umut Simsekli, and Stefano Peluchetti. Latent-Augmented Discrete Diffusion Models. In *4th Workshop on Structured Probabilistic Inference & Generative Modeling (SPIGM) at ICML 2026*, 2026. URL https://arxiv.org/abs/2510.18114.

[326] Jeff Shen, Zhejian Zhou, Ryan Lee, Jonathan May, and Shirley Ho. FlashMap: Flow Map Speculative Decoding. arXiv preprint arXiv:2610.24365, 2026. URL https://arxiv.org/abs/2610.24365.

[327] Junzhe Shen, Jieru Zhao, Ziwei He, and Zhouhan Lin. CoDAR: Continuous Diffusion Language Models are More Powerful Than You Think. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 23172–23187, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.emnlp-main.1007/.

[328] Yingdong Shi, Ruiming Zhang, Changming Li, Zhiyu Yang, Kaixing Zhang, Jingyi Yu, and Kan Ren. UniSteer: Text-Guided Flow Matching in Activation Space for Versatile LLM Steering. arXiv preprint arXiv:2605.30076, 2026. URL https://arxiv.org/abs/2605.30076.

[329] Ziqiang Shi, Rujie Liu, Shanshan Yu, Satoshi Munakata, and Koichi Shirahata. SchröMind: Mitigating Hallucinations in Multimodal Large Language Models via Solving the Schrödinger Bridge Problem. In *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 11887–11891. IEEE, 2026. doi: 10.1109/ICASSP55912.2026.11463152. URL https://arxiv.org/abs/2602.09528.

[330] Jeongwan Shin, Hyeyoung Park, and Hyun-Je Song. Factuality Guided Diffusion-Based Abstractive Summarization. *IEEE Access*, 12:139302–139315, 2024. doi: 10.1109/ACCESS.2024.3467063. URL https://doi.org/10.1109/ACCESS.2024.3467063.

[331] Makoto Shing, Masanori Koyama, and Takuya Akiba. DiffusionBlocks: Block-wise Neural Network Training via Diffusion Interpretation. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/994c9bc68bc30954f6e5995864f1d92d-Abstract-Conference.html.

[332] Raphael Shu, Jason Lee, Hideki Nakayama, and Kyunghyun Cho. Latent-Variable Non-Autoregressive Neural Machine Translation with Deterministic Inference Using a Delta Posterior. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8846–8853, 2020. doi: 10.1609/aaai.v34i05.6413. URL https://arxiv.org/abs/1908.07181.

[333] Zhenyu Shu, Jiawei Wen, Shiyang Li, Shiqing Xin, and Ligang Liu. Diff-3DCap: Shape Captioning with Diffusion Models. *IEEE Transactions on Visualization and Computer Graphics*, 31(10):8129–8142, 2025. doi: 10.1109/TVCG.2025.3564664. URL https://arxiv.org/abs/2509.23718.

[334] Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, and Gust Verbruggen. CodeFusion: A Pre-trained Diffusion Model for Code Generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 11697–11708, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.716. URL https://aclanthology.org/2023.emnlp-main.716/.

[335] Raghav Singhal, Zachary Horvitz, Ryan Teehan, Mengye Ren, Zhou Yu, Kathleen McKeown, and Rajesh Ranganath. A General Framework for Inference-time Scaling and Steering of Diffusion Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 55810–55827. PMLR, 2025. URL https://proceedings.mlr.press/v267/singhal25b.html.

[336] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=St1giarCHLP.

[337] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=PxTIG12RRHS.

[338] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency Models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 32211–32252. PMLR, 2023. URL https://proceedings.mlr.press/v202/song23a.html.

[339] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, Yuwei Fu, Jing Su, Ge Zhang, Wenhao Huang, Mingxuan Wang, Lin Yan, Xiaoying Jia, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Yonghui Wu, and Hao Zhou. Seed Diffusion: A Large-Scale Diffusion Language Model with High-Speed Inference. arXiv preprint arXiv:2508.02193, 2025. URL https://arxiv.org/abs/2508.02193.

[340] Hannes Stark, Bowen Jing, Chenyu Wang, Gabriele Corso, Bonnie Berger, Regina Barzilay, and Tommi Jaakkola. Dirichlet Flow Matching with Applications to DNA Sequence Design. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 46495–46513. PMLR, 2024. URL https://proceedings.mlr.press/v235/stark24b.html.

[341] Robin Strudel, Corentin Tallec, Florent Altché, Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, and Rémi Leblond. Self-conditioned Embedding Diffusion for Text Generation. arXiv preprint arXiv:2211.04236, 2022. URL https://arxiv.org/abs/2211.04236.

[342] Baiyu Su, Runlong Liao, Lizhang Chen, and Qiang Liu. Rectified Language Flow. arXiv preprint arXiv:2610.33811, 2026. URL https://arxiv.org/abs/2610.33811.

[343] Jiao Sun, Yufei Tian, Wangchunshu Zhou, Nan Xu, Qian Hu, Rahul Gupta, John Wieting, Nanyun Peng, and Xuezhe Ma. Evaluating Large Language Models on Controlled Generation Tasks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3155–3168, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.190. URL https://aclanthology.org/2023.emnlp-main.190/.

[344] Jaesung Tae, Hamish Ivison, Sachin Kumar, and Arman Cohan. TESS 2: A Large-Scale Generalist Diffusion Language Model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21171–21188. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1029. URL https://aclanthology.org/2025.acl-long.1029/.

[345] Chao-Hong Tan, Jia-Chen Gu, and Zhen-Hua Ling. DiffuSIA: A Spiral Interaction Architecture for Encoder-Decoder Text Diffusion. arXiv preprint arXiv:2305.11517, 2023. URL https://arxiv.org/abs/2305.11517.

[346] Luhan Tang, Longxuan Yu, Shaorong Zhang, and Greg Ver Steeg. Is Your Diffusion Sampler Actually Correct? A Sampler-Centric Evaluation of Discrete Diffusion Language Models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 119372–119401. PMLR, 2026. URL https://proceedings.mlr.press/v306/tang26ad.html.

[347] Sophia Tang and Pranam Chatterjee. Expanding Flow Maps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=3aX1lzdpqZ.

[348] Sophia Tang and Shiyi Wang. Discrete Beckmann Transport Models for One-Step Language Modeling and Reasoning. arXiv preprint arXiv:2609.15903, 2026. URL https://arxiv.org/abs/2609.15903.

[349] Zecheng Tang, Pinzheng Wang, Keyan Zhou, Juntao Li, Ziqiang Cao, and Min Zhang. Can Diffusion Model Achieve Better Performance in Text Generation? Bridging the Gap between Training and Inference! In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 11359–11386, Toronto, Canada, jul 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.721. URL https://aclanthology.org/2023.findings-acl.721/.

[350] Zineng Tang, Ziyi Yang, Chenguang Zhu, Michael Zeng, and Mohit Bansal. Any-to-Any Generation via Composable Diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 16083–16099, 2023. doi: 10.52202/075280-0707. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/33edf072fe44f19079d66713a1831550-Abstract-Conference.html.

[351] Hongyuan Tao, Xinggang Wang, Lianghui Zhu, Yongkang Li, Yunchao Wei, Bin Feng, Shaoyu Chen, Qian Zhang, Chang Huang, and Kai Yu. Multimodal Flow: Unified Flow Modeling of Language and Vision in Embedding Spaces. arXiv preprint arXiv:2609.40362, 2026. URL https://arxiv.org/abs/2609.40362.

[352] Atula Tejaswi, Litu Rout, Constantine Caramanis, Sanjay Shakkottai, and Sujay Sanghavi. Entropy Aware Reward Guidance for Diffusion Language Model Alignment. In *Workshop on Non-Autoregressive Language Models for Fast and Flexible Text Generation (NonAR-LM) at COLM 2026*, 2026. URL https://arxiv.org/abs/2602.05000.

[353] Xinyu Tian and Xiaotong Shen. Conditional Data Synthesis Augmentation. *Journal of the American Statistical Association*, 121(554):1128–1140, 2026. doi: 10.1080/01621459.2025.2586772. URL https://arxiv.org/abs/2504.07426.

[354] Xinyu Tian and Xiaotong Shen. Generative Distribution Prediction: A Unified Approach to Multimodal Learning. *Machine Learning*, 115(9):209, 2026. doi: 10.1007/s10994-026-07148-1. URL https://link.springer.com/article/10.1007/s10994-026-07148-1.

[355] Peter Toth and Dan Oprisa. Decentralised AI Training and Inference with BlockTrain. arXiv preprint arXiv:2606.24722, 2026. URL https://arxiv.org/abs/2606.24722.

[356] Gilad Turok, Yair Schiff, Chris De Sa, and Volodymyr Kuleshov. DUEL: Exact Likelihood for Masked Diffusion via Deterministic Unmasking. In *ICML 2026 Workshop on Foundations of Deep Generative Models: Understanding Memorization, Generalization, and Reasoning (FoGen)*, 2026. URL https://openreview.net/pdf/2e5c27c6314014b54b7671cb76dc55072f4e39fc.pdf.

[357] Roy Uziel, Omer Belhasin, Itay Levy, Akhiad Bercovich, Ran El-Yaniv, Ran Zilberstein, and Michael Elad. CRoCoDiL: Continuous and Robust Conditioned Diffusion for Language. arXiv preprint arXiv:2603.20210, 2026. URL https://arxiv.org/abs/2603.20210.

[358] Paul Le Van Kiem, Dario Shariatian, Umut Simsekli, and Alain Durmus. Distribution Matching Distillation for Continuous Diffusion Language Models. arXiv preprint arXiv:2609.40235, 2026. URL https://arxiv.org/abs/2609.40235.

[359] Chao Wang, Hongjian Guo, and Fanxi Xia. Knowledge-guided diffusion for emotion and topic controllable text generation. *iScience*, 29(6):115946, 2026. doi: 10.1016/j.isci.2026.115946. URL https://doi.org/10.1016/j.isci.2026.115946.

[360] Chenyu Wang, Masatoshi Uehara, Yichun He, Amy Wang, Avantika Lal, Tommi Jaakkola, Sergey Levine, Aviv Regev, Hanchen Wang, and Tommaso Biancalani. Fine-Tuning Discrete Diffusion Models via Reward Optimization with Applications to DNA and Protein Design. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/

paper_files/paper/2025/hash/771e09dd204ea339da0d8114c48afd21-Abstract-Conference.html.

[361] Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking Discrete Diffusion Models with Inference-Time Scaling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 147282–147339. Curran Associates, Inc., 2025. doi: 10.52202/085713-4927. URL https://openreview.net/forum?id=IJryQAOy0p.

[362] Hao Wang, Hao Li, Junda Zhu, Xinyuan Wang, Chengwei Pan, Minlie Huang, and Lei Sha. DiffusionAttacker: Diffusion-Driven Prompt Manipulation for LLM Jailbreak. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 22182–22194. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1128. URL https://aclanthology.org/2025.emnlp-main.1128/.

[363] Huazheng Wang, Daixuan Cheng, Haifeng Sun, Jingyu Wang, Qi Qi, Jianxin Liao, Jing Wang, and Cong Liu. How Does Diffusion Influence Pretrained Language Models on Out-of-Distribution Data? In *Proceedings of the European Conference on Artificial Intelligence (ECAI 2023)*, volume 372 of *Frontiers in Artificial Intelligence and Applications*, pages 2459–2466. IOS Press, 2023. doi: 10.3233/FAIA230549. URL https://arxiv.org/abs/2307.13949.

[364] Jian Wang, Dongding Lin, and Wenjie Li. Dialogue Planning via Brownian Bridge Stochastic Process for Goal-directed Proactive Dialogue. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 370–387, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.25. URL https://aclanthology.org/2023.findings-acl.25/.

[365] Junbo Wang, Liangyu Fu, Yining Zhu, Qiangguo Jin, Hongsong Wang, Yuke Li, Xuecheng Wu, and Kun Hu. MirrorDiff: Learning Mirror Diffusion for Image Captioning via Regeneration. In *Proceedings of the 2025 International Conference on Multimedia Retrieval*, pages 1331–1339. Association for Computing Machinery, 2025. doi: 10.1145/3731715.3733389. URL https://doi.org/10.1145/3731715.3733389.

[366] Junbo Wang, Liangyu Fu, Yuke Li, Yining Zhu, Ya Jing, Xuecheng Wu, and Jiangbin Zheng. DiffVC: A Non-autoregressive Framework Based on Diffusion Model for Video Captioning. arXiv preprint arXiv:2604.08084, 2026. URL https://arxiv.org/abs/2604.08084.

[367] Prince Zizhuang Wang and William Yang Wang. Riemannian Normalizing Flow on Variational Wasserstein Autoencoder for Text Modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 284–294. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1025. URL https://aclanthology.org/N19-1025/.

[368] Renzhi Wang, Jing Li, and Piji Li. InfoDiffusion: Information Entropy Aware Diffusion Process for Non-Autoregressive Text Generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 13757–13770, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.919. URL https://aclanthology.org/2023.findings-emnlp.919/.

[369] Rose E. Wang, Esin Durmus, Noah Goodman, and Tatsunori B. Hashimoto. Language Modeling via Stochastic Processes. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=pMQwKL1yctf.

[370] Ruoyu Wang, Yangfan He, Tengjiao Sun, Xiang Li, and Tianyu Shi. UniTMGE: Uniform Text-Motion Generation and Editing Model via Diffusion. In *Proceedings of the Winter Conference on Applications of Computer Vision (WACV)*, pages 6104–6114, 2025. doi: 10.1109/wacv61041.2025

.00595. URL https://openaccess.thecvf.com/content/WACV2025/html/Wang_UniTMGE_Uniform_Text-Motion_Generation_and_Editing_Model_via_Diffusion_WACV_2025_paper.html.

[371] Weitian Wang, Lianlei Shan, Shubham Rai, Cecilia De La Parra, and Akash Kumar. x-Prediction Flow: Efficient Continuous Decoding for Masked Diffusion Language Models. arXiv preprint arXiv:2606.29066, 2026. URL https://arxiv.org/abs/2606.29066.

[372] Wenlin Wang, Zhe Gan, Hongteng Xu, Ruiyi Zhang, Guoyin Wang, Dinghan Shen, Changyou Chen, and Lawrence Carin. Topic-Guided Variational Autoencoders for Text Generation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 166–177. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1015. URL https://aclanthology.org/N19-1015/.

[373] Yuchi Wang, Shuhuai Ren, Rundong Gao, Linli Yao, Qingyan Guo, Kaikai An, Jianhong Bai, and Xu Sun. LaDiC: Are Diffusion Models Really Inferior to Autoregressive Counterparts for Image-to-Text Generation? In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6699–6715, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.373. URL https://aclanthology.org/2024.naacl-long.373/.

[374] Zhongyao Wang. Why Do Few-Step Text Latents Fail When Image Latents Work? Non-Commitment at Sharp Categorical Readouts. arXiv preprint arXiv:2606.30705, 2026. URL https://arxiv.org/abs/2606.30705.

[375] Jędrzej Warczyński, Ondřej Dušek, and Mateusz Lango. One-step Nonautoregressive Natural Language Generation with Shortcut Flow Matching Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 646–655. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-short.53. URL https://aclanthology.org/2026.acl-short.53/.

[376] Penghui Wei. Ideation: Diversifying Ad Text Generation in Sponsored Search with Latent Diffusion. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 1394–1397. Association for Computing Machinery, 2025. doi: 10.1145/3701716.3715571. URL https://doi.org/10.1145/3701716.3715571.

[377] Yujie Wei and Kui Jin. Conditional Question Generation Model Based on Diffusion Model. *Advances in Computer, Signals and Systems*, 8(7):37–42, 2024. doi: 10.23977/acss.2024.080705. URL https://clausiuspress.com/article/14491.html.

[378] Daniel Wesego and Pedram Rooshenas. Multimodal ELBO with Diffusion Decoders. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 7644–7653, 2026. URL https://openaccess.thecvf.com/content/CVPR2026W/A2A-MML/html/Wesego_Multimodal_ELBO_with_Diffusion_Decoders_CVPRW_2026_paper.html.

[379] Bernardo Williams, Victor M. Yeom-Song, Marcelo Hartmann, and Arto Klami. Simplex-to-Euclidean Bijections for Categorical Flow Matching. In Emtiyaz Khan, Yingzhen Li, Arno Solin, and Aaditya Ramdas, editors, *Proceedings of the 29th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 300 of *Proceedings of Machine Learning Research*, pages 280–288. PMLR, 02–05 May 2026. URL https://proceedings.mlr.press/v300/williams26a.html.

[380] Junde Wu, Minhao Hu, Jiayuan Zhu, Yuyuan Liu, Tianyi Zhang, Kang Li, Jingkun Chen, Jiazhen Pan, Qirong Ho, Min Xu, and Yueming Jin. Evo: Autoregressive–Diffusion Large Language Models with Evolving Balance. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 3108–3121, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.emnlp-main.134/.

[381] Shengguang Wu, Mei Yuan, and Qi Su. DiffuVST: Narrating Fictional Scenes with Global-History-Guided Denoising Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1885–1896, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.126. URL https://aclanthology.org/2023.findings-emnlp.126/.

[382] Tong Wu, Zhihao Fan, Xiao Liu, Hai-Tao Zheng, Yeyun Gong, Yelong Shen, Jian Jiao, Juntao Li, Zhongyu Wei, Jian Guo, Nan Duan, and Weizhu Chen. AR-Diffusion: Auto-Regressive Diffusion Model for Text Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 39957–39974. Curran Associates, Inc., 2023. doi: 10.52202/075280-1737. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/7d866abba506e5a56335e4644ebe18f9-Abstract-Conference.html.

[383] Yiming Wu, Wei Ji, Kecheng Zheng, Zicheng Wang, and Dong Xu. MoTe: Learning Motion-Text Diffusion Model for Multiple Generation Tasks. arXiv preprint arXiv:2411.19786, 2024. URL https://arxiv.org/abs/2411.19786.

[384] Yunshu Wu, Jiayi Cheng, Longxuan Yu, Partha Thakuria, Rob Brekelmans, Evangelos E. Papalexakis, and Greg Ver Steeg. Discrete Stochastic Localization for Non-autoregressive Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=Jf5a5Sqnz8.

[385] Zhengxian Wu, Juan Wen, Yiming Xue, Ziwei Zhang, and Yinghan Zhou. GTSD: Generative Text Steganography Based on Diffusion Model. In *Neural Information Processing (ICONIP 2024)*, volume 2293 of *Communications in Computer and Information Science*, pages 168–183. Springer, 2025. doi: 10.1007/978-981-96-7005-5_12. URL https://doi.org/10.1007/978-981-96-7005-5_12.

[386] Kejing Xia, Mingzhe Li, Lixuan Wei, Zhenbang Du, Xiangchi Yuan, Dachuan Shi, Qirui Jin, and Wenke Lee. MetaState: Persistent Working Memory Enhances Reasoning in Discrete Diffusion Language Models. In *Conference on Language Modeling (COLM)*, 2026. URL https://proceedings.colm.cc/paper_files/paper/2026/hash/d5a655b8b373737b4f2aea8f78e5e754-Abstract-Conference.html.

[387] Jianxiang Xiang, Zhenhua Liu, Haodong Liu, Yin Bai, Jia Cheng, and Wenliang Chen. DiffusionDialog: A Diffusion Model for Diverse Dialog Generation with Latent Space. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 4912–4921, Torino, Italia, May 2024. ELRA and ICCL. URL https://aclanthology.org/2024.lrec-main.440/.

[388] Tianyu Xie, Shuchen Xue, Zijin Feng, Tianyang Hu, Jiacheng Sun, Zhenguo Li, and Cheng Zhang. Variational Autoencoding Discrete Diffusion with Enhanced Dimensional Correlations Modeling. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=yh7MV2V0ba.

[389] Dancheng Xin, Jiawei Yuan, and Yang Li. Diffusion Based Counterfactual Augmentation for Dual Sentiment Classification. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 4901–4911. ELRA and ICCL, 2024. URL https://aclanthology.org/2024.lrec-main.439/.

[390] Dancheng Xin, Kaiqi Zhao, Jingyun Sun, and Yang Li. CDA^2: Counterfactual Diffusion Augmentation for Cross-Domain Adaptation in Low-Resource Sentiment Analysis. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 61–72. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.coling-main.6/.

[391] Fan Xu, Sumin Liu, Keyu Yan, and Baishun Li. Diversify Question Generation Based on Diffusion Model in Dialogue. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E108.A(12):1687–1697, 2025. doi: 10.1587/transfun.2025EAP1042. URL https://www.jstage.jst.go.jp/article/transfun/E108.A/12/E108.A_2025EAP1042/_article.

[392] Manjie Xu, Chenxing Li, Yong Ren, Xinyi Tu, Ruibo Fu, Wei Liang, and Dong Yu. Towards Diverse and Efficient Audio Captioning via Diffusion Models. In *Proceedings of Interspeech 2025*, pages 191–195, 2025. doi: 10.21437/Interspeech.2025-79. URL https://www.isca-archive.org/interspeech_2025/xu25_interspeech.html.

[393] Minkai Xu, Tomas Geffner, Karsten Kreis, Weili Nie, Yilun Xu, Jure Leskovec, Stefano Ermon, and Arash Vahdat. Energy-Based Diffusion Language Models for Text Generation. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/535baca36883a4e0450423b26b150a48-Abstract-Conference.html.

[394] Shitong Xu. CLIP-Diffusion-LM: Apply Diffusion Model on Image Captioning. arXiv preprint arXiv:2210.04559, 2022. URL https://arxiv.org/abs/2210.04559.

[395] Weijie Xu, Wenxiang Hu, Fanyou Wu, and Srinivasan H. Sengamedu. DeTiME: Diffusion-Enhanced Topic Modeling using Encoder-decoder based LLM. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9040–9057. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.606. URL https://aclanthology.org/2023.findings-emnlp.606/.

[396] Xingqian Xu, Zhangyang Wang, Gong Zhang, Kai Wang, and Humphrey Shi. Versatile Diffusion: Text, Images and Variations All in One Diffusion Model. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7754–7765, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Xu_Versatile_Diffusion_Text_Images_and_Variations_All_in_One_Diffusion_ICCV_2023_paper.html.

[397] Yuanfeng Xu, Yuhao Chen, Liang Lin, and Guangrun Wang. Bridging the Discrete-Continuous Gap: Unified Multimodal Generation via Coupled Manifold Discrete Absorbing Diffusion. arXiv preprint arXiv:2601.04056, January 2026. URL https://arxiv.org/abs/2601.04056.

[398] Kaiwen Xue, Yuhao Zhou, Shen Nie, Xu Min, Xiaolu Zhang, Jun Zhou, and Chongxuan Li. Unifying Bayesian Flow Networks and Diffusion Models through Stochastic Differential Equations. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 55656–55681. PMLR, July 2024. URL https://proceedings.mlr.press/v235/xue24d.html.

[399] Bing Yan, Ge Sun, Juan J. de Pablo, Stefano Martiniani, and Kyunghyun Cho. ReasonFlow: Flow Matching that Thinks Step by Step. In Hung-Yi Lee, Sunipa Dev, Jessy Li, and Desmond Elliott, editors, *Findings of the Association for Computational Linguistics: EMNLP 2026*, pages 48554–48571, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.findings-emnlp.2268/.

[400] Da-Hee Yang and Joon-Hyuk Chang. Latent-Level Enhancement with Flow Matching for Robust Automatic Speech Recognition. *IEEE Signal Processing Letters*, 33:589–593, 2026. doi: 10.1109/LSP.2026.3653238. URL https://arxiv.org/abs/2601.04459.

[401] Haoning Yang and Lili Pan. GRAFT: Restoring the Missing Sampling Interface in Continuous Diffusion Language Models. arXiv preprint arXiv:2610.29136, 2026. URL https://arxiv.org/abs/2610.29136.

[402] Ningyuan Yang, Weihua Du, Weiwei Sun, Andre Wang He, Sean Welleck, and Yiming Yang. SimpLM: Simplex-Based Flow Language Modeling via Autoregressive Adaptation. In *Workshop on Non-Autoregressive Language Models for Fast and Flexible Text Generation (NonAR-LM) at COLM 2026*, 2026. URL https://openreview.net/forum?id=30EfdF7US1.

[403] Tianyu Yang, Thy Thy Tran, and Iryna Gurevych. Dior-CVAE: Pre-trained Language Models and Diffusion Priors for Variational Dialog Generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4718–4735, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.313. URL https://aclanthology.org/2023.findings-emnlp.313/.

[404] Yu Yang and Xiaotong Shen. Boosting Summarization with Normalizing Flows and Aggressive Training. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2727–2751. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.165. URL https://aclanthology.org/2023.emnlp-main.165/.

[405] Zhihan Yang, Wei Guo, Jean-Marie Lemercier, Simon Welker, Yonggan Fu, Mohammad Mahdi Kamani, Sajad Norouzi, Julius Berner, Tomas Geffner, Karsten Kreis, Yongxin Chen, Molei Tao, John Thickstun, Pavlo Molchanov, Ante Jukić, Arash Vahdat, and Morteza Mardani. Large Language Continuous Diffusion Models. arXiv preprint arXiv:2610.02665, 2026. URL https://arxiv.org/abs/2610.02665.

[406] Zhihan Yang, Wei Guo, Shuibai Zhang, Subham Sekhar Sahoo, Yongxin Chen, Arash Vahdat, Morteza Mardani, and John Thickstun. Continuous Diffusion Scales Competitively with Discrete Diffusion for Language. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://arxiv.org/abs/2605.18530v3. Updated results from arXiv v3 (24 October 2026).

[407] Jiacheng Ye, Shansan Gong, Liheng Chen, Lin Zheng, Jiahui Gao, Han Shi, Chuan Wu, Xin Jiang, Zhenguo Li, Wei Bi, and Lingpeng Kong. Diffusion of Thought: Chain-of-Thought Reasoning in Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 105345–105374, 2024. doi: 10.52202/079017-3343. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/be30024e7fa2c29cac7a6dafcbb8571f-Abstract-Conference.html.

[408] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion Large Language Models. arXiv preprint arXiv:2508.15487, 2025. URL https://arxiv.org/abs/2508.15487.

[409] Jiasheng Ye, Zaixiang Zheng, Yu Bao, Lihua Qian, and Mingxuan Wang. DINOISER: Diffused Conditional Sequence Learning by Manipulating Noises. *Transactions of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2302.10025.

[410] Yushi Ye, Feng Hong, Huangjie Zheng, Xu Chen, Zhiyong Chen, Yanfeng Wang, and Jiangchao Yao. Rejection Mixing: Fast Semantic Propagation of Mask Tokens for Efficient DLLM Inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17619–17629, 2026. URL https://openaccess.thecvf.com/content/CVPR2026/html/Ye_Rejection_Mixing_Fast_Semantic_Propagation_of_Mask_Tokens_for_Efficient_CVPR_2026_paper.html.

[411] Zihuiwen Ye, Elle Michelle Yang, and Phil Blunsom. Latent Diffusion for Document Generation with Sequential Decoding. In *Workshop on Diffusion Models at NeurIPS 2023*, 2023. URL https://neurips.cc/virtual/2023/74876.

[412] Jeong Hun Yeo, Chae Won Kim, Hyeongseop Rha, and Yong Man Ro. Diffusion Large Language Models for Visual Speech Recognition. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 41596–41610. Association for Computational Linguistics, 2026. URL https://aclanthology.org/2026.emnlp-main.1826/.

[413] Kyeongmin Yeo and Minhyuk Sung. MUNITE: Unified Multimodal Latent Inference for Any-to-Any Multimodal Generation. arXiv preprint arXiv:2610.09866, 2026. URL https://arxiv.org/abs/2610.09866.

[414] Kyeongmin Yeo, Yunhong Min, and Minhyuk Sung. MUNI: Multimodal Unified Latent Diffusion for Coherent Any-to-Any Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=AWvbbkHyEX.

[415] Qiuhua Yi, Xiangfan Chen, Chenwei Zhang, Zehai Zhou, Linan Zhu, and Xiangjie Kong. Diffusion models in text generation: a survey. *PeerJ Computer Science*, 10:e1905, 2024. doi: 10.7717/peerj-cs.1905. URL https://doi.org/10.7717/peerj-cs.1905.

[416] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step Diffusion with Distribution Matching Distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6613–6623, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Yin_One-step_Diffusion_with_Distribution_Matching_Distillation_CVPR_2024_paper.html.

[417] Jaehoon Yoo, Wonjung Kim, Floor Eijkelboom, Chanhyuk Lee, Nicholas M. Boffi, Seunghoon Hong, and Jinwoo Kim. Self-conditioned Flow Map Language Models via Fixed-point Flows. arXiv preprint arXiv:2607.00714, 2026. URL https://arxiv.org/abs/2607.00714v1.

[418] Jaehoon Yoo, Wonjung Kim, Chanhyuk Lee, and Seunghoon Hong. Infinite Mask Diffusion for Few-Step Distillation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pages 149532–149558. PMLR, 2026. URL https://proceedings.mlr.press/v306/yoo26b.html.

[419] Hanlin Yu, Sibylle Marcotte, Arto Klami, Christian A. Naesseth, and Omar Chehab. Faster Density Estimation and Inference-Time Steering for Flow Language Models. In *Workshop on Beyond Next Token Prediction: Diffusion and Flow Models for Next-Generation Decoding (BeNTo) at NeurIPS 2026*, 2026. URL https://openreview.net/forum?id=gD97exlJSH.

[420] Longxuan Yu, Yunshu Wu, Yu Fu, Siheng Xiong, Rob Brekelmans, Hui Liu, Yue Dong, and Greg Ver Steeg. DSL-LLaDA: Scaling Continuous Denoising to 8B Masked Diffusion LMs. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, pages 44727–44752, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.findings-emnlp.2101/.

[421] Peiyu Yu, Sirui Xie, Xiaojian Ma, Baoxiong Jia, Bo Pang, Ruiqi Gao, Yixin Zhu, Song-Chun Zhu, and Ying Nian Wu. Latent Diffusion Energy-Based Model for Interpretable Text Modeling. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 25702–25720. PMLR, 2022. URL https://proceedings.mlr.press/v162/yu22h.html.

[422] Runpeng Yu, Qi Li, and Xinchao Wang. Discrete Diffusion in Large Language and Multimodal Models: A Survey. *Transactions on Machine Learning Research*, 2026. URL https://openreview.net/forum?id=0DsqnkP8Cp.

[423] Xiaofei Yu, Yitong Li, Jie Ma, Chang Li, and Hanlin Wu. Diffusion-RSCC: Diffusion Probabilistic Model for Change Captioning in Remote Sensing Images. *IEEE Transactions on Geoscience and Remote Sensing*, 63:1–13, 2025. doi: 10.1109/TGRS.2025.3554360. URL https://arxiv.org/abs/2405.12875.

[424] Xinlei Yu, Zhangquan Chen, Yongbo He, Tianyu Fu, Guanting Dong, Cheng Yang, Chengming Xu, Yue Ma, Xiaobin Hu, Zhe Cao, Jie Xu, Guibin Zhang, Jiale Tao, Jiayi Zhang, Siyuan Ma, Kaituo Feng, Haojie Huang, Youxing Li, Ronghao Chen, Huacan Wang, Chenglin Wu, Zikun Su, Xiaogang Xu, Kelu Yao, Kun Wang, Chen Gao, Yue Liao, Ruqi Huang, Tao Jin, Zhucun Xue, Cheng Tan, Jiangning Zhang, Wenqi Ren, Yanwei Fu, Yong Liu, Yu Wang, Xiangyu Yue, Yu-Gang Jiang, and Shuicheng Yan. The Latent Space: Foundation, Evolution, Mechanism, Ability, and Outlook. arXiv preprint arXiv:2604.02029, 2026. URL https://arxiv.org/abs/2604.02029.

[425] Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Fei Huang, and Songfang Huang. Text Diffusion Model with Encoder-Decoder Transformers for Sequence-to-Sequence Generation. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 22–39, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.2. URL https://aclanthology.org/2024.naacl-long.2/.

[426] Yulin Yuan, Ying Zhang, and Xiangming Meng. One Latent, Many Tokens: Jointly Learning Compressed Embeddings for Efficient Language Diffusion. arXiv preprint arXiv:2609.33698, 2026. URL https://arxiv.org/abs/2609.33698.

[427] Yiming Zeng, Jinghan Cao, Zexin Li, Yiming Chen, Tao Ren, Zhuochun Li, Dawei Xiang, Xidong Wu, Shangqian Gao, and Tingting Yu. TreeDiff: AST-Guided Code Generation with Diffusion LLMs. In *Proceedings of the First Workshop on Structured Understanding, Retrieval, and Generation in the LLM Era (SURGeLLM 2026)*, pages 93–106. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.surgellm-1.5. URL https://aclanthology.org/2026.surgellm-1.5/.

[428] Chenlu Zhan, Yu Lin, Gaoang Wang, Hongwei Wang, and Jian Wu. MedM2G: Unifying Medical Multi-Modal Generation via Cross-Guided Diffusion with Visual Invariant. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11502–11512, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Zhan_MedM2G_Unifying_Medical_Multi-Modal_Generation_via_Cross-Guided_Diffusion_with_Visual_CVPR_2024_paper.html.

[429] Andrew Zhang, Anushka Sivakumar, Chia-Wei Tang, and Chris Thomas. Flexible-length Text Infilling for Discrete Diffusion Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 31344–31359. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1597. URL https://aclanthology.org/2025.emnlp-main.1597/.

[430] De Shuai Zhang. When Latent Geometry Is Not Enough: Draft-Conditioned Latent Refinement for Non-Autoregressive Text Generation. arXiv preprint arXiv:2605.15557, 2026. URL https://arxiv.org/abs/2605.15557.

[431] Haopeng Zhang, Xiao Liu, and Jiawei Zhang. DiffuSum: Generation Enhanced Extractive Summarization with Diffusion. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13089–13100, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.828. URL https://aclanthology.org/2023.findings-acl.828/.

[432] Juntao Zhang, Yuehuai Liu, Yu-Wing Tai, and Chi-Keung Tang. C3Net: Compound Conditioned ControlNet for Multimodal Content Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26886–26895, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Zhang_C3Net_Compound_Conditioned_ControlNet_for_Multimodal_Content_Generation_CVPR_2024_paper.html.

[433] Runzhe Zhang, Letian Chen, Wenpeng Zhang, Zhouhan Lin, and Peilin Zhao. FlowLM: Few-Step Language Modeling via Diffusion-to-Flow Adaptation. In *World Artificial Intelligence Conference Academic (WAICA 2026)*, 2026. URL https://arxiv.org/abs/2605.20199.

[434] Ruqian Zhang, Yijiao Zhang, Juan Shen, Zhongyi Zhu, and Annie Qu. Syntax-Guided Diffusion Language Models with User-Integrated Personalization. arXiv preprint arXiv:2510.01028, 2025. URL https://arxiv.org/abs/2510.01028.

[435] Shaorong Zhang, Longxuan Yu, Rob Brekelmans, Luhan Tang, Salman Asif, and Greg Ver Steeg. Generation Order and Parallel Decoding in Masked Diffusion Models: An Information-Theoretic Perspective. arXiv preprint arXiv:2602.00286, 2026. URL https://arxiv.org/abs/2602.00286.

[436] Shuai Zhang, Zijie Chen, Huachuan Qiu, Hongliang He, Lun Du, and Zhenzhong Lan. A Dominant Self-Conditioning Direction Drives Repetition in Unconditional Continuous Diffusion Language Models. arXiv preprint arXiv:2607.00588, 2026. URL https://arxiv.org/abs/2607.00588.

[437] Shujian Zhang, Lemeng Wu, Chengyue Gong, and Xingchao Liu. LanguageFlow: Advancing Diffusion Language Generation with Probabilistic Flows. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3893–3905, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.215. URL https://aclanthology.org/2024.naacl-long.215/.

[438] Yaojie Zhang, Linfeng Zhang, Bin Cui, and Xupeng Miao. DFlow: Enabling Verifier Information Flow in Block Diffusion Speculative Decoding. arXiv preprint arXiv:2609.06498, 2026. URL https://arxiv.org/abs/2609.06498.

[439] Yijie Zhang, Yiyang Shen, and Weiran Wang. Disentanglement of Variations with Multimodal Generative Modeling. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/581e1a06fa20f2c079dc5fb2db236335-Abstract-Conference.html.

[440] Yingji Zhang, Danilo S. Carvalho, and André Freitas. Learning Disentangled Semantic Spaces of Explanations via Invertible Neural Networks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2113–2134. Association for Computational Linguistics, August 2024. doi: 10.18653/v1/2024.acl-long.116. URL https://aclanthology.org/2024.acl-long.116/.

[441] Yizhe Zhang, Jiatao Gu, Zhuofeng Wu, Shuangfei Zhai, Josh Susskind, and Navdeep Jaitly. PLANNER: Generating Diversified Paragraph via Latent Language Diffusion Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 80178–80190, 2023. doi: 10.52202/075280-3513. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/fdba5e0a9b57fce03e89cc0cad0a24e9-Abstract-Conference.html.

[442] Zekai Zhang, Yunjie Tian, Yanjin He, Xiaoyan Zhang, Dongdi Zhao, Qing Qu, and Di Fu. Scaling and Distilling Text Embeddings for Better Diffusibility. arXiv preprint arXiv:2610.01016, 2026. URL https://arxiv.org/abs/2610.01016.

[443] Zihan Zhang, Jinqiao Dai, and Pin Yang. Flow-based Text Style Transfer Model. *Advanced Engineering Sciences*, 57(1):201–212, 2025. doi: 10.15961/j.jsuese.202200639. URL https://jsuese.scu.edu.cn/en/article/doi/10.15961/j.jsuese.202200639/.

[444] Zuming Zhang, Jie He, Yizhe Zhang, and Jeff Z. Pan. SkillFM: Generating Skills for LLM Agents via Latent Flow Matching. arXiv preprint arXiv:2609.39382, 2026. URL https://arxiv.org/abs/2609.39382.

[445] Xiangyu Zhao, Bo Liu, Qijiong Liu, Guangyuan Shi, and Xiao-Ming Wu. EasyGen: Easing Multimodal Generation with BiDiffuser and LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1351–1370. Association for Computational Linguistics, August 2024. doi: 10.18653/v1/2024.acl-long.74. URL https://aclanthology.org/2024.acl-long.74/.

[446] Guangting Zheng, Yiyuan Zhang, Tao Yang, Yunpeng Chen, Rui Zhu, Jiajun Deng, and Yanyong Zhang. GenFirst: Generation Before Reconstruction for Stable End-to-End Latent Generative Modeling. arXiv preprint arXiv:2608.29335, 2026. URL https://arxiv.org/abs/2608.29335.

[447] Guantian Zheng. StyleShield: Exposing the Fragility of AIGC Detectors through Continuous Controllable Style Transfer. arXiv preprint arXiv:2605.00924, 2026. URL https://arxiv.org/abs/2605.00924.

[448] Huangjie Zheng, Shansan Gong, Ruixiang Zhang, Tianrong Chen, Jiatao Gu, Mingyuan Zhou, Navdeep Jaitly, and Yizhe Zhang. Continuously Augmented Discrete Diffusion model for Categorical Generative Modeling. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/39b77b5e422b4e070e2811b73ea9bcf7-Abstract-Conference.html.

[449] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked Diffusion Models are Secretly Time-Agnostic Masked Models and Exploit Inaccurate Categorical Sampling. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/9e3b203e72c4e058de26d02a92a81844-Abstract-Conference.html.

[450] Shutian Zheng, Zhihong Yu, Linye Wei, Zeyang Zhang, and Meng Li. A Step in the Right Place: Understanding and Optimizing Timestep Placement in Continuous Diffusion Language Models. arXiv preprint arXiv:2610.15527, 2026. URL https://arxiv.org/abs/2610.15527.

[451] Linhao Zhong, Linyu Wu, Bozhen Fang, Tianjian Feng, Chenchen Jing, Wen Wang, Jiaheng Zhang, Hao Chen, and Chunhua Shen. Beyond Hard Masks: Progressive Token Evolution for Diffusion Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6936–6959, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.315. URL https://aclanthology.org/2026.acl-long.315/.

[452] Cai Zhou, Chenyu Wang, Dinghuai Zhang, Shangyuan Tong, Yifei Wang, Stephen Bates, and Tommi Jaakkola. Next Semantic Scale Prediction via Hierarchical Diffusion Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025. URL https://openreview.net/forum?id=kUzBGEuu7w.

[453] Cai Zhou, Chenxiao Yang, Yi Hu, Chenyu Wang, Chubin Zhang, Muhan Zhang, Lester Mackey, Tommi Jaakkola, Stephen Bates, and Dinghuai Zhang. Coevolutionary Continuous Discrete Diffusion: Make Your Diffusion Language Model a Latent Reasoner. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine*

Learning Research, pages 166143–166171. PMLR, 2026. URL https://proceedings.mlr.press/v306/zhou26bd.html.

[454] Fan Zhou and Tim Van de Cruys. Non-autoregressive Modeling for Sign-gloss to Texts Translation. In *Proceedings of Machine Translation Summit XX: Volume 1*, pages 220–230. European Association for Machine Translation, June 2025. URL https://aclanthology.org/2025.mtsummit-1.17/.

[455] Fan Zhou, Chang Tian, and Tim Van de Cruys. Controllable Stylistic Text Generation with Train-Time Attribute-Regularized Diffusion. arXiv preprint arXiv:2510.06386, 2025. URL https://arxiv.org/abs/2510.06386.

[456] Kun Zhou, Yifan Li, Wayne Xin Zhao, and Ji-Rong Wen. Diffusion-NAT: Self-Prompting Discrete Diffusion for Non-Autoregressive Text Generation. In Yvette Graham and Matthew Purver, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1438–1451, St. Julian's, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.eacl-long.86. URL https://aclanthology.org/2024.eacl-long.86/.

[457] Wangchunshu Zhou, Yuchen Eleanor Jiang, Ethan Wilcox, Ryan Cotterell, and Mrinmaya Sachan. Controlled Text Generation with Natural Language Instructions. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 42602–42613. PMLR, 2023. URL https://proceedings.mlr.press/v202/zhou23g.html.

[458] Ying Zhou, Xinyao Wang, Yulei Niu, Yaojie Shen, Lexin Tang, Fan Chen, Ben He, Le Sun, and Longyin Wen. DiffLM: Controllable Synthetic Data Generation via Diffusion Language Models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20638–20658, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1061. URL https://aclanthology.org/2025.findings-acl.1061/.

[459] Kangchen Zhu, Zhiliang Tian, Jingyu Wei, Ruifeng Luo, Yiping Song, and Xiaoguang Mao. StyleFlow: Disentangle Latent Representations via Normalizing Flow for Unsupervised Text Style Transfer. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 15384–15397. ELRA and ICCL, May 2024. URL https://aclanthology.org/2024.lrec-main.1336/.

[460] Qinglin Zhu, Yizhen Yao, Runcong Zhao, Yanzheng Xiang, Siya Qi, Amrutha Saseendran, Chen Jin, Philip A. Teare, Bin Liang, Yulan He, and Lin Gui. Latent Refinement Decoding: Enhancing Diffusion Language Models by Refining Belief States. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://openreview.net/forum?id=poXCPd4Q9D.

[461] Rui-Jie Zhu, Tianhao Peng, Tianhao Cheng, Xingwei Qu, Jinfa Huang, Dawei Zhu, Hao Wang, Kaiwen Xue, Xuanliang Zhang, Yong Shan, Tianle Cai, Taylor Kergan, Assel Kembay, Andrew Smith, Chenghua Lin, Binh Nguyen, Yuqi Pan, Yuhong Chou, Zefan Cai, Zhenhe Wu, Yongchi Zhao, Tianyu Liu, Jian Yang, Wangchunshu Zhou, Chujie Zheng, Chongxuan Li, Yuyin Zhou, Zhoujun Li, Zhaoxiang Zhang, Jiaheng Liu, Ge Zhang, Wenhao Huang, and Jason Eshraghian. A Survey on Latent Reasoning. arXiv preprint arXiv:2507.06203, 2025. URL https://arxiv.org/abs/2507.06203.

[462] Xiaochen Zhu, Georgi Karadzhov, Chenxi Whitehouse, and Andreas Vlachos. Segment-Level Diffusion: A Framework for Controllable Long-Form Generation with Diffusion Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4163–4183. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.210. URL https://aclanthology.org/2025.acl-long.210/.

[463] Yichen Zhu, Xiaoming Shi, Peng Zhao, Weiyu Chen, Debing Zhang, and James Kwok. Dynamic Chunking for Diffusion Language Models. arXiv preprint arXiv:2605.15676, 2026. URL https://arxiv.org/abs/2605.15676.

[464] Yonggang Zhu, Aidong Men, and Li Xiao. Diffusion-based diverse audio captioning with retrieval-guided Langevin dynamics. *Information Fusion*, 114:102643, 2025. doi: 10.1016/j.inffus.2024.102643. URL https://doi.org/10.1016/j.inffus.2024.102643.

[465] Yonggang Zhu, Yiming Zhang, Li Xiao, Wenwu Wang, and Aidong Men. Zero-shot diverse audio captioning with diffusion models. *Knowledge-Based Systems*, 335:115205, 2026. doi: 10.1016/j.knosys.2025.115205. URL https://doi.org/10.1016/j.knosys.2025.115205.

[466] Zhiyu Zhu and Junhui Hou. Geometry-aware Euclidean Diffusion Language Generation. Public manuscript on OpenReview, 2025. URL https://openreview.net/forum?id=v394xbZFlu.

[467] Shaobin Zhuang, Yuang Ai, Jiaming Han, Xiaohui Li, Huaibo Huang, Xiangyu Yue, Xuefeng Hu, Kun Xu, Yali Wang, and Hao Chen. BitLM: Unlocking Multi-Token Language Generation with Bitwise Continuous Diffusion. arXiv preprint arXiv:2605.11577, 2026. URL https://arxiv.org/abs/2605.11577.

[468] Yufan Zhuang, Liyuan Liu, Chandan Singh, Jingbo Shang, and Jianfeng Gao. Mixture of Inputs: Text Generation Beyond Discrete Token Sampling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025. doi: 10.52202/085713-0889. URL https://proceedings.nips.cc/paper_files/paper/2025/hash/25c61012c845908bff2ea6223cdc6192-Abstract-Conference.html.

[469] Zachary M. Ziegler and Alexander M. Rush. Latent Normalizing Flows for Discrete Sequences. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 7673–7682. PMLR, 2019. URL https://proceedings.mlr.press/v97/ziegler19a.html.

[470] Hao Zou, Zae Myung Kim, and Dongyeop Kang. A Survey of Diffusion Models in Natural Language Processing. arXiv preprint arXiv:2305.14671, 2023. URL https://arxiv.org/abs/2305.14671.

[471] Hao Zou, Zachary Horvitz, Chandhru Karthick, Zhou Yu, and Kathleen McKeown. Detect, Remask, Repair: Diffusion Editing for Faithful Summarization of Evolving Contexts. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 42763–42784, Budapest, Hungary, October 2026. Association for Computational Linguistics. URL https://aclanthology.org/2026.emnlp-main.1876/.

[472] Wei Zou, Ziyuan Zhuang, Xiang Geng, Shujian Huang, Jia Liu, and Jiajun Chen. Improved Paraphrase Generation via Controllable Latent Diffusion. *Frontiers of Computer Science*, 20(1): 2001302, 2026. doi: 10.1007/s11704-025-40633-9. URL https://arxiv.org/abs/2404.08938.